Report

1.0

Customer

Epineon



Fourth Review Round

NAPQES v4 — AEAD Scheme Audit

October 27, 2026

ABDK Consulting

Contents

1. Document Revisions **3**

2. Introduction **3**

3. System Overview **4**

4. Methodology **9**

5. Findings **10**

5.1. Notes for Users 11

5.2. Major 13

5.3. Moderate 31

5.4. Minor 115

6. Review of Fixes at Commit acbf9d0 **184**

6.1. Moderate 186

6.2. Minor 221

7. Conclusion **286**

8. Out-of-Scope Observations (Code Not Reviewed) **288**

9. Evidence Provenance: The Section 9 Statistical Report **290**

About Us **291**

Contacts **291**

1. Document Revisions

#	Date	Author	Description
0.1	September 21, 2026	Dmitry Khovratovich	Initial report draft.
0.2	October 20, 2026	Dmitry Khovratovich	Added the review of fixes.
1.0	October 27, 2026	Dmitry Khovratovich	Final report with client comments.

2. Introduction

ALL MODIFICATIONS TO THIS DOCUMENT ARE PROHIBITED. VIOLATORS WILL BE PROSECUTED TO THE FULL EXTENT OF THE U.S. LAW.

IT IS IMPORTANT TO NOTE THAT A SECURITY AUDIT IS NOT A GUARANTEE OF ABSOLUTE SECURITY BUT RATHER A SNAPSHOT OF THE SYSTEM'S SECURITY POSTURE AT A SPECIFIC POINT IN TIME. WHILE WE STRIVE TO UNCOVER ALL POTENTIAL ISSUES, THE EVOLVING NATURE OF BLOCKCHAIN TECHNOLOGY AND DEFI MEANS NEW VULNERABILITIES CAN EMERGE.

3. System Overview

NAPQES (Noise-Augmented Post-Quantum Encryption Scheme) is an Authenticated Encryption with Associated Data (AEAD) scheme whose security is claimed to reduce entirely to the pseudorandomness of `HMAC-SHA256`, deliberately avoiding the finite-field and permutation structures used by `AES-GCM` and `ChaCha20-Poly1305`. **This report audits the Rust reference implementation of NAPQES**, delivered for the first time in this engagement. The version 4 paper is used here only as the reference against which the delivered code was checked. The specification itself was audited separately, and those findings are reported in a separate document.

The specification of NAPQES v4. Because most of what follows is a comparison, the specified scheme has to be on the page. Version 4 uses two independently sampled secrets. The arithmetic-layer key $\mathbf{k} = (k_1, \ldots, k_K)$ is an ordered tuple of K distinct primes drawn uniformly without replacement from the set $\mathcal{P}$ of primes in $[10^6,\ 9.9\times10^6]$ (default $K = 10$), serialised as five-byte big-endian encodings; the derivation secret $\mathsf{sk} \in \{0,1\}^{256}$ is drawn uniformly and independently of it. The paper states that sk is the only secret that ever keys an HMAC call, and that **k** is used only, and un-keyed, by the arithmetic token map, so that no security theorem depends on the min-entropy of **k**; the first half of that statement does not hold for the delivered implementation, and we return to it below. The size of $\mathcal{P}$ is stated exactly, as $|\mathcal{P}| = 579{,}947$: we reproduced the count by sieve and confirm it, together with $H_\infty(\mathbf{k}) = 191.46$ bits and the post-Grover figure of $2^{95.73}$ at the default K.

All internal values are produced by a domain-separated derivation

$$\mathsf{Derive}_d(\mathsf{sk}, N, \mathsf{ctx}) = \mathsf{HMAC}(\mathsf{sk},\ d \,\|\, N \,\|\, \mathsf{ctx})$$

over nine domains covering the noise schedule, the padding codepoints, the per-position addends, the keystream and the authentication tag. The tag input is factored into a single named quantity, $\mathsf{TagIn}(A, \mathcal{B}) = \mathsf{be8}(|A|) \,\|\, A \,\|\, \mathcal{B}$, so the encryptor and the decryptor cannot disagree on the width of the length prefix. The nonce is synthetic, in the manner of RFC 5297 SIV:

$$N = \mathsf{HMAC}(\mathsf{sk},\ \texttt{0x0A} \,\|\, \mathsf{be8}(|A|) \,\|\, A \,\|\, M)[0{:}16],$$

so encryption is a pure function of the associated data and the message, and accidental nonce reuse across distinct inputs is a collision event rather than a sampling failure.

A plaintext of n codepoints is padded to a block size B given by a padding profile, a replaceable deployment parameter rather than a hard-wired rule. The normative default

is the power-of-two bucket

$$B = \mathsf{max}(16,\ 2^{\lceil \log_2(n+1) \rceil}),$$

with a stride-thinned ladder and a fixed-frame profile as alternatives. All three take values in the same thirteen-element set, so the set of legal ciphertext lengths is profile-independent and no decryptor needs to know which profile a sender used. Padding yields $R = B + 2$ real tokens after a two-codepoint length prefix. Each padded codepoint c becomes a token $c \cdot k + a$ for a prime $k \in \mathbf{k}$ and a derived addend $a \in [1, k-1]$, with pseudorandom noise tokens interleaved among the real ones. Runs of consecutive noise tokens are capped at MAX_NOISE_RUN $= 19$ and the stream is then padded with filler tokens to exactly $20R$ tokens, so the token count is a deterministic function of the padding bucket alone. Tokens are serialised in fixed-width eight-byte fields, masked with a keystream derived from domain `0x07`, and authenticated by a 32-byte tag over TagIn. The ciphertext is $C = N \,\|\, \mathbf{B} \,\|\, T$, of length $48 + 160(B + 2)$ bytes, a minimum of $2{,}928$ bytes for any message of at most fifteen codepoints. The message space is Unicode scalar values, excluding the surrogate range. Decryption carries two structural checks, both placed after tag verification: the recovered real-token count must equal $B+2$ for one of the thirteen reachable block sizes, and the recovered length prefix must not exceed the padded buffer.

The implementation as delivered. The Rust crate implements two wire formats side by side. The v8 format is the one the paper specifies; the v7 format is its predecessor, retained for compatibility, and differs in the key schedule, in the width of the associated-data length prefix and in the absence of the noise-run cap. Both live in `src/lib.rs`, which at 827 non-test nSLOC is 84% of the reviewable logic in scope and contains the entire AEAD: prime generation and key validation, the nine derivation domains, the padding and token map, the emission loop, the masking keystream, the tag, and the inverse of all of it. The v8 entry points are `encrypt_bytes_v8` and `decrypt_bytes_v8`; six further public functions expose the legacy v7 path, and `encrypt_raw` and `decrypt_raw` expose it to the crate's own transport modules.

Around `lib.rs` sit four smaller in-scope files. `src/self_test.rs` is the FIPS 140-3 power-on self-test module, executed at crate startup rather than under `cargo test`; 380 of its lines are nSLOC but 299 of those are a single embedded hexadecimal ciphertext constant, leaving 81 lines of logic. `src/kat_cross_check.rs` is the known-answer parity harness that consumes the JSON corpora and is the evidence behind the paper's byte-parity claim; it is gated behind `#[cfg(test)]`, so it compiles only under `cargo test`. `fuzz/fuzz_targets/decode_bytes.rs` and `examples/dudect_harness.rs` back the paper's fuzzing and constant-time claims respectively. `Cargo.toml` and `README.md` complete the in-scope set.

What the code computes, and what the specification requires. Our main conclusion is that the code and the specification compute different functions. Both v8 entry points begin by deriving a format subkey under a domain byte `0x0B` that appears nowhere in the paper,

$$\mathsf{sk}_{\mathsf{fmt}} = \mathsf{HMAC}\big(\mathsf{sk},\ \texttt{0x0B||0x01}\big),$$

and then thread $\mathrm{sk}_{\mathrm{fmt}}$, not sk, through every one of the nine tabulated domains — the synthetic nonce, the noise threshold and positions, the padding codepoints, both addend domains, the noise characters, the keystream and the tag. On the v8 path sk itself keys exactly one HMAC call per operation, for a domain the paper never mentions.

We verified this experimentally. An independent re-implementation of Enc, written from the paper's text only, was run against all twelve positive known-answer test vectors delivered for v8: keyed by sk exactly as the specification and its Table 1 caption require, **0 of 12** reproduce, and the disagreement is visible in the first sixteen bytes. With the single substitution $\mathrm{sk} \to \mathrm{sk}_{\mathrm{fmt}}$ and no other change, **12 of 12** reproduce byte for byte, ciphertext and tag included, across every padding bucket, every key size and every associated-data length in the test vectors. The implemented scheme is therefore exactly the specified scheme under one uniform key substitution: every byte string, field order and width, every truncation and modular reduction, the fixed-point noise-threshold arithmetic, the run cap and the tag-input layout are correct. The key derivation is the only difference between the two.

The extra layer binds every derivation to one wire format, so that a tag produced under one format cannot verify under another format that shares the same key material. All three implementations apply it consistently, so it is the specification that should be updated to describe it. Two corollaries follow and are developed in the findings below: the byte-identical conformance requirement of Section 10 cannot be satisfied by any implementation faithful to Table 1, so the specification's own conformance test rejects the specification; and because $\mathrm{sk}_{\mathrm{fmt}}$ is a pseudorandom output rather than a uniform draw, each of the four security theorems needs one additional $\mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC}}(\mathcal{B}_0)$ term for the extra hop. The composition is sound and the added term is negligible, so there is no loss of security. However, the theorems are proved for a slightly different function from the one the code computes.

/ (root directory)
Cargo.toml README.md

/docs/

/examples/
dudect_harness.rs

/fuzz/

/fuzz/fuzz_targets/
decode_bytes.rs

/src/
kat_cross_check.rs lib.rs self_test.rs

Scope of the review. The reviewed surface is the Rust reference implementation at commit `b8c3054`, restricted to the seven files named by the client in the scope note: `src/lib.rs`, `src/self_test.rs`, `src/kat_cross_check.rs`, `fuzz/fuzz_targets/decode_bytes.rs`, `examples/dudect_harness.rs`, `Cargo.toml` and `README.md`. That is **1,283 reviewable non-test Rust nSLOC**, or **984** once the 299-line embedded hexadecimal known-answer

constant in `self_test.rs` is set aside as a single reviewable fact rather than 299 lines of logic. `src/lib.rs` alone is 827 nSLOC, 84% of that reviewable logic. A further 508 lines of in-scope Rust are test code; they are not nSLOC, but they are audited surface under the scope note and were read in full. We record one discrepancy for the client's attention: the scope note sizes `lib.rs` at 1,420 lines and the delivered file is 1,530, so the file grew by 7.7% between the scope being agreed and the snapshot we received.

The specification is not the subject of this report. The version 4 paper, `napseq-eprint-v4.tex` (2,281 lines), was audited separately, and its findings are reported elsewhere. They are not restated here, and nothing in this report should be read as a certificate of correctness for the specification. The paper is used here only as the reference against which the code was checked. The findings that cite it describe places where the code and the specification differ, and they quote the paper to show what the code was meant to do.

What was not reviewed. The delivered tree is substantially larger than the reviewed surface, and the remainder was not audited. Outside scope, and not reviewed, are `src/kem.rs`, `src/kem_exchange.rs`, `src/ot_frame.rs`, the protocol adapters under `src/protocols/`, and the three binaries under `src/bin/` — together 1,983 non-test nSLOC, 1.55 times the in-scope figure, of which the 1,420 nSLOC of binaries carry no tests at all. Also outside scope are the Python reference suite under `tests/` (1,844 code lines, the reference from which the known-answer test vectors were generated) and the test-vector files themselves. This has two consequences. First, `src/lib.rs` — an in-scope file — declares the unreviewed modules as public modules of the audited crate, so a consumer of the crate sees reviewed and unreviewed cryptographic code as one public surface. Second, the byte-parity evidence the paper relies on establishes that the Rust code agrees with the Python reference implementation, not that either agrees with the specification; the reference implementation that would settle that has not been reviewed.

The two undelivered implementations. The paper names three reference implementations and asserts properties of all three. Only the Rust one has ever been delivered: `napqes.py` and `C/napqes.c` have not been supplied at any point in this engagement. Every claim in this report about agreement or disagreement between the implementations therefore rests on the delivered Rust source, on the stored known-answer test vectors, and on what the paper and the crate's own comments say about the other two implementations. Their code was not available to us. Where a finding turns on that distinction, it says so.

How the review was conducted. Two auditors reviewed the same files independently, and their results were then combined into the single set of findings presented below. Before a claim was kept, we checked it against the delivered source code: duplicates were merged into one finding, related but distinct claims were kept separate, and claims that could not be confirmed were dropped. The findings are the work of the team as a whole.

In addition, we compared the paper with the code in a dedicated conformance check, covering four areas in turn: the parameters and key material; the key derivations and the synthetic nonce; the padding, the placement of noise and the token encoding; and the wire format and the three algorithms. In each area, we matched every normative statement of the paper with the code that implements it and marked it as conforming or not. The great majority of statements conform, and the conformance findings below describe the differences. The most important of them is the extra key step described in the system overview: an independent re-implementation of Enc written from the paper alone reproduces **0 of 12** positive known-answer test vectors when it uses the key that the specification prescribes, and **12 of 12** with the key that the code actually uses. We ran this experiment twice, independently, with the same result.

Verification method. No client code was compiled, executed, fuzzed or benchmarked at any point in this engagement. Every statement in this report about the implementation was established by reading the code. Where we make a numeric claim, such as prime counts and densities, entropy figures, iteration counts, token sizes or collision cases, we recomputed it independently, in code written for this review and run on data generated for it, never on client artifacts. The known-answer experiment described above reproduces the client's stored vectors with an independent encryptor; it does not run the client's encryptor.

Reviewed Commits

Repository	Commit	Role
`Synergeon-epineon/NAPQES`	`b8c3054`	Baseline
`Synergeon-epineon/NAPQES`	`acbf9d0`	With fixes

The baseline, `b8c3054ac256d10e26d4b77c849a3abf9a158974`, is the code reviewed in Section 5. The commit with fixes, `acbf9d05bd4ce93a77d91e2d2a5e9a6ed4a302e1`, contains the client's fixes for the findings in Section 5 and is the code reviewed in Section 6. Fixes the client made after it, for the findings in Section 6, have not been reviewed. For both commits, every in-scope file delivered for review matches the published commit byte for byte.

4. Methodology

The methodology utilized by ABDK Consulting is a combination of established audit tactics and methods specifically tuned for the requirements and technological stack of the NAPQES v4 — AEAD Scheme Audit project. This approach is flexible and adapted based on the specific project structure, technologies used, and the client's expectations for the security review.

Our audit process incorporates several distinct phases of analysis:

General Code Assessment
The codebase is first reviewed for clarity, consistency, style, and adherence to established best practices. This phase includes checking indentation standards, naming conventions (e.g., following a style guide), appropriate commenting, code duplication, and confusing or irrelevant naming. At this phase, we establish an understanding of the overall system structure and codebase organization.

Entity Usage Analysis
Usages of various entities (contracts, libraries, functions, state variables) defined in the code are analyzed. This includes both internal usages from other parts of the system as well as potential external interactions. We verify that entities are defined with appropriate visibility scopes (internal, external, public, private) and access levels. At this phase, we understand the overall system architecture and how different parts of the code are related and interact with external interfaces.

Access Control Analysis
For those entities that can be accessed externally or manage sensitive operations (like data submission or role management), access control measures are rigorously analyzed. We check that access control mechanisms are correctly implemented, relevant to the intended function, and robust against circumvention. At this phase, we understand the defined user roles, permissions logic, and what digital assets the system ought to protect.

Code Logic Analysis
The specific code logic of critical functions is analyzed for correctness, efficiency, and security. We verify that the code actually performs its intended function as described in the system overview, that algorithms are optimal for on-chain execution, and that proper data types are used to prevent overflow/underflow issues. We also ensure that external libraries used are up to date and relevant. At this phase, we understand the data structures used and their specific purposes.

We classify all identified issues uncovered during these phases by the following severity levels:

Critical
Directly affects the smart contract functionality and may cause a significant loss of assets (e.g., permanent lock of funds, unauthorized minting, total system compromise) or a complete failure of the platform's core invariants.

Major
Is either a solid performance problem or a sign of significant misuse potential: a slight code modification or environment change may lead to loss of funds or data integrity issues. Sometimes it is an abuse of unclear code behavior which should be double-checked by the client for specific edge-case business logic.

Moderate
Is not an immediate functional problem but rather suboptimal performance in edge cases, an obviously bad code practice, or a situation where the code is correct only in certain expected business flows (lacking robust handling of all possible inputs).

Minor
Covers code style improvements, adherence to best practices that do not immediately present a security risk, and general suggestions for code clarity, gas efficiency, or future-proofing the system.

5. Findings

We have completed our technical audit of the NAPQES v4 — AEAD Scheme Audit codebase and provided the client with a detailed list of recommendations and identified vulnerabilities. The following statistics summarize the issues discovered and their current resolution status at the time of this report's finalization.

	Found	Fixed
Major	3	2
Moderate	17	8

	Found	Fixed
Minor	**17**	**15**

Fixed 25 out of 37 issues reported.

5.1. Notes for Users

This subsection collects what the findings of this report, and the client's comments on them, mean for developers who integrate the NAPQES Rust crate and for operators who deploy it: what the reviewed code does that a user might not expect, and how to act accordingly. The findings describe commit `acbf9d0`. The client has since reported changes for every finding in Section 6; where an item below describes such a change, it is the client's statement, which ABDK has not verified. The finding cited in each item gives the detail.

Use the block format for protected data.

The specification makes no security claim for the v8 streaming format (format identifier `0x02`). In the reviewed code, its closing sentinel carried the exact plaintext length in the clear, and an empty stream did not authenticate its frame size (CVF-V5-1, CVF-V5-2, CVF-V5-9). The Rust stream decryptor holds the whole stream in memory, about 160 bytes per plaintext codepoint, and returns nothing until the stream has verified (CVF-V5-10). The client reports that streaming now compiles only with the off-by-default `experimental-stream` feature, is outside the audited claims, and has the length and header issues fixed. Data that needs the guarantees the specification proves should be encrypted with the block format. The client also reports that its Python stream decryptor releases each chunk before the stream has verified, so a consumer of that port must not act on any chunk until the whole stream has.

Use v7 only to read archived data.

In the reviewed code, the OT gateway sealed every frame with the legacy v7 format, whose emission loop has no noise-run cap and whose key schedule derives everything from the prime tuple alone (CVF-V5-4, CVF-V5-7). The client reports that the gateway now uses v8, that the v7 encryptors compile only with the off-by-default `legacy-v7` feature, and that the v7 decryptors remain for archived data under a relaxed key policy (CVF-V5-16). The v7 tag does not record whether a plaintext was encoded as bytes or as codepoints. Decrypting with the function that does not match the encryptor therefore succeeds silently and returns a different plaintext for characters up to U+00FF, so `decrypt_raw` must be used only for the output of `encrypt_raw` (CVF-V5-25).

Hold keys in `NapqesKey`.
A function that takes a key as a bare slice validates it on every call by trial division, whose running time depends on the secret primes; in the reviewed code this also ran on every v7 decryption before the tag was checked (CVF-V5-12, CVF-V5-29). A key held in `NapqesKey` is validated once, when it is constructed. The client reports that the streaming functions now take a `NapqesKey`, that the bare-slice v8 functions are deprecated, that v7 decryption checks the tag first, and that the additional unwiped copies of key material found in the reviewed code are now wiped (CVF-V5-11).

Build with `fips_gate` where the self-tests must gate use.
In the reviewed code the power-on self-test gate was off by default. Even when enabled, it left the v7 entry points and key generation ungated, allowed a failed self-test to be cleared by running it again, and did not record a failure of the continuous random-number-generator test (CVF-V5-8, CVF-V5-19). The client reports that under `fips_gate` every entry point now requires the self-test and a failure persists until the process restarts, but the feature stays off by default: a default build operates whether or not the self-tests have run or passed. The client also reports that the software-integrity test compares the executable with a reference value written after linking by the `napqes_integrity` tool, and that a missing reference value fails the test only with the `fips_integrity` feature (CVF-V5-5). A deployment relying on that test must run the tool as part of its release process and enable both features.

Agree on the key parameters.
The interval for key elements was widened from $[10^6, 9.9 \times 10^6]$ to $[10^6, 1.5 \times 10^7)$, and the default number of key elements raised from 10 to 13 (CVF-V5-3). The client reports that it keeps these values and now labels the two intervals as parameter sets P1 and P2. An implementation that follows an earlier revision of the specification rejects keys with an element above 9.9×10^6, so all parties should use the same revision. For the v8 format the security theorems rest on the separate 256-bit secret sk, not on the entropy of the prime tuple; that entropy matters for v7, where the tuple is the whole key.

The crate contains code this review did not cover.
Every build of the crate includes the key-establishment, gateway and industrial-protocol modules (`kem`, `kem_exchange`, `ot_frame` and `protocols`), which were outside the scope of this review (CVF-V5-6). The client reports that it has removed some optional components but still compiles these modules unconditionally, so a consumer that needs only the AEAD links them as well.

5.2. Major

CVF-11: fixed Category: **behavior**

Summary

The Rust implementation encrypts data differently from the way the specification describes, so software written from the specification cannot work with it.

Before it uses its secret key, the code passes the key through an extra step that the specification does not mention, and every value the scheme computes depends on the result. An implementation written only from the specification therefore cannot decrypt data encrypted by this code, and the code cannot decrypt data encrypted by such an implementation. We confirmed this experimentally with the client's own test vectors: following the specification reproduced none of the twelve, and adding the undocumented step reproduced all twelve. The scheme is no weaker for it; the problem is that the specification does not describe what the code does.

Section 3.3 defines every keyed derivation in the scheme as

$$\mathrm{Derive}_d(\mathrm{sk}, N, \mathrm{ctx}) = \mathrm{HMAC}(\mathrm{sk},\ d \| N \| \mathrm{ctx})$$

, and the caption of Table 1 makes the claim explicitly: "Every domain is keyed by sk only."

The Rust implementation never uses sk directly as the key of any of these derivations. Both v8 entry points first compute a format subkey under a domain byte `0x0B`:

```
fn derive_format_subkey(sk: &[u8], format_id: u8) -> [u8; 32] {
    hmac_digest(sk, &[0x0B, format_id])
}
```

and then use `sk_fmt`, not `sk`, as the key of every derivation: the synthetic nonce (`0x0A`), the noise threshold (`0x02`), the padding codepoints (`0x06`), the noise positions (`0x00`), both addend domains (`0x01`, `0x05`), the noise characters (`0x04`), the keystream (`0x07`) and the authentication tag (`0x03`). The key that effectively protects every message is therefore $\mathrm{HMAC}(\mathrm{sk},\ \texttt{0x0B} \| \texttt{0x01})$.

Domain `0x0B` does not appear in the specification. The 2,281-line source contains no occurrence of `0x0B`, of "subkey" or of "format subkey", and Table 1 lists only the domains `0x00` to `0x07` and `0x0A`. The only pointer the code gives is to documents that were not submitted for review: the module comment cites `docs/napseq-eprint-preprint.tex` and `docs/CAVEATS.md` for the "V8 Key Schedule and Synthetic Nonce" specification.

Every derivation uses the extra key. We checked every place in the code where one of the derivations listed in Table 1 is computed, on both the encryption and the decryption side. All of them use `sk_fmt`:

- `0x00`, noise positions: `lib.rs:1160` (encrypt), `lib.rs:580` (decrypt);
- `0x01`, real-token addends: `lib.rs:1169` and `lib.rs:592`;
- `0x02`, noise threshold: `lib.rs:1150` and `lib.rs:571`;
- `0x03`, authentication tag: `lib.rs:1193` and `lib.rs:1217`;
- `0x04`, noise characters: `lib.rs:1162` and `lib.rs:1182`;
- `0x05`, noise-token addends: `lib.rs:1163` and `lib.rs:1183`;
- `0x06`, padding codepoints: `lib.rs:1152`;
- `0x07`, keystream: `lib.rs:1188` and `lib.rs:1223`;
- `0x0A`, synthetic nonce: `lib.rs:1144`.

On the v8 path, sk itself is the key of exactly one HMAC call per operation: the two-byte input `0x0B||0x01` at `lib.rs:1078`, for a domain the paper never mentions. There is no place in the code where sk is used directly, as the specification requires.

We confirmed the difference experimentally. We re-implemented `Enc` from the paper alone (`Definition 3.4`, Table 1, Section 3.4, Section 3.5, `Definition 3.6`, Listing 1, `Definition 3.8` and Section 3.9) in an independent script, and ran it against all twelve positive test vectors in `tests/kat/v8_vectors.json`. With sk as the key, exactly as the caption of Table 1 specifies, **0 of 12** reproduce. The difference is visible in the first sixteen bytes: for vector `W001`, the nonce computed from the paper is `be219a83d40477dc37de106c68d81dc3`, while the vector's nonce is `e5bb595068e24fd5630555256b5d3446`. With the single substitution $\mathsf{sk} \rightarrow \mathsf{sk}_{\mathsf{fmt}} = \mathrm{HMAC}(\mathsf{sk},\ \texttt{0x0B} \| \texttt{0x01})$ and no other change, **12 of 12** reproduce byte for byte, ciphertext and tag included, across every bucket ($B \in \{16, 32, 64\}$), every key size ($K \in \{1, 4, 10\}$) and every associated-data length in the test vectors. The implemented scheme is therefore exactly the specified scheme with one uniform change of key: every byte string, field order and width, every truncation and modular reduction, the $\theta(N)$ fixed-point arithmetic, the run cap, the token ceiling and the `TagIn` layout are all correct. The key is the only difference.

This has four consequences.

- **Three statements in the paper are incorrect:**

0x0A Synthetic nonce $\mathrm{Dec}(|T|/\|T\|\|T)$ 16 bytes

Table 1: Domain-separated HMAC derivation schedule. Every domain is keyed by sk only. The nonce N is applied uniformly by Derive_d for $d \neq \texttt{0x0A}$ and is therefore omitted from the Context column above.

Reference PDF p. 6 — Table 1's caption states the property the implementation does not have: "Every domain is keyed by sk *only."*

> function of its inputs (Definition 3.10), no nonce field appears and none is needed. Conforming implementations must produce byte-identical ciphertext for every positive vector and reject every negative vector. The generator `tests/gen_kats_v8.py` regenerates the file and verifies parity on

Reference PDF p. 25 — Section 10 requires that "Conforming implementations must produce byte-identical ciphertext for every positive vector" — a requirement no Table-1-faithful implementation can meet.

- the caption of Table 1, "Every domain is keyed by sk only" (lines 387–388);
- the abstract, "a uniform 256-bit secret sk that keys every HMAC domain derivation used for noise injection, plaintext padding, keystream masking, and message authentication" (lines 66–67);
- the Section 12 caveat *Scope of* sk *and* **k** *compromise*, "every HMAC domain derivation — the noise schedule, the padding, the per-position addends, the keystream, and the tag — is keyed by sk alone (Section 3.3)" (lines 2066–2070).

None of the three holds for any of the three implementations. `Lemma 3.5` is also affected, but the problem there has a different nature. The lemma states that the derivation inputs of different domains can never coincide, "on its *entire* argument domain", and its proof lists the domains it covers, $\{\texttt{0x00}, \texttt{0x01}, \texttt{0x02}, \texttt{0x04}, \texttt{0x05}, \texttt{0x06}, \texttt{0x07}\}$ plus `0x03` plus `0x0A` (lines 415–417), as if that list were complete. It is not: `0x0B` is missing from it. The lemma's conclusion still holds, because `0x0B||0x01` is a two-byte input, shorter than the input of every other domain (at least 17 bytes for the domains that take a nonce, and 9 for `0x0A`), so it cannot collide with any of them. The list in the proof should nevertheless include `0x0B` once Section 3.3 is fixed.

- **Section 10's conformance requirement cannot be met as written.** Lines 1992–1993 state that "Conforming implementations must produce byte-identical ciphertext for every positive vector". No implementation that follows Table 1 can meet that requirement, as we established experimentally: 0 of 12 vectors reproduce under the specified key. So the conformance test that Section 10 defines rejects any implementation of the paper's own specification. Because a different key changes every byte of the ciphertext, the test also gives an implementer no hint of the cause: the failure looks exactly like an ordinary programming error.
- **Every byte of every v8 ciphertext differs from the specified construction.** The difference lies in the key of the outermost pseudorandom function, so it reaches the nonce, the noise schedule, the addends, the padding, the keystream and the tag at once. As a result, the two constructions share only their byte layout.
- **Each of the four security bounds is missing a term.** $\mathrm{sk}_{\mathrm{fmt}}$ is the output of a pseudorandom function, not a uniformly random key, so `Assumption 4.2`, which is stated for a uniformly random key, does not apply to it directly. It applies after one extra step, in which the single call $\mathrm{HMAC}(\mathrm{sk}, \texttt{0x0B} \| \texttt{0x01})$ is replaced by a random value. `Theorem 4.8`, `Theorem 4.12`, `Theorem 4.18` and `Theorem 4.20` should therefore each carry an additional term $\mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC}}(\mathcal{B}_0)$ for that step. Adding such a step is a standard and safe construction, and the extra term is as small as HMAC's own security margin, so there is no loss of security. However, the theorems are proved for a slightly different function from the one the code computes.

We emphasise that the defect is in the paper; the implementation behaves correctly. The extra layer binds every derivation to one wire format, so that a tag produced under one format cannot verify under another format that shares the same $(\mathbf{k}, \mathsf{sk})$. It is also consistent across the three implementations: `derive_format_subkey` is documented as matching `napqes.py::_derive_format_subkey` and `C/napqes.c::derive_format_subkey`, and `kat_cross_check.rs` records that the Rust code once lacked it and that the omission "went undetected" because the v7 test vectors exercise no v8 code. The three implementations therefore agree with one another, and it is the paper that needs to be corrected.

The format identifiers. The code declares two identifiers, `FORMAT_BLOCK_V8: u8 = 0x01` and `FORMAT_STREAM_AE_V8: u8 = 0x02` (`lib.rs:1066` and `:1068`); there is no `FORMAT_AEAD_V8`. `FORMAT_BLOCK_V8` is used in exactly two places, `lib.rs:1143` and `:1213`. `FORMAT_STREAM_AE_V8` is not used anywhere in the delivered code: a search across `src/`, `tests/`, `examples/` and `fuzz/` finds only its declaration. The format identifier is never written into the ciphertext, since $C = N \| \mathcal{B} \| T$ carries no format byte; it enters only the key derivation, so a ciphertext produced under a different format subkey simply fails the tag check, exactly as if the key were wrong.

The three `api != "stream_ae"` filters in the parity harness (`kat_cross_check.rs:65`, `:126`, `:191`) are therefore not evidence of a second v8 wire format. The actual situation is the following. `tests/kat/v8_vectors.json` declares `spec_version: "v8"`, has no `api` field on any of its 20 vectors, and contains no streaming vector. All six `stream_ae` vectors are in `tests/kat/v6_vectors.json`, the older v6/v7 test vectors that the paper itself places out of scope, and `tests/gen_kats.py:5` records that those vectors "are unaffected and still use LEB128 tokens". In other words, the streaming format uses the older variable-width encoding, which was retired because the length of the encrypted data revealed information about the plaintext. The streaming API exists only in Python (`napqes.encrypt_stream_ae` / `decrypt_stream_ae`), and there is no Rust streaming entry point; the harness filters those vectors out because the Rust code cannot run them. So `0x02` is an allocated identifier with no v8 format behind it: the separation between formats that the subkey provides is real, but it is not tested yet, because only one v8 format exists.

Recommendation

Specify domain `0x0B` and the format-subkey layer in the paper, and then restate the derivations in terms of the key that is actually used.

Concretely, in Section 3.3:

- Add `0x0B` to Table 1 as the format-subkey domain, with context `format_id` and a 32-byte output, and note that it takes neither a nonce nor any other input.
- Give $\mathsf{sk}_{\mathsf{fmt}} = \mathrm{HMAC}(\mathsf{sk},\ \texttt{0x0B} \| \mathsf{format_id})$ its own definition, in the same way as `Definition 3.4` defines TagIn, and fix $\mathsf{format_id} = \texttt{0x01}$ for the block format specified here.
- Redefine the derivations as $\mathrm{Derive}_d(\mathsf{sk}_{\mathsf{fmt}}, N, \mathsf{ctx})$ for every $d \neq \texttt{0x0B}$, and replace the caption sentence of Table 1, "Every domain is keyed by sk only", with a statement that every domain other than `0x0B` is keyed by $\mathsf{sk}_{\mathsf{fmt}}$, and that `0x0B` alone is keyed by sk.
- Make the same correction in the two other places where the paper says that sk

is the key: the abstract (lines 66–67) and the Section 12 caveat *Scope of* sk *and* **k** *compromise* (lines 2066–2070). The point that caveat makes still holds: an attacker who learns sk computes $\mathsf{sk}_{\mathsf{fmt}}$ with a single HMAC call, and the trial-division search over $\mathcal{O}(|\mathcal{P}|)$ candidates then works exactly as described. Only the sentence about the key needs correcting.

- Add `0x0B` to the list of domains in `Lemma 3.5` (lines 415–417) and to its proof. The lemma's conclusion holds unchanged, because a two-byte input cannot coincide with the input of any other domain, all of which are at least nine bytes long.
- Explain why the layer exists, namely to keep tags of different formats apart, since that is the property it provides and a reader currently cannot see it.
- List the format identifiers explicitly, recording `0x01` for the block format and `0x02` for the streaming format, and say that the streaming format is outside the scope of this paper. Since `0x02` is currently declared in the Rust crate without a Rust implementation or any v8 test vectors, either describe it as reserved for future use or remove the constant; a declared but unused constant invites the reader to conclude that a second v8 format exists.

  Then carry the change through Section 4. Each theorem's reduction currently starts from HMAC keyed by a uniformly random sk; with the subkey layer it needs one more step, and the bounds of `Theorem 4.8`, `Theorem 4.12`, `Theorem 4.18` and `Theorem 4.20` each gain one term $\mathsf{Adv}^{\mathsf{PRF}}_{\mathsf{HMAC}}(\mathcal{B}_0)$ for the single `0x0B` call. `Assumption 4.2` already provides what is needed, since sk is uniform on $\{0,1\}^{256}$ and $\mathsf{sk}_{\mathsf{fmt}}$ is then pseudorandom; we recommend stating the extra step explicitly instead of leaving the reader to infer it.

  Finally, since Section 8.2 names `rust/src/lib.rs` as a reference implementation and Section 10 requires byte-identical output, add a known-answer test vector that a reader can reproduce from the paper's text alone: at minimum the format subkey for `format_id = 0x01` under a stated sk, and ideally also the synthetic nonce of vector `W001`. An independent implementer could then locate a `0x0B` mismatch in the first sixteen bytes, instead of facing a ciphertext that differs everywhere.

/docs/napseq-eprint-v4.tex

```
prime-indexed token cipher, and a uniform 256-bit secret
$\mathsf{sk}$ that
keys every HMAC domain derivation used for noise injection, plaintext
```

same file

```
All internal values are computed as
\[
  \mathrm{Derive}_d(\mathsf{sk}, N, \mathsf{ctx}) =
\mathrm{HMAC}(\mathsf{sk},\ d \| N \| \mathsf{ctx}),
\]
```

same file

```
\caption{Domain-separated HMAC derivation schedule. Every domain is
keyed
by $\mathsf{sk}$ only. The nonce $N$ is applied uniformly by
```

same file

```
context encoding. For domains
$\{\texttt{0x00},\texttt{0x01},\texttt{0x02},\texttt{0x04},\texttt{0x05},\texttt
the context is a fixed-width positional counter (or empty), trivially
```

same file

```
\[

\mathrm{Adv}^{\mathrm{IND\text{-}CPA\text{-}det}}_{\Pi}(\mathcal{A})
  \;\leq\;
  \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC\text{-}SHA256}}(\mathcal{B}_1)
  + \frac{q^2}{2^{128}},
\]
```

same file

```
\[
  \mathrm{Adv}^{\mathrm{LH\text{-}IND\text{-}CPA\text{-}det}}_{\Pi}(\mathcal{A})
  \;\leq\;
  \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC\text{-}SHA256}}(\mathcal{B}_1)
  + \frac{q^2}{2^{128}},
\]
```

same file

```
\[
  \mathrm{Adv}^{\mathrm{INT\text{-}CTXT}}_{\Pi}(\mathcal{A})
  \leq
  \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC\text{-}SHA256}}(\mathcal{B}_2)
  + \frac{q_v}{2^{256}}.
\]
```

same file

```
\[

\mathrm{Adv}^{\mathrm{IND\text{-}CCA\text{-}det}}_{\Pi}(\mathcal{A})
  \leq
  \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC\text{-}SHA256}}(\mathcal{B}_1)
  +
  \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC\text{-}SHA256}}(\mathcal{B}_2)
  + \frac{q^2}{2^{128}}
  + \frac{q_d}{2^{256}}.
\]
```

same file

```
needed. Conforming implementations must produce byte-identical ciphertext
for every positive vector and reject every negative vector. The generator
```

same file

```
  \item[Scope of $\mathsf{sk}$ and $\mathbf{k}$ compromise.] $\mathsf{sk}$
        is the sole cryptographic secret in this construction: every HMAC
        domain derivation — the noise schedule, the padding, the
        per-position addends, the keystream, and the tag — is keyed by
        $\mathsf{sk}$ alone (Section~\ref{sec:domains}). Consequently, an
```

/src/kat_cross_check.rs

```
        .filter(|v| v["kind"] == "positive" && v["api"] != "stream_ae")
```

same file

```
        .filter(|v| v["kind"] == "positive" && v["api"] != "stream_ae")
```

same file

```
        .filter(|v| v["kind"] == "negative" && v["api"] != "stream_ae")
```

same file

```
//
// These vectors exist because the v7 corpus above does not exercise v8 at
// all, which is how the Rust port's missing domain-0x0B format subkey went
// undetected (docs/CAVEATS.md, V3-CVF1 Residual 4).
```

/src/lib.rs

```
    let noise_theta = derive_noise_threshold_v8(kb, nonce);
```

same file

```
            && is_noise_pos_v8(kb, nonce, ct_pos, noise_theta)
```

same file

```
        let addend = derive_addend(kb, nonce, real_idx, k);
```

same file

```
pub const FORMAT_BLOCK_V8: u8 = 0x01;
```

```
/// Domain `0x0B` format-subkey identifier for the v8 streaming-AE format.
pub const FORMAT_STREAM_AE_V8: u8 = 0x02;
```

same file

```
fn derive_format_subkey(sk: &[u8], format_id: u8) -> [u8; 32] {
    hmac_digest(sk, &[0x0B, format_id])
}
```

same file

```
    let sk_fmt = derive_format_subkey(sk, FORMAT_BLOCK_V8);
```

same file

```
    let nonce = synthetic_nonce(&sk_fmt, aad, message.as_bytes());
```

same file

```
    let noise_theta = derive_noise_threshold_v8(&sk_fmt, &nonce);
```

same file

```
    let padded = pad_to_block(&codepoints, &sk_fmt, &nonce, block_size);
```

same file

```
            if noise_run < MAX_NOISE_RUN && is_noise_pos_v8(&sk_fmt, &nonce, ct_pos, noise_theta) {
```

same file

```
                let nc = derive_noise_char(&sk_fmt, &nonce, ct_pos);
```

same file

```
                let na = derive_noise_token_addend(&sk_fmt, &nonce, ct_pos, k);
```

same file

```
                let addend = derive_addend(&sk_fmt, &nonce, real_idx, k);
```

same file

```
        let nc = derive_noise_char(&sk_fmt, &nonce, ct_pos);
```

same file

```
        let na = derive_noise_token_addend(&sk_fmt, &nonce, ct_pos, k);
```

same file

```
    let ks = varint_keystream(&sk_fmt, &nonce, blob.len());
```

same file

```
1193     let tag = compute_auth_tag(&sk_fmt, aad, &payload,
AAD_LEN_WIDTH_V8);
```

same file

```
1213     let sk_fmt = derive_format_subkey(sk, FORMAT_BLOCK_V8);
```

same file

```
1217     let calc_tag = compute_auth_tag(&sk_fmt, aad, payload,
AAD_LEN_WIDTH_V8);
```

same file

```
1223     let ks = varint_keystream(&sk_fmt, nonce, masked.len());
```

same file

```
1226     let codepoints = decrypt_core_v8(nonce, &tokens, primes,
&sk_fmt)?;
```

CVF-13: fixed Category: **flaw**

Summary

Six legacy functions accept an empty ciphertext as valid under any key, so anyone can make the receiver accept an empty message without knowing the key.

When the legacy (v7) encryption functions are given an empty message, they return an empty result straight away instead of encrypting it, and the matching decryption functions accept an empty input as a correctly encrypted empty message. No key, tag or other check is involved, so an attacker who replaces a ciphertext with nothing gets a successful decryption instead of an error. The crate's own network message layer passes such input to these functions, so the problem can be reached over the network. The newer v8 functions are not affected.

Every legacy entry point begins by returning early on empty input. On the encrypt side:

```
pub fn encrypt_bytes(message: &str, key: &[u64], aad: &[u8]) ->
  Result<Vec<u8>, String> {
    if message.is_empty() {
        return Ok(Vec::new());
    }
```

with the same three lines in `encrypt_str`, `encrypt_raw` and `encrypt_bytes_with_nonce`. On the decrypt side:

```
pub fn decrypt_bytes(ciphertext: &[u8], key: &[u64], aad: &[u8]) ->
  Result<String, String> {
```

```
    if ciphertext.is_empty() {
        return Ok(String::new());
    }
```

with the same in `decrypt_str` and `decrypt_raw`.

The early return on the decryption side is what causes the problem. It returns `Ok` before the key is used, before the tag is computed and before the associated data is examined. So for any key and any associated data, the zero-length ciphertext is accepted as a valid encryption of the empty message. An attacker who truncates a ciphertext to nothing, or drops it and substitutes an empty frame, obtains a successful decryption instead of an authentication failure. The receiver cannot tell this apart from a legitimate empty message, because a legitimate empty message is encoded the same way, as nothing at all. The doc comment of `decrypt_raw` states the contract that this early return breaks: "Verifies the HMAC tag (constant-time) before decrypting. Returns `Err` on authentication failure."

This is a forgery that works for one message, the empty one, without the key. It is narrow, since the empty string is the only plaintext an attacker can produce this way, but it is exactly the event that the integrity property is meant to rule out: `Theorem 4.18` bounds the INT-CTXT advantage by $\mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC}}(\mathcal{B}_2) + q_v/2^{256}$, and here the advantage is 1 after a single verification query.

The behaviour also differs from the specification, which defines Enc for the empty message as well. Under the `bucket` profile, $B = \max(16, 2^{\lceil \log_2 1 \rceil}) = 16$, so $R = 18$ and $|C| = 48 + 160 \cdot 18 = 2{,}928$ bytes, which is the paper's own stated minimum ciphertext length. The specified scheme therefore produces a full 2,928-byte authenticated ciphertext for the empty message, while the v7 functions produce zero bytes.

The v8 path is correct and can serve as the model: `encrypt_bytes_v8_with_profile` has no early return for empty input, `bucket_block_size(0)` returns 16, and the empty message produces the full 2,928-byte ciphertext the specification requires. Test vector `W001`, "Empty message produces a full authenticated ciphertext", is exactly 2,928 bytes and confirms it. Only the legacy functions are affected.

The crate's own protocol layer exposes the problem to the network. This evidence comes from `src/ot_frame.rs`, which is outside the code reviewed in this engagement and is cited here only to show the impact; the cause lies entirely in `src/lib.rs`, in the six early returns listed above. `ot_frame::unwrap_pdu` parses the 23-byte header of an incoming message frame (a protocol data unit, or PDU). The header check covers only the two-byte magic value, the protocol identifier and the declared length of the encrypted payload, and the header itself is not authenticated. The function then calls `decrypt_raw` on the payload. A frame that declares a payload length of zero therefore reaches `decrypt_raw(&[])`, hits the early return, and is handed to the caller as a successfully authenticated, replay-checked message under any key. In other words, a 23-byte frame injected by an attacker is accepted. The header is bound only as associated data to a tag that, on this path, is never computed, so nothing else authenticates it either.

Two tests treat this behaviour as correct, so the fix will also require changing them. `lib.rs` checks the round trip:

```
#[test]
```

```
fn empty_message_roundtrip() {
    let k = test_key();
    assert_eq!(decrypt_str(&encrypt_str("", &k, b"").unwrap(), &k,
  b"").unwrap(), "");
}
```

This test passes without performing any cryptographic operation: both calls return through their early returns, so the assertion compares "" with "". The in-crate parity harness goes further and requires the empty-to-empty mapping as part of conformance:

```
if message.is_empty() {
    let ct = encrypt_bytes_with_nonce(message, &[], [0u8;
  NONCE_SIZE], b"");
    assert!(ct.is_empty(), "[{}] empty message should produce empty
  ciphertext", id);
```

and it does so under an empty key and an all-zero nonce. The test suite therefore treats the behaviour reported here as correct, and any fix will make this assertion fail. Updating the tests is part of the fix, and it is worth planning for in advance.

Recommendation

Remove the early returns on the decryption side first, since those are the ones that accept a forgery. In `decrypt_bytes`, `decrypt_str` and `decrypt_raw`, delete the `if ciphertext.is_empty() { return Ok(..) }` block and let the existing minimum-length check reject the input:

```
if ciphertext.len() < NONCE_SIZE + TAG_SIZE {
    return Err(format!(
        "Ciphertext too short: {} bytes; header+tag require at
  least {}.",
        ciphertext.len(), NONCE_SIZE + TAG_SIZE
    ));
}
```

A zero-length ciphertext then fails this check like any other truncated input, and an empty ciphertext no longer stands for the empty message. This change alone closes the exposure through the protocol layer, so it can be made first if a staged fix is preferred.

Then remove the early returns on the encryption side, so that the empty message is encrypted like any other, as on the v8 path and in the specification. `pad_message` and `bucket_block_size` already handle $n = 0$ correctly (`bit_length(0)` is 0, which is raised to `PAD_MIN_EXP`, giving $B = 16$), so no other change is needed, and the result is the specified 2,928-byte ciphertext.

If compatibility with stored v7 ciphertexts rules out changing the encryption side, keep at least the decryption-side fix, and make the encryption functions return an explicit error for the empty message, so that the two sides cannot disagree about whether the empty message can be encrypted.

Allow time for the test changes, which are a substantial part of the fix. `empty_message_roundtrip` should check that the ciphertext is a full 2,928 bytes and that it decrypts through a real tag verification; as written, it would pass even if both func-

tions were replaced by `Ok(String::new())`. The empty-message branch of the parity harness checks the very behaviour being removed and will fail as soon as the fix lands, so it should be either deleted together with the vector it serves or rewritten to check the new behaviour. It should also stop passing an empty key, which is unrelated to what the branch is meant to check and is rejected by `validate_key` on the v8 path anyway. Any Python or C test vector that encodes the same empty-to-empty mapping needs the same treatment; otherwise the three implementations will disagree after the fix.

Separately, review the order of operations in `ot_frame::unwrap_pdu`: the replay counter is updated before the payload is authenticated, so any frame with a forged tag advances it. This is outside the code reviewed here, but it is worth fixing at the same time, since an attacker reaches both problems with the same injected frame.

/src/kat_cross_check.rs

```
133
134         // Empty-message vector: nonce_hex is "" and ciphertext_hex
is ""
135         if message.is_empty() {
136             let ct = encrypt_bytes_with_nonce(message, &[], [0u8;
NONCE_SIZE], b"");
137             assert!(ct.is_empty(), "[{}] empty message should produce
empty ciphertext", id);
138             eprintln!("[PASS] {} — empty message", id);
139             tested += 1;
140             continue;
```

/src/lib.rs

```
731     if message.is_empty() {
732         return Ok(Vec::new());
733     }
```

same file

```
791     if message.is_empty() {
792         return Vec::new();
793     }
```

same file

```
832     if ciphertext.is_empty() {
833         return Ok(String::new());
834     }
```

same file

```
862     if message.is_empty() {
863         return Ok(String::new());
864     }
```

same file

```
869     if cypher.is_empty() {
```

```
870         return Ok(String::new());
871     }
```

same file

```
887     if data.is_empty() {
888         return Ok(Vec::new());
889     }
```

same file

```
930 /// Verifies the HMAC tag (constant-time) before decrypting.  Returns
931 /// `Err` on authentication failure — the caller must never use the
932 /// ciphertext for any purpose if this returns an error.
```

same file

```
934     if ciphertext.is_empty() {
935         return Ok(Vec::new());
936     }
```

same file

```
1311     fn empty_message_roundtrip() {
1312         let k = test_key();
1313         assert_eq!(decrypt_str(&encrypt_str("", &k, b"").unwrap(),
&k, b"").unwrap(), "");
1314     }
```

CVF-18: open Category: **procedural**

> **Summary**
>
> **The mandatory start-up self-test does not check the current encryption scheme, and its integrity check is a fixed comparison that will block the module after the next version change.**
>
> Before the module may be used, it must run a set of start-up self-tests, and its documentation forbids any encryption if they fail. Three of the four tests check only the older (v7) encryption format, so they would still pass if the current (v8) scheme were completely broken. The fourth test is presented as a software integrity check, but it only compares the crate's version number with the fixed value `0.1.0`. It detects no tampering, and as soon as the version number is increased in a future release it will fail and stop the module from being used.

The module presents itself as a FIPS 140-3 cryptographic module with a strict requirement before any operation:

> Call `run_power_on_self_tests()` before using the module in production. It returns `Ok(())` only if all tests pass. On any failure it returns `Err(SelfTestError)` — the caller MUST NOT perform any cryptographic operations after a self-test failure.

citing NIST SP 800-140B §4.9.1. Four tests run: KAT-1 encrypt, KAT-2 decrypt, KAT-3 tamper rejection, and INT-1 software integrity.

All three known-answer tests (KATs) use the v7 format. KAT-1 calls `crate::encrypt_bytes_with_nonce`, and KAT-2 and KAT-3 both call `crate::decrypt_bytes`. None of the four tests calls `encrypt_bytes_v8` or `decrypt_bytes_v8`, so the power-on self-test does not cover any of the following code: `derive_format_subkey` and the domain-`0x0B` subkey, `synthetic_nonce` and domain `0x0A`, `derive_noise_threshold_v8` and the integer $\theta(N)$ computation, `is_noise_pos_v8`, the capped emission loop and the token ceiling, `decrypt_core_v8` with its divisibility, bucket and length-prefix checks, and the `AAD_LEN_WIDTH_V8` tag prefix. The specified construction could be broken in any of these places and `run_power_on_self_tests()` would still return `Ok(())`, after which the module's own contract declares it fit for production use.

We can tell from the test vector itself that it belongs to the v7 format, without running it. `KAT_CIPHERTEXT_HEX` decodes to 4,784 bytes, so $N_{\text{tok}} = (4784 - 48)/8 = 592$ for the one-codepoint message `"A"`, whose padded block is $R = 18$. That is 32.9 tokens per padded codepoint, while the specification caps the ratio at 20, and 592 is not a multiple of 20, so no ceiling was applied. The specified v8 ciphertext for the same message is exactly 360 tokens and 2,928 bytes, which the delivered v8 test vectors confirm independently: `W002` has that message and that length. This vector is also the evidence for CVF-17.

The fourth test, INT-1, does not actually check integrity. Its full code is:

```
const BUILD_HASH: &str = env!("CARGO_PKG_VERSION");

fn integrity_check() -> Result<(), SelfTestError> {
    if BUILD_HASH == "0.1.0" {
        Ok(())
    } else {
        Err(SelfTestError::IntegrityCheckFailed)
    }
}
```

The constant is named `BUILD_HASH`, but it holds the crate's declared version string, and the test compares it with a fixed literal. It detects no modification of the binary, of the source or of anything else: any tampering that leaves the version field of `Cargo.toml` unchanged passes, and an attacker who modifies the module has no reason to change that field. The comment above the function acknowledges this ("This stub verifies a compile-time build metadata string rather than a full binary HMAC", with the full check deferred to "Phase 4 workstream 4.1"). However, this explanation is visible only in the source code, while `SelfTestError::IntegrityCheckFailed` and the module header present INT-1 to the operator as a software integrity check.

Now that the crate manifest has been delivered, it is settled. `Cargo.toml` declares `ver-`

`sion = "0.1.0"` at line 3, so `CARGO_PKG_VERSION` expands to exactly the literal that `integrity_check` compares against, and INT-1 returns `Ok(())` on every build of this version. The test therefore never checks anything: with the current version number it always passes, and with any other version number it always fails. The second case is a hidden risk. The first release with a version number other than `0.1.0` makes `integrity_check` return `Err`, which makes `run_power_on_self_tests` return `Err`, which by the module's stated contract forbids the caller from performing any cryptographic operation. A routine increase of the version number therefore disables the module, and the error it reports is an integrity-check failure, the most alarming message in the set, when the real cause is an outdated constant.

The client's scope file asks whether this test vector has become outdated. It has not, and the delivered test vectors now confirm this directly. We compared the v7 derivation code with the comment that records where the vector came from, and found no change since the vector was regenerated: every v7 derivation still uses `kb = key_bytes(key)` as the key, keeps the `d || N || ctx` layout that the comment names as the last change affecting the format, and uses `AAD_LEN_WIDTH_V7 = 4`. The v8 work added new functions and did not alter the v7 ones. Decoding the constant confirms this. `KAT_CIPHERTEXT_HEX` is a `concat!` of 299 sixteen-byte hex groups spread over three hundred lines of source, which is why a search for a single long hex literal does not find it, and it has 9,568 hex characters in total. The 4,784 bytes it decodes to are **byte-identical** to the `ciphertext_hex` field of vector `V002` in `tests/kat/v6_vectors.json`: the same key `[1000003, 1000033, 1000037, 1000039]`, the same nonce `9c6c0b921a83849cdbf2fe7efb743fe9`, the same message `"A"`, the same empty associated data and the same 4,784 bytes including the tag, with no differing byte anywhere.

There are two other things. First, the comment describing the vector's origin does not describe the constant accurately. It says that the vector is "derived from the current (v7, post-CVF1-fix) Python reference implementation, using the same key/nonce/message as the retired v6 vector `tests/kat/v6_vectors.json` V002", while the constant is in fact V002, byte for byte. Also, V002 is not retired, since three of the seven tests in `kat_cross_check.rs` still use the file that contains it (CVF-25). Second, the two copies are not kept in sync. The same 4,784-byte value exists both as a three-hundred-line `concat!` literal and in a JSON file, and nothing in the crate compares them. They agree today, but nothing ensures that they will continue to agree. If the test vectors are regenerated and the literal is left unchanged, the self-test goes out of date exactly as the scope file anticipated, and nothing reports it, in the one test that a FIPS operator is told to trust before doing anything else.

We rate this finding high because of the combination of the two problems. The integrity stub alone is documented in the source, but a FIPS module whose mandatory start-up check does not run a single line of the scheme that the paper specifies is a more serious matter. See also CVF-47, which reports that nothing enforces the self-test outcome at run time, so the self-test is both incomplete in what it checks and not binding on what it allows.

Recommendation

Add v8 known-answer tests to the power-on suite, and keep the v7 ones only for as long as the v7 format remains supported.

Since v8 encryption is deterministic in $(\mathbf{k}, \mathrm{sk}, A, M)$ (the nonce is a synthetic IV), a v8 KAT needs no special entry point for injecting a nonce and can call the public API directly, which is simpler than the v7 case:

```
const KAT_V8_PRIMES: &[u64] = &[/* ... */];
const KAT_V8_SK: [u8; SK_SIZE] = [/* ... */];
const KAT_V8_MESSAGE: &str = "A";
const KAT_V8_CIPHERTEXT_HEX: &str = /* 2,928 bytes for the
  empty-AAD, one-codepoint case */;
```

with the three tests mirroring the existing ones: encrypt to the reference bytes, decrypt to the reference plaintext, and reject a copy with a flipped tag. The delivered v8 test vectors already contain all four values (`W002` is exactly this case), so no new vectors need to be generated. Add a fourth test for the structural checks performed after authentication, since those are new in v4 and are not covered anywhere in the self-test: a ciphertext whose token count is a multiple of 20 but not of the form $20(B+2)$ must be rejected by the bucket check in `decrypt_core_v8`, and vector `W-N07` in the delivered test vectors is already such a ciphertext.

We recommend generating the vectors from the known-answer test files at build time instead of embedding them as hex literals. Embedding them is what created the risk of outdated vectors that the scope file warned about, and the test-vector files now exist. Using `include_str!` on the vector file, or a `build.rs` that emits the constants, keeps the self-test and the test vectors from drifting apart. If a literal must be kept for a power-on path without dependencies, add a test that compares the literal with the test-vector file, so that any mismatch fails in CI instead of going unnoticed in production. The same applies to the existing v7 literal, now that it is confirmed to duplicate `V002` byte for byte: a three-line test comparing `decode_hex(KAT_CIPHERTEXT_HEX)` with that vector's `ciphertext_hex` would keep the two copies in sync at no cost.

Correct the comment describing the vector's origin at the same time. It should say that the constant is vector `V002`, not that it shares that vector's inputs, and it should not describe as retired a vector that three active tests use.

Replace or rename INT-1. If the binary HMAC is deferred, the honest interim solution is to remove `IntegrityCheckFailed` from the results visible to the operator and to report the stub as what it is, a check of build provenance, not an integrity check. A self-test that names a property it does not test is worse than no test, because an operator reads `Ok(())` as covering that property. If the stub is kept in the meantime, stop comparing against a hard-coded version literal. In particular, do not "fix" it by editing the literal at each release: that keeps a check that tests nothing while removing the one symptom that reveals it. Compare instead against a value generated at build time, such as `option_env!` on a digest injected by the build, with the test skipped when the value is absent, so that a change of version number cannot disable the module. The `build.rs` design outlined in the comment (an HMAC over `.text` and `.rodata`, written to `OUT_DIR` and loaded with `include_bytes!`) is the right target and would make INT-1 check what its name says.

Note also that the compiled module contains an algorithm that the power-on suite does not cover at all: `pub mod kem` exposes a FrodoKEM key exchange with no self-test. Under FIPS, every approved algorithm inside the module boundary needs a

self-test, and CVF-23 describes the boundary problem that makes this easy to miss.

Make these changes before the fix for CVF-47. That finding asks for the self-test outcome to be recorded and enforced on every cryptographic entry point, which is the right end state. Enabling it while the three KATs are v7-only and INT-1 compares a version literal would block the module on the basis of tests that say nothing about the specified scheme, and would disable it completely at the first change of version number.

/Cargo.toml

```
version = "0.1.0"
```

/src/self_test.rs

```
//! FIPS 140-3 power-on self-tests for the NAPQES Cryptographic Module.
//!
//! Call `run_power_on_self_tests()` before using the module in production.
//! It returns `Ok(())` only if all tests pass.  On any failure it returns
//! `Err(SelfTestError)` — the caller MUST NOT perform any cryptographic
//! operations after a self-test failure.
//!
```

same file

```
    /// KAT-3: tampered ciphertext was not rejected.
```

same file

```
    /// INT-1: software integrity check failed.
```

same file

```
pub fn run_power_on_self_tests() -> Result<(), SelfTestError> {
    kat_encrypt()?;
    kat_decrypt()?;
    kat_tamper_rejection()?;
    integrity_check()?;
    Ok(())
}
```

same file

```
fn kat_encrypt() -> Result<(), SelfTestError> {
    let expected = decode_hex(KAT_CIPHERTEXT_HEX)?;
    let got = crate::encrypt_bytes_with_nonce(KAT_MESSAGE, KAT_KEY, KAT_NONCE, b"");
    if got == expected {
        Ok(())
    } else {
```

```
        Err(SelfTestError::KatEncryptMismatch)
    }
}
```

same file

```
fn kat_decrypt() -> Result<(), SelfTestError> {
    let ciphertext = decode_hex(KAT_CIPHERTEXT_HEX)?;
    match crate::decrypt_bytes(&ciphertext, KAT_KEY, b"") {
        Ok(pt) if pt == KAT_MESSAGE => Ok(()),
        Ok(_) => Err(SelfTestError::KatDecryptMismatch),
        Err(_) => Err(SelfTestError::KatDecryptMismatch),
    }
}
```

same file

```
fn kat_tamper_rejection() -> Result<(), SelfTestError> {
    let mut ciphertext = decode_hex(KAT_CIPHERTEXT_HEX)?;
    // Flip the last byte of the authentication tag.
    let last = ciphertext.len() - 1;
    ciphertext[last] ^= 0xFF;
    match crate::decrypt_bytes(&ciphertext, KAT_KEY, b"") {
        Err(_) => Ok(()),
        Ok(_) => Err(SelfTestError::KatTamperNotRejected),
    }
}
```

same file

```
// ── INT-1: software integrity ──────────────────────────────────
//
// The integrity check verifies that the module binary has not been modified
// since it was compiled.  The reference hash is embedded at compile time by
// `build.rs` (see below).
//
// IMPLEMENTATION STATUS:
//   This stub verifies a compile-time build metadata string rather than a
//   full binary HMAC.  Replacing it with a binary HMAC requires a build.rs
//   that:
//     1. Computes HMAC-SHA256 of the compiled `.text` + `.rodata` sections.
//     2. Writes the digest to `OUT_DIR/module_integrity.bin`.
//     3. include_bytes! pulls it in here.
```

```
//   This is a Phase 4 workstream 4.1 item.  The current implementation
//   satisfies the Level 1 pre-attestation requirement to demonstrate the
//   integrity-check mechanism is in place, pending the full binary HMAC.
```

same file

```
const BUILD_HASH: &str = env!("CARGO_PKG_VERSION");

fn integrity_check() -> Result<(), SelfTestError> {
    // In the full implementation this will compare an HMAC over the loaded
    // module binary to a reference digest embedded by build.rs.
    // For now, verify that the build version string matches a compile-time
    // constant to ensure the binary was built from this source tree.
    if BUILD_HASH == "0.1.0" {
        Ok(())
    } else {
        Err(SelfTestError::IntegrityCheckFailed)
    }
}
```

/tests/kat/v6_vectors.json

```
      "id": "V002",
```

/tests/kat/v8_vectors.json

```
      "id": "W002",
```

same file

```
      "id": "W-N07",
```

5.3. Moderate

CVF-12: fixed Category: **behavior**

Summary

The paper says that the Rust implementation compares authentication tags with a vetted library, but the code never calls that library and uses its own comparison loop, which also accepts tags of the wrong length.

When the receiver checks that a message is authentic, it compares two short secret values, and the comparison must take the same time whatever the values are, so that an attacker cannot learn them by timing. The paper says that the Rust code delegates this comparison to a well-known library. The library is listed as a dependency, which makes the claim look confirmed, but the code never uses it and relies instead on a hand-written loop. The loop is carefully written and we found no timing defect in it, but if it is ever given two values of different lengths, it reports them as equal. No current use of the loop triggers this, so there is no exploitable problem today, but the paper's description is wrong and a reviewer checking it would be misled.

Section 8.4 states, of tag comparison, that it "uses `hmac.compare_digest` (Python), `constant_time_eq` from the `subtle` crate (Rust), and an OR-accumulator byte-difference loop (C)", and then attaches a caveat to the C implementation only: "The C accumulator is not declared `volatile`, so nothing in the C standard prevents a sufficiently aggressive compiler from reintroducing an early exit; this is the usual caveat for hand-rolled constant-time C and is not addressed here."

`Cargo.toml` has now been delivered, and it does declare the dependency: `subtle = "2"` at line 11. However, no code in the crate uses it. `subtle`, `ConstantTimeEq`, `constant_time_eq` and `Choice` appear in no `use` declaration and in no expression under `src/`, `examples/` or `fuzz/`; outside the manifest, the string `subtle` occurs once, in a doc comment. Tag comparison in all three places where a tag is verified, and the CRNG nonce check, use a hand-written function instead:

```
#[inline(never)]
fn ct_eq_bytes(a: &[u8], b: &[u8]) -> bool {
    debug_assert_eq!(a.len(), b.len());
    let mut diff = 0u8;
    for (x, y) in a.iter().zip(b.iter()) {
        unsafe {
            diff |= std::ptr::read_volatile(x) ^
  std::ptr::read_volatile(y);
            std::ptr::write_volatile(&mut diff, diff);
        }
    }
    diff == 0
}
```

Its doc comment argues at length that `#[inline(never)]`, `read_volatile` and a `write_volatile` on every iteration together "prevent LLVM from generating an early-exit loop" and that the result "is a fixed-count loop". That reasoning is plausible and

considerably more careful than the reasoning behind the C accumulator. However, it is the same kind of argument: a hand-written construction that relies on what the optimiser is expected not to do. Rust gives no more of a language-level guarantee here than C does. The paper therefore presents the situation the wrong way round. It describes Rust as the case where a vetted library takes care of the requirement and C as the only hand-written exception, while in fact two of the three implementations are hand-written and only Python delegates to a standard-library function.

A declared dependency that nothing uses is worse than an absent one, and this is the main point of the finding. Cargo emits no warning for an unused dependency, so nothing in an ordinary build reveals it, and `subtle` is compiled and linked into every consumer for no purpose. More importantly, the manifest is the file a reviewer consults to check the attribution in Section 8.4. The scope for this review names `Cargo.toml` for exactly that purpose: "confirms `subtle` (constant-time tag compare claim, paper §Constant-Time)". It does not confirm it. It supports a claim that the code does not implement, and a reader who finds the dependency declared has no reason to go on and inspect the places where the comparison is called, which are the only places where the discrepancy is visible. If the dependency had simply been absent, the check would have failed at the first step.

This matters beyond the attribution. A reader reviewing the deployment for FIPS or Common Criteria purposes will look for the `subtle` dependency, find it declared, and conclude that a reviewed library takes care of the requirement, which is the one conclusion that the source does not support. In addition, the caveat that the paper does state, that a sufficiently aggressive compiler may reintroduce an early exit, is attached to the wrong implementation: it applies to the Rust loop as well, and the Rust loop is what the deployment runs.

Since `ct_eq_bytes` is the comparison that is actually used, we also assessed it on its own merits. It is sound in the respect that matters, and unsound in one respect that does not cause a problem today.

What it does correctly. There is no early return and no `break`, so the loop runs to completion on every call. The number of iterations is `min(a.len(), b.len())`, which depends on the lengths of the operands and not on their contents, so the position of the first differing byte does not affect the control flow. `read_volatile` on both operands makes every load an observable side effect that LLVM may not remove. The `write_volatile` to `diff` is a store to a local variable whose address is taken, which LLVM may not remove either, so iterations cannot be deleted or merged. `#[inline(never)]` prevents the caller's `if !ct_eq_bytes(...)` branch from being moved into the body. The accumulator is a byte-wise `|=` over an `xor`, and both operations take a single cycle and do not depend on the data on every architecture of interest. The returned `bool` does introduce a branch where the function is called, but that branch depends only on the single accumulated result and not on any tag byte, which is unavoidable and correct. As hand-written constant-time code goes, the reasoning is careful, and we found no defect in it.

What it does wrong concerns the length. The precondition is enforced only by `debug_assert_eq!`, which is removed in release builds. `Cargo.toml` declares no `[profile]` section, so the release profile uses Cargo's default `debug-assertions = false`, and the check is absent from any shipped binary (CVF-23). The loop then uses `a.iter().zip(b.iter())`, and `zip` stops at the end of the shorter operand. So when

the lengths differ, the function compares only the common prefix and returns `true` whenever that prefix matches. All four places where the function is called pass slices that have equal length by construction: in each of the three verification sites, `recv_tag` is cut at `ciphertext.len() - TAG_SIZE` and `calc_tag` is a 32-byte HMAC output, and the CRNG check compares two `[u8; 16]` values. The problem is therefore latent, and we found no path that reaches it. Still, this is a comparison function that guards tag verification, and when it is misused it reports equality, which is the unsafe direction: a caller that passes a truncated tag, or any future use with an operand of variable length, would get a forgery accepted instead of rejected.

The declared but unused dependency is exactly what would fix this. The `ConstantTimeEq` implementation of `subtle` for slices returns `Choice::from(0)` when the lengths differ, instead of comparing a prefix, so it rejects such input. The crate already includes, without using it, the dependency that provides the one property its own comparison function lacks.

There is one more point about scope. The function is named and documented as a constant-time equality test, but resistance to early exit is the only constant-time property it has or claims. It does not hide the length of its operands, which is acceptable for its four current uses but would be a hazard for a new one. In addition, the timing harness that would detect a change in its behaviour measures the whole of `decrypt_bytes` instead of the comparison alone, which, as CVF-26 describes, leaves the relevant signal well below the measurement noise.

Recommendation

Either make the paper match the code or make the code match the paper. Both options are reasonable, but the second one is cheaper and also fixes the other problems described above.

To make the code match the paper, no change to the manifest is needed, since `subtle` is already declared. Replace the body of `ct_eq_bytes` with `subtle::ConstantTimeEq`:

```
use subtle::ConstantTimeEq;

fn ct_eq_bytes(a: &[u8], b: &[u8]) -> bool {
    a.ct_eq(b).into()
}
```

`ConstantTimeEq` for slices returns `Choice::from(0)` when the lengths differ instead of comparing a prefix, so this change also fixes the truncation caused by `zip`. It removes two `unsafe` blocks and the argument about the optimiser that was needed to justify them, and it gives the dependency in the manifest a purpose.

To make the paper match the code instead, rewrite the sentence in Section 8.4 to say that Python delegates to `hmac.compare_digest` while both Rust and C use hand-written accumulator loops, and extend the compiler caveat so that it covers both. If this option is chosen, also note in the same place that the Rust loop declares its memory accesses volatile and is therefore on firmer ground than the C loop, and state explicitly that neither relies on a language-level guarantee. In the same change, remove `subtle` from `Cargo.toml`, so that the manifest no longer appears to support a claim that the paper no longer makes.

Whichever option is chosen, also make the following changes:

- Replace the `debug_assert_eq!` length check with a real check that remains in release builds and returns `false` when the lengths differ, so that the function cannot report equality for operands of different lengths. The safety of a tag comparison should not depend on a build profile setting; see CVF-23 on the missing `[profile]` section.
- Add a test for operands of different lengths. No test anywhere calls `ct_eq_bytes` with unequal lengths, which is why nobody noticed that the function accepts such input.
- Add a check for unused dependencies to CI, such as `cargo udeps` or `cargo machete`, so that a manifest entry that no code uses fails the build instead of serving as evidence. This is how the discrepancy went unnoticed: nothing in the toolchain objects to a declared dependency that is never used.
- Since the timing harness is the only thing that would detect a change in the behaviour of this function, make it measure the comparison itself, as CVF-26 recommends.

/Cargo.toml

```
subtle = "2"
```

same file

```
[lib]
name = "napqes"
path = "src/lib.rs"
```

/src/lib.rs

```
        if ct_eq_bytes(&nonce, &prev_nonce) {
```

same file

```
#[inline(never)]
fn ct_eq_bytes(a: &[u8], b: &[u8]) -> bool {
    debug_assert_eq!(a.len(), b.len());
    let mut diff = 0u8;
    for (x, y) in a.iter().zip(b.iter()) {
        unsafe {
            diff |= std::ptr::read_volatile(x) ^ std::ptr::read_volatile(y);
            // The volatile write forces this accumulated value to be
            // materialised every iteration; LLVM cannot skip iterations.
            std::ptr::write_volatile(&mut diff, diff);
        }
    }
    diff == 0
}
```

same file

```
847     if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
```

same file

```
949     if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
```

same file

```
1218    if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
```

CVF-14: open Category: **behavior**

> **Summary**
>
> **The Rust code contains both of the two incompatible ways of turning data into characters before encryption, so a message encrypted one way can be decrypted the other way and silently come out different.**
>
> Before encrypting, the scheme turns the message into a sequence of characters, and there are two different ways of doing this: one works on text, the other works on raw bytes. The paper says that the Rust implementation uses only the text-based method, which is the safe one. In fact the Rust code offers both, under the same key and with the same authentication, and nothing in the encrypted message records which method was used. A message encrypted with one method therefore decrypts successfully with the other, but the receiver gets different content from what was sent, with no error. Some decryption functions also silently drop or cut down characters they cannot represent. This makes the problem already described in CVF-3 reachable inside a single program.

This finding adds to CVF-3 and does not duplicate it. The root cause is the one that CVF-3 already identifies: the encoding from byte strings to codepoint sequences is not normative, and no domain separator distinguishes the two conventions. The recommendation of CVF-3 would close both findings. What is new here is that the paper's evidence that the Rust implementation is on the safe side does not hold.

Section 3.10 states: "The Python and Rust reference implementations realise $\mathcal{M}$ as specified: Rust's `&str` excludes surrogates by construction". When compared with the source, that claim is correct as far as it goes. `encrypt_bytes`, `encrypt_str`, `encrypt_bytes_with_nonce` and `encrypt_bytes_v8_with_profile` all take `&str` and go through `message.chars().map(|c| c as u32)`, which yields Unicode scalar values and cannot yield a surrogate, so those four entry points do realise the message space of `Definition 3.10` exactly. On the decryption side, the v8 path is careful in the same way: `decrypt_core_v8` rejects any recovered value above `0x10FFFF` or for which `char::from_u32` fails, and `decrypt_bytes_v8` then maps the values instead of filtering them, returning an error on a non-scalar value instead of dropping it.

However, `&str` is not the crate's only interface to the token layer. Two further public functions take and return raw bytes:

```
pub fn encrypt_raw(data: &[u8], key: &[u64], aad: &[u8]) ->
  Result<Vec<u8>, String> {
    ...
    let codepoints: Vec<u32> = data.iter().map(|&b| b as
  u32).collect();
```

This is the byte-to-codepoint convention, i.e. each byte becomes one codepoint. It is exactly the convention that CVF-3 identifies as the one used by the C implementation, and which the paper describes as realising $\mathcal{M}$ "faithfully only for ASCII input". Its doc comment states the convention plainly: "Each byte is treated as a codepoint in [0, 255]."

So the two encodings that the paper places in different implementations are both present in this one. They use the same key, and they share `compute_auth_tag` with the same `AAD_LEN_WIDTH_V7` prefix and the same domain byte `0x03`. Nothing in the wire format distinguishes them, exactly as CVF-3 observes for the case of two implementations. As a result, the worked example of CVF-3 no longer needs two implementations:

- Encrypt the string `é` with `encrypt_bytes`, and one codepoint, `U+00E9`, enters the token layer. Pass the resulting ciphertext to `decrypt_raw`, and it authenticates, because the tag is computed over the masked blob and the AAD and does not depend on how the caller obtained its codepoints. The caller receives the single byte `0xE9`.
- Encrypt the two bytes `0xC3 0xA9` with `encrypt_raw` and pass the ciphertext to `decrypt_bytes`, and the caller receives the two-character string `Ã©`.

Both directions succeed silently, with no error and no indication that the plaintext differs from the one that was encrypted.

`decrypt_raw` makes this worse than a symmetric disagreement about the encoding, because its inverse loses information:

```
Ok(codepoints.into_iter().map(|c| (c & 0xFF) as u8).collect())
```

Any recovered codepoint above 255 is silently truncated to its low byte instead of being rejected. So `decrypt_raw` does not just read the token stream in a different but consistent way: it discards information without reporting it, and it returns a plausible byte string for a ciphertext that encodes codepoints it cannot represent. By comparison, `decrypt_core_v8` rejects values out of range explicitly, while the v7 `decrypt_core` performs no range check at all.

The v7 `&str` decryption function has a third variant of the same defect. `decrypt_bytes` ends with

```
let s: String =
  codepoints.into_iter().filter_map(char::from_u32).collect();
```

`filter_map` drops every recovered value that is not a Unicode scalar value. So a v7 ciphertext whose token stream decodes to surrogates or to values out of range yields a *shorter* string than the one that was encrypted, successfully and with no error. The comment on the v8 path explicitly contrasts itself with this: "so this maps every recovered codepoint rather than silently dropping some."

This can be reached in practice within the delivered code: the PDU framing layer in `ot_frame.rs`, i.e. the module that would carry industrial traffic, uses `encrypt_raw` and `decrypt_raw` instead of the v8 entry points. That module is outside the scope of this review and is cited only to show the impact.

Recommendation

Adopt the main recommendation of CVF-3: make the encoding normative, and add an encoding tag with its own domain to the tag input, so that a ciphertext produced under one convention is rejected instead of being silently decrypted wrongly under the other. Note that this now has to cover a case within the crate as well as the case of two implementations. CVF-3 argues that defining the message space as byte strings is the cheaper option. If that option is chosen, `encrypt_raw` becomes the specification, and the `&str` entry points become thin wrappers that pass their UTF-8 bytes through. This also removes the question of surrogates and leaves one encoding in the crate instead of two.

Whichever encoding becomes normative, three changes to the source are needed:

- Remove the `& 0xFF` mask in `decrypt_raw` and return an error for any recovered codepoint above 255, in the way `decrypt_core_v8` already does. Silently truncating a value that cannot be represented is worse than refusing it, because the caller cannot tell the two cases apart.
- Replace `filter_map(char::from_u32)` in `decrypt_bytes` with the fallible map that `decrypt_bytes_v8` already uses, so that a non-scalar value causes an error instead of being silently deleted.
- Add a range check to the v7 `decrypt_core` similar to the one in `decrypt_core_v8`, or route both through a shared inverse function that performs the check. Currently `decrypt_core` pushes `((token - addend) / k) as u32` with no validation, and the `as u32` truncation there is a fourth instance of the same pattern of silent narrowing.

If both the byte-oriented and the codepoint-oriented interfaces are to be kept deliberately, bind the choice into the tag: give each a distinct `format_id` in the domain-`0x0B` subkey space that CVF-11 recommends specifying, so that the two are separated cryptographically and a ciphertext produced by one cannot be verified by the other. The code already has this mechanism for exactly this purpose, and currently uses it only to separate block mode from streaming mode.

Finally, narrow the sentence in Section 3.10 to what is true. Rust's `&str` does exclude surrogates by construction, but that is a property of four of the crate's entry points, not of the crate as a whole: the byte-oriented pair uses the same convention that the paper attributes only to the C implementation. As written, the sentence tells a reader that choosing Rust avoids the hazard, which is not the case.

/src/lib.rs

```
513             padded.push(((token - addend) / k) as u32);
```

same file

```
606         if cp > 0x10FFFF || char::from_u32(cp as u32).is_none() {
607             return Err(format!(
```

```
                "Malformed v8 ciphertext: recovered value {} at position {} is not a \
                 Unicode scalar value.",
                cp, ct_pos
            ));
        }
```

same file

```
    let s: String = codepoints.into_iter().filter_map(char::from_u32).collect();
```

same file

```
pub fn encrypt_raw(data: &[u8], key: &[u64], aad: &[u8]) -> Result<Vec<u8>, String> {
    if data.is_empty() {
        return Ok(Vec::new());
    }
    let nonce = generate_nonce_with_crng_check()?;
    let codepoints: Vec<u32> = data.iter().map(|&b| b as u32).collect();
```

same file

```
    Ok(codepoints.into_iter().map(|c| (c & 0xFF) as u8).collect())
```

same file

```
    let codepoints: Vec<u32> = message.chars().map(|c| c as u32).collect();
```

same file

```
    let s: String = codepoints
        .into_iter()
        .map(|c| {
            char::from_u32(c)
                .ok_or_else(|| format!("Malformed v8 plaintext: {} is not a Unicode scalar value.", c))
        })
        .collect::<Result<String, String>>()?;
```

CVF-15: fixed Category: **flaw**

Summary

Several older public decryption and encryption functions stop the whole program on bad input or on an invalid key instead of reporting an error, and the paper's claim that the code never crashes does not cover them.

The paper reports that extensive automated testing found no crashes in the decryption code. That testing actually exercised only a small part of the code, and five public decryption functions besides the main one can still crash the program. Most of these crashes can be triggered only by someone who already holds the key, but one low-level function accepts arbitrary input without any authentication check and crashes on malformed input. The older functions also accept an empty or obviously invalid key and crash instead of rejecting it. These are robustness defects, not ways to break the encryption, but a crash in a security component can stop a whole service, and the newer functions in the same code already handle all these cases correctly.

Section 11 reports that the Rust fuzz harness "targets the Rust `decrypt_bytes_v8` entry point with multiple key sizes and AAD values" and that "no panics were observed across 10^6 corpus entries". The harness has since been delivered, and it does not do that: it targets the legacy `decrypt_bytes`, and by construction it reaches nothing beyond the tag check. This is the subject of CVF-22. The reading of the source is still valid: `decrypt_bytes_v8` really is the careful path. It is written throughout in the fallible style, i.e. every failure is returned as an error: `decrypt_core_v8` returns `Result` at every branch, `fixed_decode_tokens` checks its length precondition, and `unpad_message` validates the length prefix instead of slicing without checking. However, the fuzzing result does not demonstrate any of this, since the fuzzer never enters the v8 path.

The crate exports five other decryption functions, and they are not written in that way.

The shared legacy core ends with an explicit abort, and its comment states that this is deliberate:

```
  // v7 low-level API: signature is `-> Vec<u32>`, so a malformed
padded
  // buffer panics here exactly as it did before V3-CVF8. The v8
path
  // (`decrypt_core_v8`) returns the error instead.
  unpad_message(&padded).expect("v7 decrypt: malformed padded
buffer")
```

`decrypt_core` is called by `decrypt`, `decrypt_bytes`, `decrypt_str` and `decrypt_raw`. For the three that verify a tag first, the panic can be reached only by a party who holds the key, so it is a robustness defect and not an attack surface. `decrypt` itself, however, is `pub` and takes a raw token slice without any tag verification, so for that one entry point any token vector supplied by a caller reaches the panic directly.

The same function subtracts without checking:

```
padded.push((token - addend) / k) as u32);
```

For a token stream that is not of the form $ck + a$, `token - addend` underflows, which causes a panic in debug builds and wraps around in release builds. `decrypt_core_v8` guards exactly this case, and its comment explains why: "the bare subtraction panics here in debug builds and wraps in release builds and in C." The guard was added to the v8 core, but not to the v7 core that four public functions still use. The delivered manifest confirms which behaviour a shipped build has: since there is no `[profile]` section, the release default is `overflow-checks = false`, so the wrap is the deployed behaviour and the panic happens only in debug builds (CVF-16, CVF-23).

Separately, `validate_key` is called from exactly two places, `encrypt_bytes_v8_with_profile` and `decrypt_bytes_v8`, so no legacy entry point validates its key. Two kinds of invalid key cause a panic:

- An empty key. `let kk = key.len() as u64` is zero, and the next line indexes `key[(real_idx % kk) as usize]`, so the remainder operation divides by zero. This affects `encrypt`, `encrypt_bytes`, `encrypt_raw`, `encrypt_bytes_with_nonce` and, through `decrypt_core`, all four legacy decryption functions. For example, `encrypt_bytes("x", &[], b"")` panics.
- A key element equal to 1 or 0. `derive_addend` and `derive_noise_token_addend` both compute `% (key_element - 1)`, which is a division by zero when the element is 1, and an underflow of the subtraction itself when the element is 0.

An attacker cannot trigger any of these, since the key is the caller's own. The defect is that a public cryptographic API accepts obviously invalid keys and aborts the process, instead of returning the error that its `Result` return type already allows. The v8 entry points return a descriptive error in all three cases, so the fix is to call the existing check, and no new logic is needed.

The KAT harness illustrates how easily an empty key gets passed. `kat_cross_check.rs` calls `encrypt_bytes_with_nonce(message, &[], [0u8; NONCE_SIZE], b"")` with a literal empty key. That call does not panic only because the early return for an empty message happens before the key is read; it would divide by zero for any non-empty message. See CVF-25, where the same line is a test defect for a different reason.

We also noticed an unrelated problem while reading `validate_key`. Its doc comment justifies the primality requirement by correctness: "The correctness argument recovers `c` from `c * k + a` by exact division, which needs `gcd(a, k) = 1` for every addend `a` in `[1, k - 1]` – true only when `k` is prime." This is the same claim that CVF-8 reports as incorrect in the paper, now repeated in the code. It is not correct here either: for $t = ck + a$ with $0 < a < k$, the recomputed addend is identical on both sides and $t - a = ck$ exactly, so the division is exact for a composite k as well. Invertibility requires $a < k$, not coprimality. Primality remains a requirement of the specification, because $\mathcal{P}$ is a set of primes, so the check itself should stay. Only its stated reason is wrong, and correcting it here would prevent the code from being cited as independent support for the paper's version of the claim.

Recommendation

Bring the legacy path up to the error handling of the v8 path, so that the crate does not follow two different standards.

For the abort on a malformed buffer, change `decrypt_core` to return `Result<Vec<u32>, String>` and pass the error on. The four functions that call it already return `Result`, so each needs only a `?`. `decrypt`, whose signature is `-> Vec<u32>`, is the one that needs a real decision. Since it is a public low-level entry point that accepts an arbitrary token slice without a tag check, we recommend changing its return type to `Result` instead of keeping the panic, and deprecating it if that change is too disruptive.

For the unchecked subtraction, move the guard that `decrypt_core_v8` already has into the shared inverse function:

```
if token < addend || (token - addend) % k != 0 {
    return Err(format!(
        "Malformed ciphertext: token at position {} is not of the
  form \
         codepoint * k + addend for key element {}.", ct_pos, k
    ));
}
```

Placing it once in a shared helper used by both cores is better than adding a second copy, since the divergence between the two cores is how this problem arose.

For key validation, call `validate_key(key)?` at the start of every legacy entry point that takes a key: `encrypt_bytes`, `encrypt_str`, `encrypt_raw`, `decrypt_bytes`, `decrypt_str`, `decrypt_raw` and `encrypt_bytes_with_nonce`. `validate_key` already rejects the empty key and any element below `MIN_KEY_PRIME`, which covers 0 and 1, so no new checks are required; see CVF-16 for the upper bound that it still lacks. `encrypt` and `decrypt` take `&[u64]`, return `Vec<u32>` and cannot report an error. If they are kept, document that the caller must validate the key, but we recommend deprecating them. CVF-24 recommends the same deprecation for a different reason, and one `#[deprecated]` attribute per function serves both.

Also correct the claim about fuzzing. The harness has been delivered and targets the legacy `decrypt_bytes`, not `decrypt_bytes_v8`, so Section 11 needs to be corrected on that point. The correction does not make the no-panic result valid for the legacy functions either, because every panic listed above lies either beyond the tag check or behind invalid keys that the harness's fixtures never supply. CVF-22 describes what the harness would need to do to reach them: derive the plaintext from the fuzzer input, construct a valid ciphertext, modify it while recomputing the tag, add a target for the unauthenticated `decrypt`, and include invalid keys in the fixture set. Until this is done, Section 11 should say that the no-panic result covers the initial parsing and the tag comparison, since that is what it covers.

Finally, correct the `validate_key` doc comment together with the fix for CVF-8, so that the primality check is justified by the definition of $\mathcal{P}$ and by the requirement $a < k$, instead of by a coprimality argument that does not hold.

/fuzz/fuzz_targets/decode_bytes.rs

```
29 use napqes::decrypt_bytes;
```

same file

```
/// Key fixtures — three representative sizes selected by fuzzer byte[0].
fn key_for(selector: u8) -> Vec<u64> {
    match selector % 3 {
        0 => vec![1_000_003, 1_000_033],
        1 => vec![7_999_993],
        _ => vec![
            1_000_003, 1_000_033, 1_000_037, 1_000_039,
            1_000_081, 1_000_099, 1_000_117, 1_000_121,
            1_000_133, 1_000_151,
        ],
    }
}
```

/src/kat_cross_check.rs

```
            let ct = encrypt_bytes_with_nonce(message, &[], [0u8; NONCE_SIZE], b"");
```

/src/lib.rs

```
    (u32_from_be4(&d[..4]) as u64 % (key_element - 1)) + 1
```

same file

```
    (u32_from_be4(&d[..4]) as u64 % (key_element - 1)) + 1
```

same file

```
/// Reject a prime tuple that is empty, composite, undersized or repeating.
///
/// The correctness argument recovers `c` from `c * k + a` by exact division,
/// which needs `gcd(a, k) = 1` for every addend `a` in `[1, k - 1]` -- true
/// only when `k` is prime. Called from both v8 entry points so that a caller
/// supplying a malformed key gets an error here rather than a silently
/// undecryptable ciphertext, matching `_validate_key` in the Python port.
```

same file

```
fn validate_key(key: &[u64]) -> Result<(), String> {
```

same file

```
            padded.push(((token - addend) / k) as u32);
```

same file

```
    // v7 low-level API: signature is `-> Vec<u32>`, so a malformed padded
```

```
    // buffer panics here exactly as it did before V3-CVF8. The v8 path
    // (`decrypt_core_v8`) returns the error instead.
    unpad_message(&padded).expect("v7 decrypt: malformed padded buffer")
```

same file

```
    let kk = key.len() as u64;
```

same file

```
                let k = key[(real_idx % kk) as usize];
```

same file

```
    validate_key(primes)?;
```

same file

```
    validate_key(primes)?;
```

CVF-16: fixed Category: **over-underflow**

> **Summary**
>
> **Key validation does not check the upper limit on the size of key elements that the paper requires, so a key the code accepts can cause an arithmetic overflow that corrupts messages or makes them impossible to decrypt.**
>
> The secret key is a list of prime numbers, and the paper requires every one of them to lie within a fixed range. The paper's guarantee that encrypted values always fit into their fixed-size fields depends on that upper limit. The code defines the limit but deliberately does not check it when it accepts a key, so a key with much larger primes is accepted. With such a key, the arithmetic overflows: depending on the code path, the receiver either cannot decrypt the message or silently gets wrong content. Such a key also makes every encryption and decryption very slow, because the key is re-checked each time. Only the key owner can supply such a key, but nothing in the code prevents it.

Section 3 makes the prime interval normative: $\mathcal{P}$ is the set of primes in $[10^6,\ 9.9 \times 10^6]$, and the paper now requires implementations to use this interval. `Remark 3.11` then discharges the outstanding obligation from the notation section that tokens fit into their field, by bounding the largest token, at $c = \texttt{0x10FFFF}$ and $k = 9{,}899{,}999$, as

$$1{,}114{,}112 \times 9{,}899{,}999 - 1 = 11{,}029{,}707{,}685{,}887 \approx 2^{43.33}$$

, which leaves a margin of more than 1.6×10^6 with respect to the 8-byte serialised field. The paper describes this bound as "unconditional: it holds for every" admissible input. It is unconditional in c, but it depends entirely on $k \leq 9{,}899{,}999$.

The implementation defines the bound but does not check it. `MAX_KEY_PRIME` is `9_900_000`, and its doc comment presents the omission as a decision:

> This bound constrains key *generation* only. Validation and decryption accept any prime `>= MIN_KEY_PRIME`, so keys generated before this bound was tightened remain usable.

Accordingly, `validate_key` checks four things: that the key is non-empty, that its elements are prime, at least `MIN_KEY_PRIME`, and pairwise distinct. It checks no upper bound. Tokens are then constructed as

```
cypher.push(c as u64 * k + addend);
```

with `c` at most `0x10FFFF`, so the product overflows `u64` once k exceeds roughly $2^{64}/2^{20.09} \approx 1.66 \times 10^{13}$. Every prime key element in $[1.66 \times 10^{13},\ 2^{64})$ passes `validate_key` and overflows this multiplication. The same expression appears in all five places where tokens are emitted, on both the v7 and the v8 paths.

`Cargo.toml` has now been delivered, and it settles how the overflow behaves in a deployed build. The manifest declares no `[profile]` section, so the release profile uses Cargo's defaults, `overflow-checks = false` and `debug-assertions = false`. The product therefore wraps around silently in release builds and panics only in debug builds. In other words, the safe behaviour occurs only in the configuration that is not shipped, and the behaviour that corrupts data occurs in the configuration that is. See CVF-23, which describes the missing profile section and its second consequence, for the tag comparison.

What happens next differs between the two paths, and both outcomes are bad:

- On v8, `decrypt_core_v8` checks that each token really has the form $ck + a$, so a wrapped token fails the check `(token - addend) % k != 0` and the ciphertext is rejected. The sender therefore produces authenticated ciphertexts that the receiver cannot decrypt, and the error message points to a malformed ciphertext instead of to the key.
- On v7, `decrypt_core` performs no such check and computes `((token - addend) / k) as u32` on the wrapped value, which yields an arbitrary codepoint and a silently corrupted plaintext.

Nothing in the API guides a caller towards the normative interval either. `generate_prime_numbers` and `generate_v8_key` both take `min_val` and `max_val` as parameters, so the bounds are chosen by the caller at generation time, in addition to being unchecked at validation time. The constants exist, but they are only conventions.

A second consequence of the missing bound is a sharp drop in performance on the decryption path. `is_prime` uses trial division:

```
let mut i: u64 = 3;
while i.saturating_mul(i) <= n {
```

```
    if n % i == 0 { return false; }
    i += 2;
}
```

This is correct, since `saturating_mul` avoids the overflow that the naive form would have, but it costs $\Theta(\sqrt{n})$. Inside the normative interval, that is at most about 1,600 iterations per element, which is negligible. For a key element near 2^{63}, it is of the order of 2^{31} iterations per element. Since `validate_key` runs on *every* call of `encrypt_bytes_v8` and `decrypt_bytes_v8`, instead of once when the key is loaded, a key that the validator accepts can make every decryption take minutes. Even within the intended interval, testing the primality of all K elements again for every message is avoidable work on the path that runs for every message.

Recommendation

Add the upper bound to `validate_key`, next to the existing check of the lower bound:

```
if k > MAX_KEY_PRIME {
    return Err(format!(
        "Key element at index {} ({}) exceeds the maximum of {}.",
        i, k, MAX_KEY_PRIME
    ));
}
```

This check is what makes the bound of `Remark 3.11` hold in the implementation: with it, the word "unconditional" in the remark becomes true of the code, and not only of the intended input.

The reason given in the doc comment for omitting the check, that keys generated before the bound was tightened must remain usable, should be resolved explicitly instead of by leaving validation open. If such keys exist, their elements lie in $[9{,}900{,}000,\ 10^7)$ or close to it, far below the overflow threshold. So accept a documented legacy ceiling that still preserves the bound on token width (any $k < 2^{43}$ is safe for the full codepoint range), instead of accepting every prime that fits into `u64`. Express the safe ceiling in terms of the bound it protects:

```
// c <= 0x10FFFF, so c * k + a < 2^64 requires k < 2^64 / (0x10FFFF
  + 1).
const MAX_SAFE_KEY_PRIME: u64 = u64::MAX / (0x10FFFF + 1);
```

and reject any element above it unconditionally, even if a wider legacy range is allowed below it.

Independently of this, make the arithmetic check its own precondition. Use `checked_mul` and `checked_add` in the five places where tokens are emitted, and return an error on overflow instead of relying on the validator having been called, since two of those places are on legacy paths that never call it (CVF-15). A crate that computes on secret key material should not depend on the overflow setting of the release profile for correctness, and the delivered manifest confirms that this setting is off. Adding `[profile.release] overflow-checks = true`, as CVF-23 recommends, is worth doing as an additional safeguard, but it does not replace the checks: a profile setting is not a precondition, and a crate that depends on this one can override it.

Move key validation off the path that runs for every message. Validating K primes by

trial division on every encryption and every decryption is wasted work at best, and an opportunity for denial of service at worst. We recommend a validated key type that is constructed once: a `Key` newtype whose constructor runs `validate_key`, so that any value of the type is known to be valid. `encrypt_bytes_v8` and `decrypt_bytes_v8` should then take that type instead of `&[u64]`. If the current signatures must be kept, at least bound the trial division by `MAX_KEY_PRIME`, so that its cost cannot exceed the cost for the interval.

Finally, since $\mathcal{P}$ is now normative, consider making `generate_prime_numbers` and `generate_v8_key` use the interval by default instead of taking it as a parameter, or making them reject bounds outside it. As written, neither key generation nor key validation enforces the normative interval.

/Cargo.toml

```
[lib]
name = "napqes"
path = "src/lib.rs"
```

/src/lib.rs

```
/// Upper bound of the normative prime interval, per
/// `docs/napseq-eprint-v3.tex` §Notation. `P` contains exactly 579_947
/// primes (verified by sieve), giving `P(579_947, 10) = 2^191.46` ordered
/// 10-tuples (`2^95.73` post-Grover).
///
/// This bound constrains key *generation* only. Validation and decryption
/// accept any prime `>= MIN_KEY_PRIME`, so keys generated before this bound
/// was tightened remain usable. Matches `napqes.py::MAX_KEY_PRIME` and
/// `C/napqes.h::NAPQES_MAX_KEY_PRIME`.
pub const MAX_KEY_PRIME: u64 = 9_900_000;
```

same file

```
pub fn is_prime(n: u64) -> bool {
    if n < 2 {
        return false;
    }
    if n == 2 {
        return true;
    }
    if n % 2 == 0 {
        return false;
    }
    let mut i: u64 = 3;
    while i.saturating_mul(i) <= n {
```

```
        if n % i == 0 {
            return false;
        }
        i += 2;
    }
    true
}
```

same file

```
fn validate_key(key: &[u64]) -> Result<(), String> {
    if key.is_empty() {
        return Err("Key must be a non-empty list of primes.".into());
    }
    for (i, &k) in key.iter().enumerate() {
        if !is_prime(k) {
            return Err(format!("Key element at index {} ({}) is not
prime.", i, k));
        }
        if k < MIN_KEY_PRIME {
            return Err(format!(
                "Key element at index {} ({}) is below the minimum of
{}.",
                i, k, MIN_KEY_PRIME
            ));
        }
        if key[..i].contains(&k) {
            return Err(format!(
                "Key element {} at index {} is a duplicate; all
elements must be distinct.",
                k, i
            ));
        }
    }
    Ok(())
}
```

same file

```
                cypher.push(c as u64 * k + addend);
```

same file

```
                cypher.push(c as u64 * k + addend);
```

same file

```
                cypher.push(c as u64 * k + addend);
```

same file

```
                cypher.push(c as u64 * k + addend);
```

same file

```
1170                cypher.push(c as u64 * k + addend);
```

CVF-17: open Category: **flaw**

Summary

The older (v7) encryption functions do not limit the number of noise tokens, so the length of an encrypted message varies randomly and reveals information about the message size that the newer (v8) format hides.

The scheme mixes random filler (”noise”) into every encrypted message and is designed so that the length of the encrypted message depends only on a rough size class of the original message. The newer v8 functions do this correctly. The four older v7 encryption functions, which are still public and are still used by other parts of the delivered code, do not limit the amount of filler, so the length of the encrypted message changes randomly from one encryption to the next. By observing these lengths, an attacker can learn the size class of the message, which is the leak that the client has already fixed in v8. Since the amount of filler is unlimited, a single message can also become very large and use a lot of memory. The built-in self-test vector shows this behaviour.

Scope of this finding. It concerns only the four legacy v7 encryption functions. The v8 emission loop implements both bounds correctly and is *not* affected. The v8 line ranges cited below are given as the conforming reference with which the v7 loops are compared, not as instances of the problem.

The specified emission loop has two bounds. For each real codepoint, it emits noise while $noise_run < 19$ and the position is a noise position, and afterwards it appends filler tokens until exactly

$$N_{\text{tok}} = R \cdot (\text{MAX_NOISE_RUN} + 1) = 20R$$

tokens have been emitted. `Lemma 3.9` depends on both: $|C| = 48 + 160(B(|M|) + 2)$ is a function of the padding bucket alone.

The v8 implementation applies both bounds correctly. `encrypt_bytes_v8_with_profile` keeps a `noise_run` counter, compares it with `MAX_NOISE_RUN`, and then fills up to the ceiling:

```
let ceiling = (padded.len() as u64) * (MAX_NOISE_RUN + 1);
while (cypher.len() as u64) < ceiling { /* emit filler */ }
```

Since the inner loop emits at most 19 noise tokens and one real token per codepoint, the count can never exceed the ceiling, and the filling step makes it exact. We checked the whole emission part of `encrypt_bytes_v8_with_profile` statement by statement against `Listing 1`, and it conforms on every point:

- `noise_run` is reset once per padded codepoint;
- the condition is `noise_run < MAX_NOISE_RUN && is_noise_pos_v8(...)`, which matches the listing's `while noise_run < MAX_NOISE_RUN and is_noise(ct_pos)`, including the early exit that skips the noise-position check once the run reaches 19;
- `MAX_NOISE_RUN` is `19`;
- the key element is `key[real_idx % K]` for both noise and real tokens, so the noise tokens emitted inside a run use the key element of the real token that follows them;
- `ct_pos` advances on every emission, and `real_idx` only on a real one;
- the ceiling is `padded.len() * (MAX_NOISE_RUN + 1)` with `padded.len() == R`;
- in the filler loop, `real_idx` stays **fixed** at R while `ct_pos` continues to advance, exactly as in the listing, so the filler tokens have the same structure as genuine noise tokens.

The v8 path therefore satisfies `Lemma 3.9` as written, and this finding does not ask for any change to it.

The four legacy encryption functions, `encrypt`, `encrypt_bytes`, `encrypt_bytes_with_nonce` and `encrypt_raw`, have neither bound. Each contains the same unbounded loop:

```
for &c in &padded {
    loop {
        if is_noise_pos(&kb, &nonce, ct_pos, noise_p) {
            /* emit noise, ct_pos += 1 */
        } else {
            /* emit real token, break */
        }
    }
}
```

There is no `noise_run` counter, no cap in the condition, and no filling up to a ceiling afterwards. The omission is deliberate and documented: the doc comment of `MAX_NOISE_RUN` states that the cap is "never applied to v7, matching `napqes.py` / `C/napqes.c`", and the same comment describes exactly what the missing ceiling costs, in the client's own words:

> Without this, the *natural* token count varies with the message-derived synthetic nonce even for a fixed padding bucket, letting an observer who collects several ciphertexts of one message under varying AAD average out the noise and reliably recover the padding bucket

This attack was reported in an earlier review as V2-CVF2. It was fixed for v8 but remains in v7, which is still the crate's most prominent public API and, in the delivered code, the API that the transport layers actually use.

On v7, the leak is worse than the comment describes, because the v7 nonce is a fresh random value from a CSPRNG and not a synthetic IV. The resulting noise pattern is a

deterministic function of (kb, N), so with a random N the token count is a new random variable on every call. An observer therefore does not need to collect ciphertexts of one message under varying AAD to average out the noise: the length of the ciphertext varies across encryptions of *the same message*, and the centre of its distribution is determined by the padding bucket. The length is not deterministic at all on this path, so `Lemma 3.9`, the $\log_2 \beta$ leakage bound of `Corollary 4.14`, and the whole length-hiding argument of Section 4.3 apply only to v8.

The size of the variation can be computed from the paper's own parameters. Without the cap, the expected number of tokens per padded codepoint is $1+\sum_{j\geq 1} p^j = 1/(1-p)$ with $p = \theta(N)/2^{64}$. With the cap, it is $1 + \sum_{j=1}^{19} p^j$, which ranges from 3.99 at $p = 0.75$ to 18.21 at $p = 0.99$. Since $\theta(N)$ takes values across the whole range $[\theta_{\mathsf{min}}, \theta_{\mathsf{max}}]$ as the nonce varies, the v7 ciphertext length for one fixed padding bucket varies by a factor of roughly 4.6, even before the uncapped tail is counted.

This can be demonstrated from the delivered files without running anything. `self_test.rs` contains a v7 reference ciphertext of 4,784 bytes for the one-codepoint message `"A"`. That gives $N_{\mathsf{tok}} = (4784 - 48)/8 = 592$ tokens for a padded block of $R = 18$ codepoints, i.e. 32.9 tokens per padded codepoint, while the cap allows at most 20. In addition, 592 is not a multiple of 20, so no ceiling was applied. Under the specification and under v8, the same message yields exactly 360 tokens and 2,928 bytes. The vector itself therefore shows that the v7 path implements neither bound.

Without the cap, the loop is also unbounded in principle. Since $\theta(N)/2^{64} \in [0.75, 0.99]$, the length of a run follows a geometric distribution with a mean of up to 99, and there is no upper limit on any individual run. At the top of the range, the expected expansion is roughly 800 bytes per plaintext codepoint, against 160 in the specification. A message of 65,535 codepoints produces a token vector of more than six million entries, and individual runs of thousands of tokens occur with non-negligible probability. The loop terminates with probability one but has no bound, so the memory used on the encryption path depends on a value that the caller cannot inspect.

Finally, code **outside the reviewed scope** shows how far the problem reaches. We cite it only as evidence of impact; the cause of the finding lies entirely in `src/lib.rs`. The crate's own `src/ot_frame.rs` builds its protocol data unit with `encrypt_raw`, and `src/bin/sts.rs`, which generates the SP 800-22 evidence of Section 9, calls `encrypt_bytes`. Both are v7 entry points with the unbounded loop, so the v7 path is not an unused compatibility layer in the delivered code.

Recommendation

Apply both bounds to the legacy loop, or withdraw the legacy encryption functions. No change is required on the v8 path.

The straightforward fix is to move the v8 loop into a shared helper and call it from all five encryption functions, which would then differ only in the derivation key and the width of the AAD prefix. This removes four copies of a loop that has diverged from its v8 counterpart in two ways that matter for security, and it is this divergence, more than either missing bound, that allowed the problem to arise. The helper needs the `noise_run` counter in the loop condition and the filling up to the ceiling after the loop. Nothing else in the v7 derivations has to change, so v7 ciphertexts keep the

same structure while their length becomes deterministic.

Note that this changes the v7 wire format: existing v7 ciphertexts will not decrypt with a bounded decoder, and the reverse also holds, because `decrypt_core` classifies every position independently and consumes the whole token slice, while a decoder aware of the ceiling must derive the number of real tokens from the length. Since v7 and v8 are already declared not to interoperate, and the format is already selected outside the ciphertext, the second option is cleaner: deprecate the v7 encryption functions instead of introducing a third format. The paper specifies only v8, `Section 8.2` describes the Rust implementation only in terms of `generate_v8_key`, `encrypt_bytes_v8` and `decrypt_bytes_v8`, and nothing in the specification requires v7 to exist. Keeping `decrypt_bytes` and `decrypt_raw` for reading archived ciphertexts, while removing the v7 *encryption* functions from the public API, would resolve this finding without a migration.

Whichever option is chosen, make two changes to the documentation. The `MAX_NOISE_RUN` comment currently explains the V2-CVF2 attack and then mentions in passing that the cap is "never applied to v7", without connecting the two. It should state plainly that the v7 format is exposed to the attack described in the following paragraph. In addition, if any v7 function remains public, Section 8.2 should say that the crate exports a second, unspecified wire format whose ciphertext length is not deterministic, so that a reader does not assume that the leakage bound of Section 4.3 applies to it.

Finally, if the v7 encryption functions are kept, cap the run length regardless of format compatibility. A loop with no bound, whose number of iterations is determined by a keyed derivation that the caller cannot observe, is worth bounding for robustness alone, independently of the length leak.

/src/lib.rs

```
    for &c in &padded {
        loop {
            if is_noise_pos(&kb, &nonce, ct_pos, noise_p) {
                let k = key[(real_idx % kk) as usize];
                let noise_c = derive_noise_char(&kb, &nonce, ct_pos);
               let noise_add = derive_noise_token_addend(&kb, &nonce,
ct_pos, k);
                cypher.push(noise_c * k + noise_add);
                ct_pos += 1;
            } else {
                let k = key[(real_idx % kk) as usize];
                let addend = derive_addend(&kb, &nonce, real_idx, k);
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }
```

same file

```
    for &c in &padded {
        loop {
            if is_noise_pos(&kb, &nonce, ct_pos, noise_p) {
                let k = key[(real_idx % kk) as usize];
                let nc = derive_noise_char(&kb, &nonce, ct_pos);
                let na = derive_noise_token_addend(&kb, &nonce,
ct_pos, k);
                cypher.push(nc * k + na);
                ct_pos += 1;
            } else {
                let k = key[(real_idx % kk) as usize];
                let addend = derive_addend(&kb, &nonce, real_idx, k);
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }
```

same file

```
    for &c in &padded {
        loop {
            if is_noise_pos(&kb, &nonce, ct_pos, noise_p) {
                let k = key[(real_idx % kk) as usize];
                let nc = derive_noise_char(&kb, &nonce, ct_pos);
                let na = derive_noise_token_addend(&kb, &nonce,
ct_pos, k);
                cypher.push(nc * k + na);
                ct_pos += 1;
            } else {
                let k = key[(real_idx % kk) as usize];
                let addend = derive_addend(&kb, &nonce, real_idx, k);
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }
```

same file

```
    for &c in &padded {
        loop {
            if is_noise_pos(&kb, &nonce, ct_pos, noise_p) {
```

```
                let k = key[(real_idx % kk) as usize];
                let nc = derive_noise_char(&kb, &nonce, ct_pos);
                let na = derive_noise_token_addend(&kb, &nonce,
ct_pos, k);
                cypher.push(nc * k + na);
                ct_pos += 1;
            } else {
                let k = key[(real_idx % kk) as usize];
                let addend = derive_addend(&kb, &nonce, real_idx, k);
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }
```

same file

```
/// Hard cap on consecutive noise tokens emitted before a real token
in the
/// v8 token-emission loop (never applied to v7, matching `napqes.py`
/
/// `C/napqes.c`). As of the V2-CVF2 fix, this cap also fixes the
*total*
/// per-real-token budget: every v8 ciphertext is padded with
additional
/// filler tokens up to a deterministic per-bucket ceiling of
/// `real_token_count * (MAX_NOISE_RUN + 1)` tokens (see
/// `encrypt_bytes_v8` / `decrypt_core_v8`). Without this, the
*natural*
/// token count varies with the message-derived synthetic nonce even
for a
/// fixed padding bucket, letting an observer who collects several
/// ciphertexts of one message under varying AAD average out the
noise and
/// reliably recover the padding bucket (`docs/CAVEATS.md`, V2-CVF2).
After
/// this fix, v8 ciphertext length is a deterministic function of the
/// padding bucket alone, at the cost of always paying the worst-case
~20x
/// expansion instead of the ~13.4x average case.
pub const MAX_NOISE_RUN: u64 = 19;
```

same file

```
    for &c in &padded {
        let mut noise_run: u64 = 0;
```

```
1159        loop {
1160            if noise_run < MAX_NOISE_RUN && is_noise_pos_v8(&sk_fmt,
&nonce, ct_pos, noise_theta) {
```

same file

```
1177    // V2-CVF2 fix: pad up to the fixed, bucket-only ceiling so
ciphertext
1178    // length never depends on the message-derived nonce's noise
realisation.
1179    let ceiling = (padded.len() as u64) * (MAX_NOISE_RUN + 1);
1180    while (cypher.len() as u64) < ceiling {
1181        let k = primes[(real_idx % kk) as usize];
1182        let nc = derive_noise_char(&sk_fmt, &nonce, ct_pos);
1183        let na = derive_noise_token_addend(&sk_fmt, &nonce, ct_pos,
k);
1184        cypher.push(nc * k + na);
1185        ct_pos += 1;
1186    }
```

/src/self_test.rs

```
40 // Reference ciphertext hex, v7 wire format (fixed-width tokens, CVF1
fix),
41 // regenerated under the unified domain-first HMAC derivation layout
42 // introduced by the CVF2 fix (see SPEC.md §3 and
docs/napseq-eprint-preprint.tex
43 // rem:domsep). Every derivation input changed shape (`d || N || ctx`
instead
44 // of the previous mixed nonce-first/domain-first layouts), so this
constant
45 // is not byte-compatible with any pre-CVF2 vector for the same
key/nonce/message.
46 // Decoded once at test time; storing as hex avoids large byte
literals.
47 const KAT_CIPHERTEXT_HEX: &str = concat!(
```

CVF-20: fixed Category: **algorithm**

Summary

Key generation does not use the sampling method that the paper specifies, so the keys it produces have a tiny bias that the paper says is absent, and it stops the program instead of returning an error when it fails.

The paper describes exactly how the prime numbers of the secret key must be chosen, so that every allowed prime is equally likely. The code chooses them in a slightly different way, and its choice is almost, but not exactly, uniform. The deviation is far too small to weaken security, but it contradicts what the paper states. The same function also stops the whole program, instead of reporting an error, if it cannot find enough primes.

`Remark 3.2` specifies the sampling procedure in detail, with the explicit aim of eliminating modulo bias, i.e. the slight preference for some values that appears when a random number is reduced modulo a range size that does not divide the number of possible random values. $\mathcal{P}$ is fixed as a public canonical ascending enumeration, and index sampling draws a uniform 32-bit x, sets $L = \lfloor 2^{32}/|\mathcal{P}| \rfloor \cdot |\mathcal{P}|$, redraws while $x \geq L$, and returns $idx = x \bmod |\mathcal{P}|$, "which is then exactly uniform over $\{0, \ldots, |\mathcal{P}| - 1\}$ with no modulo bias". Distinctness is obtained by redrawing collisions, and the remark ends by tying the procedure to the entropy claim: "This realises exactly the uniform distribution over ordered tuples of K distinct elements of $\mathcal{P}$ assumed by the $H_\infty(\mathbf{k})$ computation below."

The implementation uses a different procedure:

```
let span = max_val - min_val + 1;
let max_attempts = span.saturating_mul(4);
...
while primes.len() < count && attempts < max_attempts {
    let num = min_val + (rng.next_u64() % span);
    if is_prime(num) && !primes.contains(&num) {
        primes.push(num);
    }
    attempts += 1;
}
```

There are two differences, and only the second one is a defect.

First, the code samples a uniform integer from the interval and rejects composite numbers, instead of sampling a uniform index into an enumeration of $\mathcal{P}$. The two procedures give the same distribution: rejection sampling over an interval, conditioned on landing on a prime, is uniform over the primes in that interval. The entropy claim therefore still holds, and this is not a security difference. We mention it because the implementation never enumerates $\mathcal{P}$, and so it never checks that $|\mathcal{P}| = 579{,}947$ or that the interval is the normative one. The canonical enumeration that `Remark 3.2` requires to be "identical across implementations" does not exist in this implementation, so agreement between implementations on $\mathcal{P}$ rests on the two interval constants alone.

Second, the code computes `rng.next_u64() % span`. This is the modulo reduction that `Remark 3.2` is written to avoid, applied without the rejection step. With `span` $= 8{,}900{,}001$ for the normative interval, 2^{64} is not a multiple of `span`, so the lowest $2^{64} \bmod \text{span}$ residues each receive one extra preimage. The effect is negligible: about 2.07×10^{12} complete cycles fit in 2^{64}, which gives a relative deviation near 5×10^{-13} and a total variation distance from uniform of roughly 2.4×10^{-13}, and no security claim is measurably affected. We report it because the paper specifies the exact sampler and states that the result has "no modulo bias", while the implementation does not follow it in the one place where the paper makes the absence of bias a normative requirement. A reader comparing the two would expect the rejection loop to be present, and its absence is visible only in a line-by-line comparison.

There are two smaller points in the same function. First, it stops the program instead of reporting a failure:

```
if primes.len() < count {
    panic!("Could not find {} distinct primes in [{}, {}] — widen the
  range.", ...);
}
```

and it also starts with `assert!(max_val > min_val)`. A public key-generation function therefore has two panic paths, i.e. places where the program aborts, in situations where its callers, `generate_v8_key` and application code, could handle an error. CVF-15 describes the same pattern across the legacy API. Second, the attempt limit is $4 \cdot$span, which for the normative interval means 35.6 million trial-division primality tests before the panic occurs. The limit is never approached in normal use, since about one number in 15.4 in $[10^6, 9.9 \times 10^6]$ is prime and ten primes take roughly 154 draws. However, a caller who passes a narrow range without enough primes pays the full cost before learning that the call failed.

Recommendation

Implement the sampler that `Remark 3.2` specifies, or amend the remark to describe what the implementations do.

Implementing it is the better option, because the remark's canonical enumeration is what makes $\mathcal{P}$ identical across implementations, and that guarantee is currently missing. Precompute the sieve table once, then use:

```
fn sample_index(rng: &mut impl RngCore, n: u32) -> u32 {
    let l = (u32::MAX / n) * n;          // floor(2^32 / n) * n
    loop {
        let x = rng.next_u32();
        if x < l { return x % n; }
    }
}
```

and draw key elements as `P[sample_index(&mut rng, P.len() as u32) as usize]`, redrawing on a collision with an element already chosen. This is exactly uniform and removes the bias. It also removes the attempt limit and the panic that depends on it, because the loop cannot fail: every draw eventually lands below L, and $|\mathcal{P}|$ is much larger than K. Finally, it gives the implementation a concrete $|\mathcal{P}|$

that it can check against 579,947, which is a cheap check of the interval constants that the code does not currently perform.

If the interval-rejection approach is kept instead, at least remove the modulo bias. We recommend the library primitive over a hand-written loop: `rand::distr::Uniform::new_inclusive(min_val, max_val)` performs the rejection internally, is harder to get subtly wrong, and removes the `span` arithmetic entirely.

```
use rand::distr::{Distribution, Uniform};
let dist = Uniform::new_inclusive(min_val, max_val)
    .map_err(|e| format!("invalid prime interval: {e}"))?;
let num = dist.sample(&mut rng);
```

If a form without dependencies is preferred, the equivalent hand-written version is:

```
let l = (u64::MAX / span) * span;
let num = min_val + loop {
    let x = rng.next_u64();
    if x < l { break x % span; }
};
```

In that case, also amend `Remark 3.2` to say that implementations may sample uniformly from the interval and reject composite numbers, and that this gives the same distribution, so that the paper and the code describe the same procedure. The remark should then also say that the interval-rejection variant does not need the canonical enumeration, since the reader is currently told that it is required.

Separately, change the return type to `Result<Vec<u64>, String>` and replace both panics with errors, so that key generation handles errors in the same way as the v8 entry points. `generate_v8_key` should pass the error on to its caller. If the attempt limit is kept, scale it to the expected number of draws, $O(K \log |\mathcal{P}|/\text{density})$, instead of $4 \cdot \text{span}$, so that a range without enough primes is reported within milliseconds instead of after tens of millions of primality tests.

Note also that the `!primes.contains(&num)` collision check on the same line is item 2 of CVF-41, and that the sieve table recommended here overlaps item 1 of CVF-41. If both are implemented, build the table once and use it for both generation and validation.

/src/lib.rs

```
/// Generate `count` distinct primes drawn uniformly from `[min_val, max_val]`.
pub fn generate_prime_numbers(count: usize, min_val: u64, max_val: u64) -> Vec<u64> {
    assert!(max_val > min_val);
    let mut rng = rand::thread_rng();
    let span = max_val - min_val + 1;
    let max_attempts = span.saturating_mul(4);
    let mut primes: Vec<u64> = Vec::with_capacity(count);
    let mut attempts: u64 = 0;
    while primes.len() < count && attempts < max_attempts {
```

same file

```
        let num = min_val + (rng.next_u64() % span);
```

same file

```
        if is_prime(num) && !primes.contains(&num) {
            primes.push(num);
        }
        attempts += 1;
    }
```

same file

```
    if primes.len() < count {
        panic!(
            "Could not find {} distinct primes in [{}, {}] — widen the
range.",
            count, min_val, max_val
        );
    }
    primes
}
```

same file

```
pub fn generate_v8_key(count: usize, min_val: u64, max_val: u64) ->
(Vec<u64>, [u8; SK_SIZE]) {
    let primes = generate_prime_numbers(count, min_val, max_val);
    let mut sk = [0u8; SK_SIZE];
    rand::thread_rng().fill_bytes(&mut sk);
    (primes, sk)
}
```

CVF-22: open Category: **procedural**

Summary

The fuzz testing that the paper cites as evidence of robustness tests an older function than the paper says, and by its design it cannot reach any of the decryption code beyond the authentication check.

Fuzz testing feeds large numbers of random inputs to a program to find crashes. The paper says that the Rust code was fuzz-tested on its main, current decryption function a million times without a crash. The delivered fuzzing program actually tests an older decryption function. More importantly, it feeds random bytes to a function that first checks the message's authentication tag, and random bytes never pass that check, so none of the code that runs after it is ever tested. All the crashes we found in the older decryption code lie in exactly that untested part. The figure of one million inputs also cannot be traced to any delivered run, log or stored input set. The scheme's security is not directly affected, but the paper's robustness claim is not supported by the evidence.

Section 11 states:

> The Rust harness (`rust/fuzz/fuzz_targets/decode_bytes.rs`, using `cargo-fuzz`) targets the Rust `decrypt_bytes_v8` entry point with multiple key sizes and AAD values. No panics were observed across 10^6 corpus entries in either harness.

The harness has now been delivered, and it does not target `decrypt_bytes_v8`. Its only import from the crate is `use napqes::decrypt_bytes;`, and the body makes three calls, all of them to `decrypt_bytes`, i.e. the v7 entry point. `decrypt_bytes_v8` does not appear anywhere in the file. The header of the harness describes it as a target "for NAPSEQ v6 decoder paths" and reasons about "a valid NAPSEQ v6 ciphertext", so the file is consistent with itself but not with the paper. The scheme that the paper specifies has no Rust fuzz coverage at all.

The more important problem is what the harness can reach, which follows from its design and not from its choice of target. `decrypt_bytes` verifies the authentication tag before it reads the payload, so every input has exactly two ways out:

- the minimum-length check, `if ciphertext.len() < NONCE_SIZE + TAG_SIZE`, for inputs shorter than 48 bytes;
- the authentication failure, `if !ct_eq_bytes(recv_tag, calc_tag.as_ref())`, for everything else.

To get past the second, an input must carry a payload whose HMAC-SHA256 tag under the fixture key matches, i.e. the fuzzer would have to forge a tag. Everything after that line is therefore unreachable: no input that libFuzzer can construct ever enters `varint_keystream`, `fixed_decode_tokens`, `decrypt`, `decrypt_core` or `unpad_message`. The header of the harness presents this as a reassurance, while it is in fact a limitation:

```
//! Security contract:
```

```
//!   `decrypt_bytes` MUST never panic.  Any malformed or adversarial
  input
//!   must produce `Err(...)`.  A successful `Ok(plaintext)` is only
  possible
//!   when the fuzzer discovers a valid NAPSEQ v6 ciphertext, which
  requires
//!   forging an HMAC-SHA256 tag — computationally infeasible.
```

The same infeasibility that makes the ciphertext unforgeable makes the decoder impossible to fuzz with this design. Of the three stages that the header claims to cover, "full decode pipeline: auth check → LEB128 varint decode → plaintext recovery", one is exercised and two cannot be. The second of them is named after the LEB128 encoding that an earlier fix retired: the pipeline now uses a fixed width, and `b128_decode_tokens` is dead code.

The three calls made for each input are almost identical. They differ only in the key fixture and the AAD, and all three end at the same two exits, so the input space actually explored consists of the minimum-length check and a tag mismatch, three times over. The guard `if data.len() < 2 { return; }` discards only inputs of length 0 and 1. At length 2, the payload `&data[2..]` is empty, so the early return for an empty ciphertext at the start of `decrypt_bytes`, which is the behaviour that CVF-13 reports, is in fact reached. However, since it returns `Ok(String::new())` instead of panicking, a harness that only detects panics cannot see it, and the result for 10^6 entries says nothing about it.

The other half of the sentence in Section 11 is not accurate either. Section 11 says that the Python harness "targets `decrypt_bytes_v8` and the fixed-width token decoder, verifying that no exception other than `ValueError` escapes the public API", and ends with "no panics were observed across 10^6 corpus entries in either harness". The Python test suite is not itself under review here, but we read it as the reference with which the Rust claim is compared, and found the following:

- `tests/fuzz_atheris.py` targets the v7 `decrypt_bytes` and the retired LEB128 parser `_b128_decode_tokens`, and never `decrypt_bytes_v8` or a fixed-width decoder.
- It does not enforce the stated contract on exceptions: it allows `IndexError` on the varint target, and `UnicodeDecodeError` and `OverflowError` in addition to `ValueError` on the legacy-flag target, which is the one that goes through the public API.
- Its default run is 10,000 iterations, and the coverage-guided run that its docstring recommends is 10^5. The figure 10^6 appears in neither harness, in no run log and in no committed set of fuzzing inputs.

So the claim names the wrong target for both harnesses, overstates the exception contract of one of them, and cites a number of inputs that nothing in the delivery accounts for.

This is directly relevant to CVF-15. The legacy decryption function is fuzzed in name, but the code in it that can panic cannot be reached. None of the five panics described in CVF-15 could have been found by this harness:

- the abort `unpad_message(&padded).expect("v7 decrypt: malformed padded buffer")` and the unchecked subtraction `token - addend` are both inside `decrypt_core`, beyond the tag check;
- the division by zero for an empty key, at `key[(real_idx % kk) as usize]`, requires a key of length zero, while `key_for` returns tuples of 2, 1 and 10 elements, never an empty one;
- the division by zero for a key element of 0 or 1 requires such an element, while every fixture element is a prime above 10^6;
- `pub fn decrypt`, which takes a raw token slice without tag verification and is the one entry point where the abort can be reached by any caller, is never called by the harness.

So the five reachable panics in the crate are exactly the five that the harness, by its construction, cannot find, and the result for 10^6 entries says nothing about any of them.

Finally, the run cannot be reproduced. `fuzz/Cargo.toml` says that the corpus is "created on first run", while `decode_bytes.rs` says that the seed inputs are "checked in so that subsequent runs benefit from prior coverage discoveries". The two statements contradict each other, and neither holds for the delivered tree: there is no `fuzz/corpus/` directory. `.gitignore` contains only `/target` and `Cargo.lock`, so the inputs are missing because they were never committed, not because they were excluded. No run log, coverage figure, commit hash or number of seed inputs accompanies the claim of 10^6 entries, so a reader cannot tell which build produced it, from which inputs, or for which of the two wire formats. Since `Cargo.lock` is ignored as well (CVF-23), the claim cannot even be tied to a fixed set of dependencies.

Recommendation

Make the paper and the harness agree, and then make the harness able to reach the code that the claim is about. These are separate tasks, and the second one is the one that matters.

For the first task, either correct Section 11 to say that the Rust harness targets `decrypt_bytes` and the v7 wire format, or change the harness's target. Changing the target is preferable, and adding a second target instead of replacing the first is better still, since the v7 functions are the ones with reachable panics and the v8 functions are the ones the paper's claims are about. Two `[[bin]]` entries in `fuzz/Cargo.toml` cost nothing. Correct the Python half of the same sentence at the same time: name the entry point that `fuzz_atheris.py` actually calls, list the exception types it actually allows, and replace 10^6 with the figure from the recorded run.

For the second task, note that a target fed with raw bytes and aimed at a function that authenticates first can only ever test the authentication. To exercise a decoder, derive the *plaintext* from the fuzzer input, construct a valid ciphertext from it, and then modify the ciphertext and recompute its tag:

```
fuzz_target!(|data: &[u8]| {
    if data.len() < 4 { return; }
    let key = key_for(data[0]);
    let sk  = sk_for(data[1]);
    let aad = aad_for(data[2]);
```

```
    let msg = String::from_utf8_lossy(&data[3..]);

    // Round-trip: exercises the full encode and decode pipeline.
    if let Ok(ct) = encrypt_bytes_v8(&msg, &key, &sk, aad) {
        let _ = decrypt_bytes_v8(&ct, &key, &sk, aad);

        // Structure-aware mutation: flip inside the blob, then
  re-tag, so
        // the input authenticates and reaches the
  post-authentication checks.
        let mut bad = ct.clone();
        let i = NONCE_SIZE + (data[3] as usize % (bad.len() -
  NONCE_SIZE - TAG_SIZE));
        bad[i] ^= 1;
        retag_in_place(&mut bad, &sk, aad);   // test-only helper
        let _ = decrypt_bytes_v8(&bad, &key, &sk, aad);
    }
});
```

The helper that recomputes the tag is the essential part. Without it, the structural checks performed after authentication that were added in this version of the scheme, i.e. the check that the token count is a multiple, the bucket check and the check of the padded length prefix, which form step (3) of `Dec` and are the subject of `Remark 3.13`, have no fuzz coverage at all, although they are new code. The negative vectors `W-N06` to `W-N08` in the KAT test vectors are exactly such cases, which shows that the client already knows how to construct them. The harness should generate such inputs instead of relying on three fixed examples.

Add a separate target for the functions that do not authenticate their input, where a target fed with raw bytes *is* effective: `pub fn decrypt` takes a token slice without a tag check, and `decrypt_raw`, `decrypt_str` and `b128_decode_tokens` can all be reached with input shaped by an attacker in a way that the exits of the current target prevent. This is the target that would have found the aborts described in CVF-15.

Extend `key_for` so that it also returns keys that cause panics, not only well-formed keys. An empty tuple, `[0]`, `[1]`, a tuple with a repeated element and an element above `MAX_KEY_PRIME` each cause a panic or a silent overflow, as described in CVF-15 and CVF-16, and none of the five is in the fixture set. A fuzz target for `validate_key` would be cheap and would cover all of them.

Commit the fuzzing inputs and record the run. Reconcile the two contradictory comments, and commit a set of seed inputs that includes at least one valid ciphertext per KAT vector. Publish, together with the number of iterations, the target name, the commit, the total number of executions, the number of stored inputs and the percentage of code covered, so that the claim can be checked. Add `fuzz/target/` and `fuzz/artifacts/` to `.gitignore`, and stop ignoring `Cargo.lock`; see CVF-23, since a fuzzing result can be tied to a build only if the build is pinned.

Finally, correct the outdated references in the header together with CVF-21: the file is not a v6 target, and the decoder it names is not LEB128.

/.gitignore

```
/target
Cargo.lock
```

/fuzz/Cargo.toml

```
# Cargo-fuzz requires nightly.  Build with:
#   cargo +nightly fuzz run decode_bytes -- -max_total_time=300
# Corpus is stored in fuzz/corpus/decode_bytes/ (created on first run).
```

/fuzz/fuzz_targets/decode_bytes.rs

```
//! Cargo-fuzz target for NAPSEQ v6 decoder paths.
```

same file

```
//! Targets:
//!   * `decrypt_bytes` — full decode pipeline: auth check → LEB128 varint
//!     decode → plaintext recovery.  Must return `Ok(String)` or `Err(String)`;
//!     must never panic regardless of input.
//!   * Additional AAD variants to exercise the AAD-binding path.
```

same file

```
//! Corpus seeds are stored in `fuzz/corpus/decode_bytes/` and checked in
//! so that subsequent runs benefit from prior coverage discoveries.
```

same file

```
//! Security contract:
//!   `decrypt_bytes` MUST never panic.  Any malformed or adversarial input
//!   must produce `Err(...)`.  A successful `Ok(plaintext)` is only possible
//!   when the fuzzer discovers a valid NAPSEQ v6 ciphertext, which requires
//!   forging an HMAC-SHA256 tag — computationally infeasible.
```

same file

```
use napqes::decrypt_bytes;
```

same file

```
/// Key fixtures — three representative sizes selected by fuzzer byte[0].
fn key_for(selector: u8) -> Vec<u64> {
    match selector % 3 {
        0 => vec![1_000_003, 1_000_033],
        1 => vec![7_999_993],
```

```
        _ => vec![
            1_000_003, 1_000_033, 1_000_037, 1_000_039,
            1_000_081, 1_000_099, 1_000_117, 1_000_121,
            1_000_133, 1_000_151,
        ],
    }
}
```

same file

```
    if data.len() < 2 {
        return;
    }
```

same file

```
    let _ = decrypt_bytes(payload, &key, aad);
```

same file

```
    let _ = decrypt_bytes(payload, &key, b"");
```

same file

```
        let _ = decrypt_bytes(payload, &bad_key, aad);
```

/src/lib.rs

```
    if ciphertext.is_empty() {
        return Ok(String::new());
    }
```

same file

```
    if ciphertext.len() < NONCE_SIZE + TAG_SIZE {
```

same file

```
    if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
        return Err("Authentication failed: invalid HMAC tag.".into());
    }
```

CVF-23: open Category: **architecture**

Summary

The crate cannot be built as the encryption scheme alone, so its claimed certified-module boundary is undefined, and the exact build behind its test results cannot be reproduced.

The software presents itself as a cryptographic module of the kind that is certified under the US government standard FIPS 140-3. Such a module must have a clearly defined boundary, and everything inside it must be self-tested. Here, anyone who wants only the encryption scheme also receives a network runtime, a command-line parser, a logging client, a key-exchange algorithm and several protocol codecs, and the self-test covers none of the extra cryptography. The build settings also leave two safety checks switched off in the version that would actually be deployed. Finally, the exact versions of the third-party libraries are not recorded, so nobody can rebuild the software that the published test results describe.

`Cargo.toml` is now delivered. It is 42 lines long and contains a `[package]` section, a `[dependencies]` block of thirteen entries, one `[dev-dependencies]` entry, four `[[bin]]` targets and a `[lib]` section. It contains no `[features]` section and no `[profile]` section, and `cfg(feature = ...)` appears nowhere in `src/`, `examples/` or `fuzz/`. There is therefore no conditional compilation anywhere in the crate.

The first consequence concerns the module boundary. `lib.rs` declares six public modules unconditionally:

```
pub mod self_test;
pub mod kem;
pub mod kem_exchange;
pub mod ot_frame;
pub mod protocols;
pub mod vale;
```

and each of them brings its dependencies with it. `tokio` is declared with `features = ["full"]` and is used only by `kem_exchange` and the `ls_gateway` binary; `clap` and `syslog` are used only by `ls_gateway`; `hkdf`, `pqcrypto-frodo` and `pqcrypto-traits` are used only by `kem`. A downstream crate that needs only `encrypt_bytes_v8` and `decrypt_bytes_v8`, i.e. the scheme that the paper specifies, nevertheless compiles and links all of these, including Tokio's full feature set with its multi-threaded scheduler, process spawning, signal handling and filesystem APIs.

This matters more here than it would in an ordinary library, because `self_test.rs` opens by presenting the crate as a regulated artifact with a strict requirement before any operation:

> FIPS 140-3 power-on self-tests for the NAPQES Cryptographic Module.
>
> Call `run_power_on_self_tests()` before using the module in production.

A cryptographic module is defined by its boundary, and every approved algorithm in-

side that boundary has to be covered by the power-on tests. Neither condition is met. Nothing in the manifest or in the source separates the algorithm from the code around it, so the boundary is whatever the compiler happens to include: an AEAD, a FrodoKEM key exchange, an oblivious-transfer frame layer, two industrial protocol codecs in `protocols::dnp3` and `protocols::modbus`, and four binaries. At the same time, `run_power_on_self_tests()` runs three known-answer tests of the legacy v7 AEAD and one version-string comparison, and no other test (see CVF-18). As compiled, the module contains a post-quantum KEM with no self-test at all.

The second consequence comes from the missing `[profile]` section, and it answers a question that CVF-16 left open. Cargo's defaults for the release profile are `overflow-checks = false` and `debug-assertions = false`, and nothing in the manifest overrides them. In the configuration that would actually be deployed, therefore:

- the token multiplication `c as u64 * k + addend` wraps around silently instead of stopping the program when a key element large enough to overflow `u64` passes `validate_key`, which is the branch that CVF-16 identifies as the one that corrupts data; and
- the `debug_assert_eq!(a.len(), b.len())`, which is the only length precondition of `ct_eq_bytes`, is compiled out, leaving a tag comparator that returns `true` for a matching prefix when the two operands have different lengths (see CVF-12).

Both of the crate's arguments that a behaviour "is safe in debug builds" therefore hold only in a configuration that is never shipped. The `examples/dudect_harness.rs` harness illustrates this: its own instructions require `--release`, so the timing evidence for the tag comparator is collected in exactly the build where the comparator's length check is removed.

There are two smaller points in the same file. First, `subtle = "2"` is declared, but no code in the crate calls it. Cargo does not warn about an unused dependency, so nothing in a normal build reveals this, and a reader who checks the manifest against the paper's claim in Section 8.4 finds the claim apparently confirmed; this is the subject of CVF-12. Second, no `rust-version` is declared, so there is no minimum supported toolchain, even though the correctness argument of `ct_eq_bytes` is about what a particular LLVM version will not do to a volatile loop, and `fuzz/Cargo.toml` separately requires the nightly toolchain.

Finally, `.gitignore` has two lines:

```
/target
Cargo.lock
```

Because the lockfile is excluded, the thirteen caret-range requirements are resolved again on every checkout, so two builds are not guaranteed to use the same dependency versions. Cargo's own convention is to commit the lockfile for any package that produces binaries, and this package produces four. In practice, every empirical result that the paper reports (the KAT byte-for-byte agreement of Section 10, the fuzzing run of Section 11, the statistical study of Section 9 and the power-on self-test) is attributed to a build that cannot be reconstructed. It also means that the version of `subtle` that the claim in Section 8.4 refers to is not recorded, which would matter if that claim were ever made true.

Recommendation

Define the module boundary in the manifest, and make everything outside it optional.

The AEAD is what the paper specifies and what the self-test is meant to check, so it should be the default, and it should be usable without the rest of the crate. Put each of the other five modules behind a feature and mark its dependencies as optional:

```
[features]
default = []
kem      = ["hkdf", "pqcrypto-frodo", "pqcrypto-traits"]
net      = ["tokio", "serde", "serde_json"]
gateway  = ["net", "clap", "syslog"]

[dependencies]
hmac = "0.12"
sha2 = "0.10"
rand = "0.8"
base64 = "0.22"
subtle = "2"
tokio  = { version = "1", features = ["rt", "net", "io-util"],
  optional = true }
clap   = { version = "4", features = ["derive"], optional = true }
syslog = { version = "6", optional = true }
hkdf   = { version = "0.12", optional = true }
```

with `#[cfg(feature = "kem")] pub mod kem;` and similar lines in `lib.rs`, and `required-features` on the three binaries that need them. At the same time, narrow Tokio's feature list: `features = ["full"]` is a development convenience and enables far more than `kem_exchange` and `ls_gateway` use.

Then describe the boundary in the documentation as well. If the FIPS framing is kept, the module needs a named boundary, i.e. a list of the functions and files inside it, and the power-on suite needs to cover every approved algorithm inside that boundary, which it currently does not do for the KEM. If the FIPS framing describes a goal, say so where an operator will read it, in the header of `self_test.rs`, and not only in the comment above INT-1.

Add an explicit release profile instead of relying on Cargo's defaults, and turn overflow checking on:

```
[profile.release]
overflow-checks = true
debug-assertions = false
```

`overflow-checks = true` costs little for this workload and turns the silent wrap-around of CVF-16 into an abort. It does not replace the `checked_mul` that CVF-16 recommends, because a profile setting is not a precondition and a downstream crate can override it, but it removes the case where the arithmetic silently produces a wrong codepoint. Independently of this, replace the `debug_assert_eq!` in `ct_eq_bytes` with a real length check, since the correctness of a tag comparator should not depend on a profile setting (CVF-12).

Declare a minimum supported Rust version with `rust-version`, and pin the toolchain with a `rust-toolchain.toml` file, so that the constant-time argument

and the fuzzing results can be attributed to a specific compiler.

Commit `Cargo.lock`. Remove it from `.gitignore` and add the entries that belong there instead, `fuzz/target/` and `fuzz/artifacts/`, neither of which is covered by the root-anchored `/target`. Publish the lockfile's hash, or the commit, together with each empirical result that the paper reports, so that Sections 9 to 11 identify a build.

Finally, either use `subtle` or remove it, as CVF-12 recommends. A declared dependency that no code calls is worse than an absent one, because a reader of the manifest takes it as confirmation of the paper's claim.

/.gitignore

```
Cargo.lock
```

/Cargo.toml

```
[dependencies]
hmac = "0.12"
sha2 = "0.10"
base64 = "0.22"
rand = "0.8"
subtle = "2"
hkdf = "0.12"
pqcrypto-frodo = "0.4"
pqcrypto-traits = "0.3"
serde = { version = "1", features = ["derive"] }
serde_json = "1"
tokio = { version = "1", features = ["full"] }
clap = { version = "4", features = ["derive"] }
syslog = "6"
```

same file

```
subtle = "2"
```

same file

```
pqcrypto-frodo = "0.4"
pqcrypto-traits = "0.3"
```

same file

```
tokio = { version = "1", features = ["full"] }
```

same file

```
clap = { version = "4", features = ["derive"] }
```

same file

```
syslog = "6"
```

same file

```
[dev-dependencies]
dudect-bencher = "0.6"
```

same file

```
[lib]
name = "napqes"
path = "src/lib.rs"
```

/examples/dudect_harness.rs

```
//! Run (release mode required for meaningful timings):
//!   cargo run --example dudect_harness --release -- --continuous bench_tag_comparison
```

/src/lib.rs

```
pub mod self_test;
pub mod kem;
pub mod kem_exchange;
pub mod ot_frame;
pub mod protocols;
pub mod vale;
```

CVF-24: fixed Category: **documentation**

> **Summary**
>
> **The README describes only an old, small part of the library and an outdated data format, so developers who follow it will choose exactly the legacy functions that have known defects.**
>
> The README is the first, and often the only, document a developer reads before using the library. It describes five functions of an older version, out of about thirty-three public items, and does not mention the current encryption format that the paper specifies at all. It also describes an outdated layout of the encrypted data, and its only usage example does not compile. Every function it recommends is one of the legacy functions that other findings in this report show to be defective, and none of the newer, safer functions is mentioned. A developer who follows the README will therefore build on the weaker functions without knowing that better ones exist.

The scope file lists this file against the check "stated API surface vs. actual `pub fn` set". We carried out that check, and the gap between the two is large and goes in one direction.

The README's opening line is `Rust port of the v6 authenticated EpiCypher in ../napqes.py`. Both the version and the product name are outdated, in the same way that CVF-21 records for the module header of `lib.rs`: the file implements v7 and v8, and

"EpiCypher" appears nowhere in the paper, which calls the scheme NAPQES throughout. The referenced `../napqes.py` has not been delivered at any point in this engagement.

The documented API is:

```
use napqes::{encrypt_str, decrypt_str, generate_prime_numbers};
```

together with `MIN_KEY_PRIME` and `MAX_KEY_PRIME`, which are used in the example, and a "Scope" section that names `encrypt`/`decrypt`, `encrypt_bytes`/`decrypt_bytes`, `encrypt_str`/`decrypt_str` and prime generation. By contrast, `lib.rs` exports six public modules, ten public constants, one public enum with a public method, and sixteen public functions. The README does not mention any of the following:

- `generate_v8_key`, `encrypt_bytes_v8`, `encrypt_bytes_v8_with_profile` and `decrypt_bytes_v8`, i.e. the whole wire format that the paper specifies;
- `PadProfile`, `PadProfile::block_size`, `PAD_MIN_EXP` and `PAD_MAX_EXP`, the padding-profile mechanism on which Section 6 and the length-hiding results are built;
- `SK_SIZE`, `MAX_NOISE_RUN`, `FORMAT_BLOCK_V8` and `FORMAT_STREAM_AE_V8`;
- `encrypt_raw` and `decrypt_raw`, the byte-oriented pair;
- `zeroize_sk` and `zeroize_key`, the only functions in the crate that erase keys from memory;
- `is_prime`, `NONCE_SIZE` and `TAG_SIZE`;
- the six public modules `self_test`, `kem`, `kem_exchange`, `ot_frame`, `protocols` and `vale`, one of which is the FIPS power-on self-test that, according to the crate's own contract, a caller MUST invoke before any cryptographic operation.

The wire format that the README states is the retired one:

```
nonce(16) || varint_blob || hmac_sha256_tag(32)
```

The blob has had a fixed width since the change that the code calls the CVF1 fix, and `lib.rs` says so itself: the LEB128 encoder "is removed since nothing in this crate produces LEB128 ciphertexts anymore". The description also has the shape of v7 and cannot be adapted to v8, where the nonce is synthetic, i.e. computed from the inputs, and is not a supplied field. The accompanying claim that the format is "interoperable with Python ... when the same key, nonce, and AAD are used" therefore has no meaning for v8. The closing claim of the "Scope" section, that the named functions are "all byte-compatible with the Python reference", is the claim that CVF-25 examines. For the v7 functions, the test vectors that fix this compatibility were produced by the generator's own re-implementation, not by the public Python API.

The line "Out of scope: streaming API" appears next to a `pub const FORMAT_STREAM_AE_V8` and test vectors that include six `api: "stream_ae"` vectors, which is the unused-format observation that CVF-21 records.

The example does not compile. Its four lines contain three errors: `MIN_KEY_PRIME` and `MAX_KEY_PRIME` are used but are missing from the `use` declaration; `encrypt_str` returns `Result<String, String>` and its value is bound without `?` or `.unwrap()`; and `decrypt_str` then receives `&ct`, which is a `&Result<String, String>`, where

it expects `&str`. `lib.rs` does not include the README with `#![doc = include_str!("../README.md")]`, so nothing compiles this code block and no test would have detected the errors. The crate's only usage example has therefore never been run.

The main consequence is that the README leads developers to the wrong functions, which is why we do not consider this finding cosmetic. The functions that the README names are exactly the ones that CVF-13, CVF-14 and CVF-15 are about:

- CVF-13 reports that six public v7 entry points return an empty output for an empty input early, instead of producing a full authenticated ciphertext as the specification requires. `encrypt_str`, `decrypt_str`, `encrypt_bytes` and `decrypt_bytes` are four of those six, and they are four of the seven functions that the README names.
- CVF-14 reports that the crate implements both of the competing encodings of the message space. The README documents the codepoint-oriented `str` pair and does not mention the byte-oriented `raw` pair at all, so a reader cannot find out that there is a choice, let alone which option the paper specifies.
- CVF-15 reports that `validate_key` is called from exactly two places, `encrypt_bytes_v8_with_profile` and `decrypt_bytes_v8`, so no legacy entry point validates its key, and an empty key or a key element of 0 or 1 aborts the process. Neither of the two validating functions appears in the README.

Every defect that these findings identify therefore lies inside the part of the API that the README documents, and every entry point that avoids them lies in the part it omits. A developer who integrates this crate by following its README will choose the legacy-format API, with its divergent handling of empty input and its unvalidated keys, in every case, and will have no indication that an API resistant to misuse exists.

Recommendation

Rewrite the README around the format that the paper specifies, and mark the other format as legacy.

The v8 pair should be the first thing a reader sees, together with the key-generation call that produces both key components and the padding profile that controls how much the length of the ciphertext reveals:

```
use napqes::{generate_v8_key, encrypt_bytes_v8, decrypt_bytes_v8,
             zeroize_key, zeroize_sk, MIN_KEY_PRIME, MAX_KEY_PRIME};

let (mut primes, mut sk) = generate_v8_key(10, MIN_KEY_PRIME,
  MAX_KEY_PRIME);
let ct = encrypt_bytes_v8("hello", &primes, &sk, b"aad")?;
let pt = decrypt_bytes_v8(&ct, &primes, &sk, b"aad")?;
zeroize_sk(&mut sk);
zeroize_key(&mut primes);
```

Make sure that whichever example you choose compiles. Adding `#![doc = include_str!("../README.md")]` to `lib.rs` makes every code block in this file a doctest, so the three errors above become build failures and cannot come back. If that is not wanted, mark the code blocks `no_run`, but still make sure that they type-check. In either case, the current example needs the two missing imports and correct handling of the `Result` values.

Document the whole public API, or reduce the public API to what you want to document. With thirty-three public items and five described, the gap is best closed from both ends. Put behind features, or make private, what is not meant for users of the crate (the feature work recommended in CVF-23 does most of this), and list what remains, including the six public modules. State prominently that `self_test::run_power_on_self_tests()` must be called before any other operation.

Mark the v7 functions as deprecated, in addition to removing them from the documentation. This one change addresses the entry-point side of CVF-13, CVF-14 and CVF-15 at once, and it is cheaper than fixing each of them:

```
#[deprecated(since = "0.2.0", note = "legacy v7 format; use
  encrypt_bytes_v8")]
pub fn encrypt_bytes(message: &str, key: &[u64], aad: &[u8]) ->
  Result<Vec<u8>, String> { ... }
```

Apply it to `encrypt`, `decrypt`, `encrypt_bytes`, `decrypt_bytes`, `encrypt_str`, `decrypt_str`, `encrypt_raw` and `decrypt_raw`. A compiler warning wherever one of these functions is called reaches developers whom a README section does not. It does not replace the fixes that those three findings recommend, since the functions can still be called, but it stops new users from choosing them by default.

Correct the outdated text together with the changes for CVF-21, so that the two files do not have to be fixed twice: use the name NAPQES instead of EpiCypher, drop the v6 label, describe the blob as having a fixed width, describe the two wire formats separately and say which one the paper specifies, and qualify or remove the sentence about byte compatibility, which holds for v7 with a supplied nonce and has no meaning for v8. Reconcile "Out of scope: streaming API" with the public `FORMAT_STREAM_AE_V8`, with the six `stream_ae` test vectors and with the `CAV-001` caveat that CVF-21 raises.

Finally, replace the pointer to `../napqes.py` with a reference to the paper by version and date. The Python reference has not been delivered, and a README that cites an unavailable file gives a reader nothing to check.

/README.md

````
# napqes (Rust)

Rust port of the v6 authenticated EpiCypher in `../napqes.py`.
````

same file

````
## API

```rust
use napqes::{encrypt_str, decrypt_str, generate_prime_numbers};

let key = generate_prime_numbers(10, MIN_KEY_PRIME, MAX_KEY_PRIME);
let ct  = encrypt_str("hello", &key, b"");
let pt  = decrypt_str(&ct, &key, b"").unwrap();
```
````

same file

```
Wire format (interoperable with Python):

```
nonce(16) || varint_blob || hmac_sha256_tag(32)
```

String form is the base64 encoding of the above.
```

same file

```
## Scope

Implements: prime generation, `encrypt` / `decrypt`, `encrypt_bytes` /
`decrypt_bytes`, `encrypt_str` / `decrypt_str`, all HMAC derivations
(positions, addends, noise chars, noise probability, auth tag) — all
byte-compatible with the Python reference.
```

same file

```
Out of scope: streaming API, legacy v2/v3/v5 ciphertexts.
```

/src/lib.rs

```
pub mod self_test;
pub mod kem;
pub mod kem_exchange;
pub mod ot_frame;
pub mod protocols;
pub mod vale;
```

same file

```
pub fn encrypt_raw(data: &[u8], key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
```

same file

```
pub fn decrypt_raw(ciphertext: &[u8], key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
```

same file

```
pub fn generate_v8_key(count: usize, min_val: u64, max_val: u64) ->
(Vec<u64>, [u8; SK_SIZE]) {
```

same file

```
pub fn encrypt_bytes_v8(
```

same file

```
pub fn encrypt_bytes_v8_with_profile(
```

same file

```
pub fn decrypt_bytes_v8(
```

same file

```
1240 pub fn zeroize_sk(sk: &mut [u8; SK_SIZE]) {
```

same file

```
1255 pub fn zeroize_key(key: &mut [u64]) {
```

CVF-25: fixed Category: **procedural**

> **Summary**
> **The test-vector suite that is meant to prove all implementations produce identical output checks only plain English text and half of it tests a format outside the paper, so it cannot detect the known differences between implementations.**
> The paper claims that every correct implementation produces exactly the same encrypted output, and points to a shared set of test vectors as the proof. We found that the newer tests do check this properly, but only for messages in plain English characters. All the known differences between the implementations concern other characters, so these tests cannot reveal them. Half of the tests check an older format that the paper itself declares out of scope, and the one test case written to catch a known defect with empty messages is skipped by one test and checked the wrong way round by another, so it reports success. The suite also compares against a reference implementation that was not delivered for review.

Section 10 states that `rust/src/kat_cross_check.rs` uses `tests/kat/v8_vectors.json`, that "conforming implementations must produce byte-identical ciphertext for every positive vector and reject every negative vector", and that the second test-vector file, `tests/kat/v6_vectors.json`, "is retained in the repository for the earlier v7 wire format, which … is *not* the scheme analysed here; its vectors are outside the scope of this paper". Both test-vector files are now delivered.

We start with what holds. The three v8 tests are real tests of byte-for-byte agreement. `v8_positive_encrypt_matches_python` calls the public `encrypt_bytes_v8` and compares the full result with the stored `ciphertext_hex`; `v8_positive_decrypt_roundtrip` decrypts the stored bytes back to the stored message; and `v8_negative_returns_err` requires an `Err` for each tampered vector. All three end in `assert!(tested > 0, ...)`, so none of them can pass without testing anything; the defect that CVF-19 records is limited to the v7 tests. The test vectors are also arithmetically consistent with the delivered encryption code, which can be checked without running anything. Under `PadProfile::Bucket` the token ceiling is $(B+2)(\text{MAX_NOISE_RUN}+1)$ fields of `TOKEN_WIDTH = 8` bytes, plus 48 bytes for the nonce and the tag, and every positive vector matches it: 2,928 bytes for the messages of 0 to 15 codepoints, where $B = 16$; 5,488 bytes at 16 codepoints, where $B = 32$;

and 10,608 bytes at 32 codepoints, where $B = 64$. The v8 positive vectors were also generated through the public `napqes.encrypt_bytes_v8`, so for v8 the agreement is with the shipped Python API.

Four qualifications follow, and then the limits of the coverage.

First, the reference implementation was not delivered for review. Agreement is measured against `tests/gen_kats_v8.py`, which begins by adding the repository root to `sys.path` and importing a module `napqes`, and that module has not been delivered at any point in this engagement. We can confirm that the Rust implementation reproduces the bytes stored in the test vectors. We cannot confirm that those bytes are what the Python reference produces, because the Python reference is not available to us. The paper's accompanying sentence also overstates what the generator does: "The generator `tests/gen_kats_v8.py` regenerates the file and verifies parity on any port." Its `--check` mode runs the Python generator again and compares the result, as a string, with the JSON file in the repository. The file contains no code that involves another language: no subprocess call, and no reference to the Rust or C implementations. It verifies that the Python output can be reproduced from the Python module, which is a useful property, but it is not agreement with any other implementation.

Second, for v7 the reference is weaker still, since it is a re-implementation used only for testing. `gen_kats.py` does not call the public `napqes.encrypt_bytes`. It rebuilds encryption from private primitives in a local function `_encrypt_with_nonce`, and the stored v7 ciphertexts are the output of that function. The three v7 tests therefore compare the Rust implementation with the generator's re-implementation of the Python reference, not with the reference's own public entry point, and a difference between those two would be invisible to both sides.

Third, three of the six tests use the test vectors that the paper places outside its scope. `positive_decrypt_roundtrip`, `positive_encrypt_bytes_deterministic` and `negative_returns_err` all read `v6_vectors.json`, and the opening line of the test file describes it as the "Rust KAT cross-check for NAPSEQ v7". Most of the harness therefore tests a wire format that Section 10 explicitly excludes, and the paper's description of what `kat_cross_check.rs` uses applies to only three of its tests.

Fourth, the label of the v6 test-vector file contradicts its contents, and the generator acknowledges this. `spec_version` is `"v6"`, but every one of the 22 non-streaming positive vectors uses the fixed-width v7 encoding: for each of them, the ciphertext length minus 48 is an exact multiple of 8, which the retired LEB128 encoding would not produce. The docstring of `gen_kats.py` admits that block-mode vectors "use the v7 fixed-width token wire format", and that "The output file is still named `v6_vectors.json` for path stability … even though block-mode vectors are now v7", while the six `api: "stream_ae"` vectors in the same file still use LEB128. One file therefore contains two wire formats under a label that matches neither. This also answers the scope file's concern that the test vectors might be outdated: they were regenerated after the change that the code calls the CVF1 fix, so they are not. See CVF-18 for the same question about the test vector embedded in the self-test.

The empty-message vector is skipped by one test and checked the wrong way round by the other. The description of V001 in the test vectors is `Empty message produces authenticated ciphertext (not empty bytes)`, and the vector has a 16-byte

`nonce_hex` and a 1,776-byte `ciphertext_hex`. The deterministic-encryption test asserts the opposite:

```
// Empty-message vector: nonce_hex is "" and ciphertext_hex is ""
if message.is_empty() {
    let ct = encrypt_bytes_with_nonce(message, &[], [0u8;
  NONCE_SIZE], b"");
    assert!(ct.is_empty(), "[{}] empty message should produce empty
  ciphertext", id);
    eprintln!("[PASS] {} — empty message", id);
    tested += 1;
    continue;
}
```

The comment's premise is false for the delivered test vectors. The assertion is made with an empty key and an all-zero nonce, ignoring the vector's own key and nonce, so it says nothing about V001. Moreover, `encrypt_bytes_with_nonce` returns `Vec::new()` through its own early return for an empty message before it reads either argument, so the assertion holds for every possible argument, including the empty key that, as CVF-15 shows, would otherwise cause a division by zero. The test then reports `[PASS]` and increments `tested`. The other test, `positive_decrypt_roundtrip`, skips the same vector with `[SKIP] … empty message, no ciphertext`. The one vector that the test-vector file contains specifically to fix the behaviour that CVF-13 reports as divergent is therefore ignored by one test and confirmed in reverse by the other. The Python test suite checks the correct behaviour for the v8 format (`test_v8_empty_message_is_not_empty_ciphertext` asserts a 2,928-byte result), and the Rust `encrypt_bytes_v8` has no such early return, so the divergence affects only v7, which is consistent with the scope of CVF-13. This is a different defect from CVF-19: there, a guard fails to trigger when nothing was tested; here, an assertion triggers and is wrong.

What byte-for-byte agreement cannot cover by construction. Every message in the v8 test vectors is printable ASCII, with a maximum codepoint of U+0079. Across both test-vector files, exactly one character exceeds U+00FF, an em dash (U+2014) in V016, and no character exceeds U+FFFF. Consequently:

- The difference that CVF-3 reports, between the byte-oriented message space of the C reference and the specified codepoint space, cannot be detected at all, because the two encodings coincide below U+0080. All the test vectors come from the range where the disagreement is invisible.
- The two competing Rust encodings of CVF-14 agree on every vector in both files: the codepoint path in `decrypt_bytes` and the `(c & 0xFF) as u8` truncation in `decrypt_raw` return identical results for every message present.
- The `filter_map(char::from_u32)` in the v7 decryption path, which silently drops values that are not Unicode scalar values and returns a shorter plaintext with no error, cannot be reached by any vector.
- No vector comes close to the v8 length check `if codepoints.len() > 0xFFFF`; the longest message in either file is 1,000 codepoints.

The agreement result is therefore a statement about printable ASCII, while every encoding question raised in this review concerns characters above U+007F, where no vector goes.

Public entry points with no vector coverage. The six tests exercise `decrypt_bytes`, `encrypt_bytes_with_nonce`, `encrypt_bytes_v8` and `decrypt_bytes_v8`. None of the following is exercised:

- `encrypt_bytes`, the production path with a random nonce;
- `encrypt_str` and `decrypt_str`, the base64 wrappers that a caller is most likely to use, and the two that the README documents;
- `encrypt_raw` and `decrypt_raw`, the byte-oriented pair that CVF-14 is about;
- `encrypt_bytes_v8_with_profile` under any profile other than the default `Bucket`;
- `PadProfile::Frame`, the profile with no length leakage that Section 6 recommends and CVF-5 discusses.

The negative vectors are declared, not verified. In both generators, the functions that build negative vectors only construct dictionaries and never attempt a decryption, so nothing establishes that a `tampered_hex` value actually fails, or that it fails for the reason recorded in its `expected_exception` field. Three of the eight v8 negative vectors are created by monkey-patching, i.e. temporarily replacing at run time, the reference's padding function. On the Rust side, both negative tests check only `result.is_err()` and never which error is returned, so a vector intended to reach the bucket check after authentication, but in fact rejected at the tag check, passes in the same way. `W-N06` to `W-N08` exist specifically to reach the structural checks added at step (3) of `Dec` in v4, i.e. the new code, and no test confirms that they reach them.

The location of the test-vector files. Both loaders build the path by taking `CARGO_MANIFEST_DIR`, going up one level, and appending `tests/kat/…`, with the comment "repo root / rust/". The Python driver for the comparison across languages assumes the same layout: it finds the crate at `/ "rust"` and the test vectors at `/ "tests" / "kat"`. The two sides agree with each other, so the paths presumably work in the client's own repository. In the delivered snapshot, however, `Cargo.toml` and `tests/` are at the same level, and in that layout `CARGO_MANIFEST_DIR/../tests/kat/` does not exist, so `load_vectors()` panics with `Cannot read …`. We consider this a result of how the snapshot was packaged, not a defect, although we could not tell the two apart, since the enclosing repository was not delivered. Regardless of the layout, the lookup is fragile. The location is a hard-coded relative path behind `.parent().unwrap()`, with no fallback and no search; the test-vector files are outside the crate's package root, so `cargo package` cannot include them and `cargo test` on a published crate cannot find them; and a missing file causes a panic inside a helper shared by three tests, instead of a skip with a clear message.

Recommendation

Fix the empty-message inversion first, since it is the only item here that confirms a known divergence as correct.

Delete the special case and test the vector like every other positive vector, with its own key, nonce and expected ciphertext:

```
let got_ct = encrypt_bytes_with_nonce(message, &key, nonce, &aad);
assert_eq!(got_ct, expected_ct, "[{}] ciphertext differs from the
  Python KAT", id);
```

For V001, this test will fail against the delivered test vectors until the v7 early return for empty input is removed, and that is intended: the test should fail, and CVF-13 is the fix. If the divergence is to be kept instead, then V001 has to be removed from the test vectors, and the paper's claim that conforming implementations produce byte-identical output for every positive vector has to be qualified. The current arrangement, which keeps the vector and asserts its opposite, leaves both the test vectors and the harness reporting success while they contradict each other. At the same time, remove the corresponding `[SKIP]` in `positive_decrypt_roundtrip`, and with it the `skipped` counter that, as CVF-19 shows, disables that test's only guard.

Extend the test vectors to characters above U+007F. This one change would make the agreement claim meaningful, and it is cheap, since the generator already produces vectors from a list of messages. At minimum, add the following to the v8 test vectors:

- a message in the range U+0080 to U+00FF, where the codepoint and byte encodings first diverge and CVF-3 and CVF-14 become observable;
- a message in the Basic Multilingual Plane above U+00FF;
- a message above U+FFFF, which is where an implementation based on UTF-16 would break;
- on the decryption side, a lone surrogate or an out-of-range value, to reach the `filter_map(char::from_u32)` path that currently drops values silently;
- a message at the `0xFFFF` codepoint boundary.

Until such vectors exist, state in Section 10 that the test vectors contain only ASCII and that agreement between implementations is claimed only for that range, since a reader currently has no way to know it.

Cover the untested public functions. Add v8 vectors under `PadProfile::Frame` as well as `Bucket`, since `Frame` is the profile that the caveats section recommends and it has no coverage at all, and add round-trip tests through `encrypt_str`/`decrypt_str` and `encrypt_raw`/`decrypt_raw`. The `raw` pair matters most: its message space is the one that differs, and nothing exercises it at present.

Make the negative vectors verify what they claim. Have both generators attempt to decrypt each `tampered_hex` and assert that the decryption fails, with a message that matches `expected_exception`, so that the test vectors record a checked property instead of an intention. On the Rust side, check the error text in addition to `is_err()`:

```
let err = decrypt_bytes_v8(&ct, &key, &sk, &aad).unwrap_err();
let want = vec["expected_exception"].as_str().unwrap();
assert!(err.contains(want), "[{}] rejected for the wrong reason:
  {}", id, err);
```

Without this, `W-N06` to `W-N08` cannot distinguish rejection by the new structural checks from rejection by the tag check, which defeats the purpose they were written for.

Point the harness at the format that the paper specifies. Section 10 excludes the v6 test vectors, so most of the tests in this file test something that the paper does not

analyse. Either move the three v7 tests into a clearly named `legacy_v7` module, so that a reader can see which results concern the paper, or retire them together with the v7 format. In either case, correct the file's opening line: it describes the file as a v7 cross-check, while the paper's claim is about v8.

Relabel the v6 test-vector file. A `spec_version: "v6"` label on a file whose block vectors are v7 and whose streaming vectors use LEB128 is the same kind of trap that the scope file warned about for the self-test constant. Split the two formats into separate files with accurate `spec_version` fields, or add a `wire_format` field to each vector; keeping the path unchanged is not worth a mislabelled cryptographic test-vector file.

Make the location of the test vectors robust and the tests self-contained. Find the test vectors by searching upwards from `CARGO_MANIFEST_DIR` for a `tests/kat/` directory, or, better, move the files inside the crate and embed them with `include_str!`. This removes the dependency on the file system, makes the vectors part of what `cargo package` ships, and turns a missing file into a compile error instead of a run-time panic. In either case, replace `.parent().unwrap()` with proper error handling.

Finally, correct Section 10's description of the generator: `--check` verifies that the Python output can be reproduced, not that any other implementation agrees with it. If a real check of agreement between implementations is wanted, the Python side needs to run the other implementations and compare the bytes itself. At present, `test_cross_lang.py` reads a pass count reported by the Rust harness on its standard output, behind an `if` that makes the assertion optional, so the byte comparison happens inside the very code under test. Since the Python reference module itself has not been delivered, submitting it would allow the agreement claim to be checked instead of being taken on trust.

/src/kat_cross_check.rs

```
1  //! Rust KAT cross-check for NAPSEQ v7 (Phase 0, workstream 0.1 /
   ROADMAP §2.B8).
```

same file

```
3  //! Reads `tests/kat/v6_vectors.json` from the repo root, exercises
   both the
4  //! deterministic `encrypt_bytes_with_nonce` API (exact byte
   comparison) and
5  //! the `decrypt_bytes` API for positive vectors, and asserts that
   negative
6  //! vectors return an `Err`.
```

same file

```
26 fn load_vectors() -> Vec<Value> {
27     // Path is relative to the Cargo workspace root (repo root /
   rust/).
28     let manifest_dir = env!("CARGO_MANIFEST_DIR");
29     let path = Path::new(manifest_dir)
30         .parent()              // repo root
```

```
        .unwrap()
        .join("tests/kat/v6_vectors.json");
```

same file

```
    for vec in vectors
        .iter()
        .filter(|v| v["kind"] == "positive" && v["api"] !=
"stream_ae")
```

same file

```
        if message.is_empty() {
            skipped += 1;
           eprintln!("[SKIP] {} — empty message, no ciphertext", id);
            continue;
        }
```

same file

```
    for vec in vectors
        .iter()
        .filter(|v| v["kind"] == "positive" && v["api"] !=
"stream_ae")
```

same file

```
        // Empty-message vector: nonce_hex is "" and ciphertext_hex
is ""
        if message.is_empty() {
            let ct = encrypt_bytes_with_nonce(message, &[], [0u8;
NONCE_SIZE], b"");
            assert!(ct.is_empty(), "[{}] empty message should produce
empty ciphertext", id);
            eprintln!("[PASS] {} — empty message", id);
            tested += 1;
            continue;
```

same file

```
        if nonce_hex.is_empty() || ct_hex.is_empty() {
            eprintln!("[SKIP] {} — missing nonce_hex or
ciphertext_hex", id);
            continue;
        }
```

same file

```
    for vec in vectors
        .iter()
        .filter(|v| v["kind"] == "negative" && v["api"] !=
"stream_ae")
```

same file

```
        assert!(
            result.is_err(),
            "[{}] expected Err but got Ok({:?})",
            id,
            result.unwrap()
        );
```

same file

```
fn load_v8_vectors() -> Vec<Value> {
    let manifest_dir = env!("CARGO_MANIFEST_DIR");
    let path = Path::new(manifest_dir)
        .parent()
        .unwrap()
        .join("tests/kat/v8_vectors.json");
```

same file

```
        let got_ct = encrypt_bytes_v8(message, &key, &sk, &aad)
```

same file

```
        assert!(
            got_ct == expected_ct,
            "[{}] v8 ciphertext differs from the Python KAT (first differing byte at {:?})",
            id,
            got_ct.iter().zip(expected_ct.iter()).position(|(a, b)| a != b)
        );
```

same file

```
        let result = decrypt_bytes_v8(&ct, &key, &sk, &aad);
        assert!(
            result.is_err(),
            "[{}] expected Err but got Ok({:?})",
            id,
            result.unwrap()
        );
```

/src/lib.rs

```
    if message.is_empty() {
        return Vec::new();
    }
```

same file

```
    let s: String = codepoints.into_iter().filter_map(char::from_u32).collect();
```

same file

```
886 pub fn encrypt_raw(data: &[u8], key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
```

same file

```
933 pub fn decrypt_raw(ciphertext: &[u8], key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
```

same file

```
1145     let codepoints: Vec<u32> = message.chars().map(|c| c as
u32).collect();
1146     if codepoints.len() > 0xFFFF {
1147         return Err("Message too long for 2-byte length
prefix.".into());
```

/tests/kat/v6_vectors.json

```
2   "spec_version": "v6",
```

same file

```
5       "id": "V001",
6       "kind": "positive",
7       "description": "Empty message produces authenticated ciphertext
(not empty bytes)",
```

same file

```
20      "id": "V002",
```

/tests/kat/v8_vectors.json

```
2   "spec_version": "v8",
```

same file

```
5       "id": "W001",
6       "kind": "positive",
7       "description": "Empty message produces a full authenticated
ciphertext",
```

same file

```
269     "id": "W-N06",
```

same file

```
284     "id": "W-N07",
```

same file

```
299     "id": "W-N08",
```

CVF-27: open Category: **algorithm**

Summary

The specified decryption algorithm does not handle every input a key holder can produce, so an implementation written from the paper alone can crash or return wrong output, while the Rust code silently adds two checks that the paper lacks.

The paper explains that two checks in the decryption algorithm make every later step well defined, even for malformed messages. This is not quite true: someone who holds the key can create a correctly authenticated message whose content still makes a later calculation fail. The Rust code handles this case with two extra checks that the paper does not mention, but an implementation written only from the paper would not include them and could crash or produce a wrong result. Outsiders without the key cannot trigger this, so there is no security impact; the problem is correctness and agreement between implementations.

(5) if $T \neq T'$ (compared in constant time), output $\perp$; (6) if $R \notin \{B + 2 : B \in \{2^4, \ldots, 2^{16}\}\}$, output $\perp$ (Remark 3.13); (7) else unmask $B = \widetilde{B} \oplus \mathrm{ks}(N)$, deserialise the N_{tok} fixed-width tokens, and invert the token-emission loop using the same (sk, N)-derived noise-position oracle: skip at most MAX_NOISE_RUN *consecutive noise positions, then read one real token t and recover the padded codepoint as $(t - a)/k_{(\text{real_idx mod } K)+1}$, where a is the* real_idx*-th addend* $\mathrm{Derive}_{0x01}(\mathsf{sk}, N, \text{real_idx})$ *and* real_idx *counts real tokens from 0 — exactly the prime and addend* Enc *used at that index; stop once R real tokens have been extracted, discarding any*

Reference PDF p. 11 — `Dec` *step (7) recovers each padded codepoint as $(t - a)/k_{(\text{real_idx mod } K)+1}$, with no guard requiring $t \geq a$ and no divisibility condition on $t - a$.*

addend Enc *used at that index; stop once R real tokens have been extracted, discarding any trailing filler; (8) read the 2-codepoint big-endian length prefix n and, if $n > R - 2$, output $\perp$ (Remark 3.13); (9) output the n codepoints following the prefix.*

Remark 3.13 (The post-authentication structural checks)**.** *Steps (6) and (8) are performed* after *the tag comparison of step (5), so they are never reachable on an input an adversary without* sk *could construct, and they play no part in the INT-CTXT argument. They are specified nonetheless because step (7) is not total without them: R is recovered from $|C|$ and n from the decrypted buffer, so a malformed but correctly tagged ciphertext — which a key holder can trivially produce — would otherwise drive an out-of-range read or an allocation sized by a field the caller did not choose. Divisibility by* MAX_NOISE_RUN $+ 1$ *alone does not imply step (6): $R = 20$ passes step (3) and fails step (6). All three reference implementations enforce both checks, and vectors* `W-N06`–`W-N08` *of Section 10 pin them.*

Reference PDF p. 11 — `Remark 3.13` *justifies the two structural checks by asserting that step (7) "is not total without them". Steps (6) and (8) bound only R and the length prefix n; neither constrains a token value, so the totality claim is false.*

Remark 3.13 exists to justify the two structural checks, and it justifies them by an argument about totality, i.e. that step (7) is defined for every input. According to the remark, steps (6) and (8) "are specified nonetheless because step (7) is not total without them: R is recovered from $|C|$ and n from the decrypted buffer, so a malformed but correctly tagged ciphertext — which a key holder can trivially produce — would otherwise drive an out-of-range read or an allocation sized by a field the caller did not choose."

The premise is correct, but the conclusion is too weak, because steps (6) and (8) are **not sufficient**. Step (6) constrains R, which is derived from $|C|$; step (8) constrains n, which is read from the recovered prefix. Neither of them constrains the **token values**. Step (7) of Dec only specifies that the padded codepoint is recovered as $(t-a)/k_{(real_idx \bmod K)+1}$. A key holder, i.e. exactly the adversary that Remark 3.13 refers to and that Remark 3.15 describes, can produce a correctly tagged ciphertext with $|C| = 2928$, so that $R = 18$ passes step (6), and whose masked blob unmasks to arbitrary 64-bit tokens. Three failure modes then pass both structural checks:

- $t < a$. The specification computes $t - a$ over the natural numbers, where it is undefined, and a 64-bit implementation underflows.
- $(t - a) \bmod k \neq 0$. The quotient is not an integer, and the specification does not say what to do.
- The quotient exceeds 0x10FFFF or falls into the surrogate range, i.e. outside $\mathcal{M}$, and step (9) is then asked to output a value that is not a codepoint.

Step (8) cannot help with any of these: it is specified *after* step (7) and constrains only the slice taken in step (9).

The developers of the implementation found this gap, but the paper does not record it. decrypt_core_v8 performs two rejections that do not appear anywhere in Definition 3.10, and the comment above the first one states the reason explicitly: "the bare subtraction panics here in debug builds and wraps in release builds and in C":

```
if token < addend || (token - addend) % k != 0 {
    return Err(/* not of the form codepoint * k + addend */);
}
let cp = (token - addend) / k;
if cp > 0x10FFFF || char::from_u32(cp as u32).is_none() {
    return Err(/* not a Unicode scalar value */);
}
```

The first rejection handles the first two failure modes. The second rejection handles the third one, and it is stricter than Definition 3.10, which says that the arithmetic layer "is indifferent to which subset of $[0, 2^{21})$" the values come from.

There is **no forgery consequence**. All three conditions arise after authentication and cannot be reached without sk, so Theorem 4.18 is not affected and no outside adversary can trigger any of them. The consequences concern correctness and interoperability:

- A conforming implementation written from the paper alone contains the failure. In a Rust debug build, the bare subtraction panics. In a Rust release build without overflow-checks, which is the delivered configuration since Cargo.toml declares no [profile.release] section, it wraps around silently, and in C it also wraps

around silently. The paper as written therefore specifies a decryption routine that a key holder can crash or lead to a wrong answer, which is exactly the class of problem that Remark 3.13 was written to rule out.

- Three implementations of the same specification would disagree on this class of input: the Rust implementation returns an error, an implementation that uses floor division returns a plaintext, and an implementation that follows the specification's literal subtraction underflows. Section 10 requires byte-identical behaviour on test vectors that include no vector of this class, so nothing in the delivered evidence would detect the difference.

Proposition 3.14 is not affected, because correctness is stated only for honest outputs of Enc. The defect is in the totality claim, not in the correctness proof.

This finding is separate from the problem with the addend notation recorded in CVF-7, which concerns how step (7) *writes* the addend and not whether step (7) is *total*. It is also separate from the overflow caused by the missing upper bound in CVF-16, which is a multiplication on the encryption side and not a subtraction on the decryption side.

Recommendation

Extend step (7) of Definition 3.10 with the two rejections that the implementations already perform, and correct Remark 3.13 so that its totality claim matches.

In step (7), before the division, specify: if $t < a$ or $(t-a) \not\equiv 0 \pmod{k_{(real_idx \bmod K)+1}}$, output $\perp$. State this as a precondition of the division, not as prose, so that a reader implementing from the paper cannot reach the subtraction without the check. This is the same technique that Definition 3.4 uses for the tag input, and the same one recommended for the addend in CVF-7: state the constraint once, in the algorithm, so that it cannot be lost when the algorithm is translated into code.

Decide explicitly what to do about the third check. The Rust implementation rejects a recovered value that is not a Unicode scalar value, which is stricter than the message-space paragraph of Definition 3.10 allows. Either add that rejection to step (7) and delete the sentence "indifferent to which subset of $[0, 2^{21})$", or remove the check from the implementations. It should not remain in the code while being absent from the specification, because another implementation that needs to interoperate can observe this rejection.

Then rewrite the justification in Remark 3.13. As written, it says that steps (6) and (8) are what make step (7) total, which is not the case. The correct statement is that the check of the token form in step (7) itself makes the arithmetic total, while steps (6) and (8) bound the two length fields; all of these checks take place after authentication, so none of them affects the integrity argument. It is enough to keep the structure of the remark and correct only the claim, since the remark is otherwise the right place for this material.

Finally, add a negative known-answer vector for this class: a correctly tagged ciphertext of 2,928 bytes, produced by a key holder, whose blob unmasks to a token with $t < a$ at a known position. Of the structural negative vectors, this is the one that the delivered test vectors (W-N06 to W-N08) do not cover, and without it an implementation that omits the check passes the whole suite. Note that the vector must be generated with the key, since this class cannot be reached without sk; the existing "re-tagged" negative vectors already use the same construction.

/Cargo.toml

```
[package]
name = "napqes"
version = "0.1.0"
edition = "2021"
```

/docs/napseq-eprint-v4.tex

```
       exactly the set of strings that admit a UTF-8 encoding. Nothing
        weaker is needed and nothing stronger is assumed: the
arithmetic
        layer itself operates on the integer codepoint values and is
        indifferent to which subset of $[0,2^{21})$ they range over.
  \item Ciphertext space $\mathcal{C}=\{0,1\}^{*}$: every output of
```

same file

```
(Remark~\ref{rem:dec-structural});
(7) else unmask $B=\widetilde{B}\oplus\mathrm{ks}(N)$, deserialise
the
$N_{\mathrm{tok}}$ fixed-width tokens, and invert the token-emission
loop
using the same $(\mathsf{sk},N)$-derived noise-position oracle: skip
at most
$\mathrm{MAX\_NOISE\_RUN}$ consecutive noise positions, then read one
real
token $t$ and recover the padded codepoint as
$(t-a)/k_{(\mathrm{real\_idx}\bmod K)+1}$, where $a$ is the
$\mathrm{real\_idx}$-th addend
$\mathrm{Derive}_{0x01}(\mathsf{sk},N,\mathrm{real\_idx})$ and
$\mathrm{real\_idx}$ counts real tokens from $0$ — exactly the prime
and
addend $\mathsf{Enc}$ used at that index; stop once $R$ real tokens
have
```

same file

```
\begin{remark}[The post-authentication structural checks]
\label{rem:dec-structural}
Steps (6) and (8) are performed \emph{after} the tag comparison of
step (5),
so they are never reachable on an input an adversary without
$\mathsf{sk}$
could construct, and they play no part in the INT-CTXT argument. They
are
specified nonetheless because step (7) is not total without them: $R$
is
recovered from $|C|$ and $n$ from the decrypted buffer, so a
malformed but
```

```
correctly tagged ciphertext — which a key holder can trivially
produce —
would otherwise drive an out-of-range read or an allocation sized by
a field
the caller did not choose. Divisibility by
$\mathrm{MAX\_NOISE\_RUN}+1$
alone does not imply step (6): $R=20$ passes step (3) and fails step
(6).
All three reference implementations enforce both checks, and vectors
\texttt{W-N06}--\texttt{W-N08} of Section~\ref{sec:kats} pin them.
\end{remark}
```

/src/lib.rs

```
        // A genuine real token is exactly c * k + addend with addend
in
        // [1, k - 1]. Checking that explicitly, rather than
subtracting and
        // dividing, keeps the three ports in lock-step: the bare
subtraction
        // panics here in debug builds and wraps in release builds and
in C.
```

same file

```
        if token < addend || (token - addend) % k != 0 {
            return Err(format!(
                "Malformed v8 ciphertext: token at position {} is not
of the form \
                 codepoint * k + addend for key element {}.",
                ct_pos, k
            ));
        }
```

same file

```
        let cp = (token - addend) / k;
```

same file

```
        if cp > 0x10FFFF || char::from_u32(cp as u32).is_none() {
            return Err(format!(
                "Malformed v8 ciphertext: recovered value {} at
position {} is not a \
                 Unicode scalar value.",
                cp, ct_pos
            ));
        }
```

CVF-28: open Category: **architecture**

Summary

The legacy encryption functions, which are still public and used by the crate's own network layer, use the weak key design that the current version was built to replace, and the paper describes them inaccurately.

The library still offers the older (v7) encryption functions to every user, without any deprecation warning. These functions use the secret key in the way that the library's own comments describe as the root of three earlier security problems, and none of the paper's security proofs applies to them. They also record the length of the associated data in four bytes where the specification requires eight, and the paper describes them as a test-only format that takes its random value from the caller, while in fact they are the main production interface. The crate's own network transport uses these functions, so real traffic is protected by a design that the paper does not cover.

This finding reports three differences in the same group of functions, together, because one decision resolves all three.

The key derivation is the one that was discarded. Every v7 derivation uses as its key `kb = key_bytes(key)`, i.e. the serialisation of the prime tuple with five bytes per element, in seven functions: `encrypt`, `decrypt`, `encrypt_bytes`, `encrypt_bytes_with_nonce`, `decrypt_bytes`, `encrypt_raw` and `decrypt_raw`. This is exactly the structured HMAC key with low min-entropy that `Remark 3.3` and `Assumption 4.2` were rewritten to eliminate, and the crate says so itself. The module comment that introduces the v8 key derivation lists three earlier findings that "trace back to that one design choice":

- a repeated nonce under this key allows solving $k = (t_1 - t_2)/(c_1 - c_2)$ from a single known plaintext codepoint, i.e. "catastrophic key recovery, not merely a confidentiality loss";
- `key_bytes(primes)` carries only $H_\infty(\mathbf{k})$ bits of min-entropy and is therefore "a structured, non-uniform HMAC key, which is a different (and non-standard) hypothesis from the textbook uniform-key HMAC-SHA256 PRF assumption every theorem bound otherwise cites";
- the PRF step in the reductions is vacuous, i.e. it proves nothing, because the reduction must already know the primes that make up the supposedly hidden key of the PRF.

All of this describes code that is still exported. `Assumption 4.2` is stated for a uniform 256-bit key and does not apply to `key_bytes(primes)`, and `Remark 3.3` states explicitly that "sk is the *only* secret that ever keys an HMAC call in this construction", which is true for v8 and false for v7. None of `Theorem 4.8`, `Theorem 4.12`, `Theorem 4.18` or `Theorem 4.20` covers a v7 ciphertext. Nothing in the crate, the README or the paper says so: the v7 functions have no `#[deprecated]` attribute, and their doc comments contain no warning about the key derivation.

This is separate from the two v7 findings already reported. CVF-17 concerns the emission loop, which lacks the cap on noise runs and the token ceiling, and CVF-22 concerns the

fuzzing harness, which targets the v7 decoder. Neither of them states that the v7 *key derivation* is the one that the v8 redesign was built to replace.

The length prefix of the associated data has 4 bytes, not 8. `Definition 3.4` introduces the length prefix for one stated purpose, so that "encryptor and decryptor cannot diverge in the width of the length prefix", and fixes it as be8$(|A|)$. The v8 path follows this with `AAD_LEN_WIDTH_V8 = 8`, while the v7 path uses `AAD_LEN_WIDTH_V7 = 4` in all five places where it computes or checks a tag. v7 is internally consistent, so its encryptor and decryptor do not actually diverge, and v7 is not the specified scheme. However, the helper that writes the prefix, `be_len_prefix(n, width)`, takes the **low** `width` bytes without a range check, so with width 4 it silently encodes $|A|$ mod 2^{32}. Two associated-data values whose lengths differ by a multiple of 2^{32} therefore produce identical tag inputs. This requires associated data of 4 GiB, so there is no practical consequence, but it is a silent truncation in a case where an error is the correct behaviour, and it occurs in the very construct that `Definition 3.4` introduced to make the length encoding unambiguous.

Section 10 wrongly describes v7 as taking a nonce from the caller. The paper says that `v6_vectors.json` "is retained in the repository for the earlier v7 wire format, which takes an externally supplied nonce and is *not* the scheme analysed here". In the delivered crate, the only function that takes a nonce is `encrypt_bytes_with_nonce`, which is `pub(crate)` and, according to its own doc comment, "reachable only from the self-test module and the in-crate KAT cross-check, never from external consumers of this crate". Every *public* v7 entry point generates its own nonce internally: `encrypt` draws one directly from `rand::thread_rng()`, and `encrypt_bytes` and `encrypt_raw` call `generate_nonce_with_crng_check`. The sentence therefore describes v7 as a leftover test format driven by the caller, while it is in fact the crate's most prominent production API, the one that `encrypt_str`, `encrypt_bytes` and `encrypt_raw` offer to every user. This affects how a reader relates the paper's security claims to the delivered code: a reader who is told that v7 takes an external nonce concludes that it is part of the test harness and looks no further.

Reach, shown with code outside the reviewed scope. We mention this only to show the impact; the cause of all three items lies in `src/lib.rs`. The crate's own `src/ot_frame.rs` seals and opens its protocol data units with `encrypt_raw` and `decrypt_raw`, and the DNP3 and Modbus protocol modules and the `ls_gateway` binary are built on top of it. The delivered transport path therefore uses the v7 key derivation from end to end, so the shipped gateway for industrial protocols carries traffic that no theorem in the paper covers.

Recommendation

Decide whether v7 is a supported format or only a compatibility layer for reading archived data, and make the code and the paper say the same thing.

The simplest solution is to withdraw the v7 encryption functions. Nothing in the specification requires v7 to exist: the paper specifies only v8, Section 8.2 describes the Rust implementation purely in terms of `generate_v8_key`, `encrypt_bytes_v8` and `decrypt_bytes_v8`, and the crate's own module comment gives three reasons why the v7 key derivation was replaced. Keeping `decrypt_bytes` and `decrypt_raw` for reading archived ciphertexts, while removing `encrypt`, `encrypt_bytes`, `encrypt_`

`str` and `encrypt_raw` from the public API, resolves this finding without requiring any migration of stored data, and it resolves CVF-17 at the same time. It also requires moving `ot_frame::wrap_pdu` to the v8 entry points. This is the main part of the work, and it is worth doing for its own sake, since the transport layer is the one user of the crate for which the theorems ought to hold.

If v7 must remain public, three things need to be stated, in the code and in the paper:

- Mark every v7 entry point `#[deprecated]`, and state in each doc comment that the format uses `key_bytes(primes)` as the key of its derivations instead of sk, that this is the key derivation described in the module comment's discussion of CVF3, CVF8 and CVF13, and that, as a result, none of the paper's security theorems applies to a v7 ciphertext. At present, a caller must read the rationale for v8 and then notice, from what is missing, that v7 was left unchanged.
- Make `be_len_prefix` return an error, instead of truncating, when `n` does not fit in `width` bytes. The v8 path is not affected, since $|A| < 2^{64}$ always fits in eight bytes; the change only removes a silent wrap-around on the v7 path. If the four-byte v7 prefix is kept for compatibility with the wire format, add the explicit range check next to it.
- Correct the description of `v6_vectors.json` in Section 10. Replace "which takes an externally supplied nonce" with an accurate statement: in every public entry point, the v7 format draws a fresh nonce from a cryptographically secure random number generator for each message, uses the serialised prime tuple instead of sk as the key of its derivations, and is not covered by any result in Section 4. The current wording understates what is still shipped.

  In either case, add a sentence to Section 8.2 stating that the Rust crate exports a second, unspecified wire format next to the one this paper defines, so that a reader does not apply the bounds of Section 4 to it.

/docs/napseq-eprint-v4.tex

```
312 $\mathsf{sk}$ is the \emph{only} secret that ever keys an HMAC call
    in this
313 construction (Section~\ref{sec:domains}); $\mathbf{k}$ is used
    exclusively,
314 directly, and un-keyed by the arithmetic token map $c \mapsto c\cdot
    k_j+a$
315 (Section~\ref{sec:token-construction}). Consequently, none of the
    security
316 theorems of Section~\ref{sec:security} depend in any way on
317 $H_\infty(\mathbf{k})$: the PRF assumption below is stated, and used,
318 against the standard \emph{uniform}-key distribution of $\mathsf{sk}$
    only.
```

same file

```
356 \begin{definition}[Tag input]\label{def:taginput}
357 The authentication tag is always computed over the single quantity
358 \[
```

```
  \mathrm{TagIn}(A, \widetilde{B}) \;=\; \mathrm{be8}(|A|) \,\|\, A
\,\|\, \widetilde{B},
\]
so that the complete domain-$\texttt{0x03}$ HMAC input is
$\texttt{0x03}\|N\|\mathrm{TagIn}(A,\widetilde{B})$. Every occurrence
of the
tag in this paper — Table~\ref{tab:domains}, the wire format
(Section~\ref{sec:wire-format}), both $\mathsf{Enc}$ and
$\mathsf{Dec}$
(Definition~\ref{def:aead-triple}), and the integrity proof
(Theorem~\ref{thm:int-ctxt}) — denotes this one quantity, so
encryptor and
decryptor cannot diverge in the width of the length prefix.
\end{definition}
```

same file

```
\begin{assumption}[Uniform-key HMAC-SHA256 PRF]\label{assum:hmac-prf}
$\mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC\text{-
}SHA256}}(\mathcal{B})$
is negligible in the security parameter for every PPT $\mathcal{B}$,
for $\kappa$ drawn uniformly from $\{0,1\}^{256}$ (matching how
$\mathsf{sk}$ is sampled, Definition~\ref{def:keygen}).
\end{assumption}
```

same file

```
\subsection{Rust Implementation}

A Rust port (\texttt{rust/src/lib.rs}) mirrors the Python
implementation
(\texttt{generate\_v8\_key}, \texttt{encrypt\_bytes\_v8},
\texttt{decrypt\_bytes\_v8}). Unit tests cover round-trip
correctness,
tamper/AAD-mismatch rejection, same-message determinism (the
disclosed
MRAE trade-off), distinct-message nonce distinctness, and
ciphertext-length determinism across varied AAD and keys
(Lemma~\ref{lem:length-det}).
```

same file

```
A second corpus, \texttt{tests/kat/v6\_vectors.json} (37 vectors), is
retained in the repository for the earlier v7 wire format, which
takes an
externally supplied nonce and is \emph{not} the scheme analysed here;
its
vectors are outside the scope of this paper and none of the results of
Section~\ref{sec:security} are asserted of them.
```

/src/lib.rs

```
43 const AAD_LEN_WIDTH_V7: usize = 4;
```

same file

```
127 fn key_bytes(key: &[u64]) -> Vec<u8> {
128     let mut out = Vec::with_capacity(key.len() * 5);
129     for &k in key {
130         let b = k.to_be_bytes(); // 8 bytes
131         out.extend_from_slice(&b[3..8]); // low 5 bytes, big-endian
132     }
133     out
134 }
```

same file

```
159 fn be_len_prefix(n: usize, width: usize) -> Vec<u8> {
160     (n as u64).to_be_bytes()[8 - width..].to_vec()
161 }
```

same file

```
467 pub fn encrypt(message: &[u32], key: &[u64]) -> ([u8; NONCE_SIZE],
Vec<u64>) {
468     let mut nonce = [0u8; NONCE_SIZE];
469     rand::thread_rng().fill_bytes(&mut nonce);
```

same file

```
470     let kb = key_bytes(key);
```

same file

```
524     let kb = key_bytes(key);
```

same file

```
736     let kb = key_bytes(key);
```

same file

```
767     let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
```

same file

```
785 pub(crate) fn encrypt_bytes_with_nonce(
786     message: &str,
787     key: &[u64],
788     nonce: [u8; NONCE_SIZE],
789     aad: &[u8],
790 ) -> Vec<u8> {
```

same file

```
794     let kb = key_bytes(key);
```

same file

```
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
```

same file

```
    let kb = key_bytes(key);
```

same file

```
    let calc_tag = compute_auth_tag(&kb, aad, payload,
AAD_LEN_WIDTH_V7);
```

same file

```
    let kb = key_bytes(key);
```

same file

```
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
```

same file

```
    let kb = key_bytes(key);
```

same file

```
    let calc_tag = compute_auth_tag(&kb, aad, payload,
AAD_LEN_WIDTH_V7);
```

same file

```
// ─── V8 key schedule + synthetic nonce (CVF3 / CVF8 / CVF13 fix)
──────────
//
// v7 (and every earlier wire format) keys *every* domain derivation
with
// `kb = key_bytes(primes)` — the serialisation of the same prime
tuple used
// for the token arithmetic (`c*k+a`) — and draws the nonce from a
CSPRNG
// independent of the message. Three audit findings trace back to
that one
```

same file

```
//   - **CVF8.** `key_bytes(primes)` has only `H_inf(k) ≈
log2(|P|!/(|P|-K)!)`
//     bits of min-entropy — a structured, non-uniform HMAC key,
which is a
//     different (and non-standard) hypothesis from the textbook
uniform-key
//     HMAC-SHA256 PRF assumption every theorem bound otherwise cites.
//   - **CVF13.** The IND-CPA/INT-CTXT reductions must simulate the
token
```

same file

```
//     in place of `key_bytes(primes)`.
```

CVF-41: open Category: **efficiency**

Summary

The code that generates and checks the secret key's primes has four avoidable inefficiencies, and fixing one of them also closes the timing leak described in CVF-29.

Every time a message is encrypted or decrypted, the code checks again that each element of the secret key is a prime number and that no element is repeated. These checks use simple but slow methods. At the default key size the cost is small, but it grows quickly with the size of the key, and the time the prime check takes depends on the secret key, which leaks information about it (CVF-29). A faster method based on a fixed table of small primes would remove both problems.

The crate checks primality and distinctness of key material with the most direct algorithm in each case. None of the four choices below is incorrect, and at the default `K = 10` none of them is expensive on its own. They are reported together for two reasons: `validate_key` runs on every call of both v8 entry points (`lib.rs:1149`, `lib.rs:1212`), so the cost is paid on every encryption and decryption instead of once per key, and the remedy for the first item is also the cheapest fix for a security finding.

1. `is_prime` divides by every odd integer up to the square root, when a fixed table of 446 primes would be enough. The loop is

```
let mut i: u64 = 3;
while i.saturating_mul(i) <= n {
    if n % i == 0 { return false; }
    i += 2;
}
```

A sieve confirms the count:

$$\lfloor \sqrt{\text{MAX_KEY_PRIME}} \rfloor = \lfloor \sqrt{9{,}900{,}000} \rfloor = 3146$$

, and there are exactly **446 primes** up to 3,146, of which 445 are odd. Trial division by that fixed table replaces up to 1,572 divisions with 445. It also removes the bound check `i.saturating_mul(i)` that is recomputed on every iteration, since the table is traversed by index.

The effect on timing matters more than the speed. Because the table has a fixed length, running it to completion every time makes the work done by `is_prime` independent of `n` for every `n` in the normative interval, which is the property that CVF-29 reports as missing. CVF-29 measures the current loop at 499 iterations for $k = 1{,}000{,}003$ and 1,572 for $k = 9{,}899{,}999$, a $3.15\times$ spread that grows with each secret prime and is leaked again on every message. A fixed table run to completion removes that spread, so this item is also the cheapest fix for CVF-29. This is also how CVF-29's advice against "a constant-time primality test" should be read: a general-purpose constant-time primality test is not needed, while a fixed table of small primes over a bounded interval is a different and cheaper solution than the current loop.

This change is safe only together with the fix for CVF-16. `validate_key` currently enforces `k >= MIN_KEY_PRIME` but omits any upper bound, so it accepts key elements far above `MAX_KEY_PRIME`, which is the subject of CVF-16. A table limited to $\sqrt{\text{MAX_KEY_PRIME}}$ would declare such an element prime without testing the divisors above 3,146. This would turn a missing check into a wrong answer: a composite key element above $3146^2 = 9{,}897{,}316$ would be accepted. The table and the upper bound of CVF-16 should therefore be introduced as one change, and adding the table without the bound is worse than making neither change.

2. `generate_prime_numbers` rejects duplicates by scanning a growing vector (`lib.rs:111`, `!primes.contains(&num)`). At $K = 10$ this is at most ten comparisons per accepted candidate and is not worth changing on its own. We mention it because it has the same remedy as item 3.

3. `validate_key` checks distinctness in $O(K^2)$ time (`lib.rs:243`, `key[..i].contains(&k)`). At $K = 10$ that is 45 comparisons. However, `validate_key` places **no upper bound on `K`** and rejects only the empty key, so a key of length L supplied by a caller costs $L(L-1)/2$ comparisons *and* L trial divisions on every message. This is a second form of the denial-of-service risk described in CVF-16. CVF-16 concerns the unbounded **size** of a single key element, while this item concerns the unbounded **length** of the key: a key of 10^6 elements costs roughly 5×10^{11} comparisons per call, even before the trial divisions are counted.

4. The square-root bound is recomputed on every iteration. `i.saturating_mul(i) <= n` performs a multiplication and a comparison per step, where a single `let bound = n.isqrt();` before the loop would be enough. This item disappears if the table from item 1 is adopted.

Recommendation

Treat items 1 and 3 as part of the key-validation rework that CVF-16 and CVF-29 already call for, instead of as separate small optimisations.

Adopt the fixed prime table **together with** the upper bound from CVF-16:

```
/// All 446 primes <= isqrt(MAX_KEY_PRIME) = 3146.
const SMALL_PRIMES: [u64; 446] = [2, 3, 5, /* ... */ 3137,
  3163_u64 /* excluded */];

fn is_prime_bounded(n: u64) -> Result<bool, String> {
    if n > MAX_KEY_PRIME {
        return Err(format!("{} exceeds MAX_KEY_PRIME ({})", n,
  MAX_KEY_PRIME));
    }
    if n < 2 { return Ok(false); }
    let mut composite = false;
    for &p in SMALL_PRIMES.iter() {
        // Run the whole table unconditionally: the work is then
  independent
        // of n, which is what closes the CVF-29 timing channel.
        composite |= (n % p == 0) & (n != p);
    }
```

```
    Ok(!composite)
}
```

Running through the whole table unconditionally is essential: an early `return false` on the first divisor would make the number of iterations depend on the input again for composite numbers. Keep the existing unbounded `is_prime` under its current name if any caller outside key validation needs it, and route `validate_key` at `lib.rs:229` through the bounded version only.

Bound `K` in `validate_key`. The paper's key space consists of ordered K-tuples of distinct elements of $\mathcal{P}$, with a stated default of $K = 10$, while the crate accepts keys of any non-empty length. An explicit limit, `MAX_KEY_ELEMENTS`, declared as a `pub const` next to `MIN_KEY_PRIME` and `MAX_KEY_PRIME`, removes the denial-of-service risk from long keys and makes the $O(K^2)$ distinctness check always cheap, so that items 2 and 3 need no algorithmic change. If no limit is acceptable, sort a scratch copy of the key and scan it once, which takes $O(K \log K)$ time. However, bounding K is the better fix, because an unbounded key length causes problems beyond the number of comparisons.

Move the whole check off the per-message path, as CVF-29 and CVF-16 both recommend, by validating the key once into a `NapqesKey` newtype. When validation is performed once, at key construction, items 1 to 4 no longer affect each message, and the timing leak is also closed for a second, independent reason. The table is still worth adopting in that case, because it makes even a single validation cheap.

Item 4 needs no separate action. Computing `n.isqrt()` once before the loop is the right change if the table is not adopted, and it is unnecessary if the table is adopted.

/src/lib.rs

```rust
pub fn is_prime(n: u64) -> bool {
    if n < 2 {
        return false;
    }
    if n == 2 {
        return true;
    }
    if n % 2 == 0 {
        return false;
    }
```

same file

```rust
    let mut i: u64 = 3;
    while i.saturating_mul(i) <= n {
        if n % i == 0 {
            return false;
        }
        i += 2;
    }
    true
}
```

same file

```
        if is_prime(num) && !primes.contains(&num) {
```

same file

```
fn validate_key(key: &[u64]) -> Result<(), String> {
    if key.is_empty() {
        return Err("Key must be a non-empty list of primes.".into());
    }
```

same file

```
    for (i, &k) in key.iter().enumerate() {
        if !is_prime(k) {
            return Err(format!("Key element at index {} ({}) is not prime.", i, k));
        }
```

same file

```
        if key[..i].contains(&k) {
            return Err(format!(
                "Key element {} at index {} is a duplicate; all elements must be distinct.",
                k, i
            ));
```

same file

```
    validate_key(primes)?;
```

same file

```
    validate_key(primes)?;
```

CVF-43: fixed Category: **flaw**

Summary

The decryptor reads the message length from two values that it never checks, so one encrypted message can have several readings and a wrong length is returned instead of an error.

Every encrypted message carries its own length at the front, written as two small numbers. When the code decrypts a message, it combines these two numbers into the length without checking that each one is small enough. As a result, several different encodings produce the same length, and some invalid encodings produce a wrong length that is accepted instead of rejected. An outside attacker cannot exploit this without the key, because the message is authenticated first. However, a sender who holds the key can produce a valid message that the receiver decodes to a wrong length, and two implementations of the scheme could disagree about what such a message contains.

`unpad_message` recovers the message length from the first two entries of the decoded codepoint buffer:

```
let n = ((padded[0] as usize) << 8) | (padded[1] as usize);
```

This reconstruction assumes that `padded[0]` and `padded[1]` hold byte values (each in $[0, 255]$), mirroring the encoder in `pad_to_block`, which writes the length as a two-codepoint big-endian prefix and masks each half:

```
out.push(((n >> 8) & 0xFF) as u32);
out.push((n & 0xFF) as u32);
```

However, `padded` holds `u32` codepoints recovered by the decryptor, and neither prefix slot is restricted to a byte. On the v7 path, the recovered codepoint is pushed with no range check at all: `padded.push(((token - addend) / k) as u32)` at `lib.rs:513`. On the v8 path, it is only required to be a valid Unicode scalar, by `cp > 0x10FFFF || char::from_u32(cp as u32).is_none()` at `lib.rs:606`. In both cases, `padded[0]` and `padded[1]` can exceed `0xFF`.

The reconstruction is not injective, which we confirmed by direct computation. Because the shifted high slot and the low slot overlap through the `|`, different pairs of codepoints give the same `n`. The pairs $(1, 0)$, $(0, 256)$ and $(1, 256)$ all give $n = 256$: the first is the canonical encoding that the encoder emits, and the other two are non-canonical encodings of the same value that `unpad_message` accepts as equivalent. So the padded buffer does not have a unique reading, and an authenticated ciphertext does not determine its plaintext through this function alone.

The length check does not catch this. The subsequent check `2 + n > padded.len()` rejects an `n` that is too large, but an `n` that is within range and parsed incorrectly passes, and `unpad_message` returns a slice `padded[2..2 + n]` of the wrong length. The boundary case can actually be reached: at the largest padded block, $B = 2^{16}$, the buffer length is $B + 2 = 65{,}538$, and the pair $(\texttt{0x100}, 0)$ gives $n = 65{,}536$ with $2 + n = 65{,}538 \leq$

$65{,}538$. The check passes, and the function returns the entire buffer, including the prefix slots, as if it were the message.

The exposure is limited to two cases. On the authenticated wrappers, the tag is verified before decryption, so a network attacker cannot push the prefix out of range without forging the tag. The practical consequences are the following.

- A sender who holds the key can produce an authentic ciphertext whose padded prefix is non-canonical, and the receiver decodes it to a plaintext of the wrong length while the tag verifies. Two different valid ciphertexts therefore decrypt to the same plaintext, and one ciphertext has a reading that the encoder never intended. Section 10 claims that "conforming implementations must produce byte-identical ciphertext for every positive vector", and for such a scheme an ambiguous decoding is a risk across implementations: two conforming implementations could disagree about what a given authentic ciphertext means.
- On the unauthenticated low-level interface, `pub fn decrypt` calls `decrypt_core` with no tag check, and `decrypt_core` ends with `unpad_message(&padded).expect(...)` at `lib.rs:520`. A prefix that *does* fail the length check therefore aborts the process at that point. This abort is one of the five panics that CVF-15 already reports, so we do not report it again here. We mention it because it is the same missing check seen from the other side, and the fix below closes both.

The missing check is also inconsistent with the defensive approach of the function itself. The doc comment of `unpad_message` says explicitly that the length prefix "is attacker-chosen in the sense that it is recovered from the decrypted blob, so it must be validated against the buffer actually present". The function then checks `n` against the buffer length, but it never checks that the two codepoints from which `n` is built are byte values. In addition, item 8 of CVF-40 records that Section 12 of the paper calls this prefix "2-byte", while `Definition 3.10` and step (8) of `Dec` correctly call it two codepoints. The code follows the paper's looser description in exactly the place where the distinction matters.

Recommendation

Validate the two prefix codepoints before reconstructing the length, and return an error in the same way as `unpad_message` already does for other failures:

```
if padded[0] > 0xFF || padded[1] > 0xFF {
    return Err(format!(
        "Malformed length prefix: codepoints ({}, {}) are not
  byte-valued.",
        padded[0], padded[1]
    ));
}
let n = ((padded[0] as usize) << 8) | (padded[1] as usize);
```

Rejecting the input is better than masking it with `& 0xFF`. Masking would accept a non-canonical buffer as a valid encoding of some `n`, and so the encoding would remain ambiguous. An explicit check keeps the length field canonical and matches the intent of the encoder, which is what a claim of byte-for-byte agreement between implementations needs.

Apply the same check in `napqes.py::_unpad_message` and in `C/napqes.c`, so that

the three implementations agree on out-of-range input as well as on well-formed input. At present they agree only on the output of the encoder; this is one of the places where they have never been compared, because no test vector reaches it.

Add a negative known-answer test vector for this case. The test vectors already include `W-N06` to `W-N08` for the structural checks performed after authentication that were added in this version of the scheme, and a vector whose padded prefix is non-canonical belongs in the same group: it is the only remaining check after authentication that has no vector. CVF-25 records that the negative vectors are currently asserted and not verified, and that the Rust side checks only `is_err()`. A vector added for this case should check the error text, otherwise it cannot be told apart from a rejection by the tag check.

Finally, note the interaction with CVF-15: turning the `expect` in `decrypt_core` into a propagated error is the other half of this fix. With the range check in place, the `expect` can no longer be reached through the v8 path, but it can still be reached through `pub fn decrypt`, so both changes are needed.

/src/lib.rs

```
407     let mut out = Vec::with_capacity(2 + block_size);
408     out.push(((n >> 8) & 0xFF) as u32);
409     out.push((n & 0xFF) as u32);
```

same file

```
450 fn unpad_message(padded: &[u32]) -> Result<Vec<u32>, String> {
451     if padded.len() < 2 {
452         return Err("Padded message too short to contain length
prefix.".into());
```

same file

```
454     let n = ((padded[0] as usize) << 8) | (padded[1] as usize);
```

same file

```
455     if 2 + n > padded.len() {
456         return Err(format!(
457             "Length prefix ({}) exceeds available data ({}
codepoints).",
458             n,
459             padded.len() - 2
460         ));
461     }
```

same file

```
513             padded.push(((token - addend) / k) as u32);
```

same file

```
520     unpad_message(&padded).expect("v7 decrypt: malformed padded
buffer")
```

same file

```
606         if cp > 0x10FFFF || char::from_u32(cp as u32).is_none() {
```

CVF-45: fixed Category: **flaw**

Summary

The function that compares authentication tags reports a match when the two tags have different lengths, and the only check against this is removed from the builds the client ships.

The code decides whether a message is authentic by comparing the tag it receives with the tag it computes. The comparison function assumes that both tags have the same length but does not enforce it in production builds: if one tag is shorter, or empty, it compares only the overlapping part and can report a match. Today every place that uses the function always passes two tags of the correct length, so this cannot be exploited now. However, the function is the code's standard tool for this kind of comparison, and any future use with a tag of variable length could accept forged messages.

`ct_eq_bytes` is the crate's equality function for authentication tags. Its doc comment describes it as a comparison of "two equal-length byte slices in constant time", and it ends with a specific claim about the generated code:

> The resulting assembly is a fixed-count loop: `jne` branches only on the loop counter (always 32 iterations), never on the tag data.

The signature accepts two slices whose lengths are arbitrary and independent, and the body checks that the lengths are equal only in a debug build:

```
#[inline(never)]
fn ct_eq_bytes(a: &[u8], b: &[u8]) -> bool {
    debug_assert_eq!(a.len(), b.len());
    let mut diff = 0u8;
    for (x, y) in a.iter().zip(b.iter()) {
        ...
    }
    diff == 0
}
```

`Iterator::zip` stops at the end of the shorter of the two operands. So when the lengths differ, the function compares the common prefix and ignores the rest of the longer slice, and `diff` keeps its initial value `0` if the prefix matches. This includes the extreme case in which one slice is empty: `ct_eq_bytes(&[], &tag)` returns **`true`** for every `tag`. The result of this function is the only condition that leads to `Err("Authentication failed:`

`invalid HMAC tag.")` in three places in the code. For such a function, returning `true` for an empty candidate tag is the wrong failure mode: a comparator of this kind must reject any input it was not designed for.

The guard does not exist in the build that the client ships. `debug_assert_eq!` is active only when `debug_assertions` is enabled, and `Cargo.toml` declares no `[profile.release]` section at all. CVF-23 reports this omission, and CVF-31 relies on it in its own analysis. In the delivered configuration, a release build therefore contains no length check of any kind, and a length mismatch goes unnoticed instead of stopping the program.

The property stated in the doc comment is already false in one place where the function is used. The comment's "always 32 iterations" holds for the three tag comparisons, but not for `generate_nonce_with_crng_check`, which calls the same function on two 16-byte nonces:

```
if ct_eq_bytes(&nonce, &prev_nonce) {
```

This call is correct, since both operands are `[u8; NONCE_SIZE]`, but the loop runs 16 times there. So the one concrete property that the doc comment states about the generated code does not hold for the function as it is used. CVF-35 mentions this call for a different reason; we cite it here because it shows that the comment is wrong.

No current use of the function can trigger the mismatch, which limits the severity. At `lib.rs:847`, `lib.rs:949` and `lib.rs:1218`, the received tag is `&ciphertext[split..]` with `split = ciphertext.len() - TAG_SIZE`, taken after a minimum-length check that guarantees `ciphertext.len() >= NONCE_SIZE + TAG_SIZE`, so it is exactly 32 bytes long; and the computed tag is a `[u8; 32]`. At `lib.rs:56`, both operands are `[u8; NONCE_SIZE]`. This is therefore a latent defect, not a vulnerability that can be exploited today.

We rate it medium instead of low for three reasons. First, the function is declared `fn`, not `pub fn`, but it is the crate's designated constant-time comparison, and a future caller would naturally reuse it, for example for a variable-length MAC, a KEM shared secret, or a tag supplied by a caller through a protocol layer such as `ot_frame` or `protocols`. Second, the type system could have enforced the length at no cost and was not used: `&[u8; 32]` would have made the requirement impossible to break. Third, the protection that was chosen is the one kind of check that the crate's own build configuration removes. CVF-31 identifies the same root cause in the `debug_assert!(block_size > n)` of `pad_to_block`, so two security-relevant preconditions in this crate are now protected only by assertions that do not exist in the shipped binary.

Recommendation

Enforce the length requirement through the type system, which removes the failure mode instead of detecting it:

```
#[inline(never)]
fn ct_eq_tag(a: &[u8; TAG_SIZE], b: &[u8; TAG_SIZE]) -> bool { ...
  }
```

All three places that compare tags can supply this type directly: `calc_tag` is already `[u8; 32]`, and `recv_tag` becomes `ciphertext[split..].try_into().map_`

`err(...)?`. This is a real improvement, because it replaces a requirement maintained by arithmetic in three separate places with one that the compiler checks. Give the nonce comparison in `generate_nonce_with_crng_check` its own comparator typed `[u8; NONCE_SIZE]`, or, since both operands are public values that appear unencrypted on the wire, use a plain `==`. CVF-35 makes the same point and notes that using the constant-time comparator there hides which comparisons are security-relevant.

If the slice signature must be kept for generality, then reject a length mismatch before the loop, in every build:

```
if a.len() != b.len() {
    return false;
}
```

An early `return false` on a length mismatch reveals only the lengths, which are public in every current and foreseeable use, and it is the standard approach: the `ConstantTimeEq` implementation for slices in `subtle` does exactly this. Do not leave the `debug_assert!` as the only check. It can stay, but it must not be the only safeguard.

Correct the doc comment in the same change. "Always 32 iterations" should say that the number of iterations is the fixed length of the tag type, and the comment should state explicitly that the comparison is constant-time with respect to the tag *contents* and not the tag *length*, since that is what a reader needs to know before reusing the function.

More broadly, CVF-12 already recommends replacing the hand-written `read_volatile`/`write_volatile` construction with `ConstantTimeEq` from `subtle`. Doing so also resolves this finding, because the slice implementation in `subtle` checks the lengths and rejects a mismatch. If that change is planned, it can close this finding instead of a separate fix. Until it is made, however, the `[u8; TAG_SIZE]` signature is a two-line change that removes the risk now.

Finally, add a `[profile.release]` section that enables `debug-assertions`, or at least `overflow-checks`, as CVF-23 recommends. This does not fix this finding, but it would have made both this problem and the `pad_to_block` precondition of CVF-31 fail visibly instead of silently.

/Cargo.toml

```
6 [dependencies]
7 hmac = "0.12"
8 sha2 = "0.10"
```

/src/lib.rs

```
56         if ct_eq_bytes(&nonce, &prev_nonce) {
```

same file

```
622 /// Compare two equal-length byte slices in constant time.
```

same file

```
635 /// The resulting assembly is a fixed-count loop: `jne` branches only on the
636 /// loop counter (always 32 iterations), never on the tag data.
```

same file

```
637 #[inline(never)]
638 fn ct_eq_bytes(a: &[u8], b: &[u8]) -> bool {
639     debug_assert_eq!(a.len(), b.len());
```

same file

```
640     let mut diff = 0u8;
641     for (x, y) in a.iter().zip(b.iter()) {
642         unsafe {
643             diff |= std::ptr::read_volatile(x) ^
std::ptr::read_volatile(y);
```

same file

```
649     diff == 0
```

same file

```
847     if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
```

same file

```
949     if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
```

same file

```
1218     if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
```

CVF-46: open Category: **architecture**

Summary

The core encryption loop is copied by hand into five functions, and the copies have already drifted apart, which is how one known defect arose and how the next one is likely to arise.

The step that produces the encrypted output, mixing real characters with random noise, is written out separately in five different encryption functions instead of being written once and shared. When a fix was made to this step, it was applied to one copy and not to the other four, and that is the cause of a defect described in another finding. All copies currently produce correct output, but each future change has to be made in five places by hand, and a change made in only some of them would make two functions of the same library produce incompatible encrypted data without any warning.

Five public or crate-visible encryption functions each contain their own copy of the NAPQES emission loop, i.e. the loop that turns the padded message into tokens: `encrypt` at `lib.rs:467`, `encrypt_bytes` at `lib.rs:730`, `encrypt_bytes_with_nonce`

at `lib.rs:785`, `encrypt_raw` at `lib.rs:886`, and `encrypt_bytes_v8_with_profile` at `lib.rs:1136`. The four v7 copies are identical line for line, apart from the names of two local variables:

```
for &c in &padded {
    loop {
        if is_noise_pos(&kb, &nonce, ct_pos, noise_p) {
            let k = key[(real_idx % kk) as usize];
            let nc = derive_noise_char(&kb, &nonce, ct_pos);
           let na = derive_noise_token_addend(&kb, &nonce, ct_pos, k);
            cypher.push(nc * k + na);
            ct_pos += 1;
        } else {
            let k = key[(real_idx % kk) as usize];
            let addend = derive_addend(&kb, &nonce, real_idx, k);
            cypher.push(c as u64 * k + addend);
            ct_pos += 1;
            real_idx += 1;
            break;
        }
    }
}
```

Three of the four are also followed by an identical sealing sequence, consisting of `fixed_encode_tokens`, `varint_keystream`, the XOR mask, the `nonce || masked` payload and `compute_auth_tag(..., AAD_LEN_WIDTH_V7)`, at `lib.rs:761-769`, `lib.rs:820-828` and `lib.rs:917-925`.

The copies have already diverged, and this divergence is the cause of CVF-17. The fifth copy, in `encrypt_bytes_v8_with_profile`, is the same loop with a noise-run counter added:

```
if noise_run < MAX_NOISE_RUN && is_noise_pos_v8(&sk_fmt, &nonce,
  ct_pos, noise_theta) {
```

and a `noise_run += 1` in the noise branch. The four v7 copies have neither, which is exactly what CVF-17 reports: the legacy emission loop applies neither the noise-run cap nor the token ceiling, so the length of its output depends on the pseudorandom noise positions instead of being a fixed function of the padded block. That defect did not come from wrong reasoning about v7. It came from a fix being applied to one of five copies of the same code, which is the risk this finding describes. `derive_noise_char` and `derive_noise_token_addend` are shared, but the loop that calls them is not.

Within each copy, the two branches repeat the same work. `let k = key[(real_idx % kk) as usize];` is the first statement of both the noise branch and the real branch, and `ct_pos += 1;` is the last operation that both perform before they diverge: `lib.rs:482` and `lib.rs:488`, `lib.rs:486` and `lib.rs:491`, with the same pairing in each of the other four copies. Both can be moved out of the branches, which shortens every copy by two lines and removes two more places where a future edit could change one branch and not the other.

The structure of the loop does not show what it does. Written as an unconditional `loop` with a `break` in the `else` branch, it reads as "emit either a noise token or a real token", while it actually does "emit zero or more noise tokens, then exactly one real token". This distinction matters here: the number of noise tokens per real token is the quantity that CVF-17 is about, the quantity that the v8 cap limits, and the quantity that the ceiling loop at `lib.rs:1179-1186` later pads out. A reader checking whether the cap is applied first has to work out how many times the loop runs, in each of five places.

None of this makes any ciphertext wrong today: the four v7 copies agree with each other, and the known-answer test vectors cover two of the five entry points. We report it because Section 10 of the paper claims byte-for-byte agreement between three implementations in different languages, and a construction kept as five copies synchronised by hand within one implementation is the weakest part of that claim. The next fix that is applied to the wrong subset of copies will make two entry points of the same library produce different wire formats without any warning, as has already happened with the noise cap in v7 and v8.

Recommendation

Write the emission loop once and call it from all five entry points.

The five copies differ in only four parameters: the derivation key (`kb` for v7, `sk_fmt` for v8), the noise predicate (`is_noise_pos` with an `f64` probability, or `is_noise_pos_v8` with an integer threshold), whether a noise-run cap applies, and whether the ceiling padding follows. This makes one function with a small parameter object, not five functions:

```
struct EmitParams<'a> {
    kb: &'a [u8],
    nonce: &'a [u8],
    key: &'a [u64],
    noise: NoiseRule,          // V7 { p: f64 } | V8 { theta: u64,
  cap: u64 }
}

fn emit_tokens(padded: &[u32], p: &EmitParams) -> Vec<u64> {
    let kk = p.key.len() as u64;
    let (mut real_idx, mut ct_pos) = (0u64, 0u64);
    let mut cypher = Vec::new();
    for &c in padded {
        let mut noise_run = 0u64;
        // Hoisted: identical in both former branches.
        while p.noise.is_noise(p, ct_pos, noise_run) {
            let k = p.key[(real_idx % kk) as usize];
            let nc = derive_noise_char(p.kb, p.nonce, ct_pos);
            let na = derive_noise_token_addend(p.kb, p.nonce,
  ct_pos, k);
            cypher.push(nc * k + na);
            ct_pos += 1;
            noise_run += 1;
```

```
        }
        let k = p.key[(real_idx % kk) as usize];
        let addend = derive_addend(p.kb, p.nonce, real_idx, k);
        cypher.push(c as u64 * k + addend);
        ct_pos += 1;
        real_idx += 1;
    }
    cypher
}
```

The `while` form makes the structure explicit, i.e. zero or more noise tokens followed by exactly one real token, so restructuring the loop and removing the duplication are the same edit, and moving `let k` and `ct_pos += 1` out of the branches follows from it. Extract the v7 sealing sequence (from `fixed_encode_tokens` to `compute_auth_tag`) into a second helper in the same way, since it also exists in three identical copies.

Coordinate this with the retirement of v7 so that the work is not done twice. CVF-17 and CVF-28 both recommend withdrawing the v7 encryption functions, and CVF-22 notes that the v7 interface is the one with the reachable panics. If v7 is being retired, deleting four of the five copies is a better solution than unifying them, and this finding is closed together with CVF-17. If v7 is kept for byte compatibility, the extraction should be done first, because the divergence of the noise cap shows that keeping five copies synchronised by hand has not worked in this code base.

In either case, add a test that checks the v7 entry points against each other on a shared input: `encrypt_bytes`, `encrypt_raw` on the same bytes, and `encrypt_bytes_with_nonce` with a fixed nonce must produce the same token stream. CVF-25 records that `encrypt_raw` and `encrypt_str` are not covered by any known-answer test, so at present nothing would detect a change made to one copy and not to the others.

/src/lib.rs

```
pub fn encrypt(message: &[u32], key: &[u64]) -> ([u8; NONCE_SIZE], Vec<u64>) {
```

same file

```
    for &c in &padded {
        loop {
            if is_noise_pos(&kb, &nonce, ct_pos, noise_p) {
                let k = key[(real_idx % kk) as usize];
                let noise_c = derive_noise_char(&kb, &nonce, ct_pos);
                let noise_add = derive_noise_token_addend(&kb, &nonce, ct_pos, k);
                cypher.push(noise_c * k + noise_add);
                ct_pos += 1;
            } else {
                let k = key[(real_idx % kk) as usize];
                let addend = derive_addend(&kb, &nonce, real_idx, k);
```

```
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }
```

same file

```
pub fn encrypt_bytes(message: &str, key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
```

same file

```
    for &c in &padded {
        loop {
            if is_noise_pos(&kb, &nonce, ct_pos, noise_p) {
                let k = key[(real_idx % kk) as usize];
                let nc = derive_noise_char(&kb, &nonce, ct_pos);
                let na = derive_noise_token_addend(&kb, &nonce,
ct_pos, k);
                cypher.push(nc * k + na);
                ct_pos += 1;
            } else {
                let k = key[(real_idx % kk) as usize];
                let addend = derive_addend(&kb, &nonce, real_idx, k);
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }
```

same file

```
    let blob = fixed_encode_tokens(&cypher);
    let ks = varint_keystream(&kb, &nonce, blob.len());
    let masked: Vec<u8> = blob.iter().zip(ks.iter()).map(|(a, b)| a ^
b).collect();
    let mut payload = Vec::with_capacity(NONCE_SIZE + masked.len());
    payload.extend_from_slice(&nonce);
    payload.extend_from_slice(&masked);
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
    payload.extend_from_slice(&tag);
    Ok(payload)
```

same file

```
pub(crate) fn encrypt_bytes_with_nonce(
```

same file

```
    for &c in &padded {
        loop {
            if is_noise_pos(&kb, &nonce, ct_pos, noise_p) {
                let k = key[(real_idx % kk) as usize];
                let nc = derive_noise_char(&kb, &nonce, ct_pos);
                let na = derive_noise_token_addend(&kb, &nonce,
ct_pos, k);
                cypher.push(nc * k + na);
                ct_pos += 1;
            } else {
                let k = key[(real_idx % kk) as usize];
                let addend = derive_addend(&kb, &nonce, real_idx, k);
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }
```

same file

```
    let blob = fixed_encode_tokens(&cypher);
    let ks = varint_keystream(&kb, &nonce, blob.len());
    let masked: Vec<u8> = blob.iter().zip(ks.iter()).map(|(a, b)| a ^
b).collect();
    let mut payload = Vec::with_capacity(NONCE_SIZE + masked.len());
    payload.extend_from_slice(&nonce);
    payload.extend_from_slice(&masked);
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
    payload.extend_from_slice(&tag);
    payload
```

same file

```
pub fn encrypt_raw(data: &[u8], key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
```

same file

```
    for &c in &padded {
        loop {
            if is_noise_pos(&kb, &nonce, ct_pos, noise_p) {
                let k = key[(real_idx % kk) as usize];
                let nc = derive_noise_char(&kb, &nonce, ct_pos);
                let na = derive_noise_token_addend(&kb, &nonce,
ct_pos, k);
                cypher.push(nc * k + na);
```

```
                ct_pos += 1;
            } else {
                let k = key[(real_idx % kk) as usize];
                let addend = derive_addend(&kb, &nonce, real_idx, k);
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }
```

same file

```
    let blob = fixed_encode_tokens(&cypher);
    let ks = varint_keystream(&kb, &nonce, blob.len());
    let masked: Vec<u8> = blob.iter().zip(ks.iter()).map(|(a, b)| a ^ b).collect();
    let mut payload = Vec::with_capacity(NONCE_SIZE + masked.len());
    payload.extend_from_slice(&nonce);
    payload.extend_from_slice(&masked);
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
    payload.extend_from_slice(&tag);
    Ok(payload)
```

same file

```
pub fn encrypt_bytes_v8_with_profile(
```

same file

```
    for &c in &padded {
        let mut noise_run: u64 = 0;
        loop {
            if noise_run < MAX_NOISE_RUN && is_noise_pos_v8(&sk_fmt, &nonce, ct_pos, noise_theta) {
                let k = primes[(real_idx % kk) as usize];
                let nc = derive_noise_char(&sk_fmt, &nonce, ct_pos);
                let na = derive_noise_token_addend(&sk_fmt, &nonce, ct_pos, k);
                cypher.push(nc * k + na);
                ct_pos += 1;
                noise_run += 1;
            } else {
                let k = primes[(real_idx % kk) as usize];
                let addend = derive_addend(&sk_fmt, &nonce, real_idx, k);
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
```

```
1172                real_idx += 1;
1173                break;
1174            }
1175        }
1176    }
```

CVF-47: open Category: **procedural**

> **Summary**
>
> **The requirement to pass the start-up self-tests before encrypting exists only in the documentation, so the module encrypts and decrypts in the same way whether the self-tests passed, failed or were never run.**
>
> The module's documentation says that its start-up self-tests must be run before any use, and that no encryption may take place if they fail, as the FIPS 140-3 standard that it cites requires. The code does not enforce this. It does not remember whether the self-tests were run or what their result was, and the encryption and decryption functions work in exactly the same way in every case. A failed self-test is reported to the caller, but the module keeps working if the caller ignores the report. An operator who relies on the module's stated FIPS behaviour is therefore not protected by it.

The self-test module begins with a requirement for the operator:

> Call `run_power_on_self_tests()` before using the module in production. It returns `Ok(())` only if all tests pass. On any failure it returns `Err(SelfTestError)` — the caller MUST NOT perform any cryptographic operations after a self-test failure.

and ends by citing the standard that it follows, NIST SP 800-140B §4.9.1. No code implements this requirement. `run_power_on_self_tests()` is a free function that returns a `Result` and changes no shared state. The declaration `pub mod self_test;` at `lib.rs:19` is the module's only connection to the rest of the crate, and no entry point refers to it. We checked every public encryption and decryption path: `encrypt_bytes`, `encrypt_str`, `encrypt_raw`, `encrypt_bytes_v8`, `encrypt_bytes_v8_with_profile` and the corresponding decryption functions each behave identically in all four states that the requirement distinguishes: self-tests never called, called and passed, called and failed, and called and failed with the error ignored by the caller.

This has three consequences.

The requirement cannot be enforced on the party it applies to. A caller who follows the instruction and a caller who never learns of it get the same behaviour from the same API, and nothing in any signature, type or doc comment of the entry points shows that a precondition exists. The requirement appears only in the documentation of a module that a user of the crate has no reason to open.

A failed self-test does not stop the module. This matters most under the standard that the module cites. FIPS 140-3 requires a module that fails a self-test to enter an error state in which all cryptographic operations and all data output are inhibited. It does not allow the module to keep operating while reporting the failure to a caller who may ignore it. As implemented, a `SelfTestError` is an ordinary return value. From the library's side, a deployment that logs the error and continues cannot be told apart from one that stops. Yet the failures of KAT-1 and KAT-2 that the self-tests are designed to catch are exactly the conditions in which continuing to encrypt does the most harm.

The problem adds to the limits of what the self-tests cover. CVF-18 shows that all three known-answer tests (KATs) use the legacy v7 format and none of them uses the v8 scheme that the paper specifies, and that INT-1 compares a version string instead of checking integrity. That finding concerns *what* the self-tests check. This one concerns *whether their result has any effect*: even a complete and correct set of self-tests would not block anything here. The two problems are independent, since fixing either one leaves the other, and both must be fixed before the module's claim of conformance with SP 800-140B holds. The same applies to `pub mod kem`, which CVF-18 notes has no self-test of its own and which would also not be blocked by one if it had.

Nothing in the paper's security argument depends on the self-tests, and no theorem is affected. The finding concerns the FIPS behaviour that the module advertises, which the crate's documentation states and on which an operator would rely when deploying it in a regulated setting.

Recommendation

Make the requirement effective by keeping the self-test result in module state and checking it on the cryptographic paths.

```
use std::sync::atomic::{AtomicU8, Ordering};

const POST_NOT_RUN: u8 = 0;
const POST_PASSED:  u8 = 1;
const POST_FAILED:  u8 = 2;

static POST_STATE: AtomicU8 = AtomicU8::new(POST_NOT_RUN);

pub fn run_power_on_self_tests() -> Result<(), SelfTestError> {
    let r = run_all();
    POST_STATE.store(
        if r.is_ok() { POST_PASSED } else { POST_FAILED },
        Ordering::SeqCst,
    );
    r
}

/// Returns `Err` unless the power-on self-tests have run and
  passed.
pub(crate) fn require_post() -> Result<(), String> {
    match POST_STATE.load(Ordering::SeqCst) {
```

```
        POST_PASSED => Ok(()),
        POST_FAILED => Err("Module is in the self-test error state;
  \
                               cryptographic operations are
  inhibited.".into()),
        _           => Err("Power-on self-tests have not been
  run.".into()),
    }
}
```

and call `require_post()?` as the first line of each public encryption and decryption entry point. Once `POST_FAILED` is set, nothing should be able to clear it except a new call of `run_power_on_self_tests()`, so that the error state persists as the standard requires.

Choose deliberately between blocking and warning, and state the choice in the documentation. Blocking, i.e. returning `Err`, is what SP 800-140B §4.9.1 requires, and it is the right default for a module that advertises conformance. Warning is weaker than the standard requires, and should be chosen only if the crate also stops claiming conformance with §4.9.1. The one arrangement that should not remain is the current one, in which the documentation states the strong requirement and the code implements neither option.

There are two further points. First, if blocking by default would break existing applications that embed the crate, put it behind a `fips` Cargo feature that is enabled by default. CVF-23 notes that the manifest declares no features at all, and this would be a natural first one; a crate compiled without the feature should then not carry the SP 800-140B text. Second, whichever option is chosen, `pub mod kem` needs the same treatment as the AEAD entry points, since it is inside the module boundary that the self-test documentation describes.

Make this change after the fix for CVF-18, not before. Enabling the block while the three KATs all use the legacy v7 format would stop the module on the basis of tests that say nothing about the v8 scheme, and INT-1 in its current form would disable it completely at the first change of the version number away from `0.1.0`.

/src/lib.rs

```
19 pub mod self_test;
```

same file

```
730 pub fn encrypt_bytes(message: &str, key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
```

same file

```
886 pub fn encrypt_raw(data: &[u8], key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
```

same file

```
1120 pub fn encrypt_bytes_v8(
```

same file

```
1136 pub fn encrypt_bytes_v8_with_profile(
```

/src/self_test.rs

```
1 //! FIPS 140-3 power-on self-tests for the NAPQES Cryptographic
  Module.
2 //!
3 //! Call `run_power_on_self_tests()` before using the module in
  production.
4 //! It returns `Ok(())` only if all tests pass.  On any failure it
  returns
5 //! `Err(SelfTestError)` — the caller MUST NOT perform any
  cryptographic
6 //! operations after a self-test failure.
```

same file

```
8 //! Tests performed:
9 //!   KAT-1  Encrypt a known plaintext → compare to reference
  ciphertext.
10 //!   KAT-2  Decrypt the reference ciphertext → compare to original
  plaintext.
11 //!   KAT-3  Decrypt a tampered ciphertext → confirm authentication
  failure.
12 //!   INT-1  Software integrity check via embedded compile-time build
  hash.
```

same file

```
27 //! Reference: NIST SP 800-140B §4.9.1 (power-on self-tests).
```

5.4. Minor

CVF-19: fixed Category: **procedural**

Summary

Four tests cannot fail: two compatibility tests report success even when they have checked no test vectors, and two unit tests check conditions that are true whatever the code does.

The paper relies on the test suite as evidence that the implementations agree with each other and that the code was written carefully. Four of these tests would pass even if the code they are meant to check were broken. Two tests that compare the Rust code with the reference test vectors report success even if no vector was actually checked, so a missing or renamed field in the test-vector file would go unnoticed. Two other tests check facts that are always true, for example that a 32-byte value differs from a 50-byte value. This does not affect the security of the scheme directly, but it means that the tests give less assurance than they appear to.

Section 10 presents the known-answer test vectors as the evidence that the implementations agree, and Section 8.2 lists the Rust unit tests as evidence of careful implementation. Reading the source shows that four of those tests cannot report a failure.

The positive-decryption KAT guards against a run that tests nothing, but the guard uses a conjunction that makes it ineffective:

```
    // Fail if ZERO vectors were tested successfully (indicates a
build problem)
    if tested == 0 && skipped == 0 {
        panic!("No positive vectors were tested — check vector file
path");
    }
```

`skipped` is incremented for every vector whose message is empty. So the guard does not fire, and the test passes without having decrypted anything, if the test vectors consist entirely of empty-message vectors, or if a renamed field makes `v["message"].as_str().unwrap_or("")` yield `""` for every entry. The second case is the realistic way for this to fail, since the code falls back to the empty string instead of reporting an error on a missing field. In both cases `skipped > 0` and `tested == 0`. The companion test `positive_encrypt_bytes_deterministic` gets this right with a plain `assert!(tested > 0, "No vectors were tested")`, so the two guards disagree, and the weaker one protects the decryption direction.

The negative KAT has no guard at all, and its comment says so:

```
    if tested == 0 {
        // Not a hard failure: negative vectors may also hit
SKIP-PHASE2 paths
        eprintln!("PARITY-NOTE: no negative vectors exercised");
    }
```

This is the test that checks that tampered ciphertexts are rejected. As written, it passes when no tampered ciphertext was ever submitted, and only prints a note to stderr, which

no CI system will treat as a failure. Taken together with the previous case, this means that the v7 compatibility tests can report a clean run without having decrypted a single valid vector or rejected a single invalid one.

Two unit tests in `lib.rs` check statements that hold regardless of the implementation. The first is meant to check that the two v8 key components are sampled independently:

```
#[test]
fn v8_key_generation_is_independent() {
    let (primes, sk) = generate_v8_key(10, MIN_KEY_PRIME,
  MAX_KEY_PRIME);
    assert_eq!(primes.len(), 10);
    // sk must not be derivable from key_bytes(primes) via the v7 KDF
  —
    // spot-check it does not equal the v7 HMAC key material shape.
    assert_ne!(sk.to_vec(), key_bytes(&primes));
}
```

`sk` is `SK_SIZE = 32` bytes long, while `key_bytes` returns $5K$ bytes, here 50. Equality of `Vec<u8>` values compares the lengths first, so the assertion holds for every K, since $5K = 32$ has no integer solution. It would still hold if `generate_v8_key` were changed to derive `sk` directly from the primes, which is exactly the property that the test is named after. The independence of the two random draws, on which the security argument for the v8 key relies, is therefore not tested.

The second is `empty_message_roundtrip`, which is discussed in CVF-13: both calls take their early return for empty input, so the assertion compares `""` with `""` without performing any cryptographic operation.

There are also two smaller gaps in the same area, which we mention here instead of reporting them separately. First, `primes_are_prime` checks that the generated elements are prime, but not that they are below `MAX_KEY_PRIME` or that they are distinct, which are the two properties that CVF-16 and the claim about the entropy of the key space depend on. Second, both tamper tests flip the last byte, which is always a byte of the tag. No test modifies the nonce or the masked blob, so the fact that the tag covers the blob is never tested.

By contrast, the v8 compatibility tests are guarded correctly: all three end with `assert!(tested > 0, ...)`, and `v8_positive_encrypt_matches_python` compares the full ciphertext byte for byte. The problem is limited to the v7 tests and the two unit tests described above.

Recommendation

Make each of the four tests capable of failing.

For the two compatibility guards, replace both with the unconditional form that the companion test already uses, and check the expected number of vectors as well as checking that it is not zero, so that a test-vector file that silently loses vectors makes the test fail:

```
assert!(tested > 0, "No positive vectors were tested");
```

```
assert_eq!(tested + skipped, expected_vector_count, "corpus size
  changed unexpectedly");
```

Remove the `skipped` term from the positive-decryption guard entirely, since a skipped vector is not evidence of anything. If the empty-message vector is kept at all, handle it with an explicit assertion about the intended behaviour, instead of with a `continue` that increments a counter which then disables the guard. See also CVF-13, which recommends removing that behaviour together with the vector.

For the negative tests, turn the note into a failure. A test that checks that forgeries are rejected must fail when it has rejected nothing. If some negative vectors are legitimately out of scope for the Rust implementation, filter them out explicitly by an `api` or `phase` field and check that the number of remaining vectors is not zero, as the `stream_ae` filter already does elsewhere in the same file.

For `v8_key_generation_is_independent`, test the property itself instead of the length of the values. Comparing a 32-byte value with a 50-byte value cannot detect the failure the test is meant to catch. A test that can detect it computes what the v7 derivations would have produced and checks that `sk` differs from it, with a fixed seed:

```
let (primes, sk) = generate_v8_key(10, MIN_KEY_PRIME,
  MAX_KEY_PRIME);
// sk must not be any HMAC of the prime material under the v7 or
  subkey schedules
assert_ne!(sk, hmac_digest(&key_bytes(&primes), &[]));
assert_ne!(sk, derive_format_subkey(&key_bytes(&primes),
  FORMAT_BLOCK_V8));
```

A more useful alternative is a statistical test that two calls with the same `primes` yield different values of `sk`. This is the observable consequence of independent sampling, and it would detect an accidental derivation directly.

For `empty_message_roundtrip`, see CVF-13: check that the ciphertext length is the specified 2,928 bytes, so that the test goes through the tag computation.

Two further additions are worth making in the same file. Strengthen `primes_are_prime` so that it also checks `p <= MAX_KEY_PRIME` and that the elements are pairwise distinct. Add one tamper test that flips a byte inside the masked blob and one that flips a byte of the nonce, so that the tag's coverage of both is tested instead of assumed.

/src/kat_cross_check.rs

```
68          let message = vec["message"].as_str().unwrap_or("");
```

same file

```
73          if message.is_empty() {
74              skipped += 1;
75              eprintln!("[SKIP] {} — empty message, no ciphertext", id);
76              continue;
77          }
```

same file

```
109     // Fail if ZERO vectors were tested successfully (indicates a
build problem)
```

```
    if tested == 0 && skipped == 0 {
        panic!("No positive vectors were tested — check vector file path");
    }
```

same file

```
    eprintln!("\nRust KAT deterministic encrypt: {} passed", tested);
    assert!(tested > 0, "No vectors were tested");
```

same file

```
    eprintln!("\nRust KAT negative: {} passed", tested);
    if tested == 0 {
        // Not a hard failure: negative vectors may also hit SKIP-PHASE2 paths
        eprintln!("PARITY-NOTE: no negative vectors exercised");
    }
```

/src/lib.rs

```
        let last = ct.len() - 1;
        ct[last] ^= 0x01;
```

same file

```
    #[test]
    fn empty_message_roundtrip() {
        let k = test_key();
        assert_eq!(decrypt_str(&encrypt_str("", &k, b"").unwrap(), &k, b"").unwrap(), "");
    }
```

same file

```
    #[test]
    fn primes_are_prime() {
        let ps = generate_prime_numbers(10, MIN_KEY_PRIME, MAX_KEY_PRIME);
        assert_eq!(ps.len(), 10);
        for p in &ps {
            assert!(is_prime(*p));
        }
    }
```

same file

```
        let last = ct.len() - 1;
        ct[last] ^= 0x01;
```

same file

```
    #[test]
    fn v8_key_generation_is_independent() {
```

```
1418        let (primes, sk) = generate_v8_key(10, MIN_KEY_PRIME,
MAX_KEY_PRIME);
1419        assert_eq!(primes.len(), 10);
1420        // sk must not be derivable from key_bytes(primes) via the v7
KDF —
1421       // spot-check it does not equal the v7 HMAC key material shape.
1422        assert_ne!(sk.to_vec(), key_bytes(&primes));
1423    }
```

CVF-21: fixed Category: **documentation**

> **Summary**
> **The comments at the top of the Rust code describe an outdated version of the scheme, and every reference to the specification in the code points to an older document instead of the paper under review.**
> Developers rely on the comments in the code to understand what it does and where the rules it follows are written down. In this code, the introductory comment describes an older version of the scheme under a different product name, and every pointer to the specification refers to earlier documents that were not submitted for review, never to the current paper. This is how an important design element came to be documented in the code but missing from the paper (CVF-11). The code also contains leftover pieces of retired or unspecified data formats, and one declared module was not submitted at all, so it has not been reviewed. None of this changes how the code behaves, but it makes the code and the paper harder to keep consistent.

First, a note on a module that was not delivered. The list of module declarations ends with

```
pub mod vale;
```

and neither `src/vale.rs` nor a `src/vale/` directory is present in the delivered code. Three delivered files nonetheless expect the module: the declaration itself, the `[[bin]] name = "vale_demo"` target in `Cargo.toml`, which points at `src/bin/vale_demo.rs`, and that binary's own `use napqes::vale::{...}`. We therefore cannot tell whether the module is really missing from the client's source tree or whether the file was lost when the submission was assembled, and the evidence favours the second explanation. A tree without `vale.rs` could not have built the `vale_demo` binary that was delivered and declared, and this submission is known to have been extracted from a larger repository: both the Rust KAT loaders and the Python cross-language driver resolve paths relative to a `rust/` subdirectory and an enclosing repository root that are not present (CVF-25). We do not claim that the crate fails to compile in the client's own tree, and `src/vale.rs` is outside the scope of this review. We record only that the module was not submitted,

so nothing in it has been read, and whatever `pub mod vale` exposes is part of the crate's public API and has not been reviewed.

The file header describes an earlier version of the scheme:

> Rust port of `napqes.py` — v6 authenticated EpiCypher.
>
> Wire format (binary): nonce(16) || masked_blob || hmac_sha256_tag(32) where masked_blob = varint_blob XOR varint_keystream(key_bytes, nonce, len).

Three things in it are out of date:

- The version is given as v6, while the file implements v7 and v8.
- "EpiCypher" is a product name that appears nowhere in the paper, which calls the scheme NAPQES throughout.
- The serialisation is described as a "varint_blob" masked by a "varint_keystream". This is the LEB128 encoding that an earlier fix retired, and the code itself says that it is gone: the encoder "is removed since nothing in this crate produces LEB128 ciphertexts anymore". The blob now has a fixed width (`fixed_encode_tokens`), and only the function name `varint_keystream` still contains the old term.

The header also states that the derivations "match the reference Python implementation byte-for-byte, so ciphertexts are interoperable between languages when the same key, nonce, and AAD are used", which no longer describes v8, where the nonce is derived and not supplied. The delivered `README.md` repeats all three defects word for word, which is the subject of CVF-24.

More importantly, every pointer to the specification in the file cites a document other than this paper:

- `docs/napseq-eprint-v3.tex` is cited four times: for the prime interval in the notation section, for the section on noise probability, for the section on padding profiles, and for the padding-profile API.
- `docs/napseq-eprint-preprint.tex`, i.e. the original preprint, is cited as the specification of the V8 key schedule and synthetic nonce, which is the material that CVF-11 shows to be missing from the current paper.
- `docs/CAVEATS.md` is cited four times and `docs/SPEC.md` once.

None of these documents was delivered, and none of them is `napseq-eprint-v4.tex`. So the code's own account of where its specification is written points to documents two versions old, and a reader comparing the code with the paper finds no pointer to the paper in the source. This is how the problem of CVF-11 arose: the `0x0B` layer is documented, at length and accurately, in a comment that directs the reader to a superseded document instead of to the current specification, and as a result it is missing from the latter. The same pattern appears in the two delivered files that serve as evidence: `examples/dudect_harness.rs` cites "ROADMAP §5 NF-6" for its pass threshold, and `fuzz/fuzz_targets/decode_bytes.rs` describes itself as a v6 target. Neither reference can be checked against anything that was submitted.

Two leftover format elements complete the picture. First, `FORMAT_STREAM_AE_V8: u8 = 0x02` is declared `pub` next to the identifier of the block mode and is never used anywhere

in the crate, while `kat_cross_check.rs` filters test vectors on `api != "stream_ae"` in three places. The delivered test vectors show what those filters are for: `tests/kat/v6_vectors.json` contains six vectors with `api: "stream_ae"`, four positive and two negative, and they are the only vectors in either file that still use the retired LEB128 token encoding. Together, these confirm that the streaming AE format still exists in the family of implementations and in the test vectors. This is relevant to the caveat `CAV-001` about unauthenticated streaming, which v3 of the paper dropped together with the streaming format, and which the current paper neither addresses nor contradicts. The code and the test vectors both indicate that the format did not disappear when the caveat did.

Second, `b128_decode_tokens` remains in the code under `#[allow(dead_code)]`. It is a decoder for the retired LEB128 format, kept "only for potential legacy-format tooling", and the public API does not use it. Counting it, the v7 format and the streaming identifier, the crate contains three elements of data formats that the paper does not specify.

There is also one smaller item. The doc comment of `TOKEN_WIDTH` justifies the 8-byte field as "sized to comfortably hold the largest realistic token", which was the right level of precision before `Remark 3.11` gave an exact bound. The bound is now $2^{43.33}$, with a margin of more than 1.6×10^6, and stating it here would connect the constant to the guarantee that depends on it. See CVF-16, where the missing upper bound on key elements means that this margin is not enforced.

On the positive side, the doc comment of `MAX_KEY_PRIME` is accurate and up to date: it gives $|\mathcal{P}| = 579{,}947$ "verified by sieve", together with the derived figures $2^{191.46}$ and $2^{95.73}$, all of which match the exact values in the current paper.

Recommendation

For the `vale` module, the action concerns the delivery and not the code: include `src/vale.rs` in the next submission, or confirm that no such file exists in the source tree, so that the question is settled instead of inferred. Since `pub mod vale` is a public module of the audited crate and `vale_demo` is a declared binary target, whatever the module exposes is part of the crate's public API and has not been reviewed so far.

Point every specification reference at the current paper. Replace the citations of `docs/napseq-eprint-v3.tex` and `docs/napseq-eprint-preprint.tex` with `napseq-eprint-v4.tex` and, where the cited material has moved, with the current section or environment label. The reference at line 1042 is the most important one: it directs the reader to the original preprint for the V8 key schedule, and once CVF-11 is fixed it should name the new `0x0B` definition in the paper instead. Consider adding a single `//! Specification:` line to the module header that names the paper, its version and its date, so that the correspondence is stated once instead of being inferred from eleven scattered citations. Either deliver `docs/CAVEATS.md` and `docs/SPEC.md` together with the code, or replace the references to them with references to the paper, since a comment citing a document that was not delivered cannot be checked. Do the same for `ROADMAP §5 NF-6` in the dudect harness (CVF-26).

Rewrite the module header so that it describes what the file implements. It should use the name NAPQES instead of EpiCypher, state that the file contains two wire formats and which of them the paper specifies, describe the blob as fixed-width instead of varint, and remove or qualify the sentence about byte-for-byte interoperability, which

holds for v7 with a supplied nonce but not for v8. Fix `README.md` in the same change, since it has the same three defects; CVF-24 covers the rest of it.

Decide explicitly what to do with the two leftover format elements instead of leaving them in the code as constants. If the streaming AE format is still supported, it needs a specification, and the caveat `CAV-001` that v3 of the paper dropped needs to be restored or replaced, since the format it applied to has clearly outlived the caveat's removal: in the code as a public constant, and in the test vectors as six live vectors. If the format is not supported, remove `FORMAT_STREAM_AE_V8` and the three `stream_ae` filters, and remove the corresponding vectors from the test-vector file. Similarly, delete `b128_decode_tokens` unless a named tool uses it. An unused decoder for a retired format is a liability in a cryptographic crate, and `#[allow(dead_code)]` suppresses the warning that would otherwise prompt its removal.

Finally, justify `TOKEN_WIDTH` in terms of the bound of `Remark 3.11` instead of "the largest realistic token", and add a cross-reference to the upper bound on key elements that the guarantee requires.

/Cargo.toml

```
[[bin]]
name = "vale_demo"
path = "src/bin/vale_demo.rs"
```

/README.md

```
# napqes (Rust)

Rust port of the v6 authenticated EpiCypher in `../napqes.py`.
```

same file

```
Wire format (interoperable with Python):

```
nonce(16) || varint_blob || hmac_sha256_tag(32)
```
```

/src/kat_cross_check.rs

```
        .filter(|v| v["kind"] == "positive" && v["api"] !=
"stream_ae")
```

same file

```
        .filter(|v| v["kind"] == "positive" && v["api"] !=
"stream_ae")
```

same file

```
        .filter(|v| v["kind"] == "negative" && v["api"] !=
"stream_ae")
```

/src/lib.rs

```
//! Rust port of `napqes.py` — v6 authenticated EpiCypher.
//!
```

```
//! Wire format (binary): nonce(16) || masked_blob || hmac_sha256_tag(32)
//! where masked_blob = varint_blob XOR varint_keystream(key_bytes, nonce, len).
//! String wrapper: base64(binary).
//!
//! All HMAC derivations (noise positions, real-token addends, noise chars,
//! noise-token addends, noise probability, padding codepoints, varint
//! keystream, and auth tag) match the reference Python implementation
//! byte-for-byte, so ciphertexts are interoperable between languages when
//! the same key, nonce, and AAD are used.
```

same file

```
pub mod vale;
```

same file

```
/// `docs/napseq-eprint-v3.tex` §Notation. `P` contains exactly 579_947
```

same file

```
/// integers (docs/napseq-eprint-v3.tex, Section "Noise Probability").
```

same file

```
/// (docs/napseq-eprint-v3.tex, Section "Padding Profiles").
```

same file

```
// ─── Base-128 varint (retired for new ciphertexts — see CVF1 fix below) ──────
// The variable-length LEB128 encoding gave each token a byte-length that grew
// with its magnitude (token = codepoint * key_element + addend), so the
// serialised blob length leaked plaintext content even between messages of
// equal padded length, breaking the IND-CPA hiding argument (audit finding
// CVF1). `encrypt_bytes`/`encrypt_raw` now use `fixed_encode_tokens` instead.
// The encoder is removed since nothing in this crate produces LEB128
// ciphertexts anymore; the decoder is kept only for potential legacy-format
// tooling and is currently unused by the public API.
```

same file

```
#[allow(dead_code)]
fn b128_decode_tokens(data: &[u8]) -> Result<Vec<u64>, String> {
```

same file

```
/// Width in bytes of each fixed-width token field, sized to comfortably hold
/// the largest realistic token (codepoint * key_element + addend).
const TOKEN_WIDTH: usize = 8;
```

same file

```
// See `docs/napseq-eprint-preprint.tex` (new subsection, "V8 Key Schedule
// and Synthetic Nonce") and `docs/CAVEATS.md` (CVF3/CVF8/CVF13 follow-ups)
// for the full specification and updated security argument.
```

same file

```
pub const FORMAT_BLOCK_V8: u8 = 0x01;
/// Domain `0x0B` format-subkey identifier for the v8 streaming-AE format.
pub const FORMAT_STREAM_AE_V8: u8 = 0x02;
```

same file

```
/// (docs/napseq-eprint-v3.tex, Section "Padding Profiles").
```

/tests/kat/v6_vectors.json

```
  "spec_version": "v6",
```

CVF-26: open Category: **procedural**

Summary

The paper's timing-safety claim for the Rust implementation is not backed by any reported measurement, and the delivered timing test is too coarse to detect the leak it looks for.

The paper claims that comparing authentication tags takes the same time whatever the tag contains, so that an attacker cannot learn anything by timing it. As evidence it cites a timing study of the C implementation in a document that was not delivered. The delivery instead contains a timing test for the Rust implementation, which the paper does not mention and whose results are not recorded anywhere. That test is carefully designed, but it times a much larger operation than the comparison itself, so a small timing leak would be lost in the noise. It also does not cover any of the operations that the paper admits are not constant-time, and it tests the older data format.

Section 8.4 ends its treatment of timing with:

> A dudect-style timing study of the C port is reported in `docs/DUDECT_ATTESTATION.md`; it covers the tag comparison only.

The delivered artifact is something else. `examples/dudect_harness.rs` is a Rust harness for `napqes::decrypt_bytes`, and the paper does not mention a Rust timing study anywhere; `docs/DUDECT_ATTESTATION.md` has not been delivered. The paper therefore cites a measurement of one implementation that we cannot see, while the delivery contains a harness for another implementation that the paper does not cite and whose results are not recorded anywhere: there is no t-statistic, no sample count, no platform and no attestation. Since the Rust comparator is the one written by hand (CVF-12), the implementation whose constant-time property most needs measuring is the one with no recorded measurement.

The harness itself is well constructed. `Class::Left` flips the first tag byte and `Class::Right` flips the last one, so a comparator with an early exit would return after one byte in one class and after thirty-two in the other, which is the right way to distinguish the two cases for the property being tested. Both classes fail authentication, so `decrypt_bytes` returns at the same `return Err` immediately after `ct_eq_bytes`, and the two measurement windows are structurally identical up to the comparison. The class is drawn before `run_one`, and the doc comment explicitly discusses the memory-layout confounding factor, the bias from the order of measurements within a call and the branch-predictor effect of the branch that selects the class. This is more careful than most such harnesses.

Nevertheless, the harness has serious limits, and none of them is stated.

First, most of the measured time is spent on work that is not under test. `run_one` wraps the whole `decrypt_bytes` call, which, before reaching the comparison, performs `key_bytes(key)` and then `compute_auth_tag`, i.e. an HMAC-SHA256 over the whole payload. The base ciphertext is produced by `encrypt_bytes` from a 42-character message

under a ten-element key, and the v7 path applies no token ceiling, so the payload is several kilobytes long. The signal that a comparator with an early exit would produce is about thirty-one byte comparisons, i.e. tens of cycles, inside a window of tens of thousands of cycles that has its own variance. The harness's stated criterion, $|t| < 4.5$, can therefore be met by a comparator that really does leak, at any realistic number of samples, so a result showing no leak is weak evidence. The note that batches below 200 k are noisy and that 2 M measurements should be accumulated addresses the sample size. The problem here, however, is the signal-to-noise ratio, which more samples improve only as $\sqrt{n}$, against a handicap of three or four orders of magnitude.

Second, the harness cannot reach any of the behaviours that Section 8.4 admits are not constant-time. That section's "What is not" list has five items: loop iteration counts that vary with the actual noise pattern, division by a key element k whose latency depends on the data, a padding loop whose cost depends on the exact codepoint count n, the absence of any guarantee in the Python reference, and the structural checks at steps (6) and (8) of `Dec`, which "reject at different points and in different time". Because both of the harness's classes fail authentication, the call returns before `varint_keystream`, `fixed_decode_tokens` and `decrypt_core` are entered, so no token inversion and no division by k ever happens inside the measurement window. The padding loop is on the encryption side, which the harness does not time at all. The harness thus measures exactly the one operation that the paper already claims is constant-time, and none of the five that it admits are not.

Third, the harness targets the legacy format. `decrypt_bytes` is the v7 entry point. `decrypt_bytes_v8` calls the same `ct_eq_bytes`, so the result for the comparator carries over, but the v8 path is the one whose checks after authentication are the fifth item of Section 8.4, and none of them is measured. CVF-22 records the same problem for the fuzzing harness: both supporting test tools in this submission target the wire format that the paper excludes, and neither exercises the one it specifies.

Fourth, the harness measures a build in which the comparator's only length check is absent. The harness requires `--release`, and since the manifest has no `[profile]` section, that build has `debug-assertions = false` (CVF-23), so the `debug_assert_eq!(a.len(), b.len())` in `ct_eq_bytes` is not in the binary being measured. That is the right build to measure, but the consequence should be recorded: the harness confirms the timing behaviour of the comparator for operands of equal length, while its behaviour for operands of different lengths is unchecked and, as CVF-12 shows, fails open, i.e. returns `true` when the shorter operand matches a prefix of the longer one.

There are two smaller points. The threshold is attributed to "ROADMAP §5 NF-6", another document that was not delivered, which is the kind of unresolvable reference that CVF-21 records throughout the crate. Also, the harness is an `example`, not a test or a benchmark, so no build depends on $|t| < 4.5$: the criterion exists only in a comment and is enforced only when someone remembers to run the harness.

Recommendation

Narrow the measurement window to the operation under test. The property being checked is that the time of the comparison does not depend on the position of the first differing byte, and that is a property of thirty-two bytes of work; timing an HMAC over several kilobytes around it loses most of the resolution. Expose the comparator

to the harness behind a gate used only for testing, and measure it directly:

```
#[doc(hidden)]
#[cfg(feature = "ct-bench")]
pub fn __ct_eq_bytes_for_bench(a: &[u8], b: &[u8]) -> bool {
  ct_eq_bytes(a, b) }
```

with the two classes differing only in which byte of a 32-byte operand is flipped. You may keep the existing end-to-end harness as a second, coarser check, since it does test that nothing else in `decrypt_bytes` leaks the position in the tag, but the main measurement should be the one that contains the signal.

Report a result, since a harness with no recorded result is not evidence. Publish, for each pair of classes, the platform and CPU, the compiler and its flags, the number of samples, the observed t and the decision against the $|t| < 4.5$ threshold, and cite these in Section 8.4 instead of, or together with, the C attestation. Deliver `docs/DUDECT_ATTESTATION.md` as well: the paper's only timing claim rests on that document, and it is not in the submission.

Correct Section 8.4 to describe what exists. It should say that a Rust dudect harness covering the tag comparison is provided, give its result, and state that the C study is separate. As written, a reader concludes that only the C implementation was measured and that the comparator of the Rust implementation, which the same section wrongly attributes to `subtle` (CVF-12), needed no study. Also state the harness's scope explicitly: it covers the tag comparison on the v7 decryption path; both classes fail authentication, so the token inversion and the division behaviour of the "What is not" list are outside the measurement; and timing on the encryption side, including the padding loop, is not measured at all.

Add a v8 harness, or explain why the v7 one is enough. The comparator is shared, so a short sentence explaining that the result for the tag comparison carries over would be enough for the comparator. However, the structural checks at steps (6) and (8) of `Dec` exist only in v8, are new in v4, and are named by Section 8.4 as rejecting input in different amounts of time. If the paper relies on the argument that these differences do not give an outside attacker a usable signal, it should make that argument where the harness's scope is stated, instead of leaving it to the reader.

Move the harness to a place where a build can enforce it. As a `#[bench]`, or as an example run by a CI job that fails on $|t| \geq 4.5$, the threshold becomes a gate; as an example with the threshold in a comment, it is only documentation. Given the crate's FIPS framing, a change that makes the comparator leak is exactly the kind of change that should not pass unnoticed.

Finally, replace the "ROADMAP §5 NF-6" reference with a reference to the paper's own Section 8.4, or deliver the ROADMAP, together with the cleanup of references that CVF-21 recommends.

/examples/dudect_harness.rs

```
1 //! Constant-time timing analysis for `napqes::decrypt_bytes`.
```

same file

```
3 //! Run (release mode required for meaningful timings):
```

```
//!   cargo run --example dudect_harness --release -- --continuous
bench_tag_comparison
```

same file

```
//! The test compares decryption timing when the HMAC-SHA256 tag
differs at the
//! FIRST byte (Class::Left) versus the LAST byte (Class::Right).  A
naive
//! byte-by-byte comparator would exit early on Left (faster),
producing a
//! detectable t-statistic.  A correct constant-time implementation
scans all 32
//! bytes unconditionally, so both classes should be
timing-indistinguishable.
```

same file

```
//! Target: |t| < 4.5 (TVLA threshold per ROADMAP §5 NF-6).
```

same file

```
//! Note: allow the benchmark to accumulate at least 2 M measurements
before
//! drawing conclusions.  Early batches (n < 200 k) are noisy.
```

same file

```
use napqes::{decrypt_bytes, encrypt_bytes, TAG_SIZE};
```

same file

```
/// A single valid ciphertext produced once at startup.  Both classes
derive
/// from this by cloning (allocator reuses the same heap slot between
calls)
/// and corrupting one tag byte in-place.
fn base_ct() -> &'static Vec<u8> {
```

same file

```
        encrypt_bytes(
            "napqes dudect constant-time timing harness",
            fixed_key(),
            b"",
        )
```

same file

```
fn bench_tag_comparison(runner: &mut CtRunner, rng: &mut BenchRng) {
    let key = fixed_key();
    let base = base_ct();
    let n = base.len();
```

same file

```
    let mut ct = base.clone();
```

same file

```
    if rng.gen::<bool>() {
        // Left: tag byte 0 wrong (early-exit comparator fails immediately)
        ct[n - TAG_SIZE] ^= 0xFF;
        runner.run_one(Class::Left, || decrypt_bytes(&ct, key, b""));
```

same file

```
    } else {
        // Right: tag byte 31 wrong (early-exit comparator scans all 31 bytes)
        ct[n - 1] ^= 0xFF;
       runner.run_one(Class::Right, || decrypt_bytes(&ct, key, b""));
    }
```

/src/lib.rs

```
#[inline(never)]
fn ct_eq_bytes(a: &[u8], b: &[u8]) -> bool {
    debug_assert_eq!(a.len(), b.len());
```

same file

```
    if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
        return Err("Authentication failed: invalid HMAC tag.".into());
    }
```

CVF-29: open Category: **flaw**

Summary

The code checks that every secret key element is prime again on every encryption and decryption, and the time this takes reveals information about the secret key, which the paper's list of timing leaks does not mention.

Each time a message is encrypted or decrypted, the code checks again that every element of the secret key is a prime number. The time this check takes grows with the size of each secret prime, so an attacker who can measure how long operations take learns something about the key with every message. The paper carefully lists the operations whose timing varies, but this one is missing from the list. The practical impact is limited, because the part of the key that leaks is not the one on which the security proofs rely, but the paper's list should be complete, and the repeated check is unnecessary work in any case.

`validate_key` runs on **every** call to `encrypt_bytes_v8_with_profile` and `decrypt_bytes_v8`, and for each key element it calls `is_prime`, which uses trial division:

```
let mut i: u64 = 3;
while i.saturating_mul(i) <= n {
    if n % i == 0 { return false; }
    i += 2;
}
```

For a real prime the loop never returns early, so it runs to completion, and the number of iterations is $\lfloor(\lfloor\sqrt{k}\rfloor - 3)/2\rfloor + 1$, a direct and monotone function of the secret key element. Across the normative interval, this is **499 iterations at** $k = 1{,}000{,}003$ **and 1,572 at** $k = 9{,}899{,}999$, a spread of $3.15\times$, and at the default $K = 10$ the total ranges from roughly **4,990 to 15,720** 64-bit division instructions per message. An attacker who can time encryption or decryption therefore learns approximately $\sum_i \sqrt{k_i}$ once per message, at no cost: no manipulation of ciphertexts is needed, the leak occurs on the encryption side as well as on the decryption side, and it is independent of everything the noise loop does.

The difference is with **Section 8.4**, not with Section 3. The paragraph "What is not constant-time" in Section 8.4 is an explicit list, and it is careful and honest as far as it goes. It lists the iteration counts of the token-emission and token-inversion loops, the integer division by a key element, the dependence of the padding loop on the exact codepoint count, the Python implementation, and the different rejection points of steps (6) and (8) of `Dec`. **Key validation is not among these five items**, and it is not in the timing caveat of Section 12 either. The paper's list of operations that are not constant-time is therefore incomplete in the one place where the leaked quantity is key material, as opposed to a public value or a value derived from the nonce. Every item that the paper does list leaks something about (sk, N) or about n, while this one leaks **k** directly.

The security consequence is limited. Recovering $\lfloor\sqrt{k_i}\rfloor$ for each element would reduce $H_\infty(\mathbf{k})$ from 191.46 bits to roughly the logarithm of the number of primes per $\sqrt{\cdot}$ bucket,

summed over the tuple, which is a large reduction. However, Remark 3.3 and Section 6 state explicitly that no theorem depends on $H_\infty(\mathbf{k})$, and Section 12 acknowledges that the secrecy of **k** "adds **no** independent confidentiality margin once sk is known". The leak does not affect sk. This is therefore mainly a gap in the documentation, on top of a timing leak that is real but not catastrophic.

The repeated check is also unnecessary. Checking again, on every single message, that a fixed key supplied by the caller consists of primes brings no benefit: the property does not change during the lifetime of the key, and the check costs between five thousand and sixteen thousand divisions per message to confirm something that was already true.

CVF-16 recommends the right remedy for a different reason: it asks for validation to be moved off the per-message path into a validated key type, because trial division of a large prime supplied by the caller can be used for denial of service. Neither CVF-16 nor any other finding identifies the timing leak or the omission in Section 8.4. The correction on the paper side (extending the list in Section 8.4) is also separate from the correction on the code side, which is why this finding is reported separately.

Recommendation

Validate the key once, when it is constructed, and record the timing leak in Section 8.4 until then.

On the code side, introduce a validated key type, so that primality is established when the key enters the system and never again:

```
pub struct NapqesKey(Vec<u64>);

impl NapqesKey {
    pub fn new(primes: Vec<u64>) -> Result<Self, String> {
        validate_key(&primes)?;
        Ok(NapqesKey(primes))
    }
}
```

with `encrypt_bytes_v8*` and `decrypt_bytes_v8` taking `&NapqesKey` instead of `&[u64]`. This removes the trial division from every message, which closes the timing leak and removes the cost per message, and it is the same change that CVF-16 asks for to prevent denial of service, so one refactoring resolves both findings. If the API cannot change yet, at least move `validate_key` out of the per-message path inside the crate, and document that callers must validate the key once.

Note that a constant-time primality test is *not* the right fix here and should not be attempted. The solution is to stop repeating the test, not to make it constant-time: even the cheapest constant-time primality test is far more expensive than performing the check once, and a Miller-Rabin variant would add a probabilistic failure mode to what is currently an exact, deterministic check.

On the paper side, add a sixth item to the list "What is not constant-time" in Section 8.4, in the same style as the existing five: key validation performs trial division up to $\sqrt{k}$ for each key element on every encryption and decryption call, so the number of iterations is a monotone function of each secret prime, and an attacker who can measure timing learns approximately $\sum_i \sqrt{k_i}$ per message. Add the context that the

paper already establishes elsewhere and that limits the impact, i.e. that no theorem depends on $H_\infty(\mathbf{k})$ and that Section 12 acknowledges that **k** adds no confidentiality margin once sk is known, so that a reader can judge the size of the leak. The list is otherwise the most candid part of the paper, and this is the only item missing from it that we found. If the change to a validated key type is made first, the item can instead say that key validation is performed once, when the key is constructed, and is therefore not repeated for each message.

/docs/napseq-eprint-v4.tex

```
\paragraph{What is not.} The token-emission and token-inversion loops are
\emph{not} constant-time, and are not intended to be:
\begin{itemize}
```

same file

```
  \item The number of loop iterations depends on the realised noise
        pattern, which is a deterministic function of $(\mathsf{sk},N)$.
        The total ciphertext length is padded to a fixed ceiling
        (Lemma~\ref{lem:length-det}) and so leaks nothing, but the
        \emph{time} to produce or consume it varies with the number of
        noise tokens actually emitted before the run cap engages.
```

same file

```
  \item Both directions use arbitrary-precision or 64-bit integer
        multiplication and division by a key element $k$. Division latency
        is data-dependent on many microarchitectures.
```

same file

```
  \item The padding loop of Section~\ref{sec:padding} performs $B-n$
        derivations, so encryption time depends on the \emph{exact}
        codepoint count $n$, not merely on its padding bucket $B$. The
        padding-bucket caveat below concedes a $\log_2 13\approx3.70$-bit
        length leak through ciphertext \emph{length}; a timing channel can
        in principle recover more than that.
```

same file

```
  \item The Python reference implementation offers no timing guarantees of
        any kind: it uses \texttt{int}, list resizing, and dictionary
        lookups throughout.
```

same file

```
  \item The structural checks added at $\mathsf{Dec}$ steps (6) and (8)
```

```
        (Remark~\ref{rem:dec-structural}) reject at different points and in
        different time. They are reachable only after the tag has verified,
        i.e.\ only by a party already holding $\mathsf{sk}$, so this
        difference is not an oracle for an outside adversary.
```

same file

```
        noise-dependent number of times and perform integer division by a
        key element, both of which are data-dependent on real hardware, and
        the Python reference implementation offers no timing guarantees at
        all. Section~\ref{sec:constant-time} states precisely what is and is
        not covered. Deployments in which an adversary can measure
        encryption or decryption time on inputs it influences should treat
        this as an unresolved gap; a TVLA or dudect study of the full
        encode/decode path, rather than of the tag comparison alone, remains
        future work.
```

/src/lib.rs

```rust
pub fn is_prime(n: u64) -> bool {
    if n < 2 {
        return false;
    }
    if n == 2 {
        return true;
    }
    if n % 2 == 0 {
        return false;
    }
```

same file

```rust
    let mut i: u64 = 3;
    while i.saturating_mul(i) <= n {
        if n % i == 0 {
            return false;
        }
        i += 2;
    }
    true
}
```

same file

```
229 fn validate_key(key: &[u64]) -> Result<(), String> {
230     if key.is_empty() {
231         return Err("Key must be a non-empty list of primes.".into());
232     }
233     for (i, &k) in key.iter().enumerate() {
234         if !is_prime(k) {
235             return Err(format!("Key element at index {} ({}) is not
prime.", i, k));
236         }
```

same file

```
1149     validate_key(primes)?;
```

same file

```
1212     validate_key(primes)?;
```

CVF-30: fixed Category: **behavior**

> **Summary**
>
> **The paper states that the scheme uses no floating-point arithmetic, but the legacy encryption functions do, so two builds of the same software can disagree and fail to decrypt each other's messages.**
>
> The paper promises that one key calculation is done with whole numbers only, so that every implementation on every platform gets exactly the same result. The current (v8) code keeps this promise. The older (v7) functions, which are still public, perform the same calculation with floating-point numbers, whose results can differ slightly between computers and compiler settings. This does not weaken security, but a message encrypted by one build may fail to decrypt in another.

Section 3.4 opens the definition of the noise probability with an unconditional sentence: "The noise probability is realised entirely with fixed-width integer arithmetic; no floating-point representation is used anywhere in this construction." The paragraph then explains why this matters: $\theta(N)$ is computed "with a single 128-bit intermediate product and one integer division, so that $\theta(N)$ is bit-identical across all conforming implementations regardless of platform or language."

The v8 path follows this exactly. `derive_noise_threshold_v8` widens to `u128`, multiplies, shifts right by 64 and narrows the result, and `is_noise_pos_v8` compares the first eight digest bytes, as a `u64`, with the resulting `u64` threshold. We checked the arithmetic from end to end: $\theta_{\text{min}} = 13{,}835{,}058{,}055{,}282{,}163{,}712$ (exactly $3 \cdot 2^{62}$, so the floor is

exact) and $\theta_{\max} = 18{,}262{,}276{,}632{,}972{,}456{,}099$ are both below 2^{64}, so the `as u64` casts lose nothing; $\tau \cdot \Delta < 2^{128}$, so the product cannot overflow; and `>> 64` is exactly the specified floor division. The v8 path therefore matches the specification.

The v7 path works in a completely different way:

```
const TWO_POW_64: f64 = 18446744073709551616.0_f64;

fn derive_noise_p(kb: &[u8], nonce: &[u8]) -> f64 {
    ...
    let t = u64_from_be8(&d[..8]) as f64 / TWO_POW_64;
    0.75 + t * (0.99 - 0.75)
}

fn is_noise_pos(kb: &[u8], nonce: &[u8], ct_pos: u64, noise_p: f64)
  -> bool {
    ...
    let v = u64_from_be8(&d[..8]) as f64 / TWO_POW_64;
    v < noise_p
}
```

The threshold is an `f64`, the value compared with it is an `f64`, and the comparison is a floating-point comparison. This code is used by every v7 entry point: `encrypt`, `encrypt_bytes`, `encrypt_bytes_with_nonce`, `encrypt_raw` and `decrypt_core`.

Two mechanisms make the result depend on the implementation, in addition to being imprecise. First, the `u64 as f64` conversion discards the low 11 bits of a 64-bit value, so roughly one draw in 2^{53} that should fall on one side of the threshold falls on the other. Second, `0.99 - 0.75` is a subtraction of two values that are not both exactly representable in binary (the literal `0.75` is exact, but `0.99` is not), evaluated at whatever precision and in whatever order the compiler chooses, and the fused multiply-add in `0.75 + t * (0.99 - 0.75)` may or may not be contracted, depending on the target features. This is exactly the kind of platform dependence that the sentence in Section 3.4 rules out.

The consequence concerns interoperability, not security. The threshold is secret and the noise positions are masked by the keystream, so no observable quantity depends on the floating-point result. However, two conforming v7 implementations built for different targets, or with different optimisation settings, can disagree about whether a given position is a noise position. Since the v7 decoder classifies every position independently with the same predicate, such a disagreement produces a ciphertext that does not decrypt correctly. This is exactly the reproducibility defect that the doc comment above `derive_noise_threshold_v8` describes as removed: it was removed for v8, but left in place for v7.

The sentence in Section 3.4 is unqualified and refers to "this construction". The paper's Π is v8, and Section 10 does exclude the v7 test vectors as "not the scheme analysed here", so a generous reading limits the sentence to v8. However, the crate ships the floating-point path as a public API that is not deprecated, and a reader who takes the sentence literally and then reads the crate finds it contradicted at the first v7 entry point.

This finding is separate from CVF-17, which concerns the missing cap on noise runs and the missing token ceiling in the same v7 loop, and from CVF-28, which concerns the v7 key derivation and the width of the length prefix of the associated data. The arithmetic used to compute the threshold is a third, independent difference on that path.

Recommendation

Compute the v7 threshold with integers, or retire the v7 encryption functions.

If the v7 encryption functions are withdrawn as part of the fixes for CVF-17 or CVF-28, this finding is resolved with them and no separate work is needed. Note only that `decrypt_core` must keep the arithmetic under which archived v7 ciphertexts were produced, so a v7 *decryption* function that is kept cannot simply switch to the integer path without breaking the very ciphertexts it exists to read.

If v7 stays, replace `derive_noise_p` and `is_noise_pos` with the v8 pair. `derive_noise_threshold_v8` and `is_noise_pos_v8` already take the derivation key as a parameter, so they can be called with `kb = key_bytes(key)` unchanged; the only difference between the two paths would then be the derivation key, which is where the difference belongs. Be explicit that this changes the v7 wire format: the set of noise positions changes for a small fraction of positions, so existing v7 ciphertexts will not decrypt correctly under the new predicate. Give the change a new version number instead of changing the behaviour silently.

Whichever option is chosen, qualify the sentence in Section 3.4. "No floating-point representation is used anywhere in this construction" should say what "this construction" means when the reference crate exports two of them, for example by naming Π explicitly: the scheme specified in this paper realises the noise probability entirely in fixed-width integer arithmetic. The reasoning in the surrounding paragraph, i.e. that the result is bit-identical across platforms and languages, is the purpose of the requirement and is worth keeping prominent, because it is what a second implementer will rely on.

Finally, if any floating-point arithmetic remains on any derivation path, add it to the list in Section 8.4 together with the other notes on how the implementations behave, so that the paper's description of what the implementations actually do remains complete.

/docs/napseq-eprint-v4.tex

```
The noise probability is realised entirely with fixed-width integer
arithmetic; no floating-point representation is used anywhere in this
construction. Fix the public integer constants
```

same file

```
The per-message noise threshold is the 64-bit integer
\[
  \theta(N) = \theta_{\min} + \left\lfloor \frac{\tau \cdot
(\theta_{\max} - \theta_{\min})}{2^{64}} \right\rfloor,
\]
computed with a single 128-bit intermediate product and one integer
division, so that $\theta(N)$ is bit-identical across all conforming
implementations regardless of platform or language. Every informal
```

/src/lib.rs

```
const TWO_POW_64: f64 = 18446744073709551616.0_f64;
```

same file

```
fn is_noise_pos(kb: &[u8], nonce: &[u8], ct_pos: u64, noise_p: f64)
-> bool {
    let mut buf = Vec::with_capacity(1 + nonce.len() + 5);
    buf.push(0x00);
    buf.extend_from_slice(nonce);
    buf.extend_from_slice(&be5(ct_pos));
    let d = hmac_digest(kb, &buf);
    let v = u64_from_be8(&d[..8]) as f64 / TWO_POW_64;
    v < noise_p
}
```

same file

```
fn derive_noise_p(kb: &[u8], nonce: &[u8]) -> f64 {
    let mut buf = Vec::with_capacity(1 + nonce.len());
    buf.push(0x02);
    buf.extend_from_slice(nonce);
    let d = hmac_digest(kb, &buf);
    let t = u64_from_be8(&d[..8]) as f64 / TWO_POW_64;
    0.75 + t * (0.99 - 0.75)
}
```

same file

```
fn derive_noise_threshold_v8(kb: &[u8], nonce: &[u8]) -> u64 {
    let mut buf = Vec::with_capacity(1 + nonce.len());
    buf.push(0x02);
    buf.extend_from_slice(nonce);
    let d = hmac_digest(kb, &buf);
    let tau = u64_from_be8(&d[..8]) as u128;
    THETA_MIN + ((tau * (THETA_MAX - THETA_MIN) as u128) >> 64) as u64
}
```

same file

```
fn is_noise_pos_v8(kb: &[u8], nonce: &[u8], ct_pos: u64, theta: u64)
-> bool {
    let mut buf = Vec::with_capacity(1 + nonce.len() + 5);
    buf.push(0x00);
    buf.extend_from_slice(nonce);
    buf.extend_from_slice(&be5(ct_pos));
    let d = hmac_digest(kb, &buf);
    u64_from_be8(&d[..8]) < theta
}
```

same file

```
471     let noise_p = derive_noise_p(&kb, &nonce);
```

same file

```
505     let noise_p = derive_noise_p(kb, nonce);
```

same file

```
737     let noise_p = derive_noise_p(&kb, &nonce);
```

same file

```
795     let noise_p = derive_noise_p(&kb, &nonce);
```

same file

```
893     let noise_p = derive_noise_p(&kb, &nonce);
```

CVF-31: fixed Category: **over-underflow**

Summary

A public padding function can return block sizes outside the documented set, and the check that would stop a block smaller than the message is removed in release builds.

The library pads every message to one of a fixed set of sizes, and both the specification and the code's own documentation promise that the size always comes from that set. One public function that computes the size does not enforce the upper end of the set, and for extremely large inputs it can even return a size smaller than the message. The internal code that uses this value checks it only in test builds, not in the builds that are shipped. None of this can be triggered through the library's encryption functions today, because they reject long messages first, so the risk is limited to other software that calls the size function directly and trusts its documentation.

Section 3.6 promises that all padding profiles share one set of block sizes: "All three profiles take values in the same 13-element set, so the set of legal real-token counts $R = B + 2$ — and hence the set of legal ciphertext lengths — is identical for all of them." This is what allows a decryptor to work without knowing the sender's profile and to check R against one fixed set. The crate restates it as an invariant in the doc comment on `PAD_MIN_EXP`/`PAD_MAX_EXP`: "Every padding profile takes values in this same 13-element set, so the set of legal token counts is profile-independent and a decryptor never needs to know which profile the sender used."

`PadProfile::block_size` is `pub`, but it enforces only the lower end of that set:

```
let e = bit_length(n).max(PAD_MIN_EXP);
```

```
match self {
    PadProfile::Bucket   => Ok(bucket_block_size(n)),          // 1 <<
  bit_length(n).max(4)
    PadProfile::Coarse(g) => { ...; Ok(1usize << (PAD_MIN_EXP + g *
  steps)) }
    PadProfile::Frame(f)  => { /* range-checks both ends */ }
}
```

Neither `Bucket` nor `Coarse` is capped at `PAD_MAX_EXP`. `PadProfile::Bucket.block_size(70_000)` and `PadProfile::Coarse(1).block_size(70_000)` both return `Ok(131_072)`, i.e. 2^{17}, which is outside $\{2^4, \ldots, 2^{16}\}$ and would give $R = 131{,}074$, a real-token count that no conforming decryptor accepts at `Dec` step (6). Only `Frame(F)` checks both ends of the range.

The shift itself is not guarded either. `bit_length(n)` is `usize::BITS - n.leading_zeros()`, so for $n \geq 2^{63}$ it is 64, and `1usize << 64` is a shift overflow. It panics in a debug build. In a release build with `overflow-checks` off, which is the delivered configuration since `Cargo.toml` declares no `[profile.release]` section, the shift amount is masked modulo 64 and the expression evaluates to `1`. That gives a block size *smaller* than the message, which violates the one precondition that Section 3.6 places on a profile, "$B(n) > n$ throughout the message space".

The function that uses this value assumes that the precondition holds, and enforces it with an assertion that is removed in release builds:

```
fn pad_to_block(msg: &[u32], kb: &[u8], nonce: &[u8], block_size:
  usize) -> Vec<u32> {
    let n = msg.len();
    debug_assert!(block_size > n, "padding block must exceed the
  message");
    let pad_len = block_size - n;
    ...
    for i in 0..pad_len {
```

In a release build, `debug_assert!` is compiled out and, with no `overflow-checks`, the `usize` subtraction wraps around to a value near 2^{64}. The following `for i in 0..pad_len` loop then attempts to fill that many positions with codepoints derived from HMAC, with one `Vec` push and one HMAC-SHA256 computation per iteration.

Neither problem can be reached today, which is why the severity is low. `pad_to_block` is private, so no downstream crate can pass it a bad `block_size` directly. The only public route is `encrypt_bytes_v8_with_profile`, which rejects `codepoints.len() > 0xFFFF` *before* it calls `pad_profile.block_size(...)`. Therefore `n` never exceeds 65,535, and all three profiles then return a B in the legal set with $B > n$. The v7 path calls `bucket_block_size` through `pad_message`, which has its own `assert!(n <= 0xFFFF)`. So this is a latent defect: it is present in the code, but it cannot currently be triggered.

We report it because `PadProfile::block_size` is a public function whose doc comment tells a caller that the result is confined to a 13-element set, and this is not the case.

A downstream crate that relies on that invariant, for example to size a buffer, to precompute the set of legal R values, or to validate a configuration before passing it to `encrypt_bytes_v8_with_profile`, relies on something that the function does not guarantee, although the paper makes the same guarantee. The two problems add up: the public function can produce a B that is out of range or too small, and the only protection of the internal function against a too-small B disappears in the build configuration that is shipped.

This finding shares a root cause with CVF-16 and CVF-23: the manifest does not enable `overflow-checks`, so an arithmetic mistake wraps around instead of aborting the program. However, the location and the mechanism are different. CVF-16 is an unbounded multiplication in the token map, caused by a missing bound in key validation, while this finding is an unbounded subtraction in the padding code, caused by a missing upper limit in the padding profiles.

Recommendation

Enforce the invariant in the function whose documentation declares it, and replace the debug-only precondition check with one that also runs in release builds.

Add an upper limit that applies to both profiles, so that `block_size` cannot return a value outside the 13-element set:

```
pub fn block_size(self, n: usize) -> Result<usize, String> {
    if n >= 1usize << PAD_MAX_EXP {
        return Err(format!(
            "message of {} codepoints exceeds the largest reachable
  block size {}.",
            n, 1usize << PAD_MAX_EXP
        ));
    }
    let e = bit_length(n).max(PAD_MIN_EXP);
    ...
}
```

The early return also removes the `1usize << 64` shift overflow, because `bit_length(n)` can no longer exceed `PAD_MAX_EXP` when either shift is computed. A single check at the start of `block_size` covers both `Bucket` and `Coarse`, and is consistent with what `Frame` already does for its own parameter. The check also makes the `codepoints.len() > 0xFFFF` check in `encrypt_bytes_v8_with_profile` redundant, since

$$2^{16} - 1 = \text{MAX_PLAINTEXT_CODEPOINTS}$$

. This is the better place for the bound, because it puts the bound next to the invariant it protects, instead of in one of the two functions that happen to call `block_size` with a checked length today.

Replace the `debug_assert!` in `pad_to_block` with a check that also runs in release builds. Since the function returns `Vec<u32>` and not a `Result`, the smallest change is `assert!`, which fires in release builds too. A better option is to change the signature to `Result<Vec<u32>, String>` and to pass the error on with `?` in the two places where the function is called, which matches the way the v8 entry points already han-

dle errors. In either case, a release build must not reach the subtraction with `block_size <= n`.

Enable `overflow-checks` in `[profile.release]`, as CVF-23 recommends. This would make both this subtraction and the multiplication of CVF-16 abort the program instead of silently wrapping around. It is an additional safety measure and does not replace the two changes above.

Finally, add a unit test that checks the invariant for every profile. The existing `pad_profiles_share_one_reachable_set` test checks that the results belong to the set over the reachable range of inputs. Extend it to check that `block_size` *returns an error* for every profile at $n = 2^{16}$ and above, so that the upper limit is tested in the same way as the bounds of `Frame` are already tested by `pad_profile_frame_is_constant_and_range_checked`.

/Cargo.toml

```
[dependencies]
hmac = "0.12"
sha2 = "0.10"
```

/docs/napseq-eprint-v4.tex

```
parameter rather than as a hard-wired constant. A \emph{padding profile} is
any function $B(\cdot)$ from codepoint counts to block sizes satisfying
$B(n)>n$ throughout the message space. This paper specifies three:
```

same file

```
All three profiles take values in the same $13$-element set, so the set of
legal real-token counts $R=B+2$ — and hence the set of legal ciphertext
lengths — is identical for all of them. A decryptor consequently needs
```

/src/lib.rs

```
/// Exponent range of the reachable block sizes `{2^4, ..., 2^16}`.
///
/// Every padding profile takes values in this same 13-element set, so the set
/// of legal token counts is profile-independent and a decryptor never needs to
/// know which profile the sender used.
pub const PAD_MIN_EXP: u32 = 4;
pub const PAD_MAX_EXP: u32 = 16;
```

same file

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum PadProfile {
```

```
    /// Default: smallest power of two strictly above `n`, floored at 16.
    /// 13 reachable sizes, leaking at most `log2(13) ~= 3.70` bits of length.
    Bucket,
    /// [`PadProfile::Bucket`] thinned by a stride `g` dividing 12, leaving
    /// `12 / g + 1` reachable sizes.
    Coarse(u32),
    /// Every message padded to the single size `F`, leaking exactly zero bits.
    /// Requires `n < F`.
    Frame(u32),
}
```

same file

```
fn bit_length(n: usize) -> u32 {
    usize::BITS - n.leading_zeros()
}
```

same file

```
fn bucket_block_size(n: usize) -> usize {
    1usize << bit_length(n).max(PAD_MIN_EXP)
}
```

same file

```
    pub fn block_size(self, n: usize) -> Result<usize, String> {
        let e = bit_length(n).max(PAD_MIN_EXP);
        match self {
            PadProfile::Bucket => Ok(bucket_block_size(n)),
            PadProfile::Coarse(g) => {
                let span = PAD_MAX_EXP - PAD_MIN_EXP;
                if g == 0 || span % g != 0 {
                    return Err(format!(
                        "coarse stride g={} must divide {}.",
                        g, span
                    ));
                }
                let steps = (e - PAD_MIN_EXP + g - 1) / g; // ceil division
                Ok(1usize << (PAD_MIN_EXP + g * steps))
            }
```

same file

```
fn pad_to_block(msg: &[u32], kb: &[u8], nonce: &[u8], block_size: usize) -> Vec<u32> {
    let n = msg.len();
```

```
    debug_assert!(block_size > n, "padding block must exceed the message");
    let pad_len = block_size - n;
```

same file

```
    for i in 0..pad_len {
```

same file

```
    if codepoints.len() > 0xFFFF {
        return Err("Message too long for 2-byte length prefix.".into());
    }
```

same file

```
    let block_size = pad_profile.block_size(codepoints.len())?;
```

same file

```
    fn pad_profiles_share_one_reachable_set() {
        let legal: std::collections::HashSet<usize> =
            (PAD_MIN_EXP..=PAD_MAX_EXP).map(|e| 1usize << e).collect();
```

CVF-32: fixed Category: **procedural**

> **Summary**
>
> **Two of the three padding modes in the specification have no test vectors, so independent implementations of them can disagree without any test noticing.**
>
> The specification defines three ways of padding a message before encryption, and it recommends one of the two non-default ones for the users with the strictest privacy needs. The client's test vectors, which every implementation must reproduce byte for byte, cover only the default mode, and their format has no way to say which mode a vector uses. The library's own tests check the non-default modes only against themselves. Two implementations could therefore implement these modes slightly differently, both pass every test, and still produce different ciphertexts. This would not weaken security, but deployments that believe they hide message lengths in the same way could in fact behave differently.

Section 3.6 specifies three padding profiles normatively: `bucket`, `coarse(g)` and `frame(F)`. Section 12 recommends the third one, directing deployments "requiring zero length leakage" to select frame(F). Section 10 then states the conformance requirement for the whole scheme: "Conforming implementations must produce byte-identical ciphertext for every positive vector and reject every negative vector."

The test vectors cannot establish that requirement for two of the three profiles. Reading `tests/kat/v8_vectors.json` as data, we found that all twelve positive vectors (`W001`–`W012`) are default-`bucket` ASCII messages of 0, 1, 7, 12, 14, 15, 16 and 32 codepoints, with $K \in \{1, 4, 10\}$ and associated-data (AAD) lengths of 0, 8, 12, 16 and 64 bytes. **No vector uses `coarse(g)` or `frame(F)`.** Moreover, the vector format could not describe such a vector: the union of keys across all twenty vectors is `id`, `kind`, `description`, `key`, `sk_hex`, `message`, `aad_hex`, `ciphertext_hex`, `expected_exception` and `tampered_hex`. There is no profile field, and no vector carries one.

The tests inside the crate do not fill the gap either, because none of them fixes an expected byte string. `pad_profile_bucket_matches_legacy_ladder`, `pad_profiles_share_one_reachable_set`, `pad_profile_coarse_stride_must_divide_twelve` and `pad_profile_frame_is_constant_and_range_checked` all test the arithmetic of `block_size` and membership in the reachable set, which are properties of the profile function and not of the ciphertext. `v8_frame_profile_hides_length_and_round_trips` and `v8_frame_profile_rejects_oversized_message` test one implementation against itself. The first checks that two messages under `frame` produce ciphertexts of equal length that decrypt correctly, which any self-consistent implementation passes, and the second checks an error condition. No test anywhere fixes the ciphertext bytes for a non-default profile.

So two of the three normative profiles have no reference that implementations can be compared against, and the two disagreements most likely to arise are exactly the ones that the test vectors cannot catch:

- The exponent of `coarse(g)` is $4 + g\lceil (e(n) - 4)/g \rceil$. The Rust implementation computes the ceiling division as `(e - PAD_MIN_EXP + g - 1) / g`. A second implementation that rounds differently at a boundary, or that computes $e(n)$ as $\lceil \log_2(n+1) \rceil$ without the $\max(4, \cdot)$ floor, or with a floating-point `log2`, produces a different B for a small set of values of n, and therefore a different ciphertext length and different padding codepoints.
- The admissibility boundary of `frame(F)` is $n < F$ in Section 3.6 and `n >= f => Err` in the Rust implementation. An implementation that uses $n \leq F$ accepts one message length that the reference implementation rejects. For that length it produces $B = F$ with **pad** of length zero, i.e. a padded buffer with no padding at all, which the encoder never produces in any other case.

Neither difference is a security defect. Both are silent interoperability failures between implementations that each pass the delivered test suite, in a profile that Section 12 recommends for the deployments with the strongest requirements against traffic analysis. Since the profile is never transmitted and the decryptor by design does not need to know it, a disagreement about the profile on the sender's side does not show up as a decryption failure. Instead, two deployments believe that they leak the same information about message lengths when they do not.

This finding is related to CVF-25, but it is a different problem. CVF-25 concerns whether the byte-for-byte agreement of the three implementations can be checked at all, given that two of them were not delivered. This finding concerns the coverage of the test vectors themselves: even with all three implementations available, the test vectors would fix the output of only one of the three normative profiles.

Recommendation

Add a profile field to the test-vector format, and generate positive vectors for the two profiles that are not covered.

The change to the format is small: an optional `"pad_profile"` key, absent or `"bucket"` for the existing twelve vectors so that they are unaffected, and `{"coarse": 3}` / `{"frame": 1024}` for the new ones. `tests/gen_kats_v8.py` regenerates the file, and both consumers, `tests/test_kats.py` and `src/kat_cross_check.rs`, need one branch each to select the profile before encrypting. The Rust harness already selects its behaviour from vector fields, so this follows the existing pattern.

Choose the new vectors so that they fix the two boundaries where implementations can actually differ; a large number of vectors is not needed:

- For `coarse(g)`, add at least one vector at a stride boundary where the ceiling division rounds up and one just below it. For example, `coarse(3)` at $n = 16$ and $n = 17$, which lie on either side of the change $e(n) = 5 \to 6$ and must both give $B = 128$, and one at $n = 15$, which must give $B = 16$. Add a `coarse(12)` vector as well, since it is the extreme case that Section 3.6 names explicitly, in which only $\{2^4, 2^{16}\}$ are reachable.
- For `frame(F)`, add one positive vector at $n = F - 1$, the largest admissible message, and one negative vector at $n = F$ with `expected_exception` set. Together they fix the strict inequality that Section 3.6 states and the implementation enforces. Without the negative vector, the boundary is not tested in either direction.

  Update the description in Section 10 of what the test vectors cover so that it names the profiles, in the same style as the existing sentence that lists the message lengths, key sizes and AAD lengths. As written, a reader is told that the test vectors cover "the block boundaries at 15, 16 and 32 codepoints" and may reasonably conclude that all profiles are covered too, since Section 3.6 specifies all three normatively and Section 12 recommends one of them.

  If generating vectors for the non-default profiles that all implementations must reproduce is not practical for the current version, say so explicitly in Section 10: state that the test vectors fix only the `bucket` profile, and that conformance of different implementations for `coarse(g)` and `frame(F)` is not tested. The reader then does not have to infer the scope of the byte-for-byte requirement from the vector file.

/docs/napseq-eprint-v4.tex

```
472 \subsection{Padding Profiles}
473 \label{sec:pad-profiles}
474
475 The map $n\mapsto B$ above is the \emph{bucket} profile, and it is the
476 normative default. Because it is the sole source of the length-hiding
477 property proved in Section~\ref{sec:length-hiding}
478 (Remark~\ref{rem:coarsening-not-inflation}), we specify it as a
    replaceable
479 parameter rather than as a hard-wired constant. A \emph{padding
    profile} is
```

```
any function $B(\cdot)$ from codepoint counts to block sizes satisfying
$B(n)>n$ throughout the message space. This paper specifies three:
```

same file

```
  \item[$\mathsf{coarse}(g)$.] $B(n)=2^{\,4+g\lceil(e(n)-4)/g\rceil}$ where
        $e(n)=\max\bigl(4,\lceil\log_2(n+1)\rceil\bigr)$ and $g$ divides
        $12$: the same ladder thinned by a stride $g$, with $12/g+1$
        reachable values. $\mathsf{coarse}(1)=\mathsf{bucket}$, and
        $\mathsf{coarse}(12)$ leaves only $\{2^4,2^{16}\}$.
```

same file

```
  \item[$\mathsf{frame}(F)$.] $B(n)=F$ for a fixed
        $F\in\{2^4,\ldots,2^{16}\}$, on the restricted message space $n<F$:
        every message pads to one common block size, so exactly one value is
        reachable.
\end{description}
```

same file

```
The corpus for the scheme specified in this paper is
\texttt{tests/kat/v8\_vectors.json}. It contains 20 vectors: 12 positive
(ids \texttt{W001}--\texttt{W012}) covering the empty message, a single
character, the block boundaries at 15, 16 and 32 codepoints, 1-element and
10-element prime tuples $\mathbf{k}$, empty, binary and 64-byte associated
data (the last pinning the 8-byte $\mathrm{be8}$ AAD length prefix of
Definition~\ref{def:taginput}), and mixed-case punctuation; and 8 negative.
Of the negative vectors, \texttt{W-N01}--\texttt{W-N05} exercise
authentication (flipped tag byte, flipped masked-blob byte, wrong $A$,
wrong $\mathsf{sk}$) and the 48-byte parse floor, while
```

same file

```
needed. Conforming implementations must produce byte-identical ciphertext
for every positive vector and reject every negative vector. The generator
```

same file

```
2056        fixed per bucket, it does not vary with $\mathsf{sk}$, the
nonce, or
2057        the associated data, so no averaging attack across repeated
2058        encryptions of one message under varying AAD can extract more
than
2059        the bucket itself. Deployments requiring strictly less than
this
2060        should select a coarser padding profile, and those requiring
zero
2061        length leakage should select $\mathsf{frame}(F)$
2062        (Section~\ref{sec:pad-profiles}), which needs no change to
the wire
2063        format and no cooperation from the decryptor; no external
```

/src/kat_cross_check.rs

```
65          .filter(|v| v["kind"] == "positive" && v["api"] !=
"stream_ae")
```

same file

```
126         .filter(|v| v["kind"] == "positive" && v["api"] !=
"stream_ae")
```

same file

```
191         .filter(|v| v["kind"] == "negative" && v["api"] !=
"stream_ae")
```

/src/lib.rs

```
351     pub fn block_size(self, n: usize) -> Result<usize, String> {
352         let e = bit_length(n).max(PAD_MIN_EXP);
353         match self {
```

same file

```
366             PadProfile::Frame(f) => {
367                 if !f.is_power_of_two()
368                     || f.trailing_zeros() < PAD_MIN_EXP
```

same file

```
1443    fn pad_profiles_share_one_reachable_set() {
1444        let legal: std::collections::HashSet<usize> =
1445            (PAD_MIN_EXP..=PAD_MAX_EXP).map(|e| 1usize <<
e).collect();
```

same file

```
1462    fn pad_profile_coarse_stride_must_divide_twelve() {
1463        assert!(PadProfile::Coarse(5).block_size(3).is_err());
1464        assert!(PadProfile::Coarse(0).block_size(3).is_err());
```

same file

```
1473     fn pad_profile_frame_is_constant_and_range_checked() {
1474         for n in [0usize, 1, 100, 511] {
1475             assert_eq!(PadProfile::Frame(512).block_size(n).unwrap(),
512);
```

same file

```
1487     fn v8_frame_profile_hides_length_and_round_trips() {
1488         let (primes, sk) = generate_v8_key(10, MIN_KEY_PRIME,
MAX_KEY_PRIME);
1489 
```

same file

```
1523     fn v8_frame_profile_rejects_oversized_message() {
1524         let (primes, sk) = generate_v8_key(10, MIN_KEY_PRIME,
MAX_KEY_PRIME);
1525         let msg = "a".repeat(600);
```

CVF-33: fixed Category: **procedural**

> **Summary**
>
> **The paper reports a passed statistical randomness test for its ciphertexts, but the test was run on the output of the older encryption format and not on the scheme the paper specifies.**
>
> The paper says that the scheme's encrypted output passed a standard battery of statistical randomness tests published by NIST. The program that produced the tested data, however, uses the older (v7) encryption functions, which differ from the scheme described in the paper in several ways that affect the structure of the output. The code also cites the same tests to justify a design choice in a data format that the tests never saw. We have no reason to expect the current scheme to fail these tests; the problem is that the paper presents evidence about one format as evidence about another.

Section 9 states the evidence without qualification: "We applied the full NIST SP 800-22 Rev. 1a Statistical Test Suite to 10^7 bits of NAPQES ciphertext output generated by encrypting a cycling corpus of printable-ASCII plaintext with a 10-element prime key. All 15 eligible SP 800-22 tests passed at the $\alpha = 0.01$ significance level."

The generator is `src/bin/sts.rs`, and the call in it that produces the ciphertext is `napqes::encrypt_bytes(msg, &STS_KEY, b"")`, i.e. the **v7** entry point. It never calls `encrypt_bytes_v8` or `encrypt_bytes_v8_with_profile`. The module header confirms this: the file is titled "NIST SP 800-22 Rev 1a Statistical Test Suite — NAPSEQ v6 (Rust)", and the comment above the generator loop describes the input as "NAPSEQ v6

ciphertexts with STS_KEY". src/bin/ is **outside the code reviewed in this engagement**, so we cite it only as evidence of what produced the bitstream. The inaccurate claim is in Section 9, which is in scope.

The bitstream that passed was therefore not produced by the specified scheme. It differs from a v8 bitstream in four ways at once, each of which is one of the differences recorded in other findings of this review:

- the derivations use key_bytes(primes) as the key, not sk_{fmt} (CVF-28, CVF-11);
- the emission loop applies neither the noise-run cap nor the token ceiling, so the number of tokens per padded codepoint is unbounded and depends on the nonce, while in v8 it is fixed at 20 (CVF-17);
- the noise threshold is an IEEE-754 floating-point number, not the integer $\theta(N)$ (CVF-30);
- the tag uses a 4-byte prefix for the length of the associated data (AAD), not an 8-byte one (CVF-28).

The first three change the statistical structure of the byte stream directly, since the token population, its length distribution and the noise density all differ.

This matters more than an ordinary question of how far some evidence reaches, because the design rationale relies on it. The reason the crate gives for masking the token blob at all is statistical: the comment on varint_keystream says that "XOR-masking the raw LEB128 blob eliminates the 3:1 MSB continuation-bit bias that otherwise causes systematic NIST SP 800-22 failures." So SP 800-22 is used to justify a decision about the wire format, but the format it justifies is not the one on which the tests were run. The LEB128 encoding that produced the 3:1 bias has since been retired, and the delivered v8 blob consists of fixed-width 8-byte tokens, for which the bias argument does not apply in the same form and has not been re-established.

Nothing here suggests that the v8 bitstream would fail SP 800-22. It is masked by a keystream derived from HMAC, so it should pass comfortably, and it is likely the better-behaved of the two streams. The defect is that the paper, in a section about evidence, reports a result about one artifact as a result about another, while Contribution 8 promises to "state in each case what the evidence does and does not establish".

This is the same substitution as in CVF-22, where the fuzz harness targets decrypt_bytes instead of decrypt_bytes_v8, here in a second place, and it is a concrete case of the general problem that CVF-10 describes. We report it separately because the fix is specific and differs from both: regenerating a bitstream is a different task from retargeting a fuzz harness or rewording the list of contributions.

Recommendation

Regenerate the bitstream with the v8 encryptor and run the test suite again, or change Section 9 to say what was actually tested.

Regenerating is straightforward and is the better outcome, because it makes the evidence match the claim, instead of narrowing the claim to match the evidence. The generator needs a change in one line, napqes::encrypt_bytes_v8(msg, &STS_KEY_PRIMES, &STS_SK, b"") in place of napqes::encrypt_bytes(msg, &STS_

`KEY, b""`), with a fixed `sk` added to the test constants, and a corresponding update of the module header, which still describes the file as a v6 artifact.

Plan for one consequence of this change. Because the v8 nonce is synthetic, i.e. computed from the inputs, encrypting a *cycling* set of plaintexts produces identical ciphertexts for repeated plaintext blocks, whereas the v7 nonce, drawn from a cryptographically secure random number generator (CSPRNG), made every block different. The input must therefore not repeat: vary the plaintext or the AAD for each chunk. Otherwise the bitstream will contain exact duplicate segments of 2,928 bytes, and several SP 800-22 tests will fail for a reason that has nothing to do with the keystream. This is a normal property of deterministic AEAD and not a defect, but it will appear as soon as the entry point is switched.

If the suite is not run again for the current version, Section 9 must say which format was tested, in the same register that the paper already uses elsewhere. For example: the SP 800-22 results were obtained on the legacy v7 wire format, which differs from the scheme specified here in its key derivation, its emission bounds and its arithmetic for the noise threshold, and no statistical evidence is offered for the v8 bitstream. This is a considerably weaker statement than the one currently printed, and stating such limits explicitly is exactly what Contribution 8 undertakes to do.

Separately, update the doc comment of `varint_keystream`. It justifies the mask by an SP 800-22 failure of the LEB128 encoding, which has been retired; the blob it masks today has fixed width. Either restate the justification for the current format, or remove the reference to SP 800-22 from the comment. A sound justification for the current format exists and does not depend on the retired encoding: the unmasked fixed-width token stream is a sequence of `c*k+a` values with a small key element, so its high-order bytes are far from uniform. As written, the comment points a reader to evidence that concerns neither the current encoding nor the current entry point.

/docs/napseq-eprint-v4.tex

```
181   \item We report NIST SP 800-22 Rev.\ 1a statistical test results on
182         ciphertext output (Section~\ref{sec:sts}), the
cross-implementation
183         KAT corpora
184        (Section~\ref{sec:kats}), and a fuzz harness covering the full
185         decoder pipeline (Section~\ref{sec:fuzz}), and state in each
case
186         what the evidence does and does not establish.
```

same file

```
1960 We applied the full NIST SP 800-22 Rev.\ 1a Statistical Test
1961 Suite~\cite{sp800-22} to $10^7$ bits of NAPQES ciphertext output
generated
1962 by encrypting a cycling corpus of printable-ASCII plaintext with a
1963 10-element prime key. All 15 eligible SP 800-22 tests passed at the
1964 $\alpha=0.01$ significance level. Results are reproducible using
1965 \texttt{tests/sts\_pipeline.py} in the reference implementation
1966 repository.
```

/src/lib.rs

```
422 /// HMAC-CTR keystream for masking the varint blob — domain byte 0x07.
423 ///
424 /// Each 32-byte block is `HMAC(key_bytes, 0x07 || nonce ||
    uint32_be(block))`.
425 /// XOR-masking the raw LEB128 blob eliminates the 3:1 MSB
    continuation-bit
426 /// bias that otherwise causes systematic NIST SP 800-22 failures.
```

same file

```
730 pub fn encrypt_bytes(message: &str, key: &[u64], aad: &[u8]) ->
    Result<Vec<u8>, String> {
```

CVF-34: fixed Category: **behavior**

> **Summary**
>
> **The paper promises that a message over the size limit is rejected with an error and that nothing is ever silently cut off, but the legacy functions crash the program on such a message and can silently drop characters when decrypting.**
>
> The paper's list of caveats says that a message longer than the maximum size is rejected with an error, and that the scheme never shortens a message without saying so. The current (v8) functions keep both promises. The older (v7) functions, which are still public, keep neither: an over-long message stops the whole program instead of producing an error, and decryption can quietly remove characters and still report success. The crash can be used to take down a server that passes unchecked input to these functions. The silent removal cannot be triggered by an attacker without the key, but it can hide data corruption. Users who rely on the caveat to decide how to limit their input are misled.

The first caveat of Section 12 makes two unqualified promises about the cap of 65,535 codepoints: exceeding it raises an error immediately, and there is no silent truncation. The v8 path keeps both. `encrypt_bytes_v8_with_profile` checks `codepoints.len() > 0xFFFF` and returns `Err("Message too long for 2-byte length prefix.")`. `decrypt_bytes_v8` rejects any recovered value that is not a Unicode scalar inside `decrypt_core_v8` instead of dropping it, and a comment says exactly that: "`decrypt_core_v8` has already rejected any non-scalar value, so this maps every recovered codepoint rather than silently dropping some."

On the legacy path, neither promise holds, and the two failures go in opposite directions.

The promised error is a panic. The v7 padding routine enforces the cap with a plain assertion:

```
fn pad_message(msg: &[u32], kb: &[u8], nonce: &[u8]) -> Vec<u32> {
    let n = msg.len();
    assert!(n <= 0xFFFF, "Message too long for 2-byte length prefix");
```

Unlike `debug_assert!`, `assert!` also fires in release builds, so this aborts the process instead of returning an error. It is reached from four public entry points: from `encrypt`, `encrypt_bytes` and `encrypt_raw` directly, and from `encrypt_str` through `encrypt_bytes`. It is also reached from the `pub(crate)` function `encrypt_bytes_with_nonce`. A caller who passes a string of 70,000 codepoints to `encrypt_bytes`, which returns `Result<Vec<u8>, String>` and so indicates that it reports failures as values, gets a panic instead. In a server, this is a denial of service that any caller can trigger unless the caller itself limits the message length, since the panic unwinds through a function whose signature promised an `Err`.

The same pattern appears once more on the v7 decryption side, where `decrypt_core` ends with

```
unpad_message(&padded).expect("v7 decrypt: malformed padded buffer")
```

and its own comment acknowledges it: "a malformed padded buffer panics here exactly as it did before V3-CVF8. The v8 path (`decrypt_core_v8`) returns the error instead."

The promise of no silent truncation does not hold on the decryption side. `decrypt_bytes` ends with

```
let s: String =
  codepoints.into_iter().filter_map(char::from_u32).collect();
```

`filter_map` **drops** every recovered value in the surrogate range or above `U+10FFFF` instead of rejecting the ciphertext. So `decrypt_bytes`, and `decrypt_str`, which delegates to it, can return `Ok` with a plaintext strictly shorter than the one that was encrypted, with no error and no sign that anything was discarded. This is silent truncation, on the decryption side of the same construction whose caveat says there is none. It cannot be reached without a valid tag, so it does not allow a forgery: it is produced by a key holder or by a ciphertext corrupted in storage. The v8 path was explicitly changed to avoid exactly this.

This finding adds something to CVF-15: a **normative caveat is stated unconditionally but holds only for v8**. CVF-15 records that the paper's statement that the code does not panic covers one entry point while several others panic, and it covers all the places where the code panics. It does not record that the caveat of Section 12 about the plaintext cap, which is written as a property of "the block padding scheme" and not of one implementation or one format, is false for four public entry points, nor that the crate performs exactly the silent truncation that the caveat rules out. Both statements are read by someone deploying the scheme who is deciding how to limit the input, and this is the audience Section 12 is written for.

Recommendation

Make the two v7 places where the caveat fails behave as the caveat says, or qualify the caveat.

On the encryption side, the difficulty is that `pad_message` returns `Vec<u32>` and cannot report a failure. The clean fix is to move the length check into the four public entry

points, whose signatures already allow an error, and to delete the assertion:

```
if codepoints.len() > 0xFFFF {
    return Err("Message too long for 2-byte length
  prefix.".into());
}
```

This is the same check that `encrypt_bytes_v8_with_profile` already performs. It should use a shared named constant instead of a repeated `0xFFFF` literal (see item 9 of CVF-40 on `MAX_PLAINTEXT_CODEPOINTS`, which is a named parameter in the paper but a bare literal in both existing checks; this is why the two checks ended up as `assert!` in one place and `Err` in the other). `encrypt` returns `([u8; NONCE_SIZE], Vec<u64>)` and not a `Result`, so it needs either a change of signature or a documented panic. Since CVF-17 and CVF-28 both recommend withdrawing the v7 encryptors, changing the signature is worth doing only if they are kept.

On the decryption side, replace the `filter_map` with a map that rejects invalid values, so that a value that is not a Unicode scalar causes an error instead of being deleted:

```
let s: String = codepoints
    .into_iter()
    .map(|c| char::from_u32(c).ok_or_else(|| format!(
        "Malformed v7 ciphertext: recovered value {} is not a
  Unicode scalar value.", c)))
    .collect::<Result<String, String>>()?;
```

`decrypt_bytes` already returns `Result<String, String>`, so this needs no change of signature, and it makes the v7 path behave like the v8 one.

Replace `unpad_message(&padded).expect(...)` in `decrypt_core` with error propagation. This requires `decrypt_core` to return a `Result`, which is a larger change and belongs to CVF-15. We mention it here because it is the third place where a panic replaces the error that the caveat promises.

If none of the v7 changes is made, for instance because the v7 encryptors are being withdrawn instead, qualify the caveat in Section 12. It should say that the cap is enforced by an error in the scheme specified in this paper, and it should describe the legacy behaviour, so that a reader deploying the scheme does not have to infer it. A section of caveats that overstates the robustness of the shipped code is worse than one that says less.

/docs/napseq-eprint-v4.tex

```
2031   \item[Plaintext length cap.] The block padding scheme stores the
2032         plaintext length as a 2-byte big-endian integer, capping
messages
2033         at 65{,}535 codepoints. Exceeding the cap raises an error
2034         immediately; there is no silent truncation. Callers requiring
2035         larger messages must split at the application layer.
```

/src/lib.rs

```
395 fn pad_message(msg: &[u32], kb: &[u8], nonce: &[u8]) -> Vec<u32> {
396     let n = msg.len();
397     assert!(n <= 0xFFFF, "Message too long for 2-byte length prefix");
```

same file

```
    let padded = pad_message(message, &kb, &nonce);
```

same file

```
    // v7 low-level API: signature is `-> Vec<u32>`, so a malformed padded
    // buffer panics here exactly as it did before V3-CVF8. The v8 path
    // (`decrypt_core_v8`) returns the error instead.
    unpad_message(&padded).expect("v7 decrypt: malformed padded buffer")
```

same file

```
    let padded = pad_message(&codepoints, &kb, &nonce);
```

same file

```
    let padded = pad_message(&codepoints, &kb, &nonce);
```

same file

```
    let s: String = codepoints.into_iter().filter_map(char::from_u32).collect();
```

same file

```
    let padded = pad_message(&codepoints, &kb, &nonce);
```

same file

```
    if codepoints.len() > 0xFFFF {
        return Err("Message too long for 2-byte length prefix.".into());
    }
```

same file

```
    // `decrypt_core_v8` has already rejected any non-scalar value, so this
    // maps every recovered codepoint rather than silently dropping some.
    let s: String = codepoints
```

CVF-35: fixed Category: **behavior**

Summary

The random-number health test that the code cites from FIPS 140-3 is skipped by one of the public legacy encryption functions, and the test is easy to mistake for a stronger guarantee than it gives.

The older (v7) encryption functions draw a random value, called a nonce, for every message, and reusing a nonce under the same key is catastrophic for that format. The code includes a health test, required by the FIPS 140-3 standard, that checks that the random generator has not produced the same value twice in a row. Two public functions run this test, but a third public function draws its nonce without it. In addition, the test only detects a generator that is stuck; it does not guarantee that nonces never repeat for a given key, although its error message may suggest otherwise. No claim in the paper depends on this, and the current (v8) format does not use random nonces at all.

The crate implements a continuous random-number-generator test and cites the standard for it:

```
static PREV_NONCE: Mutex<Option<[u8; NONCE_SIZE]>> =
  Mutex::new(None);

/// Generate a cryptographically random 16-byte nonce, verifying it
  differs
/// from the previous one (FIPS 140-3 continuous RNG test, SP
  800-140B §4.9.2).
fn generate_nonce_with_crng_check() -> Result<[u8; NONCE_SIZE],
  String> {
    let mut nonce = [0u8; NONCE_SIZE];
    rand::thread_rng().fill_bytes(&mut nonce);
    let mut prev = PREV_NONCE.lock().map_err(|_| "CRNG mutex
  poisoned".to_string())?;
    if let Some(prev_nonce) = *prev {
        if ct_eq_bytes(&nonce, &prev_nonce) {
            return Err("CRNG failure: consecutive identical nonces —
  DRBG may be compromised".into());
        }
    }
    *prev = Some(nonce);
    Ok(nonce)
}
```

The specification says nothing about this test. The v8 nonce is synthetic, and there is no nonce drawn from a cryptographically secure random number generator (CSPRNG) anywhere in Π, so `generate_nonce_with_crng_check` is a construct of the implementation that only the v7 path uses. We make two observations about how it is applied.

One of the crate's own public entry points bypasses it. `encrypt_bytes` and `encrypt_raw` both call it. `encrypt`, which is also `pub`, does not; it draws its nonce directly:

```
pub fn encrypt(message: &[u32], key: &[u64]) -> ([u8; NONCE_SIZE],
  Vec<u64>) {
    let mut nonce = [0u8; NONCE_SIZE];
    rand::thread_rng().fill_bytes(&mut nonce);
```

with no health check of any kind. So a caller who uses the low-level `encrypt` gets none of the protection that the crate advertises for `encrypt_bytes`, and nothing in either signature or doc comment tells them so. A repeated v7 nonce is very costly: the crate's own module comment says that it allows recovery of k from two ciphertexts that share a nonce, "catastrophic key recovery, not merely a confidentiality loss". A health check on exactly this value is therefore worth applying uniformly. (`encrypt_bytes_with_nonce` is not affected: it takes its nonce from the caller and is `pub(crate)`, so there is nothing to check.)

The single global slot makes this a test of the random generator, not a guarantee that nonces are unique. `PREV_NONCE` is one slot for the whole process, so each call compares its nonce with the one written by the most recent call, across every thread, every key and every message in the process. For a continuous test in the sense of SP 800-140B §4.9.2, this is exactly right, and the implementation is correct for that purpose: the test is meant to catch a stuck deterministic random bit generator (DRBG), and one previous output is enough for that. However, it must not be read as a check that nonces are unique for each key, because it is not one. It detects only identical outputs that are *immediately consecutive*, it can be defeated by two encryptions from different threads that interleave, and it says nothing about a nonce that repeats at any distance greater than one. Neither the code nor the documentation explains this limit, and the error string, "DRBG may be compromised", invites the stronger reading.

A third, smaller point is that the comparison uses `ct_eq_bytes`, the crate's constant-time comparator, on two values that are both public: a freshly generated nonce and the previous nonce, both of which are sent unencrypted. There is nothing wrong with this, but it is a second place where the comparator whose origin CVF-12 discusses is used, and it should be included when the recommendation of CVF-12 to adopt `subtle` is applied.

No security claim in the paper depends on any of this, and no theorem covers the v7 path in any case. We report it because the crate advertises a named FIPS control and applies it to only two of its three public entry points that generate their own nonce, leaving out the lowest-level one.

Recommendation

Apply the check in every public function that generates a nonce, or document where it applies.

Draw the nonce of `encrypt` through `generate_nonce_with_crng_check`, as the other two functions do. The obstacle is the signature: `encrypt` returns `([u8; NONCE_SIZE], Vec<u64>)` with no way to report an error, so it cannot pass on a CRNG failure. Change it to `Result<([u8; NONCE_SIZE], Vec<u64>), String>`. This is a breaking change to a low-level v7 function, and it is worth making only if the v7 encryptors are kept. CVF-17 and CVF-28 both recommend withdrawing them, and in that case this problem is resolved together with them. If the signature must

stay, say so in the doc comment: `encrypt` performs no continuous RNG test, and callers who require one must use `encrypt_bytes`.

Document what the check does and does not do. One sentence above `generate_nonce_with_crng_check` would prevent the stronger reading: it should state that `PREV_NONCE` is a single slot for the whole process, that the function implements the SP 800-140B §4.9.2 continuous test for a stuck generator, and that it does **not** guarantee that nonces are unique for each key. For the same reason, consider a more modest error string: "consecutive identical DRBG outputs" is what was observed, while "DRBG may be compromised" is only one possible conclusion from it.

If nonces that are unique for each key are actually wanted on the v7 path, a single global slot cannot provide this, and it should not be extended to try. The right solution is the one that v8 already uses: a synthetic nonce bound to the message, which makes repetition impossible by construction instead of detecting it after the fact. This is a further argument for moving v7 users to the v8 entry points instead of hardening the v7 nonce code.

Finally, when the recommendation of CVF-12 is applied and `ct_eq_bytes` is replaced by `ConstantTimeEq` from `subtle`, remember this use as well as the three tag comparisons. It compares two public values, so a plain `==` would also be correct here, and it would make the difference between the security-relevant comparisons and this one visible in the code.

/src/lib.rs

```
47 static PREV_NONCE: Mutex<Option<[u8; NONCE_SIZE]>> =
Mutex::new(None);
```

same file

```
49 /// Generate a cryptographically random 16-byte nonce, verifying it
differs
50 /// from the previous one (FIPS 140-3 continuous RNG test, SP
800-140B §4.9.2).
51 fn generate_nonce_with_crng_check() -> Result<[u8; NONCE_SIZE],
String> {
52     let mut nonce = [0u8; NONCE_SIZE];
53     rand::thread_rng().fill_bytes(&mut nonce);
54     let mut prev = PREV_NONCE.lock().map_err(|_| "CRNG mutex
poisoned".to_string())?;
55     if let Some(prev_nonce) = *prev {
56         if ct_eq_bytes(&nonce, &prev_nonce) {
57             return Err("CRNG failure: consecutive identical nonces —
DRBG may be compromised".into());
58         }
59     }
60     *prev = Some(nonce);
61     Ok(nonce)
62 }
```

same file

```
56          if ct_eq_bytes(&nonce, &prev_nonce) {
```

same file

```
467 pub fn encrypt(message: &[u32], key: &[u64]) -> ([u8; NONCE_SIZE],
Vec<u64>) {
468     let mut nonce = [0u8; NONCE_SIZE];
469     rand::thread_rng().fill_bytes(&mut nonce);
```

same file

```
734     let nonce = generate_nonce_with_crng_check()?;
```

same file

```
890     let nonce = generate_nonce_with_crng_check()?;
```

CVF-36: fixed Category: **documentation**

> **Summary**
>
> **The paper says that the implementations warn when a key has fewer than seven elements, but the delivered code never warns, and most of the client's own test vectors use such keys.**
>
> The secret key of the scheme includes a list of prime numbers, and the paper explains that the length of this list is not a security parameter. It then adds that the implementations still display a warning when the list has fewer than seven elements. The delivered code has no such warning, and eleven of the twelve test vectors that encrypt a message use shorter lists, including one with a single element that the paper itself lists as a feature of the test vectors. This has no security impact, since nothing depends on the length of the list, but it is a false statement about the delivered software, and a user who relies on it will not be warned.

`Remark 3.3` is careful on this point. It spends a paragraph establishing that K has no minimum required by a security theorem, that "$K = 10$ is an interoperability default, not a security parameter", and that "any $K \geq 1$ is permitted by the construction". It then makes a factual claim about the artifacts: "The reference implementations emit a warning below $K = 7$; that threshold is inherited from the earlier single-secret schedule, in which $H_\infty(\mathbf{k})$ did enter the bound, and no theorem in this paper requires it."

No such warning exists in the delivered Rust implementation. `validate_key` tests four conditions, namely that the key is non-empty, that its elements are prime, that they are at or above `MIN_KEY_PRIME`, and that they are pairwise distinct, and none of them concerns K. There is no `eprintln!`, no `log` call, no `#[deprecated]`, no diagnostic behind a feature flag and no `warn` of any kind anywhere in `src/lib.rs`. The library does not declare any logging dependency: `syslog` appears in the manifest, but only for the `ls_gateway`

binary. `generate_v8_key(count, min_val, max_val)` accepts any `count` the caller passes, including 1, and returns without comment. The plural in "the reference implementations" cannot be checked further, because `napqes.py` and `C/napqes.c` were not delivered.

The delivered test vectors make the gap clearly visible. Of the twelve positive vectors in `tests/kat/v8_vectors.json`, **one uses $K = 1$, nine use $K = 4$, and only two use $K = 10$.** So eleven of the twelve positive vectors are below the stated warning threshold, as are all eight negative vectors, which use $K = 4$. Moreover, Section 10 presents the $K = 1$ case as a deliberate feature of the test vectors, listing "1-element and 10-element prime tuples **k**" among the properties that the twelve positive vectors cover. If the reference implementations warned below $K = 7$, generating and checking the test vectors would produce a warning on eighteen of the twenty vectors.

There is **no security consequence**. `Remark 3.3` states explicitly and correctly that no theorem depends on $H_\infty(\mathbf{k})$ or on a minimum K, and Section 12 goes further, stating that the secrecy of **k** "adds **no** independent confidentiality margin once sk is known". A missing warning about a parameter that has no security requirement costs nothing directly.

We report it because it is a false factual claim about the delivered artifacts, in a paper that elsewhere takes visible care to limit its claims to what the evidence shows: Contribution 8 undertakes to "state in each case what the evidence does and does not establish". The sentence also affects how the paragraph is read. It reassures a reader who has seen the $K \geq 7$ rule elsewhere that the implementations still check the old threshold, even though the theory no longer needs it. A reader who believes this, sets $K = 2$ in a deployment and expects to be warned will not be.

Recommendation

Either implement the warning or delete the sentence. Deleting it is the better choice, and the paragraph is stronger without it.

`Remark 3.3` already establishes that K is a parameter for interoperability and not for security. The sentence about the warning works against that conclusion: it tells the reader that the implementations treat $K < 7$ as worth reporting, immediately after arguing that nothing requires $K \geq 7$. Removing it keeps the argument intact and removes the claim about the artifacts. If the historical threshold is worth recording, the other half of the sentence, that it "is inherited from the earlier single-secret schedule, in which $H_\infty(\mathbf{k})$ did enter the bound", can stand on its own as a historical note, phrased in the past tense.

If the warning is to be implemented instead, implement it in all three implementations and say where it appears. In Rust, the natural place is `generate_v8_key`, not `validate_key`. A warning belongs at key *generation*, where the caller chooses K, and not in the validation performed for every message, where it would fire on every encryption and decryption for as long as the key is used. The library declares no logging framework, and adding one for this purpose would be disproportionate; a `#[must_use]` result, a documented `Result` variant or a `debug_assert!` would all be better than `eprintln!` from a library. Whatever mechanism is chosen, the test vectors must then be regenerated or the generator exempted, because eighteen of the twenty vectors would otherwise trigger it.

In either case, correct the description of the test vectors in Section 10 if the status of the $K = 1$ vector changes. Independently of this finding, note that this sentence is one of several claims in the paper about "the reference implementations" while only one of the three was delivered. That broader gap is described in CVF-25.

/docs/napseq-eprint-v4.tex

```
agree on $\mathbf{k}$ itself. The reference implementations emit a warning
below $K=7$; that threshold is inherited from the earlier single-secret
schedule, in which $H_\infty(\mathbf{k})$ did enter the bound, and no
theorem in this paper requires it. What the arithmetic layer is retained
```

same file

```
The corpus for the scheme specified in this paper is
\texttt{tests/kat/v8\_vectors.json}. It contains 20 vectors: 12 positive
(ids \texttt{W001}--\texttt{W012}) covering the empty message, a single
character, the block boundaries at 15, 16 and 32 codepoints, 1-element and
10-element prime tuples $\mathbf{k}$, empty, binary and 64-byte associated
```

/src/lib.rs

```
fn validate_key(key: &[u64]) -> Result<(), String> {
    if key.is_empty() {
        return Err("Key must be a non-empty list of primes.".into());
    }
    for (i, &k) in key.iter().enumerate() {
        if !is_prime(k) {
            return Err(format!("Key element at index {} ({}) is not prime.", i, k));
        }
        if k < MIN_KEY_PRIME {
            return Err(format!(
                "Key element at index {} ({}) is below the minimum of {}.",
                i, k, MIN_KEY_PRIME
            ));
        }
        if key[..i].contains(&k) {
            return Err(format!(
                "Key element {} at index {} is a duplicate; all elements must be distinct.",
                k, i
```

```
            ));
        }
    }
    Ok(())
}
```

same file

```
pub fn generate_v8_key(count: usize, min_val: u64, max_val: u64) -> (Vec<u64>, [u8; SK_SIZE]) {
    let primes = generate_prime_numbers(count, min_val, max_val);
    let mut sk = [0u8; SK_SIZE];
    rand::thread_rng().fill_bytes(&mut sk);
    (primes, sk)
}
```

CVF-37: fixed Category: **flaw**

> **Summary**
>
> **The library's functions for erasing secret keys from memory are never used, and the key that actually protects every message is never erased at all.**
>
> Good practice, and the standards that the code itself cites, require secret keys to be erased from memory as soon as they are no longer needed, so that they cannot be recovered later from a memory dump or a similar source. The library provides two functions for this, but none of its own code calls them. More importantly, every message is protected by a key derived from the main secret, and this derived key is enough on its own to read and forge messages. The erasure functions do not cover it, and the library leaves a copy of it in memory after every encryption and decryption. Exploiting this requires a separate way to read the program's memory, so there is no direct attack, but the code does not do what its documentation says.

The crate takes key destruction seriously enough to cite the standards for it. The doc comment of `zeroize_key` reads: "Uses `ptr::write_volatile` to prevent the compiler from eliding the writes as dead-code optimisations, consistent with the `ct_eq_bytes` approach. Call this as soon as the key is no longer needed. Reference: FIPS 140-3 / SP 800-57 Part 1 Rev 5 §8.3 (key destruction)." Both helpers are correct as written, since `write_volatile` on each element is the right primitive.

However, neither of them is ever called. The only places in the whole crate where `zeroize_key` and `zeroize_sk` are called are the two unit tests that test them, `zeroize_key_clears_memory` and `v8_zeroize_sk_clears_memory`. No library function calls either of them, no type that holds key material has a `Drop` implementation, and `Cargo.toml` declares neither the `zeroize` crate nor any `Zeroizing` wrapper.

The more serious gap is what the helpers do not cover. `zeroize_sk` erases sk, but, as CVF-11 shows, sk is used as a key only for one two-byte HMAC computation. The key of every derivation is `sk_fmt = HMAC(sk, 0x0B||0x01)`, a local `[u8; 32]` value computed at the start of `encrypt_bytes_v8_with_profile` and of `decrypt_bytes_v8` and dropped without being overwritten at the end of each. **`sk_fmt` is sufficient on its own to encrypt, decrypt and forge**, without any knowledge of sk, since it is the key of every domain in Table 1. So a deployment that follows the crate's own advice and calls `zeroize_sk` as soon as the key is no longer needed erases the value that is used only for that one HMAC computation, and leaves on the stack of every call it ever made a copy of the value that is sufficient to break the scheme. The protection thus misses the value it is meant to protect.

There are three smaller points in the same area:

- `generate_v8_key` returns `[u8; SK_SIZE]` **by value**, and `[u8; 32]` is `Copy`, so the function's own temporary value on the stack and every later assignment leave copies that are not erased and that `zeroize_sk` on the caller's variable does not reach.
- `key_bytes(key)` allocates a new `Vec<u8>` holding the serialised prime tuple in each of the seven legacy places where it is called, and drops it without erasing it, so the v7 HMAC key is left in freed memory after every v7 operation.
- `zeroize_key` takes `&mut [u64]`, so it cannot erase the spare capacity of a `Vec<u64>` or any copy that the caller made, and nothing documents this limitation.

The specification offers no guidance against which any of this could be measured, which is also worth noting. The 2,281-line source contains **zero** occurrences of "zeroiz", "erase", "destruct" or "key destruction". The caveat in Section 12 about key material says only that **k** and sk "must be protected with equal rigor, as a single unit", and says nothing more. So the code chooses its own policy for the lifetime of keys, cites FIPS 140-3 and SP 800-57 for it, and then does not apply it, while the paper refers to FIPS 140-3 elsewhere.

No exploit follows directly. Recovering a dropped stack frame or freed memory requires a memory-disclosure defect elsewhere, a core dump, a swap file or a virtual-machine snapshot, and none of the paper's theorems is stated in a model that covers any of these. The problem is that the crate's stated intent and its behaviour do not match, and that the mismatch goes in the direction that matters: the value that is erased is the one whose compromise is no longer sufficient to break the scheme, and the value that is left behind is the one whose compromise is sufficient.

Recommendation

Erase `sk_fmt`, and make erasure automatic instead of leaving it to the caller.

The minimal change is to erase `sk_fmt` at the end of both v8 entry points, on every exit path including the error paths, since a rejected decryption has already computed it. Doing this by hand before each `return` is error-prone. The usual Rust solution is a guard type that erases its contents when it is dropped:

```
struct Secret32([u8; 32]);
impl Drop for Secret32 {
    fn drop(&mut self) {
```

```
        for b in self.0.iter_mut() { unsafe {
  std::ptr::write_volatile(b, 0u8) }; }
    }
}
```

with `sk_fmt` held as a `Secret32` and passed as `&sk_fmt.0`. This covers every exit path automatically and costs one type. Applying the same wrapper to the `Vec<u8>` returned by `key_bytes` solves the v7 problem in all seven places where it is called at once.

A better option is to add the dependency: `Zeroizing<T>` from the `zeroize` crate does exactly this, is a single well-reviewed dependency, handles the spare capacity of a `Vec`, which a hand-written guard does not, and would allow `zeroize_sk` and `zeroize_key` to be removed instead of being documented as functions that nothing calls. The manifest already has fifteen dependencies, so one more does not add much to the code that needs reviewing.

Change `generate_v8_key` to return the secret in a wrapper instead of as a bare `[u8; 32]`, so that the `Copy` semantics no longer create silent copies of it. If the signature must stay for compatibility, document that the returned array is `Copy` and that the caller is responsible for every copy it makes; at present, the doc comment implies that `zeroize_sk` is sufficient.

In either case, update the doc comment of `zeroize_sk`. As written, it describes the function as erasing "a v8 HMAC subkey", which is exactly what it does not do: it erases the master secret, and the subkey is what remains. Once CVF-11 is resolved and $\mathsf{sk}_{\mathsf{fmt}}$ is a named quantity in the specification, this distinction will be easy to state.

Finally, add one paragraph on the lifetime of keys to Section 12, next to the existing sentence "protected with equal rigor, as a single unit". It does not need to be long. It should say that **k**, sk and any derived subkey should be erased when they are no longer needed, that implementations should use a volatile write or an equivalent that the compiler cannot remove, and that the paper offers no guarantees against side channels or memory disclosure. At present the specification says nothing on this, so each implementation is free to choose its own policy, and the delivered implementation chose a policy that it does not apply.

/Cargo.toml

```
[dependencies]
hmac = "0.12"
sha2 = "0.10"
base64 = "0.22"
rand = "0.8"
subtle = "2"
```

/docs/napseq-eprint-v4.tex

```
  \item[Scope of $\mathsf{sk}$ and $\mathbf{k}$ compromise.] $\mathsf{sk}$
        is the sole cryptographic secret in this construction: every HMAC
        domain derivation — the noise schedule, the padding, the
```

```
2069        per-position addends, the keystream, and the tag — is keyed by
2070        $\mathsf{sk}$ alone (Section~\ref{sec:domains}).
Consequently, an
```

/src/lib.rs

```rust
fn key_bytes(key: &[u64]) -> Vec<u8> {
    let mut out = Vec::with_capacity(key.len() * 5);
    for &k in key {
        let b = k.to_be_bytes(); // 8 bytes
        out.extend_from_slice(&b[3..8]); // low 5 bytes, big-endian
    }
    out
}
```

same file

```rust
pub fn generate_v8_key(count: usize, min_val: u64, max_val: u64) ->
(Vec<u64>, [u8; SK_SIZE]) {
    let primes = generate_prime_numbers(count, min_val, max_val);
    let mut sk = [0u8; SK_SIZE];
    rand::thread_rng().fill_bytes(&mut sk);
    (primes, sk)
}
```

same file

```rust
    let sk_fmt = derive_format_subkey(sk, FORMAT_BLOCK_V8);
```

same file

```rust
    let sk_fmt = derive_format_subkey(sk, FORMAT_BLOCK_V8);
```

same file

```rust
/// Securely erase a v8 HMAC subkey by overwriting it with zero.
pub fn zeroize_sk(sk: &mut [u8; SK_SIZE]) {
    for x in sk.iter_mut() {
        unsafe { std::ptr::write_volatile(x, 0u8) };
    }
}
```

same file

```rust
// ── Key zeroization ──────────────────────────────────────────

/// Securely erase key material by overwriting each element with zero.
///
/// Uses `ptr::write_volatile` to prevent the compiler from eliding
the writes
/// as dead-code optimisations, consistent with the `ct_eq_bytes`
approach.
```

```
/// Call this as soon as the key is no longer needed.
///
/// Reference: FIPS 140-3 / SP 800-57 Part 1 Rev 5 §8.3 (key destruction).
pub fn zeroize_key(key: &mut [u64]) {
    for x in key.iter_mut() {
        unsafe { std::ptr::write_volatile(x, 0u64) };
    }
}
```

same file

```
    fn zeroize_key_clears_memory() {
```

same file

```
    fn v8_zeroize_sk_clears_memory() {
```

CVF-38: fixed Category: **documentation**

> **Summary**
> **The paper specifies a byte encoding of the key that the current scheme never uses, and does not specify the key encoding that another implementation needs in order to load the test vectors.**
> The paper defines how one part of the secret key is converted to bytes, and presents this as part of key generation. In the delivered code, only the older (v7) format uses this encoding; the current scheme does not use it anywhere. At the same time, the paper never says how a complete key is written down, although anyone who wants to check another implementation against the client's test vectors must read keys in exactly the format those vectors use. A related helper also silently cuts off key values that are too large, which cannot happen through the public functions once another finding is fixed. None of this weakens security; it makes the specification harder to implement correctly.

`Definition 3.1` ends with a normative serialisation: "The serialised arithmetic key is

$$\mathsf{key_bytes}(\mathbf{k}) = \mathsf{be5}(k_1) \| \mathsf{be5}(k_2) \| \cdots \| \mathsf{be5}(k_K)$$

." It is stated as an output of KeyGen, in the same way as **k** and sk.

The Rust implementation implements it faithfully, with five bytes per element, big-endian, in tuple order, but it does not use it anywhere in the specified scheme. `key_bytes` is called in exactly seven places, all on the legacy v7 path: `encrypt`, `decrypt`, `encrypt_bytes`, `encrypt_bytes_with_nonce`, `decrypt_bytes`, `encrypt_raw` and `decrypt_`

raw, and in each of them it supplies the HMAC key. **No v8 function calls it.** On the v8 path, sk_fmt is the key of every derivation, and **k** is used only directly, without any HMAC, in the arithmetic token map, which is exactly the role that Remark 3.3 describes for it.

So key_bytes is specified normatively as part of the scheme, but nothing in the scheme uses it. Its only use in the delivered crate is as the key of the derivations in the format that the paper places out of scope. This picture becomes consistent once CVF-11 is resolved and the 0x0B subkey layer is documented. As the paper stands, however, a reader who implements Definition 3.1 produces a byte string that nothing in Definition 3.10 uses, and the text gives them no way to find this out.

The encoding that is actually needed is the one that is missing. A second implementation must be able to load the key pair used by the known-answer test vectors, and the paper never says how a key pair is serialised. The crate has no combined encoding either: there is no parser, no Key type and no round-trip test. generate_v8_key returns a bare (Vec<u64>, [u8; SK_SIZE]), and every entry point takes the two halves as separate arguments. The test vectors use a third, ad-hoc format of their own: a JSON array of decimal integers for **k** and a hex string for sk. Since the key pair is never transmitted, this is a gap in completeness and not an interoperability failure in the usual sense. However, Section 10 requires that different implementations produce identical bytes, and the key encoding of the test vectors is the one thing that a reader must reproduce exactly but that the paper does not define. As written, Definition 3.1 specifies an encoding that the scheme does not use and omits the encoding on which the conformance test depends.

There is also a latent hazard in key_bytes itself, which we record here because it is in the same three lines. Neither key_bytes nor the shared be5 helper checks that its argument fits in five bytes:

```
let b = k.to_be_bytes();           // 8 bytes
out.extend_from_slice(&b[3..8]);   // low 5 bytes, big-endian
```

The top three bytes are discarded without any error. Since validate_key enforces no upper bound on key elements (CVF-16), any accepted prime at or above 2^{40} is silently reduced modulo 2^{40} when it is serialised. On the legacy path, two different keys that are congruent modulo 2^{40} therefore give the **identical** HMAC key, while the token arithmetic uses the full 64-bit values, so encryption and decryption then silently disagree. The Notation section of the paper states the precondition ("each field is fixed-width; x is assumed to fit"), and Remark 3.11 shows that it holds for the *token* field only, never for the be5 key field. Enforcing MAX_KEY_PRIME in validate_key, as CVF-16 recommends, makes this case unreachable, which is why we record it here instead of reporting it as a separate finding.

Recommendation

State what key_bytes is for, and specify the key-pair encoding that the conformance test vectors require.

In Definition 3.1, describe the role of key_bytes(**k**) instead of deleting it. Once the subkey layer of CVF-11 is documented, the natural description is that it is an encoding of the arithmetic key for storage and transport, and not an input to any algorithm in Definition 3.10. One clause is enough: note that **k** enters the construction

only through the token map $c \mapsto c \cdot k_j + a$, so that key_bytes plays no part in Enc or Dec and is given here for serialisation only. This removes the false impression that a conforming implementation must compute it, and it is consistent with what Remark 3.3 already says about the roles of the two secrets. If the intent is instead that key_bytes has no role at all in the specified scheme, delete it from Definition 3.1 and note in Section 8.2 that the legacy v7 format uses it as its HMAC key.

Add the key-pair encoding to Section 10, where the test vectors are described, and not to Section 3, since it concerns the test vectors and not the construction. State that a key pair is given in the test vectors as a JSON array of decimal prime values in tuple order, together with a lowercase hex encoding of the 32-byte sk, matching the key and sk_hex fields that the vectors actually use. These two sentences remove the only ambiguity that a second implementer faces when loading the vectors, and Section 10 already asks them to reproduce those vectors byte for byte.

In the code, if a Key newtype is introduced for the reasons given in CVF-16 and CVF-29, give it a pair of functions for parsing and serialising at the same time, together with one round-trip test. This would replace three encodings, i.e. the be5 concatenation, the loose (Vec<u64>, [u8; 32]) tuple and the JSON of the test vectors, with one, and would give the crate a place to enforce the five-byte bound that key_bytes currently assumes.

Finally, make be5 and key_bytes reject an element that is out of range instead of truncating it, or at least add a debug_assert!(k < 1u64 << 40). Enforcing MAX_KEY_PRIME in validate_key as CVF-16 recommends makes this case unreachable through the public API, but the helpers are also used for the ct_pos and real_idx counters, and a bound stated in the encoder is easier to keep correct than one inferred from every function that calls it.

/docs/napseq-eprint-v4.tex

```
  \item $\mathrm{be4}(x)$, $\mathrm{be5}(x)$, $\mathrm{be8}(x)$ denote the
        4-, 5-, and 8-byte big-endian encodings of the non-negative integer
        $x$ (each field is fixed-width; $x$ is assumed to fit).
```

same file

```
function relates $\mathsf{sk}$ to $\mathbf{k}$. The serialised arithmetic
key is
\[
  \mathsf{key\_bytes}(\mathbf{k}) = \mathrm{be5}(k_1) \| \mathrm{be5}(k_2) \| \cdots \| \mathrm{be5}(k_K).
\]
```

same file

```
claiming more codepoints than the padded buffer holds. Each positive vector
fixes $(\mathbf{k},\mathsf{sk},A,M)$ and the expected
ciphertext; because $\mathsf{Enc}$ is a pure function of its inputs
```

```
(Definition~\ref{def:aead-triple}), no nonce field appears and none is
needed. Conforming implementations must produce byte-identical ciphertext
for every positive vector and reject every negative vector. The generator
```

/src/lib.rs

```rust
fn key_bytes(key: &[u64]) -> Vec<u8> {
    let mut out = Vec::with_capacity(key.len() * 5);
    for &k in key {
        let b = k.to_be_bytes(); // 8 bytes
        out.extend_from_slice(&b[3..8]); // low 5 bytes, big-endian
    }
    out
}
```

same file

```rust
fn be5(n: u64) -> [u8; 5] {
    let b = n.to_be_bytes();
    let mut out = [0u8; 5];
    out.copy_from_slice(&b[3..8]);
    out
}
```

same file

```rust
    let kb = key_bytes(key);
```

same file

```rust
    let kb = key_bytes(key);
```

same file

```rust
    let kb = key_bytes(key);
```

same file

```rust
    let kb = key_bytes(key);
```

same file

```rust
    let kb = key_bytes(key);
```

same file

```rust
    let kb = key_bytes(key);
```

same file

```rust
    let kb = key_bytes(key);
```

same file

```
pub fn generate_v8_key(count: usize, min_val: u64, max_val: u64) ->
(Vec<u64>, [u8; SK_SIZE]) {
    let primes = generate_prime_numbers(count, min_val, max_val);
    let mut sk = [0u8; SK_SIZE];
    rand::thread_rng().fill_bytes(&mut sk);
    (primes, sk)
}
```

CVF-39: fixed Category: **documentation**

Summary

A code comment states the average ciphertext size before the length fix as about 13.4 times the padded message, while the correct figure is about 8.4, so the comment understates the bandwidth cost of the fix.

The code always makes encrypted messages a fixed size, so that their length reveals nothing about the content. A comment in the code explains this choice and gives its price by comparing the fixed size with the average size before the change. The average it quotes is wrong: the real average is lower, so the fixed size costs about 2.4 times more bandwidth than before, not about 1.5 times as the comment suggests. This affects only the comment. The design choice is correct, and no security claim depends on the number.

The doc comment on `MAX_NOISE_RUN` is the only quantitative statement in the crate about the ciphertext expansion before the token ceiling was introduced. It correctly sets out the reasoning behind the ceiling, which was introduced to stop the ciphertext length from revealing information, and then ends: "After this fix, v8 ciphertext length is a deterministic function of the padding bucket alone, at the cost of always paying the worst-case ~20x expansion instead of the **~13.4x average case**."

When we recompute the average under the distribution that the paper itself establishes, the figure does not match. Two different quantities give numbers close to 13.4, and the comment has used the wrong one.

With the run cap in force, the token count per padded codepoint is $1 + \sum_{j=1}^{19} p^j$ with $p = \theta(N)/2^{64}$. `Remark 3.7` establishes that $\theta(N)$ is within total variation distance ≈ 0.033 of uniform on $[\theta_{\min}, \theta_{\max}]$. We recomputed that distance and confirm that it is exactly $1/30$, so p is approximately uniform on $[0.75, 0.99]$. The mean of the capped count over that range, computed in exact rational arithmetic, is **8.4208** (8.42075 exactly). Some point values for orientation: 3.99 at $p = 0.75$, 18.21 at $p = 0.99$, and 7.218 at the midpoint $p = 0.87$.

The figure 13.4 can be obtained in two ways, and neither is the capped mean. First, it is the mean of the **uncapped** count $1/(1-p)$ over the same range, which is 13.412, i.e. the expansion of a scheme with no run cap at all. Second, by coincidence, it is also the capped count evaluated at the single point $p = 0.955$, which is 13.374; this treats the noise probability as if it were near the top of its range instead of at its mean. Both readings give the printed number, but neither is the average that the sentence claims.

As a result, the comment understates what the ceiling costs. The true ratio is $20/8.42 = 2.38\times$, not $20/13.4 = 1.49\times$, so the fix costs roughly sixty percent more bandwidth than the comment says. The comment exists to record the price of the ceiling, so a reader deciding whether to keep it, or comparing the expansion of NAPQES with that of an alternative scheme, is given the wrong number.

This is a defect in the code documentation only. The paper states no average expansion: Section 7 and Section 12 both give the fixed figure of 20 tokens, or 160 bytes, per padded codepoint, which is correct and is what `Lemma 3.9` requires. No security claim, bound or theorem depends on the average before the fix. The design decision that the comment justifies is correct on other grounds: the length-hiding argument needs a deterministic length, and no figure for the average expansion could change that.

Recommendation

Replace "~13.4x average case" with the capped mean, and say which quantity is being averaged.

The clearest form states both figures, since both are informative and the confusion between them is what caused the error:

> at the cost of always paying the worst-case 20x expansion. Under the run cap alone, the mean token count per padded codepoint is about 8.4 for $\theta(N)$ approximately uniform on $[\theta_{\min}, \theta_{\max}]$ (`Remark 3.7`), ranging from 3.99 at $p = 0.75$ to 18.21 at $p = 0.99$; without the run cap it would be about 13.4. The ceiling therefore costs roughly 2.4x in bandwidth over the capped natural mean, and buys length determinism.

This is a small edit, and it makes the trade-off easy to follow: the reader sees the cost of the ceiling separately from the cost of the run cap, which the current sentence mixes together.

If the intent was in fact to quote the uncapped mean, as a comparison with a hypothetical scheme that has neither bound, say so explicitly ("without the run cap, the mean would be 13.4"). As printed, the sentence attributes the figure to the code as it stood after the run cap was introduced and before the ceiling was.

Consider also adding the derivation, or a pointer to it, in one line: the capped count is $1+\sum_{j=1}^{19} p^j$ and the uncapped one is $1/(1-p)$, both averaged over p approximately uniform on $[0.75, 0.99]$. The crate's comments are unusually thorough elsewhere, and this is the only quantitative claim in them that a reader cannot check from the surrounding text.

No code change is required, and the ceiling should be kept. This finding concerns the number in the comment, not the design decision it justifies.

/docs/napseq-eprint-v4.tex

```
The noise threshold $\theta(N)$ (Section~\ref{sec:noiseprob}) deserves an
explicit correction here: it is \emph{not} bias-free. Although it uses exact
integer arithmetic and no modulo reduction, the fixed-point scaling
$\theta_{\min}+\lfloor\tau(\theta_{\max}-\theta_{\min})/2^{64}\rfloor$ maps
$2^{64}$ equally likely draws of $\tau$ onto approximately
$0.24\cdot2^{64}$ outputs, i.e.\ about $4.17$ preimages per output. Since
each output receives either $4$ or $5$ preimages, individual outputs
deviate from uniform on the range by up to $+20\%$ in relative terms, and
the total variation distance from the uniform distribution on
$[\theta_{\min},\theta_{\max}]$ is $\approx0.033$. This is harmless — the
only property required of $\theta(N)$ is that it land in the public
interval $[\theta_{\min},\theta_{\max}]$, which it does by construction,
and that $\mathsf{Dec}$ recompute it identically, which it does — but it
should not be described as exempt. The prime-index sampling of
Remark~\ref{rem:sampling} is genuinely bias-free, by rejection.
```

same file

```
To keep ciphertext length independent of the message-derived nonce's
particular noise realisation (Section~\ref{sec:wire-format}), the number
of \emph{consecutive} noise tokens emitted before each real token is capped
at a fixed constant $\mathrm{MAX\_NOISE\_RUN}=19$, and the total token
count is padded, with additional filler tokens structurally identical to
genuine noise tokens, up to a fixed ceiling of exactly
$R\cdot(\mathrm{MAX\_NOISE\_RUN}+1)$ tokens per message:
```

/src/lib.rs

```
/// Hard cap on consecutive noise tokens emitted before a real token in the
/// v8 token-emission loop (never applied to v7, matching `napqes.py`/
/// `C/napqes.c`). As of the V2-CVF2 fix, this cap also fixes the *total*
/// per-real-token budget: every v8 ciphertext is padded with additional
```

```
1053 /// filler tokens up to a deterministic per-bucket ceiling of
1054 /// `real_token_count * (MAX_NOISE_RUN + 1)` tokens (see
1055 /// `encrypt_bytes_v8` / `decrypt_core_v8`). Without this, the
     *natural*
1056 /// token count varies with the message-derived synthetic nonce even
     for a
1057 /// fixed padding bucket, letting an observer who collects several
1058 /// ciphertexts of one message under varying AAD average out the
     noise and
1059 /// reliably recover the padding bucket (`docs/CAVEATS.md`, V2-CVF2).
     After
1060 /// this fix, v8 ciphertext length is a deterministic function of the
1061 /// padding bucket alone, at the cost of always paying the worst-case
     ~20x
1062 /// expansion instead of the ~13.4x average case.
1063 pub const MAX_NOISE_RUN: u64 = 19;
```

CVF-40: fixed Category: **readability**

> **Summary**
>
> **The specification contains nine small inconsistencies that could mislead an implementer, and one of them, a mixed counting convention, would make an implementation built from the text incompatible with the reference code.**
>
> We collected nine small errors in the parts of the specification that an implementer follows step by step. None of them weakens security, and each can be fixed with a one-line edit. Most only make the text harder to follow, but one of them matters more: two places describe the same value with different counting conventions, one starting from zero and the other from one. A developer who follows the wrong one would build software that cannot decrypt anything produced by the reference code, with no error message to point to the cause. One item concerns the code: a limit that the specification names is written as a bare number in two places in the code, which is how those two places came to behave differently.

We checked each item against the source and recomputed every figure involved. The items are grouped because each is a one-line or one-sentence repair and none justifies a separate finding. We report them because all of them are in text that a conforming implementer reads as normative.

1. Section 3.3 claims more length-prefixing than the construction has, and the proof of `Lemma 3.5` contradicts it. The derivation display is introduced with "every variable-width field inside ctx is length-prefixed (e.g. be8$(|A|)\|A$ for AAD A)". This is not true of

two fields: $\mathcal{B}$ in domain `0x03` and M in domain `0x0A` are both variable-width and carry no prefix. Injectivity for both comes from their being **last**, which is exactly how the proof of `Lemma 3.5` argues it: "because be8$(|A|)$ fixes $|A|$, the split between A and $\mathcal{B}$ is unambiguous". The general sentence therefore contradicts the lemma it introduces, and it states a stronger property than the construction has or needs.

2. The output column of Table 1 uses an index that is never defined. Rows `0x01` and `0x05` give the output range as $[1, k_i - 1]$. Neither the table, its caption nor the surrounding text says what i ranges over or which key element k_i denotes. `Definition 3.6` resolves this by taking k as a free parameter, but the table is the summary that a reader consults first.

3. Section 3.7 indexes the key tuple from 0, while the rest of the paper indexes it from 1. Section 3.7 writes the key element as $k = \mathbf{k}[i \bmod K]$: bracket notation, indexed from 0, and with i itself undefined, as in item 2. Step (7) of `Dec` writes the same quantity as $k_{(\text{real_idx mod } K)+1}$: subscript notation, explicitly indexed from 1, over the tuple $(k_1, \ldots, k_K)$ that `Definition 3.1` declares. Both mean the same element, and the code agrees with the form indexed from 1 and with Listing 1 (`primes[(real_idx % kk) as usize]`). However, a reader who implements Section 3.7 as written, using the paper's own tuple indexed from 1, selects the wrong prime, one position off, for every token. The resulting implementation fails to interoperate with any correct one, and nothing in its output shows why.

4. Table 1 does not record that the modulus of domain `0x05` comes from a different counter than its context. The context of the noise-token addend is be5(ct_pos), but the modulus k that it is reduced by is selected by `real_idx`, not by `ct_pos`: the noise tokens emitted inside a run use the key element of the real token they precede. Only Listing 1 records this, on one line. `Definition 3.6` takes k as a free parameter and so does not settle the question, and Table 1 shows only the context. The code does the right thing.

5. θ_{max} can never be reached, so the stated interval is in fact half-open. `Remark 3.7` says that the only property required of $\theta(N)$ "is that it land in the public interval $[\theta_{\text{min}}, \theta_{\text{max}}]$", and Table 1 gives the range as $[0.75, 0.99]$. We recomputed the maximum: with

$$\Delta = \theta_{\text{max}} - \theta_{\text{min}} = 4{,}427{,}218{,}577{,}690{,}292{,}387$$

, the largest attainable value is

$$\theta_{\text{min}} + \lfloor (2^{64} - 1)\Delta / 2^{64} \rfloor = \theta_{\text{min}} + \Delta - 1 = \theta_{\text{max}} - 1$$

, so $p \in [0.75, 0.99)$ strictly. The closed interval contains the attainable range, so nothing is wrong, but there is a related minor point in the same remark. The total variation distance is measured "from the uniform distribution on $[\theta_{\text{min}}, \theta_{\text{max}}]$", which contains $\Delta + 1$ integers, while the image of the map contains only Δ. The difference is $O(1/\Delta)$ and does not change any printed digit: we recomputed the distance in exact rationals as $1/30 = 0.0333\ldots$, which confirms the paper's ≈ 0.033.

6. The phrase "agreed out of band, exactly like k and K" contradicts the sentence before it. Section 3.6 says that the decryptor "needs *no* knowledge of the sender's profile", and two sentences later that "The profile is a deployment parameter agreed out of band, exactly like **k** and K". The comparison is wrong in the respect that matters: **k** and K *must*

be agreed, or decryption fails, while the profile need not be agreed at all, and a mismatch is undetectable and does not affect correctness. The crate's own comment describes this correctly, "The profile is a **sender-side** deployment parameter agreed out of band. It is never transmitted and `decrypt_bytes_v8` is profile-agnostic", and `decrypt_bytes_v8` indeed takes no profile argument. The more substantive consequence, i.e. that the profile is neither authenticated nor transmitted, so the guarantee of `Corollary 4.14` depends only on the sender's configuration, is the subject of CVF-5. The misleading comparison is a separate one-line fix.

7. The parenthetical about the two message classes in Table 2 understates the short class. The text says that the two message classes "fall in different buckets (16 and 512) whatever the expansion factor". We recomputed the buckets. Under `bucket`, the short class (12 to 20 codepoints) spans **two** buckets: $B = 16$ for $n \in [12, 15]$ and $B = 32$ for $n \in [16, 20]$. Under `coarse(3)` it spans $B = 16$ and $B = 128$. The long class (300 to 500 codepoints) is in $B = 512$ under `bucket` and in $B = 1024$ under `coarse(3)`. The conclusion is unaffected, since the sets of buckets of the two classes remain disjoint and spanning a boundary does not make the distinguisher any weaker. However, the parenthetical describes the short class as occupying a single bucket, which is wrong for both non-trivial profiles in the table. In addition, the 50% figure in the `frame(1024)` row depends on $500 < 1024$, which the text does not say.

8. Section 12 describes the length prefix as "2-byte", while it consists of two codepoints. `Definition 3.10` and step (8) of `Dec` both correctly say "2-codepoint big-endian length prefix". The caveat in Section 12 says that the scheme "stores the plaintext length as a 2-byte big-endian integer". On the wire, the prefix is two *codepoints*, which become two real tokens, which occupy up to 40 token slots and 320 bytes. Both descriptions give the same cap of 65,535, so the conclusion of the caveat is correct, but its description of the mechanism understates the cost on the wire by a factor of 160.

9. `MAX_PLAINTEXT_CODEPOINTS` is a named parameter in the paper but a bare literal in the code. `Definition 3.10` names

$$\text{MAX_PLAINTEXT_CODEPOINTS} = 2^{16} - 1$$

, and the crate gives a named `pub const` counterpart to every other public parameter: `MIN_KEY_PRIME`, `MAX_KEY_PRIME`, `MAX_NOISE_RUN`, `PAD_MIN_EXP`, `PAD_MAX_EXP`, `TOKEN_WIDTH`, `NONCE_SIZE`, `TAG_SIZE` and `SK_SIZE`. This parameter has none: it appears as `0xFFFF` in two places, and those two places have **come to behave differently on failure**, since one panics and the other returns an error. That difference is the subject of CVF-34; the missing constant is what allowed it to happen.

We also checked two related points and found no problem with them. First, `Enc` derives the synthetic nonce over the whole message before it rejects an over-long one, so a rejected 100 MB input costs a full-message HMAC first. This is wasted work on an error path with no security consequence, and a conforming implementation may perform the two steps in either order, since both are checks on the input. Second, the blanket statement in the Notation section, "each field is fixed-width; x is assumed to fit", never states the limit of 2^{40} that be5 imposes on `ct_pos` and `real_idx`. We confirmed that the condition holds at the current parameters, since the largest reachable `ct_pos` is $20 \times 65{,}538 = 1{,}310{,}760 \ll 2^{40}$, but it holds because of a bound that the paper does not state.

Recommendation

Make nine small edits: seven in the paper and two that span both the paper and the code.

1. Narrow the sentence in Section 3.3 to what `Lemma 3.5` actually proves: every variable-width field inside ctx is either length-prefixed or placed last. This covers A (prefixed) as well as $\mathcal{B}$ and M (placed last), and it matches the proof exactly.
2. Define the index in Table 1. Writing the output range as $[1, k-1]$ for the key element k selected at that position, and adding one clause to the caption saying which counter selects it, removes the undefined variable without making the table longer.
3. Make Section 3.7 use the paper's own tuple notation, indexed from 1: $k = k_{(\text{real_idx mod } K)+1}$, matching step (7) of `Dec` and Listing 1. Fix this item first: it is the only one in this finding whose misreading produces a completely incorrect implementation, and the fix is two symbols.
4. Add a note to the `0x05` row of Table 1, or a sentence to `Definition 3.6`, saying that the modulus of the noise-token addend is the key element selected by `real_idx`, while its context is `be5(ct_pos)`. At present a reader must work this out from Listing 1.
5. In `Remark 3.7`, write the attainable range as $[\theta_{\min}, \theta_{\max})$ and note that $\theta_{\max}$ itself is not attained, and change the range column of Table 1 to $[0.75, 0.99)$ for consistency. The total variation figure needs no change, since ≈ 0.033 is correct and the exact value is $1/30$, but the sentence that measures it against the uniform distribution on the closed interval can say "on the image of the map" instead.
6. Replace "exactly like **k** and K" with the distinction that the code already makes: the profile is a sender-side parameter that need not be agreed, unlike **k** and K, which must be. This takes one clause.
7. Correct the parenthetical so that it names the buckets that the classes actually occupy, $\{16, 32\}$ against $\{512\}$ under `bucket` and $\{16, 128\}$ against $\{1024\}$ under `coarse(3)`, and add half a sentence explaining that the 50% of `frame(1024)` follows from 1024 exceeding the length of both classes.
8. Change "2-byte big-endian integer" in Section 12 to "2-codepoint big-endian prefix", matching `Definition 3.10`. If the caveat is meant to convey the storage cost, say that the two codepoints become two real tokens and occupy up to 320 bytes on the wire, since that is the number a deployer needs when sizing a channel.
9. Add `pub const MAX_PLAINTEXT_CODEPOINTS: usize = 0xFFFF;` to the crate and use it in both existing places. This item gives the most benefit for its cost: naming the bound is what keeps the two checks from diverging again, and they have already diverged once.

   In addition, consider adding one sentence to the Notation section saying that be5 limits any counter it encodes to 2^{40}, and that `ct_pos` and `real_idx` stay within this limit because MAX_PLAINTEXT_CODEPOINTS and the token ceiling together limit `ct_pos` to $20(2^{16}+2) = 1{,}310{,}760$. The blanket "assumed to fit" carries real weight here, and at present the reader is left to check the condition alone.

/docs/napseq-eprint-v4.tex

```
  \item $\mathrm{be4}(x)$, $\mathrm{be5}(x)$, $\mathrm{be8}(x)$ denote the
        4-, 5-, and 8-byte big-endian encodings of the non-negative integer
        $x$ (each field is fixed-width; $x$ is assumed to fit).
```

same file

```
where $d$ is a 1-byte domain separator and every variable-width field
inside $\mathsf{ctx}$ is length-prefixed (e.g.\ $\mathrm{be8}(|A|) \| A$
for AAD $A$). Domain $\texttt{0x0A}$ is the single exception: it does not
```

same file

```
\texttt{0x01} & Real-token addend & $\mathrm{be5}(real\_idx)$ & $[1, k_i - 1]$ \\
```

same file

```
\texttt{0x05} & Noise-token addend & $\mathrm{be5}(ct\_pos)$ & $[1, k_i - 1]$ \\
```

same file

```
for domain $\texttt{0x03}$,
$\mathsf{ctx}=\mathrm{TagIn}(A,\widetilde{B})=\mathrm{be8}(|A|)\|A\|\widetilde{B}
and because $\mathrm{be8}(|A|)$ fixes $|A|$, the split between $A$ and
$\widetilde{B}$ is unambiguous, so distinct $(A,\widetilde{B})$ pairs never
collide. Within domain $\texttt{0x0A}$, whose input is
$\texttt{0x0A}\|\mathrm{be8}(|A|)\|A\|M$ with no nonce field at all, the
same length-prefix argument shows $\mathrm{be8}(|A|)\|A\|M$ is injective in
$(A,M)$. Every case is a statement about byte layout that holds for
```

same file

```
\emph{no} knowledge of the sender's profile: it recovers $R$ from the token
count (Section~\ref{sec:wire-format}) and the true length $n$ from the
```

same file

```
check against the same fixed set. The profile is a deployment parameter
agreed out of band, exactly like $\mathbf{k}$ and $K$; it is never
transmitted and does not appear in the wire format.
```

same file

```
the total variation distance from the uniform distribution on
```

```
$[\theta_{\min},\theta_{\max}]$ is $\approx0.033$. This is harmless — the
only property required of $\theta(N)$ is that it land in the public
interval $[\theta_{\min},\theta_{\max}]$, which it does by construction,
```

same file

```
Real and noise tokens are integers $c\cdot k + a$ for codepoint $c$
(or a noise character), key element $k=\mathbf{k}[i \bmod K]$, and addend
$a$ from Definition~\ref{def:token-prims}. Because $k$ is prime and every
```

same file

```
        $\mathrm{MAX\_PLAINTEXT\_CODEPOINTS}=2^{16}-1$, the
        limit imposed by the 2-codepoint big-endian length prefix written
        during padding (Section~\ref{sec:padding}). The surrogate range is
```

same file

```
        excluded because the synthetic-nonce derivation
        (Definition~\ref{def:synthnonce}) hashes the UTF-8 encoding of $M$,
        and UTF-8 is undefined on unpaired surrogates; $\mathcal{M}$ is thus
```

same file

```
token $t$ and recover the padded codepoint as
```

same file

```
addend $\mathsf{Enc}$ used at that index; stop once $R$ real tokens have
been extracted, discarding any trailing filler; (8) read the 2-codepoint
big-endian length prefix $n$ and, if $n>R-2$, output $\bot$
```

same file

```
the two classes fall in different buckets ($16$ and $512$) whatever the
```

same file

```
  \item[Plaintext length cap.] The block padding scheme stores the
        plaintext length as a 2-byte big-endian integer, capping messages
```

/src/lib.rs

```
fn be5(n: u64) -> [u8; 5] {
```

```
    let b = n.to_be_bytes();
    let mut out = [0u8; 5];
    out.copy_from_slice(&b[3..8]);
    out
}
```

same file

```
/// The profile is a sender-side deployment parameter agreed out of band. It is
/// never transmitted and [`decrypt_bytes_v8`] is profile-agnostic.
```

same file

```
   assert!(n <= 0xFFFF, "Message too long for 2-byte length prefix");
```

same file

```
    let nonce = synthetic_nonce(&sk_fmt, aad, message.as_bytes());
    let codepoints: Vec<u32> = message.chars().map(|c| c as u32).collect();
```

same file

```
    if codepoints.len() > 0xFFFF {
```

CVF-42: fixed Category: **naming**

> **Summary**
>
> **Three fixed values of the specification are written as bare numbers in the older (v7) code while the newer (v8) code gives them names, which makes it easy for a later edit to change the format by accident.**
>
> The code gives a name to almost every fixed value of the scheme, so that each value is defined once and is easy to find. Three values are exceptions: in the older encryption functions they appear as bare numbers. Two very similar numbers, 95 and 96, appear a few lines apart, and both are correct, but only one of them has a comment explaining it. A developer who later "corrects" one to match the other would silently change the encrypted output. Everything is correct today and there is no security impact; this finding is about keeping it that way.

The crate gives a named `pub const` to almost every public parameter of the scheme: `MIN_KEY_PRIME`, `MAX_KEY_PRIME`, `MAX_NOISE_RUN`, `PAD_MIN_EXP`, `PAD_MAX_EXP`, `TOKEN_WIDTH`, `NONCE_SIZE`, `TAG_SIZE` and `SK_SIZE`. Three values that `Definition 3.6` fixes normatively are exceptions, and in two of the three cases the *same* quantity is a named constant on the v8 path and a literal on the v7 path.

The endpoints of the noise probability are the clearest example. `derive_noise_p`, the v7 form, writes them inline:

```
let t = u64_from_be8(&d[..8]) as f64 / TWO_POW_64;
0.75 + t * (0.99 - 0.75)
```

while the v8 form, five lines below, declares them:

```
const THETA_MIN: u64 = ((75u128 << 64) / 100) as u64;
const THETA_MAX: u64 = ((99u128 << 64) / 100) as u64;
```

and `derive_noise_threshold_v8` uses those names. So θ_{min} and θ_{max} are named in one derivation, while in the other they are four floating-point literals: `0.75`, `0.99`, and `0.75` again inside the difference. A change to either endpoint has to be made in two notations in two places, and one of those places does not mention the parameter's name.

The noise-character alphabet is the second example. `derive_noise_char` ends with `(u32_from_be4(&d[..4]) as u64 % 96) + 32`, with no constant and no comment. The similar expression in `pad_to_block` ends with `(u32_from_be4(&d[..4]) % 95) + 32`, with the comment `// [32, 126]`.

We checked whether the difference between the two moduli is itself a defect, and it is not. `Definition 3.6` specifies mod 96 for `noise_char`, which gives $[32, 127]$, and mod 95 for the padding codepoint, which gives $[32, 126]$, and both places conform. However, the two functions are five lines apart, their unexplained moduli differ by one, and both are correct. This is exactly the situation in which a later editor "fixes" the discrepancy and silently changes the wire format, and only one of the two places carries the comment that would prevent it.

All three places match the specification, so there is no defect today and no security consequence. We report this because the ninth item of CVF-40 makes the same argument about `MAX_PLAINTEXT_CODEPOINTS`: that parameter was left unnamed and appeared as `0xFFFF` in two places, and **those two places then came to behave differently on failure**. The same precondition holds in all three cases here.

Recommendation

Name the three parameters and use the names in every place, following the pattern that `THETA_MIN` and `THETA_MAX` already establish on the v8 path:

```
/// Noise-character alphabet, `Definition 3.6`: `(... mod
  NOISE_ALPHABET) + NOISE_ALPHABET_BASE`.
const NOISE_ALPHABET: u64 = 96;        // yields [32, 127]
/// Padding-codepoint alphabet, `Definition 3.6`: printable ASCII
  only.
const PAD_ALPHABET: u64 = 95;          // yields [32, 126]
const ASCII_PRINTABLE_BASE: u64 = 32;
```

and express the v7 endpoints through named constants, as the v8 path already does, instead of as a second set of literals in a second numeric type:

```
const THETA_MIN_F64: f64 = 0.75;
const THETA_MAX_F64: f64 = 0.99;
...
```

```
THETA_MIN_F64 + t * (THETA_MAX_F64 - THETA_MIN_F64)
```

Add a one-line comment at `derive_noise_char` that states the resulting range, as `pad_to_block` already does, and that says the difference from the padding modulus is intentional and normative. This comment is worth adding even if the constants are not introduced, because it is what prevents a future editor from reconciling 96 and 95 and changing the wire format in the process.

Name `MAX_PLAINTEXT_CODEPOINTS` in the same change. Item 9 of CVF-40 asks for it, and the two places where that value is used have already diverged. Doing all four in one change keeps the parameters declared in the crate in one-to-one correspondence with the parameters listed in `Definition 3.6`.

/src/lib.rs

```
196     (u32_from_be4(&d[..4]) as u64 % 96) + 32
```

same file

```
213     let t = u64_from_be8(&d[..8]) as f64 / TWO_POW_64;
214     0.75 + t * (0.99 - 0.75)
```

same file

```
219 const THETA_MIN: u64 = ((75u128 << 64) / 100) as u64;
220 const THETA_MAX: u64 = ((99u128 << 64) / 100) as u64;
```

same file

```
417         out.push((u32_from_be4(&d[..4]) % 95) + 32); // [32, 126]
```

CVF-44: fixed Category: **documentation**

> **Summary**
>
> **When an encrypted message is shorter than expected, the v8 decryptor returns an error message that is correct in only one of the three cases that lead to it.**
>
> When decryption finds that an encrypted message is shorter than its header implies, it returns an error message that describes the cause. Of the three situations that lead to this error, the message is accurate for only one and misleading for the other two. This affects only the error text and has no security impact, but it may send a developer investigating a problem in the wrong direction.

In the v8 decryptor, the noise-run loop, i.e. the loop that skips the noise tokens placed between real tokens, is followed by a single check for the end of the token stream, with a single message:

```
while noise_run < MAX_NOISE_RUN
```

```
    && ct_pos < n_tokens
    && is_noise_pos_v8(kb, nonce, ct_pos, noise_theta)
{
    ct_pos += 1;
    noise_run += 1;
}
if ct_pos >= n_tokens {
    return Err(
        "Truncated v8 ciphertext: token stream ended mid noise-run
  before the \
         expected real token.".into(),
    );
}
```

Three different states satisfy `ct_pos >= n_tokens` at that point, and the message describes only the first one:

1. The inner loop exited on `ct_pos < n_tokens` with `noise_run > 0`, i.e. the stream really did end in the middle of a noise run. The message is accurate.
2. The inner loop exited on `noise_run < MAX_NOISE_RUN`, after consuming a **complete** run of `MAX_NOISE_RUN` noise tokens that happened to end exactly at `n_tokens`. Nothing was cut off in the middle of the run: the run finished, and the real token that should have followed it is missing. The message says the opposite of what happened.
3. The inner loop exited on `ct_pos < n_tokens` with `noise_run == 0`, i.e. the previous real token was the last one in the stream and no noise run was entered at all. There is no noise run to be in the middle of.

In all three cases, what a reader needs to know is the same: the token stream is shorter than the token count implies. In two of the three, however, the message points them towards the noise schedule instead.

This state can only be reached by a ciphertext that has already passed the tag check, the token-count divisibility check and the bucket check, so this is a problem of diagnostic quality with no security consequence. We report it because the error strings of `decrypt_core_v8` are the only description of the structural checks performed after authentication that an operator will ever see at run time. In addition, CVF-25 records that the negative test vectors check only `is_err()` and never which error is returned. An inaccurate message is therefore not caught by any test, and a vector aimed at one of these three states cannot currently be told apart from the others.

Recommendation

Make the message describe the condition itself instead of one of its causes, and include the counters that distinguish the cases:

```
if ct_pos >= n_tokens {
    return Err(format!(
        "Truncated v8 ciphertext: the token stream ended after {}
  of {} tokens \
```

```
            with {} of {} real tokens recovered; a real token was
  expected at \
            position {} (noise run length {}).",
           ct_pos, n_tokens, real_idx, real_count, ct_pos, noise_run
    ));
}
```

The counters are already in scope, and with them the message no longer needs to guess which of the three states caused the error, since the reader can see it. If a fixed string is preferred, "Truncated v8 ciphertext: token stream ended before the expected real token" is accurate in all three cases and differs from the existing message only by two removed words.

When the recommendation of CVF-25 to check the error text in the negative test vectors is applied, add a vector for case 2: a ciphertext whose final bucket ends with a complete run of MAX_NOISE_RUN noise tokens and whose last real token has been removed. This is the case that the current message describes wrongly, and it is easy to construct with the existing generator.

/src/lib.rs

```
576     while real_idx < real_count {
577         let mut noise_run: u64 = 0;
```

same file

```
578         while noise_run < MAX_NOISE_RUN
579             && ct_pos < n_tokens
580             && is_noise_pos_v8(kb, nonce, ct_pos, noise_theta)
581         {
582             ct_pos += 1;
583             noise_run += 1;
584         }
```

same file

```
585         if ct_pos >= n_tokens {
586             return Err(
587                 "Truncated v8 ciphertext: token stream ended mid
noise-run before the \
588                  expected real token.".into(),
589             );
590         }
```

6. Review of Fixes at Commit acbf9d0

After receiving the findings in Section 5, the client fixed the code and asked us to review the fixes. This section reports that review. We checked whether each finding of Section 5 was fixed, and we looked for new problems that the fixes may have introduced. The code we reviewed is the client's commit

`acbf9d05bd4ce93a77d91e2d2a5e9a6ed4a302e1`

This commit is in the client's published repository, `Synergeon-epineon/NAPQES`. It contains the client's fixes for the findings in Section 5. Every in-scope file delivered for review matches it byte for byte; we verified this by comparing the hash of each file with the published commit.

What was reviewed. The review covered the client's seven in-scope files: the Rust sources `src/lib.rs`, `src/self_test.rs`, `src/kat_cross_check.rs`, `fuzz/fuzz_targets/decode_bytes.rs` and `examples/dudect_harness.rs`, together with `Cargo.toml` and `README.md`. We extended the scope to three files that are new in this commit, `rust/src/key.rs` and the fuzz targets `fuzz/fuzz_targets/unauth_decode.rs` and `fuzz/fuzz_targets/validate_key.rs`, because the fixes for three findings of Section 5 were moved into them. The updated specification delivered with the commit served as the reference against which the code was checked. Despite its file name, `docs/napseq-eprint-v3.tex` is the newest revision (see CVF-V5-23). The review was static: none of the client's code was compiled or executed.

How. We checked every finding of Section 5 again against the new code. Separately, we re-implemented encryption from the updated specification alone, and the re-implementation reproduced all 22 positive v8 test vectors byte for byte. This resolves the difference between the specification and the implementation reported in CVF-11: the specification now describes the scheme that the implementation computes.

Outcome for the findings of Section 5. Of the 37 findings, 25 are fixed and 12 are partially fixed. None is unresolved, and none has become worse. The statuses shown in Section 5 reflect this review: fixed findings are marked fixed, and partially fixed findings remain open. Each partially fixed finding is carried into this section as a new finding that describes what remains.

New findings. This section reports 30 findings: 8 of moderate severity and 22 minor, with none major. Eighteen of them are new. Seventeen of these were introduced by the fixes themselves, and one is an older problem reported for the first time (CVF-V5-21). The other twelve describe what remains of the partially fixed findings.

Scale of the change. Compared with the baseline commit, the fixes add 1,291 new lines of production code to the reviewed files, of which 934 are security-critical (cryptographic cores, key and nonce derivations, validation and gating). The count excludes blank lines, comments, test code and code that was only moved.

Client comments. The Client Comment under each finding in this section was supplied by the client after this review. The client reports one fix per finding, in the commit range `acbf9d0...a2fbd13`, ending at commit `a2fbd130b85a326c020def2fd48b0e5c17a11c38`. These comments are the client's statements and are reproduced as received, except that references to a commit identifier that is not part of the published history were replaced with `acbf9d0`. ABDK has not reviewed the fixes they describe. The status of every finding reflects our review of commit `acbf9d0`, not the client's later changes.

Specification. The specification delivered with commit `acbf9d0` served only as the reference against which the code was checked. Its new material, including the restated security theorems, has not been reviewed, and neither has the client's later revision of the specification (revision 4.2).

Numbering. Findings in this section carry identifiers of the form CVF-V5-N. A plain CVF-N anywhere in this report, including inside the findings below, refers to a finding in Section 5, and each such reference links to it. The source excerpts quote the client's code and comments verbatim. Some of those comments cite the client's own labels for findings of earlier reviews (for example `CVF1`, `V2-CVF11` or "CVF-3-class"). These labels do not refer to findings in this report.

	Found	Fixed
Moderate	**8**	**0**

	Found	Fixed
Minor	**22**	**0**

Fixed 0 out of 30 issues reported in this section.

6.1. Moderate

CVF-V5-1: open Category: **flaw**

Summary

The streaming encryption format added by the fixes reveals the exact length of every message, although the documentation says that message lengths are hidden.

The fixes add a way to encrypt a long message in pieces, i.e. streaming. At the end of each encrypted stream, the code writes the exact number of characters of the original message without encrypting it, so anyone who sees the encrypted data learns the length of the message. The documentation lists the streaming functions among those that hide the message length, and the code even adds filler characters to disguise it, but the filler has no effect while the exact length is written next to it. An attacker cannot change the recorded length, because it is authenticated, but anyone can read it.

The fixes introduced this problem: the baseline code contained no streaming implementation, only an unused `FORMAT_STREAM_AE_V8` constant.

Both streaming encryptors end the stream with a sentinel, i.e. a closing record, made of a zero chunk-length marker, the total number of real codepoints as an unencrypted 8-byte big-endian integer, and the sentinel tag. `encrypt_stream_ae_v8_inner` appends `total_real_cps.to_be_bytes()` directly to the output (`rust/src/lib.rs:2019-2023`), and `StreamV8Encryptor::finish` does the same (`rust/src/lib.rs:2333-2337`). The decryptor reads the value back from the clear bytes before verifying the sentinel tag (`rust/src/lib.rs:2129-2136`). The tag authenticates the count, but it does not hide it.

The crate-level documentation (`rust/src/lib.rs:17-25`) lists `encrypt_stream_ae_v8`, `decrypt_stream_ae_v8` and `StreamV8Encryptor` among the v8 entry points and then states "Length-hiding property holds per Corollary 4.14 of the paper". `SPEC.md` section 8.2 (`SPEC.md:513-594`) likewise claims "length hiding at chunk granularity", and the filler codepoints derived under domain `0x06` exist only to pad the last chunk to `F` codepoints. However, the filler cannot hide the length, because the exact count it is meant to hide is written next to it. For example, with `F = 16`, messages of 17 and 31 codepoints both produce 5,257-byte ciphertexts, but the 8 bytes at offset `len - 40` read 17 and 31 respectively. In every positive vector of `tests/kat/v8_stream_vectors.json`, this field equals the length of the message.

The test `stream_v8_length_is_function_of_f_only` (`rust/src/lib.rs:3302-3311`) compares only the byte lengths of two ciphertexts, so it passes even though the length is transmitted in clear. Independently of the sentinel, the number of chunks reveals the length to within one frame (`ceil(n/F)`). With the default `F = 128`, this is far coarser than the at most 3.70 bits that the bucketing of the block format leaks.

We verified this by reading the code and with an independent implementation of the streaming format, which reproduces all 14 stream test vectors.

Recommendation

Do not transmit the number of real codepoints in clear. Either carry it inside the encrypted token stream of the final chunk, as the block format does with its two-codepoint length prefix, or mask it with a keystream under a new domain while keeping it bound by the sentinel tag.

Then state explicitly what the number of chunks still reveals, and remove the claim of Corollary 4.14 for the streaming entry points. Finally, replace the test with one that compares the full sentinel bytes of two messages with the same number of chunks.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-1 — Fixed in 7a17584. The sentinel no longer carries the codepoint count in clear: it carries `be8(n) XOR HMAC(sk_fmt, 0x0F || HEADER || be4(C))[:8]` (new domain `0x0F`), and the sentinel tag still binds the clear `n`. Only the chunk count `ceil(n/F)` remains visible, which SPEC.md §8.2 and the README now state. The crate documentation no longer claims Corollary 4.14 for streaming. The regression test now compares the sentinel count fields of two equal-chunk-count streams under one nonce. Python, C and Rust changed together; the stream KAT corpus was regenerated and all three ports reproduce it.

/rust/src/lib.rs

```
17 //! * **v8 (normative — use this).** Domain-`0x0B` format subkey
18 //!   ([`derive_format_subkey`], per CVF-11), synthetic nonce
19 //!   ([`synthetic_nonce`], domain `0x0A`), `MAX_NOISE_RUN`-capped
   emission
20 //!   loop, per-bucket token ceiling. Wire layout is
21 //!   `nonce(16) || masked_blob(fixed-width) || tag(32)`. Entry
   points:
22 //!   [`encrypt_bytes_v8`], [`decrypt_bytes_v8`],
   [`generate_v8_key`],
23 //!   [`encrypt_bytes_v8_with_profile`], [`encrypt_stream_ae_v8`],
24 //!   [`decrypt_stream_ae_v8`], [`StreamV8Encryptor`]. Length-hiding
   property
25 //!   holds per Corollary 4.14 of the paper.
```

same file

```
2019     let sentinel_tag = compute_stream_v8_sentinel_tag(
2020         sk_fmt, nonce, chunk_idx, aad, total_real_cps);
2021     stream.extend_from_slice(&0u32.to_be_bytes());
```

```
    stream.extend_from_slice(&total_real_cps.to_be_bytes());
    stream.extend_from_slice(&sentinel_tag);
```

same file

```
        if chunk_len == 0 {
            // SENTINEL: be8(total_real_cps) || tag(32)
            if stream.len() < pos + 8 + TAG_SIZE {
                return Err("Stream truncated: need sentinel body.".into());
            }
            let mut tr = [0u8; 8];
            tr.copy_from_slice(&stream[pos..pos + 8]);
            let total_real_cps = u64::from_be_bytes(tr);
```

same file

```
        let tag = compute_stream_v8_sentinel_tag(
            &self.sk_fmt, &self.nonce, self.chunk_idx, &self.aad, self.total_real_cps);
        out.extend_from_slice(&0u32.to_be_bytes());
        out.extend_from_slice(&self.total_real_cps.to_be_bytes());
        out.extend_from_slice(&tag);
```

same file

```
    #[test]
    fn stream_v8_length_is_function_of_f_only() {
        // Two messages that both land in the "2 chunks" bucket at F=16 must
        // produce ciphertexts of equal length -- the V2-CVF11-for-streaming
        // invariant.
        let (primes, sk) = generate_v8_key(DEFAULT_KEY_COUNT, MIN_KEY_PRIME, MAX_KEY_PRIME);
        let a = encrypt_stream_ae_v8(&"A".repeat(17), &primes, &sk, b"", 16).unwrap();
        let b = encrypt_stream_ae_v8(&"B".repeat(31), &primes, &sk, b"", 16).unwrap();
        assert_eq!(a.len(), b.len());
    }
```

CVF-V5-2: open Category: **documentation**

Summary

The new streaming encryption mode is offered as a recommended interface, but it has no security analysis, its specification is incomplete, and a known risk of this kind of mode is no longer described.

The fixes add a complete new encryption mode for long messages that are sent in pieces, i.e. streaming, and the documentation recommends it for use. However, the paper does not define what security this mode should provide and does not prove any security property for it. Its specification also leaves several details open, so an independent developer has to guess them to produce compatible output. In addition, the specification lets a receiver use parts of a message before the whole message has been checked for tampering. Earlier versions of the documentation warned about this risk, but the warning has not been restated.

The fixes introduced this mode. The earlier review recorded only an unused `FORMAT_STREAM_AE_V8` constant and asked the client either to specify the format and restore or supersede caveat `CAV-001`, which described the risk of unauthenticated streaming and had been dropped from an earlier version of the documentation, or to remove the constant. The fixed code instead adds about 630 lines of streaming code. The mode has its own nonce drawn from a CSPRNG (i.e. a cryptographically secure random number generator) instead of the block format's synthetic nonce, a per-stream format subkey `HMAC(sk, 0x0B||0x02)`, three new domains (`0x0C` chunk tag, `0x0D` sentinel, `0x0E` sub-nonce), per-chunk tags and a terminating sentinel. The crate documentation lists it under "v8 (normative - use this)", and the README lists it under "Recommended (v8)".

No security claim covers the mode, as the following points show:

- The paper reserves format `0x02` for "a streaming-AE format that is outside the scope of this paper", pointing to its caveats section and to `SPEC.md` section 8.2. The caveats section never mentions streaming.
- The paper's domain table omits `0x0C` to `0x0E`.
- The abstract and section 3.3 assert separation between the block and streaming formats, but the security games query the format subkey only for `0x01`, so that separation is not proved.
- The crate's own comments disagree with each other: the streaming header calls the functions "misuse-resistant primitives", the entry point says "Not misuse-resistant: nonce reuse under the same sk is catastrophic", and the production core is labelled "Test-only".
- The README states that the ciphertext carries no format identifier, yet every stream begins with the byte `0x02`.

The specification is also not sufficient to implement the format. We wrote an independent implementation from `SPEC.md` section 8.2 and the paper alone, and it reproduced the 11 positive stream test vectors only after we tried 12 combinations of possible readings. The text does not determine:

- the base of the chunk index (0 or 1);
- the index fed to the filler derivation;
- whether the token position and real-codepoint counters restart for each chunk.

The text also states no lower bound on `F` (`F = 0` would collide with the zero sentinel marker), no valid range for the transmitted total, no per-token decoder checks for streaming, and no limit on the 32-bit chunk count. Its "verify-before-yield" rule releases each chunk before the sentinel is verified, which is the release of unverified plaintext that `CAV-001` described. The Rust decryptor buffers the whole stream, but the specification does not require it to.

In addition, `docs/CAVEATS.md` (`CAV-005`) gives the data limit per key as a number of streams. The quantity that matters for the birthday bound, i.e. the point at which random values become likely to repeat, is the total number of chunks, because every chunk uses a 128-bit truncated sub-nonce.

Recommendation

Either give the streaming format a stated security model (nonce-based online authenticated encryption with a random nonce) and at least a proof sketch that reduces it to the PRF assumption on HMAC-SHA256, covering both format subkeys with a two-query hop, or remove it from the recommended and normative lists and place it behind a feature flag until that analysis exists.

In both cases, complete `SPEC.md` section 8.2 so that it determines every byte: the base of the chunk index, the filler index, whether the counters restart for each chunk, the bounds on `F` and on the total, and the decoder checks. Add `0x0C` to `0x0E` to the paper's domain table, restate the caveat about the release of unverified plaintext, and give the data limit as a number of chunks. Finally, correct the contradictory "misuse-resistant" and "Test-only" comments and the README's statement about the format identifier.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-2 — Mitigated in 7199472. We took the second option: the streaming format is no longer normative or recommended. In Rust it compiles only with the off-by-default `experimental-stream` feature, and it is labelled everywhere as outside the audited claims, with no security model or proof. SPEC.md §8.2 was rewritten to determine every byte: 0-based chunk index, filler index and formula, per-chunk counter reset, `1 <= F <= 16384`, `C <= 2^32-1`, the valid range of `n`, the full decoder check list, the masked count and header binding. Domains `0x0C`–`0x0F` are in the paper's domain table. The abstract and §3.3 no longer claim a proved block/stream separation. CAV-001 is restated per port (Python's generator releases chunks before the sentinel), CAV-005 counts chunks (`<= 2^48` per `sk`), the contradictory comments are fixed, and the README states that streams begin with `0x02`. No proof is offered; that is why the format is demoted.

/SPEC.md

```
513 ## 8.2 Streaming AE wire format v8 (`FORMAT_STREAM_AE_V8 = 0x02`)
514
```

(napqes.py `encrypt_stream_ae_v8`, `decrypt_stream_ae_v8`; rust/src/lib.rs `encrypt_stream_ae_v8`, `decrypt_stream_ae_v8`, `StreamV8Encryptor`; C/napqes.c `napqes_encrypt_stream_ae_v8_bytes`, `napqes_decrypt_stream_ae_v8_bytes`)

The v8 streaming construction upgrades §8.1 to the full v8 primitive family (independent 256-bit subkey `sk` + per-format subkey `sk_fmt = HMAC(sk, 0x0B || FORMAT_STREAM_AE_V8)`), uses fixed-width 8-byte tokens (closing the per-token length leak retained by §8.1 for LEB128 tokens), and pads each chunk to a fixed `F * (MAX_NOISE_RUN + 1)` token ceiling so per-chunk ciphertext size is a pure function of the sender-chosen frame parameter `F`, which is public.

```
stream = 0x02 || uint32_be(F) || nonce (16 bytes)             # HEADER (21 B)
       || [ uint32_be(F * 160)                                 # CHUNK FRAME × C
            || masked_chunk (F * 160 bytes)                      # (each frame is
            || chunk_tag (32 bytes) ] × C                        #   4 + F*160 + 32 B,
                                                                 # fixed per stream)
       || uint32_be(0) || uint64_be(total_real_codepoints)       # SENTINEL (44 B)
            || sentinel_tag (32 bytes)
```

`F` = codepoints per chunk (sender-chosen; default 128, hard upper bound 16384). Every chunk carries exactly `F` real-codepoint slots; if the input stream ends mid-chunk, the remaining slots are filled with HMAC-derived filler codepoints (domain `0x06`, keyed by the per-chunk sub-nonce). The decoder strips filler using `total_real_codepoints` from the sentinel.

Sub-nonce (domain `0x0E`). Each chunk `i` is encrypted as a

self-contained v8 primitive call keyed by `(sk_fmt, sub_nonce_i)` where

```
sub_nonce_i = HMAC(sk_fmt, 0x0E || nonce || uint32_be(i))[:16]
```

so domains `0x00`-`0x07` (position oracle, addend, noise char/addend, threshold, keystream) are reused verbatim from v8 block mode.

Per-chunk tag (domain `0x0C`).

```
chunk_tag_i = HMAC(sk_fmt,
    0x0C || nonce || uint32_be(i) || uint64_be(len(aad)) || aad || masked_chunk)
```

Sentinel tag (domain `0x0D`).

```
sentinel_tag = HMAC(sk_fmt,
    0x0D || nonce || uint32_be(C) || uint64_be(len(aad)) || aad
         || uint64_be(total_real_codepoints))
```

The sentinel binds *both* the chunk count `C` (anti-truncation) and the real-codepoint count (needed to strip filler from the last chunk safely).

Security properties:

- **Verify-before-yield.** Per-chunk tag verifies before any of that chunk's plaintext contributes to the output; the sentinel verifies before the last chunk's real codepoints are emitted.
- **Anti-truncation.** Chunk count `C` is bound into `sentinel_tag`.
- **Anti-reorder.** `chunk_idx` is bound into every `chunk_tag_i`.
- **Length hiding at chunk granularity.** Per-chunk masked_*blob length is a pure function of `F`, which is public — the LEB128 residual (V2-CVF11) that §8.1 retains is closed here.*
- **Not misuse-resistant** — *the 16-byte nonce is CSPRNG-drawn per stream, not SIV-derived (SIV requires hashing the full message, which is incompatible with streaming). Nonce reuse across two streams under the*

```
  same `sk` is catastrophic (CVF3-class hazard on aligned codepoint
  positions). See `docs/CAVEATS.md` CAV-005.

**Compatibility.** v8 streams are **not** cross-compatible with:
- §5 (v7 block) — different keying and no format subkey.
- §8   (v7 streaming basic, deprecated) — different keying, no format
subkey.
- §8.1 (v7 streaming AE) — different keying, different domain bytes.
- v8 *block* mode — different format subkey (`FORMAT_STREAM_AE_V8` vs
  `FORMAT_BLOCK_V8`) and different domain bytes.

---
```

/docs/napseq-eprint-v3.tex

```
$\mathrm{format\_id}=\texttt{0x01}$ (block mode). The identifier
$\mathrm{format\_id}=\texttt{0x02}$ is reserved for a streaming-AE
format that is outside the scope of this paper
(Section~\ref{sec:caveats}); its wire format is specified in
\texttt{SPEC.md} \S 8.2. Domain $\texttt{0x0B}$ takes no nonce and
```

same file

```
The purpose of this layer is \emph{cross-format tag separation}:
because
every subsequent HMAC call is keyed by $\mathsf{sk}_{\mathrm{fmt}}$
rather than $\mathsf{sk}$, a tag or ciphertext produced under one
$\mathrm{format\_id}$ cannot verify under another sharing the same
$(\mathbf{k},\mathsf{sk})$. The ciphertext itself carries no
$\mathrm{format\_id}$ byte on the wire — format selection is an
out-of-band API contract — so a ciphertext produced under one format
subkey simply fails the tag comparison at the receiver and is
indistinguishable from a wrong-key ciphertext.

\paragraph{Per-message derivations.}
```

/rust/README.md

```
### Wire format

```text
v8 block:    nonce(16) || masked_blob(fixed-width tokens) || tag(32)
v8 streaming:                                          (see below)
```

The ciphertext carries no format identifier — format selection is an
out-of-band API contract. A v7 ciphertext handed to
`decrypt_bytes_v8`
```

same file

```
**Recommended (v8):**
`NapqesKey` with `encrypt_bytes_v8_key`, `decrypt_bytes_v8_key`,
`encrypt_bytes_v8_with_profile_key` (validates once, wipes on drop);
`encrypt_bytes_v8`, `encrypt_bytes_v8_with_profile`,
`decrypt_bytes_v8`,
`encrypt_stream_ae_v8`, `decrypt_stream_ae_v8`, `StreamV8Encryptor`,
`generate_v8_key`, `zeroize_key`, `zeroize_sk`.
```

/rust/src/lib.rs

```
//! * **v8 (normative — use this).** Domain-`0x0B` format subkey
//!   ([`derive_format_subkey`], per CVF-11), synthetic nonce
//!   ([`synthetic_nonce`], domain `0x0A`), `MAX_NOISE_RUN`-capped
emission
//!   loop, per-bucket token ceiling. Wire layout is
//!   `nonce(16) || masked_blob(fixed-width) || tag(32)`. Entry
points:
//!   [`encrypt_bytes_v8`], [`decrypt_bytes_v8`],
[`generate_v8_key`],
//!   [`encrypt_bytes_v8_with_profile`], [`encrypt_stream_ae_v8`],
//!   [`decrypt_stream_ae_v8`], [`StreamV8Encryptor`]. Length-hiding
property
//!   holds per Corollary 4.14 of the paper.
```

same file

```
// --- Streaming AE v8 (misuse-resistant primitives, streaming nonce)
-----------
//
// Wire format (FORMAT_STREAM_AE_V8 = 0x02):
//   HEADER (21 B):     0x02 || be4(F) || nonce(16, CSPRNG)
//   CHUNK FRAME * C:   be4(F*160) || masked_chunk(F*160) ||
chunk_tag(32)
//   SENTINEL (44 B):   be4(0) || be8(total_real_codepoints) ||
sentinel_tag(32)
//
// Every chunk i is a self-contained v8 primitive call keyed by
// (sk_fmt, sub_nonce_i) where
//     sk_fmt      = HMAC(sk, 0x0B || FORMAT_STREAM_AE_V8)
//     sub_nonce_i = HMAC(sk_fmt, 0x0E || nonce || be4(i))[:16]
// so domains 0x00-0x07 (position oracle, addend, noise char/addend,
threshold,
// keystream) are reused verbatim from v8 block mode.
//
// Security caveats (docs/CAVEATS.md, CAV-005):
//   * The stream nonce is CSPRNG-drawn -- not SIV-derived -- because
SIV
```

```
//     requires hashing the full message. Nonce reuse across two streams under
//     the same sk is catastrophic (CVF3-class hazard).
//   * Fixed-width 8-byte tokens and F*20 per-chunk padding close the LEB128
//     length leak (V2-CVF11) that the v7 stream_ae format retains; per-chunk
//     size is a pure function of F, which is public.
//   * Truncation-safe: the sentinel binds the chunk count.
//   * Reorder-safe: chunk_idx is bound into every per-chunk tag.
```

same file

```
/// Deterministic v8 streaming encryption with caller-supplied nonce.
///
/// **Test-only.** Used by `kat_cross_check::stream_v8_*` to compare byte
/// output against the Python KAT vectors. Production callers must draw the
/// nonce from a CSPRNG per stream (CAV-005) -- reusing a nonce across two
/// streams under the same sk is catastrophic.
```

same file

```
/// **Not misuse-resistant**: nonce reuse under the same `sk` is catastrophic
/// (CAV-005). Use the v8 *block* API (`encrypt_bytes_v8`) whenever the whole
/// message fits in memory and MRAE is desired.
```

CVF-V5-3: open Category: **algorithm**

Summary

The range from which the secret key's primes are chosen was widened although no finding asked for it, the change is not disclosed, and the paper justifies it with incorrect claims.

The fixes change two of the scheme's fixed parameters: the range of numbers from which the prime numbers of the secret key are chosen, and the default number of primes in a key. No finding asked for this change, and the client's report on the fixes does not mention it. The paper explains the change with a security argument that is incorrect and that contradicts other statements in the same paper. The change breaks no calculation, but it carries no version marker and no test vector covers it, so software written for the previous version passes every test and still rejects almost every key generated with the new settings.

What changed. CVF-16 found that `validate_key` omitted the normative upper bound `MAX_KEY_PRIME = 9_900_000` and asked for that bound to be enforced. The fixed code enforces an upper bound (lib.rs:425-430), but the same fixes also moved the bound itself: `MAX_KEY_PRIME` is now `14_999_999` (lib.rs:127-141), the paper's set $\mathcal{P}$ is now the primes in $[10^6, 1.5 \times 10^7)$ with $|\mathcal{P}| = 892{,}206$ instead of $579{,}947$ (napseq-eprint-v3.tex:275-284), and the default key length changed from $K = 10$ to $K = 13$ (lib.rs:156-160; tex 294-296). The Python and C implementations were changed in the same way (`napqes.py:101,113`, `C/napqes.h:163`). The client's remediation report describes the change for CVF-16 only as "upper bound enforced" (report §3 row 16, §4) and never mentions the new interval or the new K, and the client's own `docs/audit_mitigation_responses.md` calls it "Not an audit finding" (the "PQ-128" change).

The doc comment at lib.rs:127-136 states the actual motive: $\mathcal{P}$ is "exactly the half-open interval `[10^6, 1.5e7)` already used by `kem.rs`". The earlier review had noted, as an observation outside the scope, that `kem.rs` derives key elements from a second interval, wider than that of the AEAD. Instead of bringing `kem.rs` into line with the specification, the specification and the AEAD were moved onto the interval of `kem.rs`. The KEM layer that is now cited as the reason (tex 282-284, "the key-establishment layer (Section~\ref{sec:keygen})") is not described anywhere in the paper, and the reference resolves to the AEAD's own Key Generation section.

The paper's justification for the change is incorrect and contradicts the paper itself. Remark `rem:pq128` (tex 342-359) says that the new default "is the smallest parameter set in this family reaching the 128-bit post-quantum target" and that "Raising K alone is not sufficient". Both statements are incorrect, and the remark also conflicts with other parts of the paper:

- Over the old interval, $K = 14$ already gives $H_\infty(\mathbf{k}) = 268.04$ bits, i.e. 134.02 bits after Grover's algorithm. Raising K alone is therefore sufficient.
- With $K = 13$, the 128-bit post-quantum target is already met with an inclusive upper bound of $14{,}257{,}301$ ($|\mathcal{P}| = 847{,}186$), so 1.5×10^7 is not the smallest such interval; it was chosen to match `kem.rs`.

- The same paper says that $K = 13$ "is an interoperability default, not a security parameter" (tex 380-383) and that the **k** figure "is reported for completeness only. It is not relevant to any theorem" (tex 1919-1923). On v8, every confidentiality and integrity claim rests on sk. A post-quantum target for **k** is meaningful only for the legacy v7 format, where the prime tuple is the whole key, and the paper disclaims v7.

The change breaks interoperability without any test noticing it. The paper makes the interval normative ("Implementations MUST use this interval: a different upper bound yields a different $\mathcal{P}$ ... and hence different keys", tex 278-281), yet the change carries no version marker: the specification's `\date{July 2026}` is the same in its last three versions, and its file name says v3. An implementation that conforms to the baseline specification, and to the bound that CVF-16 requested, passes all of the current test vectors, because no vector covers the widened region: the largest key element in the 35 v8 test vectors is $7{,}999{,}993$, and no vector uses $K = 13$. That implementation nevertheless rejects $1 - (579{,}947/892{,}206)^{13} = 99.63\%$ of the keys generated with the new default. Conversely, an implementation of the fixed specification accepts keys that an implementation of the baseline specification must reject, and nothing in the known-answer tests would reveal the disagreement.

Recomputed figures. We recomputed the following figures with our own sieve, and every one of them still holds under the new interval:

- $|\mathcal{P}| = 892{,}206$; the largest prime is $14{,}999{,}981$; both 10^6 and $14{,}999{,}999$ are composite; $\log_2 |\mathcal{P}| = 19.7670$.
- $$H_\infty(\mathbf{k}) = \log_2(|\mathcal{P}|!/(|\mathcal{P}| - 13)!) = 256.9711$$
  bits at $K = 13$ (key space 2.270×10^{77}), and 128.4855 bits after Grover's algorithm. For the old interval, the figures are 248.8921 / 124.4461 at $K = 13$, and 191.4555 / 95.7278 at the old default $K = 10$.
- Token-width bound of Remark `rem:token-fit` (tex 926-942): the maximum token `0x10FFFF` $\cdot\, k + (k - 1)$ is $16{,}711{,}658{,}831{,}871 \approx 2^{43.93}$ at the largest prime and $16{,}711{,}678{,}885{,}887 \approx 2^{43.93}$ at the inclusive bound $k = 14{,}999{,}999$, against $11{,}029{,}707{,}685{,}887 \approx 2^{43.33}$ in the baseline version. The 8-byte token field still never overflows; the margin $2^{64}/t$ shrinks from 1.672×10^6 to 1.104×10^6, and the paper's "over 1.1×10^6" is correct. `MAX_SAFE_KEY_PRIME` (lib.rs:149-154) is 1.10×10^6 times the new bound.
- $k < 2^{40}$ (5-byte serialisation) and $k - 1 < 2^{24}$ (the $< 2^{-8}$ bound of the remark on modulo bias) still hold.
- The cost of `validate_key` per call at $K = 13$ rises to about 17,400 trial divisions on average (range 6,487 to 25,155), from about 11,000 at the old $K = 10$; see the findings on key validation per call.

The widening therefore breaks no arithmetic invariant. The problem is that a normative parameter of the scheme changed without notice, for a reason that the paper misstates, without a version identifier and without test vectors that would let a second implementation notice the change.

Recommendation

Decide the interval deliberately, and record the decision as a versioned change.

- If the wider interval is kept, state the real motive (alignment with the key-establishment layer), and either specify that layer or drop the reference to it; fix the `sec:keygen` cross-reference. Delete or correct "Raising K alone is not sufficient" and "the smallest parameter set in this family", and reconcile Remark `rem:pq128` with Remark `rem:key-roles` and §5, which say that K and **k** carry no security requirement. If a post-quantum figure for **k** matters at all, say that it matters only for v7.
- Give the specification a version identifier, and name the interval change in its revision history and in the remediation report, so that implementations written against the baseline specification know that they must change.
- Add known-answer test vectors that pin the new interval: at least one positive vector with $K = 13$ and an element in $(9.9 \times 10^6, 1.5 \times 10^7)$, and a negative vector with an element just above the bound (for example $15{,}000{,}017$).
- Alternatively, keep the baseline interval and change `kem.rs` (and the Python and C KEM) to sample from it. This brings the component outside the scope into line with the specification, instead of the reverse.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-3 — Fixed in `c14e0a0`. We keep `P = [10^6, 1.5e7)` and `K = 13`, and now disclose and version the change. The paper marks the interval as new in revision 4.1 (parameter set `P2`, versus `P1 = [10^6, 9.9e6]`) and gives the real motive: alignment with the reference KEM component, which the paper does not specify. The wrong `sec:keygen` reference is removed. Remark `rem:pq128` is rewritten: K = 14 over `P1` already gives 134.02 bits; 14,257,301 suffices at K = 13; the post-quantum figure for `k` matters only for v7. New KAT vectors pin the interval: `W023` (K = 13 with elements 9,900,047, 12,000,017 and 14,999,981) and `W-E02` (15,000,017 rejected). The corpus carries a `prime_interval` marker. The same false claims were corrected in `lib.rs`, `napqes.py`, `napqes.h`, CAVEATS and the mitigation responses; all figures were re-derived by sieve.

/docs/napseq-eprint-v3.tex

```
275   \item $\mathcal{P}$ denotes the set of prime integers in $[10^6,
      1.5 \times 10^7)$,
276         a fixed public system parameter with $|\mathcal{P}| =
      892{,}206$
277         (verified by sieve; the interval is half-open, equivalently
      the
278         inclusive interval $[10^6, 14{,}999{,}999]$, and both
      endpoints are
279         composite). Implementations MUST use this interval:
280         a different upper bound yields a different $\mathcal{P}$, a
281         different canonical enumeration, and hence different keys.
282         This is the same prime set from which the key-establishment
      layer
```

```
        (Section~\ref{sec:keygen}) draws, so the AEAD and the KEM share one
        parameter set.
```

same file

```
Draw $\mathbf{k} = (k_1,\ldots,k_K)$, an ordered tuple of $K$ distinct
primes sampled uniformly, without replacement, from $\mathcal{P}$ via a
cryptographically secure DRBG (default $K=13$); \emph{independently}, draw
```

same file

```
\begin{remark}[Choice of the ``PQ-128'' parameter set]\label{rem:pq128}
The default $(\mathcal{P},K)$ is the smallest parameter set in this family
reaching the $128$-bit post-quantum target. Raising $K$ alone is not
sufficient: over the earlier interval $[10^6,9.9\times10^6]$
($|\mathcal{P}|=579{,}947$, $19.1456$ bits per element) one has
$H_\infty=248.89$ bits at $K=13$, i.e.\ only $124.45$ bits post-Grover.
Widening the interval to $[10^6,1.5\times10^7)$ raises the per-element
contribution to $19.7670$ bits and clears the target at the same $K=13$,
while simultaneously unifying $\mathcal{P}$ with the interval already used
by the key-establishment layer. Since $K$ affects neither the wire format
nor the ciphertext length (Proposition~\ref{prop:expansion-neutral}), the
only cost of moving the default from $K=10$ to $K=13$ is stored key
material, which grows from $5K+32=82$ to $97$ bytes; widening the interval
changes no serialised size. Validation enforces the same interval: every reference
implementation rejects a key element outside $\mathcal{P}$ and a tuple of
more than $128$ elements. Keys provisioned under the earlier interval lie
inside $\mathcal{P}$ and remain usable.
\end{remark}
```

same file

```
(Remark~\ref{rem:coarsening-not-inflation}). $K=13$ is an interoperability
default, not a security parameter: any $K\ge1$ is permitted by the
```

```
construction (the reference implementations cap $K\le128$ to bound the
per-call cost of key validation), and both parties must agree on it out of band exactly as they
```

/rust/src/lib.rs

```rust
/// Upper bound of the normative prime interval of the "PQ-128" profile.
/// Inclusive, and chosen so that `P` is exactly the half-open interval
/// `[10^6, 1.5e7)` already used by `kem.rs` — the AEAD and the KEM now draw
/// from the same prime set. `P` contains exactly 892_206 primes (verified by
/// sieve), giving `|P|!/(|P|-13)! = 2^256.9711` ordered 13-tuples
/// (`2^128.4855` post-Grover).
///
/// **CVF-16 fix (2026-09-22).** This bound is now enforced by [`validate_key`]
/// on *every* key it sees (not just generation), matching the normative
/// interval Remark 3.1 of the paper specifies. Prior revisions accepted any
/// prime `>= MIN_KEY_PRIME` on decrypt paths, which allowed a key element
/// large enough to make the token multiplication `c * k + addend` wrap `u64`
/// silently in release builds. Matches `napqes.py::MAX_KEY_PRIME` and
/// `C/napqes.h::NAPQES_MAX_KEY_PRIME`.
pub const MAX_KEY_PRIME: u64 = 14_999_999;
```

same file

```rust
/// Number of prime key elements generated by default ("PQ-128" profile).
/// K=13 is the smallest count reaching the 128-bit post-quantum target over
/// `P`; K=13 over the earlier `[10^6, 9.9e6]` interval would have given only
/// 124.4 bits, which is why the interval was widened too.
pub const DEFAULT_KEY_COUNT: usize = 13;
```

CVF-V5-4: open Category: **flaw**

Summary

The legacy encryption format still has no limit on its noise loop and no length ceiling, and the crate's own gateway still uses it, so the length of its ciphertexts still varies randomly and the loop still has no upper bound.

The older (v7) encryption format inserts a random number of filler tokens into each encrypted message, with no upper limit. As a result, the length of an encrypted message varies randomly with the nonce, which leaks information, and the encryption step has no bound on how long it can run. The fixes mark the old functions as deprecated, but this is only a warning to developers: the crate's own network gateway switches the warning off and still encrypts every message with the old format. The problem therefore still affects all production traffic that passes through that gateway.

This finding covers what remains of CVF-17, which is partially fixed.

The fixed code marks the four public v7 encryptors as `#[deprecated]`: `encrypt` (`lib.rs:731-740`), `encrypt_bytes` (`lib.rs:1046-1052`), `encrypt_str` (`lib.rs:1182-1187`) and `encrypt_raw` (`lib.rs:1217-1225`). The token loop was also extracted into a single function, `emit_tokens_v7`.

The following problems remain:

- `emit_tokens_v7` (`lib.rs:994-1024`) still applies neither bound. Omitting them is correct for compatibility with the v7 wire format, but every v7 ciphertext produced today has the length leak and the unbounded loop that CVF-17 described.
- `#[deprecated]` is only a lint, i.e. a compiler warning. `ot_frame.rs` (outside the scope, lines 33-40 and 303) suppresses it with `#[allow(deprecated)]`, and §8.4 of the client's report confirms that the OT gateway still seals every frame with the v7 function `encrypt_raw`. The problem therefore applies to all production traffic within the crate's own code.
- The robustness cap that CVF-17 asked for regardless of format ("cap the run anyway") was not added.
- The remediation report (row 17 and §5.2) says that `encrypt_bytes_with_nonce` was deprecated. It was not, and it was already `pub(crate)` in the baseline code.

Recommendation

Move the gateway (`ot_frame`) to the v8 format, and then remove the v7 encryptors from the public API, keeping only the decryptors.

Until then, add an iteration bound to `emit_tokens_v7` that returns an error instead of looping without limit, and correct the remediation report's statement about `encrypt_bytes_with_nonce`.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-4 — Fixed in `23025ed` (gateway migration in `b619dab`). The gateway now uses v8 (CVF-V5-7). The four v7 encryptors are compiled only with the off-by-default

`legacy-v7` feature, and the decryptors remain for archived data. `emit_tokens_v7` returns `Err` after 4,096 consecutive noise positions (probability below `2^-59` per real token at `p <= 0.99`), so the loop is bounded. The remediation report's statement about `encrypt_bytes_with_nonce` is corrected: it was never deprecated and was already `pub(crate)`.

/rust/src/lib.rs

```
#[deprecated(
    since = "0.2.0",
    note = "v7 legacy format lacks the MAX_NOISE_RUN cap and per-bucket \
            ceiling (audit CVF-17): ciphertext length is a random function \
            of the CSPRNG-drawn nonce rather than of the padding bucket, so \
            the length-hiding property Corollary 4.14 proves for v8 does not \
            hold; v7 is keyed by the prime tuple, not sk, and no theorem of \
            the paper applies (CVF-28). Use encrypt_bytes_v8 / \
            encrypt_bytes_v8_with_profile."
)]
```

same file

```
fn emit_tokens_v7(
    padded: &[u32],
    kb: &[u8],
    nonce: &[u8],
    noise_p: f64,
    key: &[u64],
) -> Vec<u64> {
    let kk = key.len() as u64;
    let mut cypher: Vec<u64> = Vec::with_capacity(padded.len() * 4);
    let mut real_idx: u64 = 0;
    let mut ct_pos: u64 = 0;
    for &c in padded {
        loop {
            if is_noise_pos(kb, nonce, ct_pos, noise_p) {
                let k = key[(real_idx % kk) as usize];
                let nc = derive_noise_char(kb, nonce, ct_pos);
                let na = derive_noise_token_addend(kb, nonce, ct_pos, k);
                cypher.push(nc * k + na);
                ct_pos += 1;
            } else {
                let k = key[(real_idx % kk) as usize];
                let addend = derive_addend(kb, nonce, real_idx, k);
```

```
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }
    cypher
}
```

same file

```
#[deprecated(
    since = "0.2.0",
    note = "v7 legacy format lacks the MAX_NOISE_RUN cap and per-bucket \
            ceiling (audit CVF-17); v7 is keyed by the prime tuple, not sk, \
            and no theorem of the paper applies (CVF-28). Use \
            encrypt_bytes_v8 / encrypt_bytes_v8_with_profile."
)]
```

same file

```
#[deprecated(
    since = "0.2.0",
    note = "v7 legacy format lacks the MAX_NOISE_RUN cap and per-bucket \
            ceiling (audit CVF-17); v7 is keyed by the prime tuple, not sk, \
            and no theorem of the paper applies (CVF-28). Use encrypt_bytes_v8."
)]
```

same file

```
#[deprecated(
    since = "0.2.0",
    note = "v7 legacy format lacks the MAX_NOISE_RUN cap and per-bucket \
           ceiling (audit CVF-17), and the byte-vs-codepoint encoding \
            ambiguity of CVF-14 remains; v7 is keyed by the prime tuple, not \
            sk, and no theorem of the paper applies (CVF-28). Use \
            encrypt_bytes_v8 for text or wrap the bytes in a caller-chosen \
            text encoding."
)]
```

CVF-V5-5: open Category: **procedural**

Summary

The start-up self-test still does not check the integrity of the software, although the module's FIPS documents say that it does, and the post-quantum key exchange is only checked for internal consistency.

Before the module may be used, it must run a set of start-up self-tests. The fixes add tests for the current encryption scheme, which addresses part of the earlier problem. However, there is still no test that checks that the module's software has not been modified, while the documents written for the module's operators describe such a test. In addition, the post-quantum key exchange is only checked by confirming that its two halves agree with each other, not by comparing its output with results known to be correct.

This finding covers what remains of CVF-18, which is partially fixed.

The following parts are fixed. `run_power_on_self_tests` (`self_test.rs:560-582`) now runs the v8 known-answer tests KAT-4 to KAT-7 (encrypt, decrypt, tamper, structural reject) and a KEM derivation known-answer test, KAT-8. KAT-4 pins the SHA-256 hash of test vector W002, and we reproduced it with an independent implementation. The test `kat_constants_match_corpus` keeps the embedded constants from drifting away from the test-vector files. INT-1 was renamed to a build-provenance check and is skipped by default (`self_test.rs:715-727`), instead of comparing the version string with itself.

The following problems remain:

- There is no software-integrity test. The client defers the binary HMAC (remediation report §6 item 1), yet the operator documents `docs/fips/SECURITY_POLICY.md:180-181` and `MODULE_BOUNDARY.md:72,153-164` still describe a `build.rs` HMAC integrity check. INT-1 cannot fail in a default build, so its unit test `integrity_check_passes` checks nothing.
- KAT-8 checks only the key derivation against a known answer. FrodoKEM and the hybrid combiner get only a pairwise-consistency test, i.e. a check that encapsulation and decapsulation agree with each other.

The missing coverage of the new streaming format in the power-on self-test is reported separately, in CVF-V5-18.

Recommendation

Either implement the integrity test (an HMAC over the module binary, computed at build time and verified at start-up), or correct the Security Policy and Module Boundary documents to say that no integrity test exists and that INT-1 is a provenance stub.

In addition, add a known-answer test for the KEM primitive: a fixed-seed encapsulation and decapsulation checked against a pinned vector.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-5 — Fixed in `b98bf23`. INT-2 is a real software-integrity test: HMAC-SHA256, under a fixed public key, over the executable that contains the module, compared in constant time with `<executable>.hmac`. The new `napqes_integrity` tool writes that file after linking; a binary cannot embed its own digest at compile time, so the `build.rs` description in the Security Policy and Module Boundary documents is corrected. With the `fips_integrity` feature a missing reference value also fails. Verified end to end (unsigned fails, signed passes, a one-byte change fails) and run in CI. KAT-8 adds primitive known answers:

- FrodoKEM-640-AES decapsulation of a (sk, ct) pair generated by liboqs, an independent implementation, cross-checked in the Python tests;
- the RFC 7748 X25519 vector;
- the hybrid combiner on fixed inputs.

The vacuous `integrity_check_passes` test is replaced.

/rust/src/self_test.rs

```
560 pub fn run_power_on_self_tests() -> Result<(), SelfTestError> {
561     // CVF-47: a panicking KAT leaves POST_RUNNING, which refuses all
callers.
562     let _lock = lock_post();
563     POST_STATE.store(POST_RUNNING, Ordering::Release);
564     let _in_post = InPostFlag::set();
565     let result = (|| -> Result<(), SelfTestError> {
566         kat_encrypt()?;
567         kat_decrypt()?;
568         kat_tamper_rejection()?;
569         kat_v8_encrypt()?;
570         kat_v8_decrypt()?;
571         kat_v8_structural_reject()?;
572         kat_v8_tamper_rejection()?;
573         kat_kem()?;
574         build_provenance_check()?;
575         Ok(())
576     })();
577     match &result {
578         Ok(()) => POST_STATE.store(POST_PASSED, Ordering::Release),
579         Err(_) => POST_STATE.store(POST_FAILED, Ordering::Release),
580     }
581     result
582 }
```

same file

```
715 const BUILD_HASH: &str = env!("CARGO_PKG_VERSION");
716
717 fn build_provenance_check() -> Result<(), SelfTestError> {
718     // If the operator has pinned an attested version at build time,
enforce
```

```
719     // strict equality. Otherwise (the default), succeed — this check
    is a
720     // build-provenance signal, not an integrity check, and no
    operator
721     // signal means no claim to verify against.
722     match option_env!("NAPQES_ATTESTED_VERSION") {
723         Some(attested) if attested == BUILD_HASH => Ok(()),
724         Some(_) => Err(SelfTestError::BuildProvenanceMismatch),
725         None => Ok(()),
726     }
727 }
```

CVF-V5-6: open Category: **architecture**

> **Summary**
>
> **The crate still builds its key-exchange, network and gateway code into every project that uses it, and the fixes add two more dependencies that are always built and a feature that cannot be built from the delivered files.**
>
> A project that needs only the encryption scheme from this library still receives a large amount of other code, because the key-exchange, networking and gateway components are always included. The fixes make this worse. They add two more libraries that are always included, one of which builds native code, i.e. code that is not written in Rust, into every project that uses the crate, although nothing in the delivery uses it and the code was not reviewed. They also add an optional component whose source files were not delivered, so it cannot be built, and the client's test results for it cannot be reproduced.

This finding covers what remains of CVF-23, which is partially fixed. The build boundary, i.e. the set of code built into every project that uses the crate, has also grown since the earlier review.

The following parts are fixed: a `[profile.release]` with overflow checks, a committed `Cargo.lock`, a narrowed `tokio` dependency, and real use of `subtle`.

The following problems remain, and the client has deferred them:

- There is no feature gating: `pub mod kem`, `kem_exchange`, `ot_frame` and `protocols` are compiled unconditionally (`lib.rs:47-54`), so a consumer who wants only the AEAD still links the KEM, networking and gateway code.
- No `rust-version` (MSRV, i.e. minimum supported Rust version) is declared.
- `[profile.test]` sets nothing that differs from the default, and its comment (`Cargo.toml:52-61`) is out of date.

The fixes also widened the boundary:

- `x25519-dalek` (`Cargo.toml:33-34`) and `pqcrypto-sphincsplus` (`Cargo.toml:47`) are new unconditional dependencies. No delivered `.rs`, `.py` or `.md` file uses SPHINCS+, yet `Cargo.lock` shows that it pulls in `pqcrypto-internals` and `cc`, so unreviewed native code is built into every consumer.
- The new `vale` feature (`Cargo.toml:14-15`) and the `[[bin]] vale_demo` target (`Cargo.toml:75-78`) refer to `src/vale` and `src/bin/vale_demo.rs`, which are gitignored or absent. The test counts that the remediation report gives for `--features vale` therefore cannot be reproduced from the delivery.

Recommendation

Put `kem`, `kem_exchange`, `ot_frame` and `protocols` behind features that are off by default, together with their dependencies (`x25519-dalek`, `tokio`, the KEM crates).

Make `pqcrypto-sphincsplus` optional under `vale` (`vale = ["dep:pqcrypto-sphincsplus"]`), and either deliver `vale` or remove the feature and the bin target. Finally, declare `rust-version`, and delete or correct the `[profile.test]` block and its comment.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-6 — Partly fixed in `da7e005`. Removed:

- the `vale` feature;
- the undeliverable `vale_demo` binary;
- the unconditional `pqcrypto-sphincsplus` dependency (no SPHINCS+ native code is built into consumers any more).

Also: `rust-version = "1.88"` is declared and checked in CI, the redundant `[profile.test]` block and its stale comment are removed, and `autobins = false` is set. **Not done:** `kem`, `kem_exchange`, `ot_frame` and `protocols` are still compiled unconditionally; we accept this remainder.

/rust/Cargo.toml

```
14 # Requires the unpublished rust/src/vale sources (gitignored).
15 vale = []
```

same file

```
33 # Classical half of the hybrid key establishment (ANSSI PQC doctrine).
34 x25519-dalek = { version = "2", features = ["static_secrets"] }
```

same file

```
47 pqcrypto-sphincsplus = { version = "0.7.2", default-features = false,
features = ["std"] }
```

same file

```
52 # CVF-23: manifest hardening — release builds now trap on integer
overflow
```

```
# (belt-and-braces backstop for CVF-16's checked_mul recommendation), and
# test builds keep debug_assert! live so CVF-12's ct_eq_bytes length check
# fires in `cargo test`.
[profile.release]
overflow-checks = true
debug-assertions = false

[profile.test]
overflow-checks = true
```

same file

```
[[bin]]
name = "vale_demo"
path = "src/bin/vale_demo.rs"
required-features = ["vale"]
```

/rust/src/lib.rs

```
pub mod self_test;
pub mod kem;
pub mod kem_exchange;
pub mod ot_frame;
pub mod protocols;
// rust/src/vale is not published (gitignored); build it with `--features vale`.
#[cfg(feature = "vale")]
pub mod vale;
```

CVF-V5-7: open Category: **architecture**

Summary

The public legacy (v7) functions still derive all keys from a single secret, which the v8 format was designed to avoid, and the gateway still uses them for all of its production traffic.

In the older (v7) format, a single secret, the list of key primes, determines everything that the scheme computes, and none of the security proofs in the paper apply to this format. The newer v8 format was designed to fix this by adding a separate random secret key. The fixes mark the old encryption functions as deprecated and document their limitations, but the functions are still available, and the crate's own network gateway still uses them to protect every message it sends. The format that the paper does not cover is therefore the one that protects production traffic.

This finding covers what remains of CVF-28, which is partially fixed.

The following parts are fixed. `check_v7_aad_len` enforces the range of the 4-byte length prefix of the associated data (AAD) at every public v7 entry point, `be_len_prefix` asserts the width bound (and the assertion cannot be reached from external input), and the v7 rustdoc now states how the keys are derived and that no theorem of the paper applies (`lib.rs:1131-1133`, `1255-1258`). The v7 encryptors are also deprecated.

The architectural core of the finding remains. The v7 encryptors `encrypt_bytes` (`lib.rs:1046-1052`) and `encrypt_raw` (`lib.rs:1217-1253`) and the decryptor `decrypt_raw` (`lib.rs:1255-1314`) are still public: they are deprecated, but not withdrawn. On v7, the prime tuple is the entire key: it derives both the HMAC key and the arithmetic, so there is no independent `sk`, and none of the paper's security theorems covers this format. §8.4 of the client's report confirms that `ot_frame` still seals every gateway frame with the v7 function `encrypt_raw`, so the format that the paper disclaims is the one in production use.

Recommendation

Migrate `ot_frame` to the v8 format with a separately provisioned `sk`, and then remove the v7 encryptors from the public API. Keep only the decryptors, behind a `legacy-v7` feature, for archived data.

Until the migration is complete, state in the README and in the paper that gateway traffic uses a format with no security proof.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-7 — Fixed in `b619dab` (encryptors withdrawn in `23025ed`). `ot_frame` now seals every frame with v8 (`encrypt_bytes_v8_key`) under a validated `NapqesKey`, and PDUs are carried as Latin-1 codepoints. The KEM derives the v8 `sk` with its own HKDF label (`derive_v8_sk`, matching Python), so the prime tuple and `sk` are independent outputs of the shared secret. Pre-shared key stores hold `{primes, sk_hex}`, and keys are validated once, at load. No in-tree component encrypts with v7; the paper's §8.2 says so.

/rust/src/lib.rs

```
#[deprecated(
    since = "0.2.0",
    note = "v7 legacy format lacks the MAX_NOISE_RUN cap and per-bucket \
            ceiling (audit CVF-17); v7 is keyed by the prime tuple, not sk, \
            and no theorem of the paper applies (CVF-28). Use \
            encrypt_bytes_v8 / encrypt_bytes_v8_with_profile."
)]
```

same file

```
#[deprecated(
    since = "0.2.0",
    note = "v7 legacy format lacks the MAX_NOISE_RUN cap and per-bucket \
           ceiling (audit CVF-17), and the byte-vs-codepoint encoding \
            ambiguity of CVF-14 remains; v7 is keyed by the prime tuple, not \
            sk, and no theorem of the paper applies (CVF-28). Use \
            encrypt_bytes_v8 for text or wrap the bytes in a caller-chosen \
            text encoding."
)]
pub fn encrypt_raw(data: &[u8], key: &[u64], aad: &[u8]) -> Result<Vec<u8>, String> {
    validate_key(key)?; // CVF-15
    check_v7_aad_len(aad.len())?;
    // CVF-34: cap at MAX_PLAINTEXT_CODEPOINTS bytes (each byte becomes a
    // codepoint under the raw-encode convention, so the same 2-codepoint
    // length prefix caps the byte count).
    if data.len() > MAX_PLAINTEXT_CODEPOINTS {
        return Err(format!(
            "data exceeds MAX_PLAINTEXT_CODEPOINTS ({}) bytes",
            MAX_PLAINTEXT_CODEPOINTS
        ));
    }
    let nonce = generate_nonce_with_crng_check()?;
    let codepoints: Vec<u32> = data.iter().map(|&b| b as u32).collect();
    let kb = key_bytes(key);
    let noise_p = derive_noise_p(&kb, &nonce);
    let padded = pad_message(&codepoints, &kb, &nonce);
```

```
    let cypher = emit_tokens_v7(&padded, &kb, &nonce, noise_p, key);
// CVF-46
    let blob = fixed_encode_tokens(&cypher);
    let ks = varint_keystream(&kb, &nonce, blob.len());
    let masked: Vec<u8> = blob.iter().zip(ks.iter()).map(|(a, b)| a ^
b).collect();
    let mut payload = Vec::with_capacity(NONCE_SIZE + masked.len());
    payload.extend_from_slice(&nonce);
    payload.extend_from_slice(&masked);
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
    payload.extend_from_slice(&tag);
    Ok(payload)
}
```

same file

```
/// Decrypt binary data previously encrypted with [`encrypt_raw`].
///
/// Legacy v7 decryptor, kept for archived ciphertexts; see
[`decrypt_bytes`]
/// for the v7 caveats.
///
/// Verifies the HMAC tag (constant-time) before decrypting.  Returns
/// `Err` on authentication failure — the caller must never use the
/// ciphertext for any purpose if this returns an error.
pub fn decrypt_raw(ciphertext: &[u8], key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
    validate_key(key)?; // CVF-15
    check_v7_aad_len(aad.len())?;
    if ciphertext.len() < NONCE_SIZE + TAG_SIZE {
        return Err(format!(
            "Ciphertext too short: {} bytes; header+tag require at
least {}.",
            ciphertext.len(),
            NONCE_SIZE + TAG_SIZE
        ));
    }
    let kb = key_bytes(key);
    let split = ciphertext.len() - TAG_SIZE;
    let payload = &ciphertext[..split];
    let recv_tag = &ciphertext[split..];
    let calc_tag = compute_auth_tag(&kb, aad, payload,
AAD_LEN_WIDTH_V7);
    if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
        return Err("Authentication failed: invalid HMAC tag.".into());
    }
    let nonce = &payload[..NONCE_SIZE];
```

```
    let masked = &payload[NONCE_SIZE..];
    let ks = varint_keystream(&kb, nonce, masked.len());
    let blob: Vec<u8> = masked.iter().zip(ks.iter()).map(|(a, b)| a ^ b).collect();
    let tokens = fixed_decode_tokens(&blob)
        .map_err(|e| format!("varint decode error: {}", e))?;
    let codepoints = decrypt(nonce, &tokens, key)?;
    // CVF-14: the previous body was `.map(|c| (c & 0xFF) as u8)` — a silent
    // narrowing that discarded every recovered codepoint above 255 without
    // signalling. A ciphertext produced by `encrypt_bytes` (codepoint-oriented,
    // e.g. `é` as one U+00E9 token) handed to `decrypt_raw` would authenticate
    // and return a self-consistent-looking single byte, hiding the fact that
    // the two ports use incompatible message-space encodings. We now reject
    // any recovered value that does not fit in a byte.
    codepoints
        .into_iter()
        .enumerate()
        .map(|(i, c)| {
            if c > 0xFF {
                Err(format!(
                    "Malformed v7 raw ciphertext: recovered codepoint {} at \
                     position {} is out of byte range [0, 255]. This usually \
                     means the ciphertext was produced by `encrypt_bytes` \
                     (codepoint-oriented) rather than `encrypt_raw` \
                    (byte-oriented); the two use incompatible message \
                     encodings (CVF-14).",
                    c, i
                ))
            } else {
                Ok(c as u8)
            }
        })
        .collect()
}
```

CVF-V5-8: open Category: **procedural**

Summary

The new FIPS self-test gate is off by default, does not cover the legacy functions or key generation even when it is on, is described incorrectly in the documentation, and no test shows that it ever blocks an operation.

A FIPS module must refuse to perform cryptographic operations before its start-up self-tests have passed and after any of them has failed. The fixes add a gate that does this, but it is switched off unless the user explicitly turns it on. Even when it is on, it does not block the older encryption functions, which the crate's own gateway uses, or key generation, and a recorded failure is cleared as soon as the tests are run again. The documentation says that all cryptographic operations are blocked, so an operator would wrongly believe that the older encryption functions are blocked too. Finally, no test checks that the gate ever refuses an operation.

This finding covers what remains of CVF-47, which is partially fixed, together with two new problems in the text and tests that the fixes added.

The following parts are fixed: a four-state latch for the power-on self-test (POST), i.e. a state variable that records its result (`self_test.rs:92-130`), with no bypass through the RUNNING state and with panic handling that fails closed, and a `require_post` check at every v8 block, streaming and `_key` entry point.

The following problems remain:

- The gate is behind the opt-in `fips_gate` feature, and the default feature set is empty (`Cargo.toml:6-13`). This is the opposite of what CVF-47 recommended.
- With the gate on, all v7 entry points, including the four public encryptors `encrypt`, `encrypt_bytes`, `encrypt_str` and `encrypt_raw` (`lib.rs:741-767`, `1053-1081`, `1188-1194`, `1226-1253`) that `ot_frame` uses, and all key generation still run in the NOT_RUN and FAILED states. A FAILED state is also cleared by any later re-run of POST.
- The manifest comment (`Cargo.toml:6-10`) says that otherwise "all crypto operations" return `NotOperational`, and the README's list of ungated functions (`README.md:100-114`) omits the v7 encryptors. An operator would therefore believe that v7 encryption is inhibited in the error state.
- No test shows a refusal. Under `cfg(test)`, POST runs automatically on the first gated use (`self_test.rs:119-130`), so NOT_RUN cannot be observed in unit tests, and no fault-injection hook makes FAILED observable. The gate tests (`self_test.rs:816-890`) assert only success and would pass on a crate without the gate. The two refusal tests that the remediation report names do not exist.

The continuous random-number-generator test, whose failure does not put the module into the error state, is reported separately, in CVF-V5-19.

Recommendation

Enable `fips_gate` by default, gate the v7 entry points and key generation, and keep the FAILED state until the process restarts.

Until the v7 functions are gated, list them as ungated in `Cargo.toml` and in the README, and remove "all crypto operations".

Add an integration test under `rust/tests/`, built with `--features fips_gate`, that asserts that every gated entry point returns `NotOperational` before POST, and add a fault-injection switch for tests only, to cover the FAILED state. Finally, correct the list of tests in the remediation report.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-8 — Partly fixed in eca82e6. Under `fips_gate`, all eight v7 entry points and key generation now require POST, and FAILED is sticky until the process restarts. The new test-only `fault_injection` feature (which refuses release builds) drives `rust/tests/fips_gate.rs`. That test asserts that every gated entry point refuses before POST and after an injected failure, and that a re-run does not clear FAILED; it runs in CI. The Cargo.toml comment and the README list exactly what is and is not gated, and the remediation report's non-existent test names are corrected. **Not done:** `fips_gate` remains opt-in rather than default-on.

/rust/Cargo.toml

```
6  # CVF-47: opt-in FIPS 140-3 self-test gating. When enabled, every
   public v8
7  # entry point calls `self_test::require_post()?` before doing crypto
   — the
8  # caller MUST first call `run_power_on_self_tests()` and it MUST
   succeed,
9  # else all crypto operations return `Err(NotOperational)`. Disabled
10 # by default so existing (non-FIPS) consumers see zero behaviour
   change.
11 [features]
12 default = []
13 fips_gate = []
```

/rust/README.md

```
100
101 ## Power-on self-tests (FIPS 140-3)
102
103 `napqes::self_test::run_power_on_self_tests()` runs the SP 800-140B
    §4.9.1
104 KATs (v7 and v8 encrypt/decrypt/tamper, a v8 structural reject, and a
    KEM
105 derivation KAT plus FrodoKEM and hybrid pairwise-consistency checks)
    and
106 latches the outcome process-wide. Building with `--features
    fips_gate`
107 makes every public v8 encrypt/decrypt entry point (block, streaming,
```

```
`NapqesKey` variants) and every KEM encapsulate/decapsulate call refuse to
run (`module not operational`) until that call has returned `Ok(())`; a
failed POST keeps the module in the error state. Without the feature the
outcome is still latched but not enforced, and the feature is off by
default. Key generation, the v7 legacy decryptors, `ot_frame`, `protocols`
and `vale` are outside the gate. The bundled binaries and examples run POST
at start-up.
```

/rust/src/lib.rs

```rust
pub fn encrypt(message: &[u32], key: &[u64]) -> ([u8; NONCE_SIZE], Vec<u64>) {
    // CVF-34: uphold the length cap even on this Vec<u64>-returning legacy
    // path. The .expect() surfaces the cap consistently with the deprecation
    // attribute and matches the pattern of the CVF-15 validate_key call.
    assert!(
        message.len() <= MAX_PLAINTEXT_CODEPOINTS,
        "message length {} exceeds MAX_PLAINTEXT_CODEPOINTS ({})",
        message.len(), MAX_PLAINTEXT_CODEPOINTS
    );
    // CVF-15: this v7 API returns `Vec<u64>` and cannot report a validation
    // error. Every other entry point returns `Result` and propagates via `?`.
    // The .expect() here surfaces the same validation failure as a panic,
    // consistent with `pub fn decrypt`'s prior panic contract and matched by
    // the #[deprecated] attribute (CVF-17) that steers callers to v8.
    validate_key(key).expect("CVF-15: encrypt called with invalid key material");
    // CVF-35: route through the checked CRNG helper like every other v7
    // encryptor. Signature is `-> (nonce, tokens)` so a CRNG failure is
    // surfaced as `.expect` — consistent with the deprecation attribute
    // that steers callers to v8.
    let nonce = generate_nonce_with_crng_check()
```

```
        .expect("CVF-35: CRNG stuck-generator check failed inside encrypt");
    let kb = key_bytes(key);
    let noise_p = derive_noise_p(&kb, &nonce);
    let padded = pad_message(message, &kb, &nonce);
    let cypher = emit_tokens_v7(&padded, &kb, &nonce, noise_p, key);
    (nonce, cypher)
}
```

same file

```
pub fn encrypt_bytes(message: &str, key: &[u64], aad: &[u8]) -> Result<Vec<u8>, String> {
    validate_key(key)?; // CVF-15
    check_v7_aad_len(aad.len())?;
    // CVF-34: cap plaintext length at MAX_PLAINTEXT_CODEPOINTS at the entry
    // point (was an `assert!` in `pad_message`, which aborted release
    // builds — Section 12's "raises an error immediately; no silent
    // truncation" caveat is now honoured on v7 too).
    if message.chars().count() > MAX_PLAINTEXT_CODEPOINTS {
        return Err(format!(
            "message exceeds MAX_PLAINTEXT_CODEPOINTS ({})",
            MAX_PLAINTEXT_CODEPOINTS
        ));
    }
    let nonce = generate_nonce_with_crng_check()?;
    let codepoints: Vec<u32> = message.chars().map(|c| c as u32).collect();
    let kb = key_bytes(key);
    let noise_p = derive_noise_p(&kb, &nonce);
    let padded = pad_message(&codepoints, &kb, &nonce);
    let cypher = emit_tokens_v7(&padded, &kb, &nonce, noise_p, key);
// CVF-46
    let blob = fixed_encode_tokens(&cypher);
    let ks = varint_keystream(&kb, &nonce, blob.len());
    let masked: Vec<u8> = blob.iter().zip(ks.iter()).map(|(a, b)| a ^ b).collect();
    let mut payload = Vec::with_capacity(NONCE_SIZE + masked.len());
    payload.extend_from_slice(&nonce);
    payload.extend_from_slice(&masked);
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
    payload.extend_from_slice(&tag);
    Ok(payload)
}
```

same file

```
pub fn encrypt_str(message: &str, key: &[u64], aad: &[u8]) ->
Result<String, String> {
    // CVF-15: transitively covered by encrypt_bytes, but
early-rejecting a
    // bad key means we never allocate the base64 buffer for a doomed
op.
    validate_key(key)?;
    #[allow(deprecated)]
    Ok(STANDARD.encode(encrypt_bytes(message, key, aad)?))
}
```

same file

```
pub fn encrypt_raw(data: &[u8], key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
    validate_key(key)?; // CVF-15
    check_v7_aad_len(aad.len())?;
    // CVF-34: cap at MAX_PLAINTEXT_CODEPOINTS bytes (each byte
becomes a
    // codepoint under the raw-encode convention, so the same
2-codepoint
    // length prefix caps the byte count).
    if data.len() > MAX_PLAINTEXT_CODEPOINTS {
        return Err(format!(
            "data exceeds MAX_PLAINTEXT_CODEPOINTS ({}) bytes",
            MAX_PLAINTEXT_CODEPOINTS
        ));
    }
    let nonce = generate_nonce_with_crng_check()?;
    let codepoints: Vec<u32> = data.iter().map(|&b| b as
u32).collect();
    let kb = key_bytes(key);
    let noise_p = derive_noise_p(&kb, &nonce);
    let padded = pad_message(&codepoints, &kb, &nonce);
    let cypher = emit_tokens_v7(&padded, &kb, &nonce, noise_p, key);
// CVF-46
    let blob = fixed_encode_tokens(&cypher);
    let ks = varint_keystream(&kb, &nonce, blob.len());
    let masked: Vec<u8> = blob.iter().zip(ks.iter()).map(|(a, b)| a ^
b).collect();
    let mut payload = Vec::with_capacity(NONCE_SIZE + masked.len());
    payload.extend_from_slice(&nonce);
    payload.extend_from_slice(&masked);
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
    payload.extend_from_slice(&tag);
    Ok(payload)
}
```

/rust/src/self_test.rs

```
fn admit(state: u8, in_post: bool) -> Result<(), SelfTestError> {
    match state {
        POST_PASSED => Ok(()),
        POST_RUNNING if in_post => Ok(()),
        _ => Err(SelfTestError::NotOperational),
    }
}

/// Crate-visible gate (CVF-47). Returns `Ok(())` iff the power-on self-tests
/// have passed. A caller arriving while POST is running on another thread
/// blocks until it finishes, then sees the final outcome.
///
/// Called from every public v8 encrypt/decrypt entry point and every KEM
/// encapsulate/decapsulate function under `#[cfg(feature = "fips_gate")]`.
#[allow(dead_code)]
pub(crate) fn require_post() -> Result<(), SelfTestError> {
    if IN_POST.with(|f| f.get()) {
        return admit(POST_STATE.load(Ordering::Acquire), true);
    }
    ensure_post_for_unit_tests();
    if POST_STATE.load(Ordering::Acquire) == POST_RUNNING {
        let _guard = lock_post();
        return admit(POST_STATE.load(Ordering::Acquire), false);
    }
    admit(POST_STATE.load(Ordering::Acquire), false)
}

/// Unit tests share one process-global POST_STATE, so run POST on first use
/// instead of letting tests race to reset and re-run it.
#[cfg(test)]
fn ensure_post_for_unit_tests() {
    if POST_STATE.load(Ordering::Acquire) == POST_NOT_RUN {
        let _ = run_power_on_self_tests();
    }
}

#[cfg(not(test))]
#[allow(dead_code)]
fn ensure_post_for_unit_tests() {}
```

same file

```
    // CVF-47: latch transitions to PASSED on success and stays there on rerun.
    #[test]
    fn cvf47_post_state_transitions_to_passed() {
        run_power_on_self_tests().expect("POST failed unexpectedly");
        run_power_on_self_tests().expect("POST failed unexpectedly on rerun");
        let _guard = lock_post();
        assert_eq!(POST_STATE.load(Ordering::Acquire), POST_PASSED);
    }

    // CVF-47: only PASSED admits other threads; RUNNING admits only the POST thread.
    #[test]
    fn cvf47_gate_decision_table() {
        assert_eq!(admit(POST_NOT_RUN, false), Err(SelfTestError::NotOperational));
        assert_eq!(admit(POST_NOT_RUN, true), Err(SelfTestError::NotOperational));
        assert_eq!(admit(POST_RUNNING, false), Err(SelfTestError::NotOperational));
        assert_eq!(admit(POST_RUNNING, true), Ok(()));
        assert_eq!(admit(POST_PASSED, false), Ok(()));
        assert_eq!(admit(POST_FAILED, false), Err(SelfTestError::NotOperational));
        assert_eq!(admit(POST_FAILED, true), Err(SelfTestError::NotOperational));
    }

    // KAT-6 must fail on the structural check, not the tag.
    #[test]
    fn kat6_forgery_passes_the_tag() {
        let ct = crate::encrypt_bytes_v8(KAT_V8_MESSAGE, KAT_V8_PRIMES, &KAT_V8_SK, KAT_V8_AAD).unwrap();
        let body = &ct[..ct.len() - crate::TAG_SIZE];
        assert_eq!(
            crate::retag_block_v8(body, &KAT_V8_SK, KAT_V8_AAD)[..],
            ct[ct.len() - crate::TAG_SIZE..]
        );
    }
}

// ─── FIPS gate integration tests (opt-in via --features fips_gate)
```

```
#[cfg(all(test, feature = "fips_gate"))]
mod fips_gate_tests {
    //! With the fips_gate feature enabled, every public v8 entry point must
    //! call `require_post()?` before doing any crypto. These tests exercise
    //! the gate end-to-end. Run with:
    //!   `cargo test --lib --features fips_gate fips_gate_tests`

    use super::*;

    #[test]
    fn v8_entry_admits_after_post() {
        run_power_on_self_tests().unwrap();
        let key = crate::NapqesKey::generate().unwrap();
        let ct = crate::encrypt_bytes_v8_key("hi", &key, b"").unwrap();
        assert_eq!(crate::decrypt_bytes_v8_key(&ct, &key, b"").unwrap(), "hi");
    }

    // A second thread must wait for an in-flight POST instead of being
    // admitted mid-run (the pre-fix behaviour) or spuriously refused.
    #[test]
    fn concurrent_callers_wait_for_post() {
        let key = std::sync::Arc::new(crate::NapqesKey::generate().unwrap());
        let workers: Vec<_> = (0..4)
            .map(|_| {
                let key = key.clone();
                std::thread::spawn(move || {
                    for _ in 0..8 {
                        crate::encrypt_bytes_v8_key("hi", &key, b"").unwrap();
                    }
                })
            })
            .collect();
        for _ in 0..4 {
            run_power_on_self_tests().unwrap();
        }
        for w in workers {
            w.join().unwrap();
        }
    }
```

```
890 }
```

6.2. Minor

CVF-V5-9: open Category: **flaw**

Summary

In an empty stream, nothing authenticates the frame size written in the header, so anyone can turn a valid empty-stream ciphertext into other valid ciphertexts.

An encrypted stream starts with a header that records the size of the pieces into which the message is split. When the stream contains no data at all, nothing protects this value against modification, so anyone can change it and the result is still accepted as authentic. The impact is small: the modified ciphertext still decrypts only to the same empty message, so no content chosen by an attacker gets through. However, the scheme is meant to reject every modified ciphertext, and in this case it does not.

The fixes introduced this problem, since the streaming code is new.

The stream header is `0x02 || be4(F) || nonce`. The sentinel tag covers the domain byte, the nonce, the chunk count, the AAD and the total number of real codepoints, but neither the format byte nor `F`. The chunk tags do not include `F` either. For a non-empty stream, `F` is fixed only indirectly: the decryptor requires every chunk to be exactly `F * 160` bytes long, and the chunk bytes are the final HMAC input. A stream with zero chunks reaches the sentinel without any check of chunk length, and the value of `F` in the header is only checked to lie in `[1, 16384]`.

To reproduce the problem, take the empty-stream vector `S001` from `tests/kat/v8_stream_vectors.json` and rewrite bytes 1 to 4 to any `F` in `[1, 16384]`. The result still decrypts to the empty string under the same key and AAD. We confirmed this with both of our independent implementations, one written from the code and one from `SPEC.md`. The forged ciphertext decrypts only to the same empty plaintext, so plaintext integrity is preserved. However, integrity of ciphertexts, i.e. the property that no modified ciphertext is accepted, does not hold, and the header can be modified.

Recommendation

Bind the complete stream header (the format byte, `be4(F)` and the nonce) into the sentinel tag, and preferably into every chunk tag, and state this in `SPEC.md` section 8.2.

Also add a negative stream test vector: an empty stream with a modified F.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-9 — Fixed in 7a17584. Every chunk tag, the sentinel tag and the count mask bind the full 21-byte header `0x02 || be4(F) || nonce`, so F of an empty stream is no longer malleable. The new negative vector S-N04 rewrites F from 8 to 9 on an empty stream, and there is a Rust unit test; all three ports reject it.

/rust/src/lib.rs

```
1906 fn compute_stream_v8_sentinel_tag(
1907     sk_fmt: &[u8; 32],
1908     nonce: &[u8; NONCE_SIZE],
1909     chunk_count: u32,
1910     aad: &[u8],
1911     total_real_cps: u64,
1912 ) -> [u8; TAG_SIZE] {
1913     let mut buf = Vec::with_capacity(1 + NONCE_SIZE + 4 +
    AAD_LEN_WIDTH_V8 + aad.len() + 8);
1914     buf.push(DOMAIN_STREAM_V8_SENTINEL);
1915     buf.extend_from_slice(nonce);
1916     buf.extend_from_slice(&chunk_count.to_be_bytes());
1917     buf.extend_from_slice(&be_len_prefix(aad.len(),
    AAD_LEN_WIDTH_V8));
1918     buf.extend_from_slice(aad);
1919     buf.extend_from_slice(&total_real_cps.to_be_bytes());
1920     hmac_digest(sk_fmt, &buf)
1921 }
```

same file

```
2103     let f = u32::from_be_bytes([stream[1], stream[2], stream[3],
    stream[4]]);
2104     if f < 1 || f > STREAM_AE_V8_MAX_FRAME {
2105         return Err(format!(
2106             "Malformed v8 stream header: frame_codepoints F={} out of
    [1, {}].",
2107             f, STREAM_AE_V8_MAX_FRAME
2108         ));
2109     }
```

CVF-V5-10: open Category: **architecture**

Summary

The Rust streaming decryptor cannot process a stream piece by piece: it needs the whole encrypted stream, about 160 times the size of the message, in memory before it returns anything.

Streaming encryption exists so that very long messages can be processed in pieces, without holding the whole message in memory. In the Rust code, only encryption works this way. Decryption needs the entire encrypted stream at once, and the encrypted data is about 160 times larger than the original message, so a long stream needs a large amount of memory. The function that encrypts a whole message at once also uses a kind of protection that is less tolerant of mistakes than the one used by the standard encryption functions. Finally, the Python implementation behaves differently under the same format name.

The fixes introduced this problem, since the streaming code is new.

`decrypt_stream_ae_v8` takes the complete ciphertext as one `&[u8]` and returns only after every chunk tag and the sentinel have been verified. No incremental decryptor exists in the crate, even though `docs/CAVEATS.md` names a `StreamV8Decryptor`. The README shows only an incremental API for encryption.

Because it buffers the whole stream, the Rust implementation never releases unverified plaintext. However, a receiver cannot process a stream of 10 million codepoints (about 1.6 GB of ciphertext at 20 tokens of 8 bytes per codepoint) without holding all of it in memory. The all-at-once `encrypt_stream_ae_v8` likewise holds the whole plaintext in memory, yet it uses a random nonce instead of the misuse-resistant synthetic nonce. For messages in memory longer than 65,535 codepoints, the only option that the crate offers is therefore the one that is not misuse-resistant. The decryptor of the Python implementation, on the other hand, is incremental (it is outside the scope), so the two implementations have different semantics under the same format name.

Recommendation

Either add an incremental decryptor that releases chunk `i` only after its tag has been verified, and document that truncation is detected only at the sentinel, or document that Rust streaming is incremental on the encryption side only.

Also consider deriving the nonce synthetically in the all-at-once `encrypt_stream_ae_v8`, since that function already has the full message.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-10 — Fixed (documentation option) in 1716bcd. The rustdoc and README state that Rust streaming is incremental on the encrypt side only: `decrypt_stream_ae_v8` needs the whole stream (about 160 B per codepoint) and releases nothing before the sentinel verifies. They contrast this with the Python generator's release semantics (CAV-001), and say that the all-at-once encryptor uses a random nonce, pointing in-memory messages of up to 65,535 codepoints to the misuse-resistant

block API. No incremental decryptor and no synthetic stream nonce are added; streaming itself is experimental (CVF-V5-2).

/rust/README.md

````
## Streaming AE (v8)

```rust
use napqes::{
    generate_v8_key, encrypt_stream_ae_v8, decrypt_stream_ae_v8,
    StreamV8Encryptor,
    DEFAULT_KEY_COUNT, MIN_KEY_PRIME, MAX_KEY_PRIME,
};

napqes::self_test::run_power_on_self_tests().expect("POST failed");

let (primes, sk) = generate_v8_key(DEFAULT_KEY_COUNT, MIN_KEY_PRIME,
MAX_KEY_PRIME).unwrap();

// All-at-once
let ct = encrypt_stream_ae_v8("hello, streaming!", &primes, &sk, b"",
128).unwrap();
let pt = decrypt_stream_ae_v8(&ct, &primes, &sk, b"").unwrap();

// Chunk-at-a-time
let (mut enc, header) = StreamV8Encryptor::new(&primes, &sk, b"",
128).unwrap();
let mut buf: Vec<u8> = header;
for c in "chunk-by-chunk".chars() {
    if let Some(frame) = enc.push(c).unwrap() {
buf.extend_from_slice(&frame); }
}
buf.extend_from_slice(&enc.finish().unwrap());
```

**Streaming caveat.** v8 streaming is NOT misuse-resistant — the
16-byte
per-stream nonce is CSPRNG-drawn, not SIV-derived. Nonce reuse under
the
same `sk` is a CVF-3-class hazard. See `docs/CAVEATS.md` CAV-005. For
messages that fit in memory, prefer the SIV-protected block API
(`encrypt_bytes_v8` / `decrypt_bytes_v8`).
````

/rust/src/lib.rs

```
/// v8 streaming-AE decryption -- consumes the full ciphertext byte
stream.
///
```

```
2066 /// Convenience "all-at-once" API. Each per-chunk tag is verified
     before that
2067 /// chunk's plaintext contributes to the output; the sentinel binds
     the
2068 /// chunk count (anti-truncation) and the true real-codepoint count
     (used to
2069 /// strip filler codepoints from the last chunk).
2070 pub fn decrypt_stream_ae_v8(
2071     stream: &[u8],
2072     primes: &[u64],
2073     sk: &[u8; SK_SIZE],
2074     aad: &[u8],
2075 ) -> Result<String, String> {
2076     // CVF-47: FIPS 140-3 gate (opt-in via the `fips_gate` feature).
2077     #[cfg(feature = "fips_gate")]
2078     self_test::require_post().map_err(|e| e.to_string())?;
2079     validate_key(primes)?;
2080     let sk_fmt = Secret32::new(derive_format_subkey(sk,
     FORMAT_STREAM_AE_V8));
2081     decrypt_stream_ae_v8_inner(stream, primes, sk_fmt.as_ref(), aad)
2082 }
```

CVF-V5-11: open Category: **flaw**

> **Summary**
>
> **The fixes leave new copies of secret key material in memory without erasing them, in three places, although they were meant to close the finding about erasing secrets.**
>
> Secret keys should be erased from memory as soon as the program no longer needs them, so that someone who can later read that memory, for example from a crash dump, does not find them there. The fixes erase most copies, but they also add three new places where a copy of the key, or of the message being encrypted, stays in memory after use. One of these copies is made every time a key is checked, which happens on almost every operation. The client's remediation report states that this kind of key material is always protected, which is not the case.

CVF-37 is otherwise fixed. The fixes introduced three copies that are not wiped, i.e. not overwritten with zeros before the memory is released:

1. **A sorted copy in `validate_key`.** To fix the quadratic distinctness check (CVF-41), `validate_key` now executes `let mut sorted = key.to_vec()`, sorts the

copy, and drops it without wiping it. The baseline code used an in-place `contains` scan with no allocation. `validate_key` runs on every key the crate sees: inside `NapqesKey::new`, at the start of all nine v7 entry points (two or three times per v7 decrypt call), on every v8 call that takes the key as a bare slice, and on every stream open. For the v7 format the prime tuple is the entire key, and a sorted copy loses only the order of its elements. Thirteen elements have about 2^32.5 orderings, which an attacker can test with one known plaintext. The same code carefully wraps the v7 `key_bytes` buffer in the wipe-on-drop `SecretBytes` type for exactly this reason.

2. **`StreamV8Encryptor`.** The client's remediation report states that `sk_fmt` "is held in the `Secret32` guard on every v8 path". However, `StreamV8Encryptor` stores `sk_fmt` as a raw `[u8; 32]` field. Its constructor `new` copies a local `sk_fmt` into the struct and returns the struct by value inside a tuple, and each such move is a bitwise copy that leaves an unwiped copy on the stack. Its `Drop` implementation wipes `sk_fmt` and `primes` but not `cp_buf`, which holds up to `F - 1` plaintext codepoints between `push` calls. The memory released by `drain` in `push` is not wiped either. The streaming paths that process the whole message at once use `Secret32` correctly.

3. **`NapqesKey::new`.** When `validate_key` rejects the input, the moved `primes` vector and `sk` array are dropped without being wiped, although wiping on drop is the stated purpose of the type.

Recommendation

Wipe or avoid each of the three copies.

In `validate_key`, either wrap the scratch vector in a wipe-on-drop guard, or return to an in-place distinctness scan, which costs at most 8,128 comparisons for `K <= 128` and is negligible next to the cost of trial division.

In `StreamV8Encryptor`, hold `sk_fmt` in a boxed `Secret32`, so that moving the struct does not copy the key, and wipe `cp_buf` in `Drop` and after every drain.

In `NapqesKey::new`, wipe the inputs on the error path.

Also correct the statement in the remediation report about the coverage of `Secret32`.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-11 — Fixed in `476f226`.

- `validate_key` uses an in-place pairwise distinctness scan, so no sorted copy is made.
- `StreamV8Encryptor` keeps `sk_fmt` in a boxed `Secret32`, and `cp_buf` is wiped over its full capacity after every chunk and on Drop.
- `NapqesKey::new` wipes rejected inputs.
- The key-generation scratch index and partial tuples are wiped (`75de8cf`).

  The remediation report's statement about `Secret32` coverage is superseded by this commit.

/rust/src/key.rs

```
54    pub fn new(primes: Vec<u64>, sk: [u8; SK_SIZE]) -> Result<Self,
String> {
```

```
        validate_key(&primes)?;
        Ok(Self { primes, sk })
    }
```

/rust/src/lib.rs

```
    // Distinctness via sort-and-scan on a local clone (CVF-41).
    let mut sorted = key.to_vec();
    sorted.sort_unstable();
    for w in sorted.windows(2) {
        if w[0] == w[1] {
            return Err(format!(
                "Key element {} is a duplicate; all elements must be distinct.",
                w[0]
            ));
        }
    }
```

same file

```
pub struct StreamV8Encryptor {
    primes: Vec<u64>,
    sk_fmt: [u8; 32],
    aad: Vec<u8>,
    nonce: [u8; NONCE_SIZE],
    f: u32,
    chunk_idx: u32,
    total_real_cps: u64,
    cp_buf: Vec<u32>,
    finished: bool,
}
```

same file

```
        let sk_fmt = derive_format_subkey(sk, FORMAT_STREAM_AE_V8);
        let mut header = Vec::with_capacity(1 + 4 + NONCE_SIZE);
        header.push(FORMAT_STREAM_AE_V8);
        header.extend_from_slice(&frame_codepoints.to_be_bytes());
        header.extend_from_slice(&nonce);
        Ok((
            Self {
                primes: primes.to_vec(),
                sk_fmt,
                aad: aad.to_vec(),
                nonce,
                f: frame_codepoints,
                chunk_idx: 0,
                total_real_cps: 0,
```

```
                cp_buf: Vec::with_capacity(frame_codepoints as usize),
                 finished: false,
            },
            header,
        ))
```

same file

```
impl Drop for StreamV8Encryptor {
    fn drop(&mut self) {
        zeroize_sk(&mut self.sk_fmt);
        zeroize_key(&mut self.primes);
    }
}
```

CVF-V5-12: open Category: **efficiency**

> **Summary**
> **The legacy decryption functions now check the key before checking that a message is authentic, so forged messages cost much more to reject and the rejection time reveals information about the key.**
> Since the fixes, the legacy (v7) decryption functions check that the secret key is well formed before they check whether an incoming message is authentic, and they do so two or three times per message. This key check is slow, and how long it takes depends on the secret key. An attacker who can send forged messages to a gateway that uses these functions can therefore make it work about a hundred times harder than before to reject each one. By measuring how long each rejection takes, the attacker can also learn a little about the key. The amount of information leaked per message is small.

The fix for CVF-15 added key validation to the v7 decryptors. In the baseline code, they computed `key_bytes` and the tag first and did no validation at all.

Validation now runs at the start of `decrypt_raw` and again inside `decrypt`, which `decrypt_raw` calls after authentication. `decrypt_bytes` likewise validates first and then again through `decrypt`. `decrypt_str` validates and then calls `decrypt_bytes`, so it performs three validations.

`validate_key` performs trial division of each element by every odd number up to its square root. For primes in the widened interval, this is up to 1,935 divisions per element. For a `K = 13` key, it is about 17,400 divisions on average per validation, ranging from about 6,500 to 25,000 depending on the secret primes.

An attacker who can reach the secure-side socket of the OT gateway can send minimal 48-byte envelopes with random tags, and `ot_frame::unwrap_pdu`, which is outside the scope of this review, passes every received envelope to `decrypt_raw`. Compared with the baseline code, where rejecting a forged envelope cost one HMAC, the attacker gets roughly two orders of magnitude of CPU amplification. The attacker also gets a timing signal before authentication, which is a function of the sum of the square roots of the key elements. On the v7 format, the prime tuple is the whole key. We rate this finding low because our estimate of the information exposed per observation is at most about 12.5 bits.

This is the behaviour described in CVF-29, i.e. key validation on every call acting as a timing channel, which the `NapqesKey` type was introduced to remove. The fixes have now extended it to entry points that did not have it before.

Recommendation

Verify the tag before validating the key in `decrypt_bytes` and `decrypt_raw`. `key_bytes` and HMAC are defined for any key, and validation is needed only before the arithmetic in `decrypt_core`. Call `decrypt_core` directly instead of the validating `decrypt`, so that validation runs at most once per call.

Preferably, also add v7 decryptors that take a pre-validated key type, the counterpart of `NapqesKey`, so that a gateway validates its keys once, when it loads its key store.

Finally, extend Section 8.4 of the paper to say that the v7 decryptors validate the key before authentication.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-12 — Fixed in `4eea6fb`. `decrypt_bytes`, `decrypt_raw` and `decrypt_str` share `open_v7`, which runs in this order: AAD-length and parse checks, the tag, one key validation, then `decrypt_core`. A forged envelope therefore costs one HMAC and never reaches key validation. CVF-V5-16 further replaces trial division on this path with a relaxed check. Paper §8.4 states the order. The gateway's pre-validated path is `NapqesKey` on v8 (CVF-V5-7), so we add no separate v7 key type.

/rust/src/lib.rs

```
401 fn validate_key(key: &[u64]) -> Result<(), String> {
402     if key.is_empty() {
403         return Err("Key must be a non-empty list of primes.".into());
404     }
405     // Cheap checks first: trial division of an out-of-range u64
costs seconds.
406     if key.len() > MAX_KEY_ELEMENTS {
407         return Err(format!(
408             "Key has {} elements, exceeding MAX_KEY_ELEMENTS ({}).
Very large \
409              keys are rejected because per-message
primality/distinctness \
410              validation is O(K²) with the classic scan and O(K log K)
with \
```

```
             sort-and-scan; the cap makes the cost of legitimate use bounded.",
            key.len(),
            MAX_KEY_ELEMENTS
        ));
    }
    for (i, &k) in key.iter().enumerate() {
        if k < MIN_KEY_PRIME {
            return Err(format!(
                "Key element at index {} ({}) is below the minimum of {}.",
                i, k, MIN_KEY_PRIME
            ));
        }
        // CVF-16: enforce the normative upper bound on every key, not just on
        // generation.
        if k > MAX_KEY_PRIME {
            return Err(format!(
                "Key element at index {} ({}) exceeds MAX_KEY_PRIME ({}).",
                i, k, MAX_KEY_PRIME
            ));
        }
        // key_bytes keeps only the low 5 bytes; unreachable while
        // MAX_KEY_PRIME < 2^40, retained as a fail-safe.
        if k > MAX_SERIALISABLE_KEY_PRIME {
            return Err(format!(
                "Key element at index {} ({}) exceeds {}, the largest value \
                 representable in the 5-byte key serialisation.",
                i, k, MAX_SERIALISABLE_KEY_PRIME
            ));
        }
    }
    for (i, &k) in key.iter().enumerate() {
        if !is_prime(k) {
            return Err(format!("Key element at index {} ({}) is not prime.", i, k));
        }
    }
    // Distinctness via sort-and-scan on a local clone (CVF-41).
    let mut sorted = key.to_vec();
    sorted.sort_unstable();
    for w in sorted.windows(2) {
```

```
        if w[0] == w[1] {
            return Err(format!(
                "Key element {} is a duplicate; all elements must be distinct.",
                w[0]
            ));
        }
    }
    Ok(())
}
```

same file

```
pub fn decrypt(nonce: &[u8], cypher: &[u64], key: &[u64]) -> Result<Vec<u32>, String> {
    validate_key(key)?; // CVF-15
```

same file

```
pub fn decrypt_bytes(ciphertext: &[u8], key: &[u64], aad: &[u8]) -> Result<String, String> {
    validate_key(key)?; // CVF-15
```

same file

```
    let codepoints = decrypt(nonce, &tokens, key)?;
```

same file

```
pub fn decrypt_str(cypher: &str, key: &[u64], aad: &[u8]) -> Result<String, String> {
    validate_key(key)?; // CVF-15 — reject before base64 decode
```

same file

```
pub fn decrypt_raw(ciphertext: &[u8], key: &[u64], aad: &[u8]) -> Result<Vec<u8>, String> {
    validate_key(key)?; // CVF-15
```

same file

```
    let codepoints = decrypt(nonce, &tokens, key)?;
```

CVF-V5-13: open Category: **architecture**

Summary

A new demonstration build option silently changes the encryption format for every program that includes the library, and nothing in an encrypted message records which format was used.

The fixes add an optional build setting that is intended only for demonstrations. When it is switched on, the library produces encrypted messages in a format different from the one in the specification, which the other implementations cannot read. Because of the way Rust combines build settings, switching it on for one component of a program switches it on for every component of that program that uses the library. Nothing in an encrypted message records which format was used, so a message in the other format is rejected with a misleading error instead of a clear one. The setting is neither tested nor documented.

The fixes added the feature `noise_x10`. In the baseline code, the manifest declared no features, and `MAX_NOISE_RUN = 19` was unconditional.

The manifest now declares `noise_x10 = []` with the comment "Demo-only … Wire-incompatible with the default build and the Python/C KATs". Under the feature, `MAX_NOISE_RUN` becomes 9 instead of 19. This changes the emission cap, the per-bucket ceiling, the decoder's token-count checks and the streaming chunk size, so the ciphertext length becomes `48 + 80(B+2)` instead of the paper's `48 + 160(B+2)`. Cargo unifies features across a dependency graph, i.e. a feature that one crate enables is enabled for every crate in the same binary. If any crate in a binary enables `napqes/noise_x10`, then every user of `napqes` in that binary gets the non-conforming format, and the public `MAX_NOISE_RUN` constant changes its value without these users being aware of it.

Both variants use the same format identifier `0x01` for the format subkey, and the tag input does not include the cap. A ciphertext from one build therefore passes the HMAC check in the other. The pre-tag length check of the default build also accepts it, because the real-token count is always even. It is rejected only after authentication, by the bucket-legality check (`2(B+2)` or `(B+2)/2` real tokens is never of the form `B+2`), and the operator sees a "Malformed" error instead of an authentication or format error. The two formats are therefore kept apart only by a coincidence of parity, not by domain separation, i.e. by giving each format its own key or its own input to the tag.

The feature is not tested anywhere and is not mentioned in the README or in the remediation report, and the doc comment justifying the cap is attached only to the default variant. With `fips_gate` enabled, the power-on self-test detects the feature, because KAT-4 pins a 2,928-byte ciphertext. In that case, the module becomes permanently unusable instead of producing non-conforming output. Without `fips_gate`, which is the default, the build silently encrypts in the non-conforming format.

Recommendation

Remove `noise_x10` from the library. A demonstration that needs a different noise budget can define its own constant outside the crate.

If the switch must stay:

- give that variant its own format identifier, so that the subkey differs and ciphertexts from the other build fail at the tag check;
- make the combination with `fips_gate` a compile error;
- document the feature next to `MAX_NOISE_RUN`;
- provide matching test vectors.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-13 — Fixed in 69ed83c. The `noise_x10` feature and its `cfg` branches are removed; `MAX_NOISE_RUN` is a fixed protocol constant.

/rust/Cargo.toml

```
16 # Demo-only: MAX_NOISE_RUN 19 -> 9 (x10 tokens per codepoint instead
   of x20).
17 # Wire-incompatible with the default build and the Python/C KATs.
18 noise_x10 = []
```

/rust/src/lib.rs

```
1422 #[cfg(not(feature = "noise_x10"))]
1423 pub const MAX_NOISE_RUN: u64 = 19;
1424 /// Demo-only x10 token budget; ciphertexts are NOT interoperable
   with the default build.
1425 #[cfg(feature = "noise_x10")]
1426 pub const MAX_NOISE_RUN: u64 = 9;
```

same file

```
1428 /// Domain `0x0B` format-subkey identifier for v8 block mode.
1429 pub const FORMAT_BLOCK_V8: u8 = 0x01;
1430 /// Domain `0x0B` format-subkey identifier for the v8 streaming-AE
   format.
1431 pub const FORMAT_STREAM_AE_V8: u8 = 0x02;
```

same file

```
1440 fn derive_format_subkey(sk: &[u8], format_id: u8) -> [u8; 32] {
1441     hmac_digest(sk, &[0x0B, format_id])
1442 }
```

/rust/src/self_test.rs

```
462 /// Expected v8 ciphertext length for a one-codepoint message under
   the
463 /// default bucket profile: 16 (nonce) + 8 * R * (MAX_NOISE_RUN+1) +
   32 (tag)
464 /// where R = B + 2 = 18 for B = 16. The self-test asserts this to
   catch a
465 /// regression that changes ciphertext size independent of the tag
   pipeline.
466 const KAT_V8_EXPECTED_LEN: usize = 2928;
```

```
/// SHA-256 of W002's `ciphertext_hex`.
const KAT_V8_EXPECTED_SHA256: [u8; 32] = [
    0xba, 0x1c, 0x3f, 0x66, 0x4c, 0x62, 0xe9, 0x34,
    0x56, 0x83, 0x32, 0x74, 0xb5, 0x5f, 0x70, 0x3e,
    0xa8, 0x26, 0xf1, 0x28, 0x24, 0xa3, 0x90, 0x88,
    0x5d, 0x8e, 0x7f, 0x1e, 0xb3, 0x99, 0x4c, 0xed,
];
```

CVF-V5-14: open Category: **behavior**

> **Summary**
>
> **The key-generation functions accept a requested key size of zero or above the allowed maximum and return keys that every encryption function then rejects, contrary to their documentation.**
>
> When asked for a key with no elements, or with more elements than the library allows, the key-generation functions report success and return a key that cannot be used: every encryption and decryption function rejects it. Their documentation promises an error in that case. A very large request also takes a very long time before it returns this useless key. The impact is limited to a confusing failure later, at the moment when the key is first used.

The fixes introduced a limit of 128 key elements but did not apply it to key generation. The case of zero elements already existed in the baseline code.

`generate_prime_numbers` does not check `count`. With `count == 0`, the loop never runs and the function returns `Ok(vec![])`. With `count > 128`, it returns `Ok` after a quadratic `contains` scan: `generate_prime_numbers(1_000_000, ...)` performs about 5 x 10^11 comparisons before returning a key that no function accepts. The documentation of `generate_v8_key` says that it returns `Err` when the key "would fail `validate_key`", but the function checks only the interval. `validate_key` rejects both an empty key and a key with more than 128 elements.

Recommendation

Reject `count == 0 || count > MAX_KEY_ELEMENTS` in `generate_v8_key`, and preferably also in `generate_prime_numbers`. Alternatively, run `validate_key` on the output before returning it, so that the documented contract holds.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-14 — Fixed in 2b6db30. `generate_prime_numbers` (and therefore `generate_v8_key` and `NapqesKey::generate`) returns `Err` for `count == 0` or `count`

> MAX_KEY_ELEMENTS before drawing anything. A test covers 0, 129 and the 128 boundary.

/rust/src/lib.rs

```
pub fn generate_prime_numbers(count: usize, min_val: u64, max_val: u64) -> Result<Vec<u64>, String> {
    use rand::distributions::{Distribution, Uniform};
    if max_val <= min_val {
        return Err(format!(
            "generate_prime_numbers: empty range [{}, {}]",
            min_val, max_val
        ));
    }
    let mut rng = rand::thread_rng();
    let dist = Uniform::new_inclusive(min_val, max_val);
    let max_attempts = (count.max(1) as u64).saturating_mul(KEYGEN_DRAWS_PER_PRIME);
    let mut primes: Vec<u64> = Vec::with_capacity(count);
    let mut attempts: u64 = 0;
    while primes.len() < count && attempts < max_attempts {
        // CVF-20: unbiased sample via rand::Uniform (Lemire rejection),
        // replacing the modulo-bias line `min_val + (rng.next_u64() % span)`.
        let num = dist.sample(&mut rng);
        if is_prime(num) && !primes.contains(&num) {
            primes.push(num);
        }
        attempts += 1;
    }
    if primes.len() < count {
        return Err(format!(
           "generate_prime_numbers: could not find {} distinct primes in \
             [{}, {}] within {} draws; the range is too narrow or too prime-poor \
             for the requested count.",
            count, min_val, max_val, max_attempts
        ));
    }
    Ok(primes)
}
```

same file

```
/// Generate a v8 key pair: an arithmetic-layer prime tuple and an
/// independently-sampled, uniformly random 256-bit HMAC subkey.
```

```
///
/// The two components MUST be generated independently (never one derived
/// from the other) for the CVF8/CVF13 security argument above to hold, and
/// MUST both be treated as secret key material.
///
/// Returns `Err` (CVF-20) if `[min_val, max_val]` is not a sub-range of the
/// normative interval `[MIN_KEY_PRIME, MAX_KEY_PRIME]` (such a key would fail
/// `validate_key`), or if `count` distinct primes cannot be drawn from it.
///
/// The returned `sk` is `Copy`: wipe every copy (see [`zeroize_sk`]), or use
/// [`crate::NapqesKey::generate`], which owns both components and wipes them
/// on drop.
pub fn generate_v8_key(
    count: usize,
    min_val: u64,
    max_val: u64,
) -> Result<(Vec<u64>, [u8; SK_SIZE]), String> {
    if !(MIN_KEY_PRIME <= min_val && max_val <= MAX_KEY_PRIME) {
        return Err(format!(
            "generate_v8_key: range [{}, {}] lies outside [MIN_KEY_PRIME, MAX_KEY_PRIME]",
            min_val, max_val
        ));
    }
    let primes = generate_prime_numbers(count, min_val, max_val)?;
    let mut sk = [0u8; SK_SIZE];
    rand::thread_rng().fill_bytes(&mut sk);
    Ok((primes, sk))
}
```

CVF-V5-16: open Category: **documentation**

Summary

The legacy decryption functions are documented as able to read all existing encrypted data, but the fixes make them reject some keys and messages that were accepted before, so some archived data may no longer decrypt.

The older (v7) decryption functions are kept so that data encrypted in the past can still be read, and the documentation says so. The fixes added stricter checks of the key and of the decrypted text to these functions. Some keys and some messages that the earlier code accepted are now rejected, so some archived data may no longer decrypt, contrary to what the documentation promises. Keys produced by the project's own key generators are not affected, so only data encrypted with keys or messages from other sources is at risk.

In the baseline code, the v7 decryptors (`decrypt` at baseline lib.rs:523-526, `decrypt_bytes` at 831-859, `decrypt_raw` at 933-960) never called `validate_key`, and the baseline documentation of `MAX_KEY_PRIME` (baseline lib.rs:75-78) promised that decryption accepts "any prime ≥ MIN_KEY_PRIME, so keys generated before this bound was tightened remain usable". The baseline `pub fn generate_prime_numbers` accepted any range, and the baseline `pub fn encrypt` accepted any `u32`.

In the fixed code, every v7 decryptor runs `validate_key` (lib.rs:401-458) first: `decrypt` at line 821, `decrypt_bytes` at 1135, `decrypt_str` at 1199 and `decrypt_raw` at 1264. A key is therefore now rejected if any element is below 10^6 or above $14{,}999{,}999$, is composite or is duplicated, or if $K > 128$. In addition, `decrypt_core` (lib.rs:793-807) now returns `Err` for a recovered value above `0x10FFFF` or in the surrogate range. Such values could be produced by the baseline `pub fn encrypt` and by the Python v7 `encrypt_bytes` (`napqes.py:876`, outside the scope of this review), which has no surrogate check. These changes came from the fixes for CVF-15 and CVF-14.

Both changes are reasonable hardening, but the documentation still says the opposite. The crate header (lib.rs:27-31) says that the v7 decryptors "remain to decode existing traffic", and `README.md:146-148` says that they are retained to decrypt archived ciphertexts. The remediation report asserts that every v7 ciphertext decrypts byte for byte.

Keys produced by the generators in the repository are not affected: historical Rust keys are at most $9{,}999{,}999$, Python keys at most $15{,}000{,}000$, and keys from `kem.rs` below 1.5×10^7, all of them prime and distinct. The affected cases are keys created outside the repository (with any prime $\geq 10^6$ above the new bound, or with an element that is composite or repeated) and archived plaintexts that contain lone surrogates. We rate this finding low for that reason. The problem is that the documentation makes an unconditional compatibility claim that is no longer true.

Recommendation

State the exact range of v7 compatibility in the crate header, the README and the remediation report, i.e. which keys and which plaintexts written by earlier versions can still be decrypted.

If archives with keys outside the interval must remain readable, give the v7 decryptors

a relaxed validator that is sufficient for the arithmetic, instead of the full v8 key policy: the key is non-empty, every element is at least 2 and below 2^{40}, and the elements are distinct.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-16 — Fixed in e872c6e. The v7 decryptors use a relaxed key policy: 1–128 distinct elements, each in `[2, 2^40-1]`, with no primality or interval check. Archived keys from any earlier interval still open, and no trial division runs on the v7 decrypt path. The exact compatibility envelope (which keys and plaintexts are rejected) is documented on `decrypt_bytes`, in the crate header and in the README; the remediation report's "every v7 ciphertext decrypts" claim is corrected. A test opens a v7 ciphertext under a key with an element below 10^6, a composite element and an element above 1.5e7.

/rust/README.md

```
applies (CVF-28). The corresponding decryptors (`decrypt`, `decrypt_bytes`,
`decrypt_str`, `decrypt_raw`) remain to decrypt existing v7 ciphertexts and
are not deprecated.
```

/rust/src/lib.rs

```
//! * **v7 (legacy — decryptors retained for archived ciphertexts).**
//!   Fixed-width tokens (CVF1 fix). Wire layout same shape, but no
//!   `MAX_NOISE_RUN` cap, so ciphertext length varies with the nonce.
//!   Encryptors are `#[deprecated]` per CVF-17; decryptors ([`decrypt_bytes`],
//!   [`decrypt_str`], [`decrypt_raw`]) remain to decode existing traffic.
```

same file

```
fn validate_key(key: &[u64]) -> Result<(), String> {
    if key.is_empty() {
        return Err("Key must be a non-empty list of primes.".into());
    }
    // Cheap checks first: trial division of an out-of-range u64 costs seconds.
    if key.len() > MAX_KEY_ELEMENTS {
        return Err(format!(
            "Key has {} elements, exceeding MAX_KEY_ELEMENTS ({}). Very large \
             keys are rejected because per-message primality/distinctness \
             validation is O(K²) with the classic scan and O(K log K) with \
             sort-and-scan; the cap makes the cost of legitimate use bounded.",
```

```
            key.len(),
            MAX_KEY_ELEMENTS
        ));
    }
    for (i, &k) in key.iter().enumerate() {
        if k < MIN_KEY_PRIME {
            return Err(format!(
                "Key element at index {} ({}) is below the minimum of {}.",
                i, k, MIN_KEY_PRIME
            ));
        }
        // CVF-16: enforce the normative upper bound on every key, not just on
        // generation.
        if k > MAX_KEY_PRIME {
            return Err(format!(
                "Key element at index {} ({}) exceeds MAX_KEY_PRIME ({}).",
                i, k, MAX_KEY_PRIME
            ));
        }
        // key_bytes keeps only the low 5 bytes; unreachable while
        // MAX_KEY_PRIME < 2^40, retained as a fail-safe.
        if k > MAX_SERIALISABLE_KEY_PRIME {
            return Err(format!(
                "Key element at index {} ({}) exceeds {}, the largest value \
                 representable in the 5-byte key serialisation.",
                i, k, MAX_SERIALISABLE_KEY_PRIME
            ));
        }
    }
    for (i, &k) in key.iter().enumerate() {
        if !is_prime(k) {
            return Err(format!("Key element at index {} ({}) is not prime.", i, k));
        }
    }
    // Distinctness via sort-and-scan on a local clone (CVF-41).
    let mut sorted = key.to_vec();
    sorted.sort_unstable();
    for w in sorted.windows(2) {
        if w[0] == w[1] {
            return Err(format!(
```

```
                    "Key element {} is a duplicate; all elements must be
distinct.",
                    w[0]
                ));
            }
        }
        Ok(())
}
```

same file

```
            if token < addend || (token - addend) % k != 0 {
                return Err(format!(
                    "Malformed v7 ciphertext: token at position {} is
not of \
                     the form codepoint * k + addend for key element
{}.",
                    ct_pos, k
                ));
            }
            let cp = (token - addend) / k;
            if cp > 0x10FFFF || char::from_u32(cp as u32).is_none() {
                return Err(format!(
                    "Malformed v7 ciphertext: recovered value {} at
position \
                     {} is not a Unicode scalar value.",
                    cp, ct_pos
                ));
            }
```

CVF-V5-17: open Category: **behavior**

Summary

The deprecated legacy encryption function still accepts values that its matching decryption function now rejects, so some messages it encrypts can no longer be decrypted.

An older encryption function, which is kept but marked as deprecated, accepts any number as a character of the message, including numbers that are not valid characters. Since the fixes, the matching decryption function rejects such numbers. A message containing them can therefore be encrypted but not decrypted again. No code in the repository uses this function, so the practical impact is small, but an encryption function whose own decryption function rejects its output does not keep its basic promise.

`encrypt` (lib.rs:741-767) takes `&[u32]` and checks only the length and the key, so it encrypts surrogates, i.e. values in the range reserved in Unicode for UTF-16 encoding, and values above `0x10FFFF` without complaint. `decrypt` (lib.rs:820-824) calls `decrypt_core`, whose new guard from the fix for CVF-14 (lib.rs:793-807) returns `Err` for exactly those values. In the baseline code (`encrypt` at baseline lib.rs:467-498, `decrypt` at 504-526), `decrypt(encrypt([0xD800]))` returned `[0xD800]`.

The following call sequence shows the problem: `let (n, t) = encrypt(&[0xD800], &k); decrypt(&n, &t, &k)` returns `Err`. We rate this finding low because the function is deprecated and is not called anywhere in the repository. However, the fixes introduced a defect in its contract: an encryption function whose own decryption function rejects its output.

Recommendation

Reject input values that are not Unicode scalar values, i.e. surrogates and values above `0x10FFFF`, in `encrypt`, consistently with its other preconditions, either with a documented panic or, preferably, with a `Result` signature. Also document the accepted input domain in the rustdoc of the function.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-17 — Fixed in `2086e00`. `encrypt` now returns `Result` and rejects surrogates, values above `0x10FFFF`, over-length messages, invalid keys and CRNG failures. Its rustdoc states the accepted input domain, and a test checks the rejections and the round trip.

/rust/src/lib.rs

```
741 pub fn encrypt(message: &[u32], key: &[u64]) -> ([u8; NONCE_SIZE],
Vec<u64>) {
742     // CVF-34: uphold the length cap even on this Vec<u64>-returning
legacy
743     // path. The .expect() surfaces the cap consistently with the
deprecation
```

```
    // attribute and matches the pattern of the CVF-15 validate_key call.
    assert!(
        message.len() <= MAX_PLAINTEXT_CODEPOINTS,
        "message length {} exceeds MAX_PLAINTEXT_CODEPOINTS ({})",
        message.len(), MAX_PLAINTEXT_CODEPOINTS
    );
    // CVF-15: this v7 API returns `Vec<u64>` and cannot report a validation
    // error. Every other entry point returns `Result` and propagates via `?`.
    // The .expect() here surfaces the same validation failure as a panic,
    // consistent with `pub fn decrypt`'s prior panic contract and matched by
    // the #[deprecated] attribute (CVF-17) that steers callers to v8.
    validate_key(key).expect("CVF-15: encrypt called with invalid key material");
    // CVF-35: route through the checked CRNG helper like every other v7
    // encryptor. Signature is `-> (nonce, tokens)` so a CRNG failure is
    // surfaced as `.expect` — consistent with the deprecation attribute
    // that steers callers to v8.
    let nonce = generate_nonce_with_crng_check()
        .expect("CVF-35: CRNG stuck-generator check failed inside encrypt");
    let kb = key_bytes(key);
    let noise_p = derive_noise_p(&kb, &nonce);
    let padded = pad_message(message, &kb, &nonce);
    let cypher = emit_tokens_v7(&padded, &kb, &nonce, noise_p, key);
    (nonce, cypher)
}
```

same file

```
            if token < addend || (token - addend) % k != 0 {
                return Err(format!(
                    "Malformed v7 ciphertext: token at position {} is not of \
                     the form codepoint * k + addend for key element {}.",
                    ct_pos, k
                ));
            }
            let cp = (token - addend) / k;
```

```
            if cp > 0x10FFFF || char::from_u32(cp as u32).is_none() {
                return Err(format!(
                    "Malformed v7 ciphertext: recovered value {} at position \
                     {} is not a Unicode scalar value.",
                    cp, ct_pos
                ));
            }
```

same file

```
pub fn decrypt(nonce: &[u8], cypher: &[u64], key: &[u64]) -> Result<Vec<u32>, String> {
    validate_key(key)?; // CVF-15
    let kb = key_bytes(key);
    decrypt_core(nonce, cypher, key, &kb)
}
```

CVF-V5-18: open Category: **procedural**

> **Summary**
>
> **The mandatory start-up self-test does not check the new streaming encryption format, which repeats for this format the problem that CVF-18 reported for the v8 scheme.**
>
> Before the module may be used, it runs a set of start-up self-tests that compare its output with known correct results. The fixed code adds a new way of encrypting data piece by piece (streaming), but none of the self-tests checks it. If the streaming code were broken, the self-tests would still pass and the module would keep operating, including when its FIPS mode is enabled. The README describes the self-tests as covering encryption and decryption without any exception. Adding the missing test is cheap, because the necessary test data already exists.

`run_power_on_self_tests` (self_test.rs:560-582) runs KAT-1 to KAT-8 and INT-1, i.e. the v7 known-answer tests (KATs), the v8 block KATs and the KEM derivation. None of them exercises `encrypt_stream_ae_v8`, `decrypt_stream_ae_v8` or `StreamV8Encryptor`, which are public, are listed among the normative v8 entry points (lib.rs:17-25), and are gated by `fips_gate` (lib.rs:2057-2059, 2076-2078, 2246-2248). The code used only by streaming, i.e. the `0x02` format subkey, `derive_stream_v8_sub_nonce`, the filler codepoints and the sentinel tag, is therefore never checked at start-up. `README.md:103-106` says that the power-on self-test (POST) runs "v7 and v8 encrypt/decrypt/tamper" KATs, without any qualification.

A deterministic helper already exists, `encrypt_stream_ae_v8_with_nonce` (`pub(crate)`, lib.rs:1960-1973), and the test-vector file `tests/kat/v8_stream_vectors.json` contains `nonce_hex` for S001–S014, so a known-answer test costs only a few lines. Without such a test, a later error in the sub-nonce derivation, the sentinel binding or the filler leaves POST returning `Ok(())`, and under `fips_gate` the module keeps operating.

Recommendation

Add three streaming tests to POST:

- a streaming encryption known-answer test, i.e. a check of the hash of one S-vector produced through `encrypt_stream_ae_v8_with_nonce`;
- a streaming decryption test;
- a test that a stream with a broken sentinel or a truncated stream is rejected.

Add their constants to `kat_constants_match_corpus`, and update the description of POST in the README accordingly.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-18 — Fixed in `e8cb86f`. With streaming compiled in, POST runs three new tests on vector S002:

- KAT-9: stream encryption, checked by SHA-256 and length;
- KAT-10: decryption;
- KAT-11: rejection of the stream with its frame removed.

The constants are pinned to S002 by `kat_constants_match_corpus`, and the README's POST description lists exactly what is tested.

/rust/README.md

```
103 `napqes::self_test::run_power_on_self_tests()` runs the SP 800-140B §4.9.1
104 KATs (v7 and v8 encrypt/decrypt/tamper, a v8 structural reject, and a KEM
105 derivation KAT plus FrodoKEM and hybrid pairwise-consistency checks) and
106 latches the outcome process-wide. Building with `--features fips_gate`
```

/rust/src/self_test.rs

```
560 pub fn run_power_on_self_tests() -> Result<(), SelfTestError> {
561     // CVF-47: a panicking KAT leaves POST_RUNNING, which refuses all callers.
562     let _lock = lock_post();
563     POST_STATE.store(POST_RUNNING, Ordering::Release);
564     let _in_post = InPostFlag::set();
565     let result = (|| -> Result<(), SelfTestError> {
566         kat_encrypt()?;
567         kat_decrypt()?;
568         kat_tamper_rejection()?;
```

```
        kat_v8_encrypt()?;
        kat_v8_decrypt()?;
        kat_v8_structural_reject()?;
        kat_v8_tamper_rejection()?;
        kat_kem()?;
        build_provenance_check()?;
        Ok(())
    })();
    match &result {
        Ok(()) => POST_STATE.store(POST_PASSED, Ordering::Release),
        Err(_) => POST_STATE.store(POST_FAILED, Ordering::Release),
    }
    result
}
```

CVF-V5-19: open Category: **behavior**

> **Summary**
>
> **When the continuous test of the random number generator fails, only the current call fails and the module keeps operating, although this generator now also supplies the nonces of the streaming format.**
>
> The module checks its random number generator continuously, by making sure that it never produces the same value twice in a row. When this check fails, the module should stop all cryptographic operations until it is restarted. Instead, it fails only the current request and carries on as if nothing had happened. This matters more now, because the new streaming encryption uses this generator for values that must never repeat, and a repeated value would let an attacker learn about the encrypted data or recombine parts of it. The check itself detects only the most obvious failures, which limits the impact.

`generate_nonce_with_crng_check` (lib.rs:103-119) returns `Err` when two consecutive draws are equal and leaves `POST_STATE` unchanged: only `run_power_on_self_tests` writes the state (self_test.rs:57-62, 560-582). The next call simply draws again and proceeds, with the module still in the PASSED state. Under the standard that the crate cites, a failure of a conditional self-test, i.e. a test that runs during operation instead of at start-up, must stop operation until the module is re-initialised.

The fixes made two changes that make this matter:

- The POST latch, i.e. the recorded result of the power-on self-test that gates all operations, is new (it is the fix for CVF-47). There is now an error state that the CRNG test could set, and it does not set it.

- The v8 streaming entry points `encrypt_stream_ae_v8` (lib.rs:2050-2062) and `StreamV8Encryptor::new` (lib.rs:2240-2271) now draw their 16-byte nonce through this function. Streaming is not misuse-resistant: reusing a nonce under one `sk` reveals token differences and allows chunk splicing. A stuck DRBG therefore makes one call of `StreamV8Encryptor::new` fail, and the very next call goes ahead.

We rate this finding low because the test itself is weak: as described in CVF-35, it detects only identical consecutive draws.

Recommendation

On a failure of the continuous test, set the POST state to FAILED under `POST_LOCK`, so that every gated entry point refuses to operate until POST is run again. Document this behaviour in the POST section of the README.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-19 — Fixed in e9489a9. A CRNG continuous-test failure sets POST to FAILED under `POST_LOCK`; the state is sticky (CVF-V5-8). `rust/tests/fips_crng.rs` proves the latch with fault injection, and the README documents it.

/rust/src/lib.rs

```
103 fn generate_nonce_with_crng_check() -> Result<[u8; NONCE_SIZE],
String> {
104     let mut nonce = [0u8; NONCE_SIZE];
105     rand::thread_rng().fill_bytes(&mut nonce);
106     let mut prev = PREV_NONCE.lock().map_err(|_| "CRNG mutex
poisoned".to_string())?;
107     if let Some(prev_nonce) = *prev {
108         if ct_eq_bytes(&nonce, &prev_nonce) {
109             // CVF-35: SP 800-140B §4.9.2 "stuck-generator" check. The
110             // single process-global PREV_NONCE slot is not a per-key
111             // uniqueness guarantee — it catches only an
immediately-stuck
112            // DRBG, not restart-and-replay across processes or per-key
113             // reuse across time. The previous message overstated the
scope.
114             return Err("CRNG stuck-generator check failed:
consecutive draws matched (SP 800-140B §4.9.2)".into());
115         }
116     }
117     *prev = Some(nonce);
118     Ok(nonce)
119 }
```

same file

```
2050 pub fn encrypt_stream_ae_v8(
2051     plaintext: &str,
2052     primes: &[u64],
```

```
    sk: &[u8; SK_SIZE],
    aad: &[u8],
    frame_codepoints: u32,
) -> Result<Vec<u8>, String> {
    // CVF-47: FIPS 140-3 gate (opt-in via the `fips_gate` feature).
    #[cfg(feature = "fips_gate")]
    self_test::require_post().map_err(|e| e.to_string())?;
    let nonce = generate_nonce_with_crng_check()?;
    encrypt_stream_ae_v8_with_nonce(plaintext, primes, sk, &nonce, aad, frame_codepoints)
}
```

same file

```
    pub fn new(
        primes: &[u64],
        sk: &[u8; SK_SIZE],
        aad: &[u8],
        frame_codepoints: u32,
    ) -> Result<(Self, Vec<u8>), String> {
        // CVF-47: FIPS 140-3 gate (opt-in via the `fips_gate` feature).
        #[cfg(feature = "fips_gate")]
        crate::self_test::require_post().map_err(|e| e.to_string())?;
        validate_key(primes)?;
        validate_stream_v8_frame(frame_codepoints)?;
        let nonce = generate_nonce_with_crng_check()?;
        let sk_fmt = derive_format_subkey(sk, FORMAT_STREAM_AE_V8);
        let mut header = Vec::with_capacity(1 + 4 + NONCE_SIZE);
        header.push(FORMAT_STREAM_AE_V8);
        header.extend_from_slice(&frame_codepoints.to_be_bytes());
        header.extend_from_slice(&nonce);
        Ok((
            Self {
                primes: primes.to_vec(),
                sk_fmt,
                aad: aad.to_vec(),
                nonce,
                f: frame_codepoints,
                chunk_idx: 0,
                total_real_cps: 0,
               cp_buf: Vec::with_capacity(frame_codepoints as usize),
                finished: false,
            },
            header,
        ))
    }
```

/rust/src/self_test.rs

```
const POST_NOT_RUN: u8 = 0;
const POST_RUNNING: u8 = 1;
const POST_PASSED: u8 = 2;
const POST_FAILED: u8 = 3;

static POST_STATE: AtomicU8 = AtomicU8::new(POST_NOT_RUN);
```

same file

```
pub fn run_power_on_self_tests() -> Result<(), SelfTestError> {
    // CVF-47: a panicking KAT leaves POST_RUNNING, which refuses all callers.
    let _lock = lock_post();
    POST_STATE.store(POST_RUNNING, Ordering::Release);
    let _in_post = InPostFlag::set();
    let result = (|| -> Result<(), SelfTestError> {
        kat_encrypt()?;
        kat_decrypt()?;
        kat_tamper_rejection()?;
        kat_v8_encrypt()?;
        kat_v8_decrypt()?;
        kat_v8_structural_reject()?;
        kat_v8_tamper_rejection()?;
        kat_kem()?;
        build_provenance_check()?;
        Ok(())
    })();
    match &result {
        Ok(()) => POST_STATE.store(POST_PASSED, Ordering::Release),
        Err(_) => POST_STATE.store(POST_FAILED, Ordering::Release),
    }
    result
}
```

CVF-V5-20: open Category: **procedural**

Summary

Several new tests named after earlier findings cannot fail on the defect they are named for, although the remediation report says that every fix is covered by an automated check named after it.

The fixes add automated tests whose names refer to the earlier findings they are supposed to cover. Four of these tests would still pass if the defect were present, so they do not actually check the fix. The remediation report nevertheless presents them as proof that every fix is tested. This is the same kind of problem as the one described in CVF-19. No defect escapes today, because other test data or the code itself shows that the fixes work, but these tests would not catch the defect if it came back.

The four tests are the following:

- `cvf43_unpad_rejects_noncanonical_length_prefix` (lib.rs:2722-2735) only performs a round trip of an honest message, so it passes on the baseline code. Its comment says that the rejection path "is covered by an internal test below via a helper that mimics decrypt_core_v8's final step", but no such test or helper exists in `lib.rs`, `self_test.rs` or `kat_cross_check.rs`, and the test module never calls `unpad_message`. The actual evidence for the fix of CVF-43 is test vector W-N11, which does fail on the baseline code.
- `cvf42_named_alphabet_constants_are_correct` (lib.rs:2802-2810) compares the new constants with the literals they replaced. It fails on the baseline code only because it does not compile there. The actual evidence is the byte-for-byte agreement of the KATs, i.e. the known-answer test vectors.
- `cvf20_generate_prime_numbers_produces_valid_key` (lib.rs:2703-2719) checks primality, range, distinctness and `validate_key`. The modulo-biased sampler of the baseline code also had all of these properties, so the test cannot detect the bias that CVF-20 is about.
- `napqeskey_matches_bare_slice_api_byte_for_byte` (lib.rs:2875-2886) compares two paths that both call `encrypt_bytes_v8_core`, and carries a stray `#[allow(deprecated)]` on a call that is not deprecated.

The remediation report (§7) states that "Every finding's fix is covered by at least one automated check named after it". For these findings, this is true only of the names of the tests. We rate this finding low because no defect escapes today: the test vectors or the code itself carry the evidence.

Recommendation

Test `unpad_message` directly on non-canonical prefixes, such as `[0, 300, …]`, `[1, 256, …]` and `[0x100, 0, …]` (with a buffer of length B+2), and assert an `Err` containing "byte-valued". Delete the incorrect comment.

Drop or rename the constant test for CVF-42.

For CVF-20, either test the distribution over a deliberately small span or state that the fix is verified by inspection.

Correct the claim in §7 of the remediation report.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-20 — Fixed in 9c0c93b.

- The CVF-43 test calls `unpad_message` directly on `(0,300)`, `(1,256)` and `(0x100,0)` and requires the "byte-valued" rejection, so it fails on v4.
- The CVF-42 constant test is renamed; its real evidence is KAT parity.
- The CVF-20 test is renamed with a note that the bias fix is verified by inspection.
- The NapqesKey parity test now compares against corpus vector W009.

The remediation report's §7 claim is corrected.

/rust/src/lib.rs

```
2703     #[test]
2704     fn cvf20_generate_prime_numbers_produces_valid_key() {
2705         for _ in 0..8 {
2706             let ps = generate_prime_numbers(DEFAULT_KEY_COUNT,
MIN_KEY_PRIME, MAX_KEY_PRIME);
2707             assert_eq!(ps.len(), DEFAULT_KEY_COUNT);
2708             for &p in &ps {
2709                assert!(is_prime(p), "sampler returned non-prime: {}",
p);
2710                 assert!(p >= MIN_KEY_PRIME && p <= MAX_KEY_PRIME);
2711             }
2712             let mut sorted = ps.clone();
2713             sorted.sort();
2714             sorted.dedup();
2715             assert_eq!(sorted.len(), ps.len(), "sampler returned
duplicates: {:?}", ps);
2716             // The sampler's output must pass validate_key
unconditionally.
2717             validate_key(&ps).expect("sampler output failed
validate_key");
2718         }
2719     }
```

same file

```
2722
2723     #[test]
2724     fn cvf43_unpad_rejects_noncanonical_length_prefix() {
2725         // Directly test unpad_message via decrypt_bytes_v8 with a
hand-
2726         // crafted padded buffer would require constructing a valid
ciphertext;
```

```
        // easier to exercise via a small helper. Instead, verify by proxy:
        // any v8 round-trip must still succeed (byte-identical to before).
        // The direct rejection path is covered by an internal test below via
        // a helper that mimics decrypt_core_v8's final step.
        let (primes, sk) = generate_v8_key(DEFAULT_KEY_COUNT, MIN_KEY_PRIME, MAX_KEY_PRIME);
        let msg = "CVF-43 sanity";
        let ct = encrypt_bytes_v8(msg, &primes, &sk, b"").unwrap();
        assert_eq!(decrypt_bytes_v8(&ct, &primes, &sk, b"").unwrap(), msg);
    }
```

same file

```
    #[test]
    fn cvf42_named_alphabet_constants_are_correct() {
        // Byte-parity check: existing KATs would fail if PAD_ALPHABET/
        // NOISE_ALPHABET/ASCII_PRINTABLE_BASE were changed. This asserts
        // the constants match the paper's numeric values.
        assert_eq!(NOISE_ALPHABET, 96);
        assert_eq!(PAD_ALPHABET, 95);
        assert_eq!(ASCII_PRINTABLE_BASE, 32);
    }
```

same file

```
    #[test]
    fn napqeskey_matches_bare_slice_api_byte_for_byte() {
        // The NapqesKey path must produce byte-identical ciphertexts to the
        // bare-slice path (CVF-29 fix skips validate_key but must not change
        // the wire format).
        let k = NapqesKey::generate().unwrap();
        let msg = "byte parity check";
        let via_key = encrypt_bytes_v8_key(msg, &k, b"aad").unwrap();
        #[allow(deprecated)]
        let via_slice = encrypt_bytes_v8(msg, k.primes(), k.sk(), b"aad").unwrap();
        assert_eq!(via_key, via_slice);
    }
```

CVF-V5-21: open Category: **procedural**

Summary

The test of rejected legacy messages silently skips the test vector for an empty encrypted message, which is the shared evidence for the fix of CVF-13, and that vector would fail if it were run.

The project has a shared set of test vectors that all three implementations must pass, including examples of invalid encrypted messages that must be rejected. One of these examples, an empty message, is the evidence that the fix for CVF-13 works in every implementation. The Rust test silently skips it and still reports success, so nobody notices that this example is never checked. If it were checked, it would fail, because the expected error text in the vector does not match the one the Rust code now returns. The behaviour itself is checked by a separate Rust test, so the fix is not in doubt.

`legacy_v7_negative_returns_err` (kat_cross_check.rs:228-300) skips every vector whose `tampered_hex` is empty (lines 241-244). N009 in `tests/kat/v6_vectors.json` is exactly such a vector: it is the empty ciphertext, `"tampered_hex": ""`, so it is never exercised. The guard at lines 295-299 requires only `tested > 0`, and `corpus_sizes_are_pinned` (lines 64-80) counts the entries in the test-vector files instead of the exercised ones. The skipped vector is therefore invisible, and the harness reports a clean run.

If the skip were removed, N009 would fail: its `expected_exception` is "not a valid authenticated v7 payload", while the Rust code now returns "Ciphertext too short..." (lib.rs:1137-1143). N002 was given `expected_error_contains: "too short"` for exactly this reason, but N009 was not.

The skip already existed in the baseline code (baseline kat_cross_check.rs:197-200), so the fixes did not introduce it. It matters now because N009 is the evidence, shared by all implementations, for the fix of CVF-13. We rate this finding low because the behaviour itself is covered by the unit test `empty_ciphertext_is_rejected` (lib.rs:2432-2443).

Recommendation

Remove the skip, since an empty `tampered_hex` is the test vector itself and not a missing field. Add `expected_error_contains: "too short"` to N009, and assert the exact number of exercised negative vectors instead of `tested > 0`.

Apply the same assertion of the exact count to every harness test that has a path for skipping vectors.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-21 — Fixed in b54ae56. N009 is no longer skipped, and it gains `expected_error_contains: "too short"`. Every harness test in `kat_cross_check.rs` asserts an exact exercised count, and the remaining skip paths are hard failures. Rust, Python and C pass the regenerated corpus.

/rust/src/kat_cross_check.rs

```
#[test]
fn legacy_v7_negative_returns_err() {
    let vectors = load_vectors();
    let mut tested = 0;

    for vec in vectors
        .iter()
        .filter(|v| v["kind"] == "negative" && v["api"] != "stream_ae")
    {
        let id = vec["id"].as_str().unwrap();
       let tampered_hex = vec["tampered_hex"].as_str().unwrap_or("");
        let aad_hex = vec["aad_hex"].as_str().unwrap_or("");

        if tampered_hex.is_empty() {
            eprintln!("[SKIP] {} — no tampered_hex", id);
            continue;
        }

        let key: Vec<u64> = vec["key"]
            .as_array()
            .unwrap()
            .iter()
            .map(|v| v.as_u64().unwrap())
            .collect();

        let ct = hex_decode(tampered_hex);
        let aad = hex_decode(aad_hex);

        let result = decrypt_bytes(&ct, &key, &aad);
        match result {
           Ok(pt) => panic!("[{}] expected Err but got Ok({:?})", id, pt),
            Err(err_text) => {
                // CVF-25: if the corpus supplies an `expected_error_contains`
                // string, check that the error message text contains it —
                // this distinguishes rejection at the tag check from
               // rejection at a post-authentication structural check
                // (Dec step (3), (6), (8)). Vectors that need to reach the
                // structural checks (W-N06/W-N07/W-N08) rely on this
                // discriminator to prove they actually do.
```

```
                // Corpus emits `expected_exception` (a substring of the
                // Python ValueError message); the same words also appear in
                // the Rust error text, so we can enforce them cross-port.
                // `expected_error_contains` remains reserved for cases where
                // the two languages diverge.
                if let Some(want) = vec
                    .get("expected_error_contains")
                    .and_then(|v| v.as_str())
                    .or_else(|| vec.get("expected_exception").and_then(|v| v.as_str()))
                {
                    assert!(
                        err_text.contains(want),
                        "[{}] error message did not contain expected \
                         substring {:?}; got: {}",
                        id, want, err_text
                    );
                }
                eprintln!("[PASS] {} — correctly returned Err", id);
                tested += 1;
            }
        }
    }

    eprintln!("\nRust KAT negative: {} passed", tested);
    // CVF-19: a negative-KAT test that never submitted a tampered ciphertext
    // is a hard failure. The previous eprintln! was the auditor's headline
    // example of a test that could not report failure: "Not a hard failure"
    // is exactly the wrong disposition for a rejection-oracle test.
    assert!(
        tested > 0,
        "negative_returns_err: no tampered vectors submitted — the \
         rejection oracle was never exercised (CVF-19)"
    );
}
```

CVF-V5-22: open Category: **documentation**

Summary

Several comments written for the fixes no longer match the code, and one of them wrongly promises that the library never aborts on a bad key.

Several comments in the fixed code describe behaviour that the code does not have. The most important one sits at the top of the library and promises that no function given a key can stop the program; one older function still does exactly that, and a test in the library checks that it does. The other comments list the wrong places where a function is used, refer to a check that no longer exists, or justify a step by a format that has been removed. None of this changes how the code behaves, but a developer who relies on these comments would draw wrong conclusions about its safety.

The following comments are inaccurate:

- The crate header (`lib.rs:39-45`) says that every entry point taking a `&[u64]` key runs `validate_key` "as its first statement" and that "no panic path is reachable from `&[u64]` input". However, the deprecated `pub fn encrypt` panics by design on an over-length message (`assert!` at `lib.rs:745-749`) and on an invalid key (`validate_key(key).expect(...)` at `lib.rs:755`), and the crate's own test `cvf15_pub_encrypt_panics_on_invalid_key` (`#[should_panic]`) asserts that panic. In addition, `encrypt_stream_ae_v8` draws its nonce before validating the key, so the words "first statement" are not literally true either. The header before the fixes made no such claim.
- The inline comment in `encrypt` (`lib.rs:742-744`) says ".expect() surfaces the cap", but the cap is enforced with `assert!`. `encrypt` has no rustdoc, and none of its three panic conditions (length, key and CRNG failure) is documented.
- The `ct_eq_bytes` doc (`lib.rs:940-945`) says that the function is used at "the three tag verification sites" plus the CRNG check. It is now called from seven places, including the streaming tag checks and the self-test.
- `Cargo.toml:52-55` says that test builds keep `debug_assert!` active "so CVF-12's ct_eq_bytes length check fires". That check was removed by the fix for CVF-12, and `[profile.test]` does not set `debug-assertions` at all.
- The `varint_keystream` doc (`lib.rs:673-677`) still justifies the mask by the bias of the LEB128 continuation bit under NIST SP 800-22. The LEB128 format no longer exists, and CVF-33 asked for this justification to be replaced.

None of these comments changes the behaviour of the code, but each of them misleads a reader about the security-relevant contract of the code next to it.

Recommendation

Restate the header's guarantee together with its exception, i.e. that the deprecated `encrypt` panics, or give `encrypt` a `Result` signature and a rustdoc that lists its failure conditions.

Also correct the inline comment in `encrypt`, the list of places where `ct_eq_bytes` is called and the comment on the `Cargo.toml` profile. Rewrite the `varint_keystream` doc so that it states what the mask is for in the current format, or remove the reference to LEB128.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-22 — Fixed in 54c8544. The `ct_eq_bytes` call-site list, the `varint_keystream` justification (LEB128 removed) and the Cargo.toml profile comment are corrected. The crate header's "first statement / no panic path" claim was replaced in `e872c6e`. The `encrypt` comment went with the `Result` signature in `2086e00`.

/rust/Cargo.toml

```
52 # CVF-23: manifest hardening — release builds now trap on integer
   overflow
53 # (belt-and-braces backstop for CVF-16's checked_mul recommendation),
   and
54 # test builds keep debug_assert! live so CVF-12's ct_eq_bytes length
   check
55 # fires in `cargo test`.
```

/rust/src/lib.rs

```
39 //! # Validation
40 //!
41 //! Every entry point that accepts a `&[u64]` key runs
   [`validate_key`]
42 //! (private) as its first statement, per CVF-15. A key that is empty,
43 //! composite, below [`MIN_KEY_PRIME`], above [`MAX_KEY_PRIME`], or
   contains
44 //! duplicates is rejected with a descriptive `String` error before
   any
45 //! HMAC or arithmetic runs — no panic path is reachable from
   `&[u64]` input.
```

same file

```
673 /// HMAC-CTR keystream for masking the varint blob — domain byte 0x07.
674 ///
675 /// Each 32-byte block is `HMAC(key_bytes, 0x07 || nonce ||
   uint32_be(block))`.
676 /// XOR-masking the raw LEB128 blob eliminates the 3:1 MSB
   continuation-bit
677 /// bias that otherwise causes systematic NIST SP 800-22 failures.
```

same file

```
742     // CVF-34: uphold the length cap even on this Vec<u64>-returning
   legacy
743     // path. The .expect() surfaces the cap consistently with the
   deprecation
```

```
744     // attribute and matches the pattern of the CVF-15 validate_key
    call.
```

same file

```
940 /// Used at the three tag verification sites (v7 `decrypt_bytes`,
941 /// `decrypt_raw`, v8 `decrypt_bytes_v8`) on `[u8; TAG_SIZE]`
    operands, and
942 /// once inside `generate_nonce_with_crng_check` on `[u8;
    NONCE_SIZE]`
943 /// operands (CVF-35 flags that the latter compares public values — no
944 /// constant-time requirement — but the comparator's uniform length
    handling
945 /// makes it safe there too).
```

CVF-V5-23: open Category: **procedural**

> **Summary**
>
> **The revised specification has no version number and keeps an older file name, so a reader cannot tell which revision a citation or an implementation refers to.**
>
> The specification was revised together with the fixes, but the new text has the same title, the same date and the same file name as two earlier revisions, and it contains no version number. A reader holding a copy therefore cannot tell whether it is the current one. This matters because the revision changed a rule that every implementation must follow, so a developer working from an older copy would build software that does not match the current one. Two references inside the code also point to the wrong document.

The revised paper (2,647 lines) is not the earlier file of the same name (1,258 lines), and it differs from the specification of the earlier review in 57 places. Nevertheless, all three of these sources have the same title and `\date{July 2026}` (`napseq-eprint-v3.tex:48`), and no version string appears anywhere in the file. Only text outside the paper calls it "NAPQES v4": `lib.rs:11-13` and `README.md:161-162` say that "the filename lags the version tag one revision", but there is no version tag for the filename to lag behind. The client's report on the fixes lists the rename as still pending.

This matters more now than before, because the fixes changed a normative parameter, namely the prime interval and the default `K`. Implementations written against the text of the earlier review must therefore be able to tell that the text they hold is out of date.

Two pointers inside the repository are also out of date. `self_test.rs:143` cites "rem:domsep" in `docs/napseq-eprint-preprint.tex`, which is the May 2026 preprint describing the v7 layout, while the rest of `rust/` cites the current text under the

v3 name. Finally, the delivered `docs/napseq-eprint-v3.pdf` matches the tex source but has 32 pages, not the 31 that the client states.

Recommendation

Give the paper an explicit version identifier, in the title block or in `\date`, and a revision history that names the normative changes of each revision, including the change of the interval.

Rename the file to match that version, and update every pointer in `rust/` at the same time. Also make `self_test.rs:143` point to the remark in the current paper.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-23 — Fixed in 4c96bd8. The paper states "Specification revision 4.2 — October 2026" and has a revision history that names the normative changes of 4.0, 4.1 (`acbf9d0`: the interval and K, the Dec step 7 guards) and 4.2. It is renamed `docs/napseq-eprint-v4.tex` (the PDF is rebuilt), and every pointer outside the historical review documents is updated. `self_test.rs` now cites `lem:domsep` of the current paper.

/docs/napseq-eprint-v3.tex

```
48 \date{July 2026}
```

/rust/README.md

```
161 * Paper: EPINeon NAPQES v4 (`docs/napseq-eprint-v3.tex`; will be
renamed
162   to `napseq-eprint-v4.tex` in the next revision).
```

/rust/src/lib.rs

```
11 //! Specification: EPINeon NAPQES v4 (see `docs/napseq-eprint-v3.tex`
— the
12 //! filename lags the version tag one revision and will be renamed to
13 //! `napseq-eprint-v4.tex` in a follow-up).
```

/rust/src/self_test.rs

```
143 // introduced by the CVF2 fix (see SPEC.md §3 and
docs/napseq-eprint-preprint.tex
```

CVF-V5-24: open Category: **documentation**

Summary

The edits made to the specification for the fixes are incomplete in several places, and some of the new sentences are inaccurate.

When the specification was revised, the change of the key that the scheme uses internally was not carried through to every place that mentions it, including part of a security proof. Some of the newly added sentences are also inaccurate: one calls an event with a probability of about one in a million negligible, one cites an audit finding under an identifier that does not exist, and one says that 15 statistical tests were run when there were 14. None of these points weakens the scheme, but together they make the specification less reliable as the reference for implementations and for the security argument.

The following places in the specification need attention:

- **Outdated sk wording (CVF-11).** The format subkey `sk_fmt` is now the key of every per-message HMAC, but several places still describe `sk` in that role:
  - The §4 preamble (`napseq-eprint-v3.tex:1115-1125`) still describes a single external oracle, i.e. a function that the adversary may query, "keyed by the challenger's sk", and says that NAPQES has no MAC key "separate from sk".
  - The proof of Proposition 3.14 (lines 1044-1077) argues from "the same sk" and "a function of sk and N". It also does not cover the new rejection branches of `Dec` steps (7) and (8a).
  - Lemma nonce-collision (lines 1269-1283) is stated "under the same sk", but its game hop is made against `HMAC(sk_fmt, ·)` without a premise that `sk_fmt` is uniform. The lemma is used only inside game G_1, so no bound fails because of this.
  - `Enc` has a step (0) that derives `sk_fmt` (lines 964-978), while `Dec` has none, and `Dec` step (7) defines the addend `a` as the raw output of `Derive_0x01` (lines 1002-1004) instead of the reduced addend.
- **Remark key-roles (lines 384-390, CVF-36).** The new sentence says that the tag is "keyed by sk alone", while it is in fact keyed by `sk_fmt`. It also says that a mismatched tuple yields a wrong plaintext only "with that negligible probability". However, a real token under a differing element `k'` passes the checks of step (7) with probability about $1/k' \in [2^{-23.8}, 2^{-19.9}]$, which is small but not negligible.
- **The remark at line 548** is titled "Audit finding CV-11", and there is no finding with that identifier. Its text (lines 562-563 onward) also asserts that the theorems and tables "have all been restated", which the outdated wording listed above contradicts.
- **The noise thresholds (lines 614-615)** are written as $\lfloor 0.99 \times 2^{64} \rfloor$ without saying whether the product is computed exactly. The exact value and the IEEE-754 value differ by 163, and the known-answer test vectors do not distinguish between them. In addition, the CVF-30 footnote next to them points to an entry in `docs/CAVEATS.md` that does not exist.

- **§9 (lines 2272-2273)** says "All 15 tests, 40 scored sub-results, passed". However, `sts_report.json` contains 14 distinct tests, because the Overlapping Template Matching test is not implemented (see CVF-33).

Recommendation

Make one pass over the paper and replace `sk` by `sk_fmt` wherever the key of an HMAC is meant.

In the same pass, state Lemma nonce-collision for a uniform `sk_fmt` (or under G_0'), add a step (0) to `Dec`, and extend the proof of Proposition 3.14 to the new rejection branches.

Also make the following smaller corrections:

- In Remark key-roles, replace "negligible" with the probability per token.
- Correct "CV-11" and the claim that everything has "all been restated".
- State that $\lfloor 0.99 \times 2^{64} \rfloor$ is computed exactly, and give the resulting integer.
- Fix or remove the pointer to `docs/CAVEATS.md`.
- Either say 14 tests in §9 or implement the missing test.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-24 — Fixed in 24c26bf.

- The security preamble, the correctness proof (which now also discharges the step 7 and step 8 rejection branches) and Lemma nonce-collision (for a uniform `sk_fmt`) are stated in terms of `sk_fmt`; Dec gains step (0).
- Remark key-roles gives the per-token probability (about `1/k'`, between `2^-23.8` and `2^-19.9`) instead of "negligible".
- "CV-11" is corrected to CVF-11, and the restatement claim is corrected.
- `theta_min` and `theta_max` are given as exact integers, with a warning about the IEEE-754 value, which differs by 163.
- The CVF-30 footnote points to existing documentation.
- §9 says 14 tests.

/docs/napseq-eprint-v3.tex

```
384 agree on $\mathbf{k}$ itself. A receiver configured with a different
    tuple
385 (including a different $K$) still verifies the tag, which is keyed by
386 $\mathsf{sk}$ alone, and then fails the structural checks of
387 $\mathsf{Dec}$ step (7) with overwhelming probability as soon as a
    real
388 token uses a differing element; a mismatch therefore surfaces as a
389 decryption error, never (except with that negligible probability) as a
390 wrong plaintext.\footnote{CVF-36: an earlier revision of this
```

same file

```
548 \begin{remark}[Audit finding CV-11: undocumented $\texttt{0x0B}$
```

same file

```
implementation written from the earlier text produces \emph{none} of
the
twelve positive vectors. The defect was in the specification, not the
```

same file

```
  \theta_{\min} = \lfloor 0.75 \times 2^{64} \rfloor, \qquad
  \theta_{\max} = \lfloor 0.99 \times 2^{64} \rfloor,
```

same file

```
\paragraph{$\mathsf{Enc}(\mathbf{k}, \mathsf{sk}, A, M) \to C$.}
(0) derive $\mathsf{sk}_{\mathrm{fmt}}$
(Definition~\ref{def:fmtkey}; it may be cached per key install); (1)
compute $N=N(\mathsf{sk}_{\mathrm{fmt}},A,M)$
(Definition~\ref{def:synthnonce}); (2) pad $M$ to $P_d$
(Section~\ref{sec:padding}); (3) run the token-emission loop of
Section~\ref{sec:token-construction} over $P_d$ using
$(\mathsf{sk}_{\mathrm{fmt}},N)$ to obtain the token vector
$(\mathrm{token}[0],\ldots,\mathrm{token}[N_{\mathrm{tok}}-1])$; (4)
serialise $B$; (5) mask $\widetilde{B}=B\oplus\mathrm{ks}(N)$; (6)
compute
$T=\mathrm{Derive}_{0x03}(\mathsf{sk}_{\mathrm{fmt}},N,\mathrm{TagIn}(A,\widetil
(Definition~\ref{def:taginput});
(7) output $C=N\|\widetilde{B}\|T$. The public encryption oracle is
$\mathsf{Encrypt}(\mathbf{k},\mathsf{sk},A,M)\triangleq\mathsf{Enc}(\mathbf{k},\
— a \emph{pure function} of $(A,M)$, since $N$ is computed
synthetically
rather than sampled.
```

same file

```
implementations perform), where $a$ is the
$\mathrm{real\_idx}$-th addend
$\mathrm{Derive}_{0x01}(\mathsf{sk}_{\mathrm{fmt}},N,\mathrm{real\_idx})$
and
```

same file

```
\begin{proof}
Let $C=N\|\widetilde{B}\|T$ be the output of
$\mathsf{Enc}(\mathbf{k},\mathsf{sk},A,M)$. We first check that the
two tag
inputs coincide, rather than assuming tag agreement.

By Lemma~\ref{lem:length-det},
$|C|=16+8R(\mathrm{MAX\_NOISE\_RUN}+1)+32$
with $R=B(|M|)+2\ge1$, so $|C|\ge2928>48$ and
$N_{\mathrm{tok}}=(|C|-48)/8=R(\mathrm{MAX\_NOISE\_RUN}+1)$ is a
non-negative integer divisible by $\mathrm{MAX\_NOISE\_RUN}+1$: steps
(1)
```

```
and (3) of $\mathsf{Dec}$ do not reject, and step (3) recovers exactly the
$R$ used by $\mathsf{Enc}$. Step (2) parses the leading 16 bytes as $N$ and
the trailing 32 as $T'=T$, leaving the middle bytes as exactly the
$\widetilde{B}$ that $\mathsf{Enc}$ emitted; in particular $N$ is
\emph{read from the ciphertext}, not re-derived, so no circularity with the
synthetic-nonce derivation arises. Hence $\mathsf{Dec}$ step (4) evaluates
$\mathrm{Derive}_{0x03}$ on $\mathrm{TagIn}(A,\widetilde{B})$ with the same
$\mathsf{sk}$, the same $N$, the same caller-supplied $A$, and the same
$\widetilde{B}$ as $\mathsf{Enc}$ step (6) did
(Definition~\ref{def:taginput}). The two tag inputs are byte-identical, so
$T=T'$ and step (5) does not reject. Since $R=B(|M|)+2$ with
$B(|M|)\in\{2^4,\ldots,2^{16}\}$, step (6) does not reject either, and since
the length prefix written by $\mathsf{Enc}$ satisfies $n=|M|\le B(|M|)=R-2$,
neither does step (8).

Step (7) then unmasks with $\mathrm{ks}(N)$, the same keystream
$\mathsf{Enc}$ masked with (a function of $\mathsf{sk}$ and the recovered
$N$ alone), recovering $B$ and hence the token vector. $\mathsf{Enc}$'s
token-emission loop and $\mathsf{Dec}$'s loop-inversion step both compute
the noise-position oracle, addends, and padding codepoints as pure,
deterministic functions of $(\mathsf{sk},N,\mathrm{position})$, so with the
same $\mathsf{sk}$ and $N$ they compute identical intermediate values at
every position; emission and inversion are therefore exact inverses, and
stripping the length prefix and padding yields $M$.
\end{proof}
```

same file

```
Every theorem in this section is unconditional under the standard,
uniform-key PRF assumption on HMAC-SHA256 stated below: because
$\mathsf{sk}$ is sampled uniformly and independently of $\mathbf{k}$, every
```

```
reduction constructed in this section may sample $\mathbf{k}$ on its
own
(to run the arithmetic layer) while forwarding every HMAC call to an
external oracle keyed by the challenger's $\mathsf{sk}$ — the
reduction
never needs to extract $\mathsf{sk}$ from $\mathbf{k}$, or vice
versa, so
no simulation gap and no non-standard key-distribution assumption
arises.
NAPQES does not define a standalone MAC verification key separate from
$\mathsf{sk}$, so EUF-CMA unforgeability of the tag and INT-CTXT
unforgeability of the ciphertext coincide here.
```

same file

```
\begin{lemma}[Nonce collision]\label{lem:nonce-collision}
For any two distinct queried pairs $(A_1,M_1)\neq(A_2,M_2)$ under the
same
$\mathsf{sk}$, there is a PRF adversary $\mathcal{B}$ with similar
running
time such that
$\Pr[N(\mathsf{sk}_{\mathrm{fmt}},A_1,M_1)=N(\mathsf{sk}_{\mathrm{fmt}},A_2,M_2)
\le \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC\text{-
}SHA256}}(\mathcal{B}) +
2^{-128}$.
\end{lemma}

\begin{proof}
By Lemma~\ref{lem:domsep}, $(A_1,M_1)\neq(A_2,M_2)$ implies the two
domain
$\texttt{0x0A}$ inputs are distinct, so replacing
$\mathrm{HMAC}(\mathsf{sk}_{\mathrm{fmt}},\cdot)$
by a truly random function $\rho$ (a PRF hop of advantage
$\mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC\text{-
}SHA256}}(\mathcal{B})$)
makes the two 128-bit truncated outputs independent and uniform,
colliding
with probability exactly $2^{-128}$.
\end{proof}
```

same file

```
suite correctly rejects the repetition). All 15 tests, 40 scored
sub-results, passed at the $\alpha=0.01$ significance level; the
smallest
```

CVF-V5-25: open Category: **behavior**

Summary

The legacy functions still do not record whether the data was encrypted as text or as raw bytes, so a message encrypted one way decrypts without error but to different content the other way.

The older (v7) interface can encrypt data either as text or as a sequence of raw bytes. Nothing in the encrypted message records which of the two was used, so a message encrypted as text and decrypted as bytes, or the other way round, is accepted without any error but gives content different from what was sent. The fixes removed the worst cases, where data was silently corrupted, but the mix-up remains for characters and byte values up to 255, such as accented Latin letters. The impact is limited because the older interface is deprecated.

This is the part of CVF-14 that the fixes did not resolve.

The fixes resolved the following. `decrypt_raw` no longer masks recovered values to their low byte, and it rejects anything above `0xFF` (`lib.rs:1295-1313`). `decrypt_bytes` maps codepoints with an error path instead of dropping invalid ones (`lib.rs:1167-1179`). `decrypt_core` checks the token shape, the `0x10FFFF` bound and the validity of each scalar value. The tests `cvf14_...rejected_by_decrypt_raw_for_non_ascii` and `cvf14_decrypt_returns_result_not_panic` fail on the code before the fixes.

The main recommendation of CVF-14, i.e. making one encoding normative or separating the two conventions in the tag, was deferred by the client (the client's report on the fixes, §8.4). Both conventions still share one key and one tag construction, so the confusion remains for every value in `[0, 0xFF]`:

- `encrypt_bytes("é")` followed by `decrypt_raw` (`lib.rs:1263-1314`) returns `[0xE9]` with no error;
- `encrypt_raw([0xC3, 0xA9])` followed by `decrypt_bytes` (`lib.rs:1134-1180`) returns "Ã©".

We rate this finding low because the v7 functions are deprecated and out-of-range values are now rejected instead of being silently corrupted.

Recommendation

Bind the plaintext convention into the v7 tag input, for example with a convention byte in the authenticated data.

Alternatively, withdraw `encrypt_raw` and `decrypt_raw`, and document that `decrypt_raw` decodes only ciphertexts produced by `encrypt_raw`. At a minimum, state in the rustdoc of both functions that decryption under the other convention succeeds without an error for values up to U+00FF.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-25 — Mitigated in `6fdbec3` (with `23025ed`). `encrypt_raw` (and every v7 encryptor) is withdrawn behind `legacy-v7`. All four v7 rustdocs state that the tag

does not bind the convention, that cross-convention decryption succeeds silently for values up to U+00FF (with the two concrete examples), and that `decrypt_raw` is only for `encrypt_raw` output. A test pins this behaviour. The convention is not bound into the v7 tag, because that would break the archived format.

/rust/src/lib.rs

```
pub fn decrypt_bytes(ciphertext: &[u8], key: &[u64], aad: &[u8]) -> Result<String, String> {
    validate_key(key)?; // CVF-15
    check_v7_aad_len(aad.len())?;
    if ciphertext.len() < NONCE_SIZE + TAG_SIZE {
        return Err(format!(
            "Ciphertext too short: {} bytes; header+tag require at least {}.",
            ciphertext.len(),
            NONCE_SIZE + TAG_SIZE
        ));
    }
    let kb = key_bytes(key);
    let split = ciphertext.len() - TAG_SIZE;
    let payload = &ciphertext[..split];
    let recv_tag = &ciphertext[split..];
    let calc_tag = compute_auth_tag(&kb, aad, payload, AAD_LEN_WIDTH_V7);
    if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
        return Err("Authentication failed: invalid HMAC tag.".into());
    }
    let nonce = &payload[..NONCE_SIZE];
    let masked = &payload[NONCE_SIZE..];
    let ks = varint_keystream(&kb, nonce, masked.len());
    let blob: Vec<u8> = masked.iter().zip(ks.iter()).map(|(a, b)| a ^ b).collect();
    let tokens = fixed_decode_tokens(&blob)
        .map_err(|e| format!("varint decode error: {}", e))?;
    let codepoints = decrypt(nonce, &tokens, key)?;
    // CVF-14: the previous body was `filter_map(char::from_u32)`, which
    // silently dropped every recovered value that was not a Unicode scalar
    // (surrogates, > 0x10FFFF). That turned a malformed ciphertext into a
    // shorter-but-syntactically-valid String rather than into an error, so a
    // ciphertext produced under the byte-oriented convention (`encrypt_raw`)
```

```
    // could decrypt to a truncated codepoint-oriented string with no signal.
    // Match `decrypt_bytes_v8`'s fallible mapping: a non-scalar value is an
    // error.
    codepoints
        .into_iter()
        .enumerate()
        .map(|(i, c)| {
            char::from_u32(c).ok_or_else(|| {
                format!(
                    "Malformed v7 plaintext: recovered value {} at position {} \
                     is not a Unicode scalar value.",
                    c, i
                )
            })
        })
        .collect::<Result<String, String>>()
}
```

same file

```
pub fn decrypt_raw(ciphertext: &[u8], key: &[u64], aad: &[u8]) -> Result<Vec<u8>, String> {
    validate_key(key)?; // CVF-15
    check_v7_aad_len(aad.len())?;
    if ciphertext.len() < NONCE_SIZE + TAG_SIZE {
        return Err(format!(
            "Ciphertext too short: {} bytes; header+tag require at least {}.",
            ciphertext.len(),
            NONCE_SIZE + TAG_SIZE
        ));
    }
    let kb = key_bytes(key);
    let split = ciphertext.len() - TAG_SIZE;
    let payload = &ciphertext[..split];
    let recv_tag = &ciphertext[split..];
    let calc_tag = compute_auth_tag(&kb, aad, payload, AAD_LEN_WIDTH_V7);
    if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
        return Err("Authentication failed: invalid HMAC tag.".into());
    }
    let nonce = &payload[..NONCE_SIZE];
    let masked = &payload[NONCE_SIZE..];
    let ks = varint_keystream(&kb, nonce, masked.len());
```

```
    let blob: Vec<u8> = masked.iter().zip(ks.iter()).map(|(a, b)| a ^ b).collect();
    let tokens = fixed_decode_tokens(&blob)
        .map_err(|e| format!("varint decode error: {}", e))?;
    let codepoints = decrypt(nonce, &tokens, key)?;
    // CVF-14: the previous body was `.map(|c| (c & 0xFF) as u8)` — a silent
    // narrowing that discarded every recovered codepoint above 255 without
    // signalling. A ciphertext produced by `encrypt_bytes` (codepoint-oriented,
    // e.g. `é` as one U+00E9 token) handed to `decrypt_raw` would authenticate
    // and return a self-consistent-looking single byte, hiding the fact that
    // the two ports use incompatible message-space encodings. We now reject
    // any recovered value that does not fit in a byte.
    codepoints
        .into_iter()
        .enumerate()
        .map(|(i, c)| {
            if c > 0xFF {
                Err(format!(
                    "Malformed v7 raw ciphertext: recovered codepoint {} at \
                     position {} is out of byte range [0, 255]. This usually \
                     means the ciphertext was produced by `encrypt_bytes` \
                     (codepoint-oriented) rather than `encrypt_raw` \
                    (byte-oriented); the two use incompatible message \
                     encodings (CVF-14).",
                    c, i
                ))
            } else {
                Ok(c as u8)
            }
        })
        .collect()
}
```

CVF-V5-26: open Category: **procedural**

Summary

The fuzz tests still cannot reach deep into the decoder that handles unauthenticated input, do not cover the new streaming decryptor, and have never been recorded as run.

Fuzzing, i.e. automated testing with large numbers of random or mutated inputs, is the usual way to find crashes in code that parses untrusted encrypted messages. The fixes improved the fuzzing set-up, but one of the new fuzz tests almost never gets past the first check of the code that it is meant to test, and the new streaming decryption code has no fuzz test at all. There is also no saved set of starting inputs and no record that any fuzzing has been run, although the specification refers to fuzzing. The claim that the parsing code has been fuzzed is therefore not yet supported by evidence.

This is the part of CVF-22 that the fixes did not resolve.

The fixes resolved the following. `fuzz/fuzz_targets/decode_bytes.rs` (lines 68-124) now targets v8 and recomputes the tag of each mutated input through `fuzz_retag_block_v8`, which is compiled only under the `fuzzing` feature (`fuzz/Cargo.toml:13-15`), so the target reaches the decoding paths that run after authentication. A `validate_key` target with malformed key fixtures was also added, and it is adequate.

The following problems remain:

- The new `unauth_decode.rs` target (lines 54-67) feeds random token vectors to the decoder. However, each real token must have the form `c*k + a`, and a random input meets this condition with probability about $1/k$. The target therefore almost never passes the first real-token check, which is the same structural limit that CVF-22 described.
- The targets use only the Bucket profile, with `B <= 256`, and never mutate the nonce.
- There is no fuzz target for `decrypt_stream_ae_v8`, which is the largest new parser in the fixed code.
- No set of seed inputs is committed and no run is recorded: `docs/FUZZ_ATTESTATION.md` is unchanged and still describes the v6 target, which the client acknowledges in §8.4 of its report on the fixes. CI also does not build the fuzz crate.
- The paper's sentence on Python fuzzing (`napseq-eprint-v3.tex` lines 2358-2361) is still not supported by any evidence.

Recommendation

Make `unauth_decode` construct structurally valid tokens, so that it reaches the later checks: choose `c` and compute `c*k + a` from the fuzzer's bytes, and then perturb the result.

Also add a target for the streaming decryptor that recomputes the tags of the chunks and of the sentinel, and cover all padding profiles and mutation of the nonce. Com-

mit a set of seed inputs, build the fuzz crate in CI, and record a run in `docs/FUZZ_ATTESTATION.md` before the paper cites it.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-26 — Fixed in `471501e` (code) and `da5da58` (recorded run).

- `unauth_decode` builds structurally valid `c*k+a` token streams and mutates them.
- A new `stream_decode` target re-tags every chunk and the sentinel.
- `decode_bytes` covers all three padding profiles and nonce mutation.
- Seed corpora are committed, and CI builds the fuzz crate.

`docs/FUZZ_ATTESTATION.md` records a 600 s ASan run of each of the four targets at `471501e`: 5,946 to 378,097 executions per target and no crashes. It also states the limits of that evidence. The paper no longer claims the Python harness covers v8.

/rust/fuzz/Cargo.toml

```
[dependencies.napqes]
path = ".."
features = ["fuzzing"]
```

/rust/fuzz/fuzz_targets/decode_bytes.rs

```
fuzz_target!(|data: &[u8]| {
    // Header: [key_sel, aad_sel, sk_sel, flags, mutate_index_lo,
    //          mutate_index_hi, mutate_xor_byte, msg_len, ..msg..]
    // flags bit0: flip a byte; bits1..3: drop that many token groups.
    if data.len() < 8 {
        return;
    }
    let key = key_for(data[0]);
    // Sanity: keys start above MIN_KEY_PRIME by construction.
    if key.iter().any(|&k| k < MIN_KEY_PRIME) {
        return;
    }
    let aad = aad_for(data[1]);
    let sk = sk_for(data[2]);
    let flip = data[3] & 1 == 1;
    let drop_groups = ((data[3] >> 1) & 0x07) as usize;
    let mut_lo = data[4] as usize;
    let mut_hi = data[5] as usize;
    let mut_xor = data[6];
    let msg_len = (data[7] as usize).min(data.len() - 8);
    let msg_bytes = &data[8..8 + msg_len];

    // Restrict to valid UTF-8 slices — v8 encrypts &str. Fall back to a
    // trivial ASCII message on invalid UTF-8 to keep coverage on the
```

```
    // encrypt/decrypt path.
    let message: &str = match std::str::from_utf8(msg_bytes) {
        Ok(s) => s,
        Err(_) => "A",
    };

    // Encrypt once — the fuzzer proves this cannot panic on any
    // validate_key-accepting key with any UTF-8 message.
    let ct = match encrypt_bytes_v8(message, &key, &sk, aad) {
        Ok(ct) => ct,
        Err(_) => return, // e.g. message too long — accepted refusal
    };

    // Unmutated round-trip: must succeed and reach every branch of
the
    // decrypt pipeline. Any panic here is a bug.
    let rt = decrypt_bytes_v8(&ct, &key, &sk, aad).expect("v8 round
trip failed");
    assert_eq!(rt, message);

    if !flip && drop_groups == 0 {
        return;
    }
    let mut payload = ct[..ct.len() - TAG_SIZE].to_vec();
    let masked_len = payload.len() - NONCE_SIZE;
    let drop = (drop_groups * TOKEN_GROUP).min(masked_len);
    payload.truncate(payload.len() - drop);
    if flip && payload.len() > NONCE_SIZE {
        let idx = NONCE_SIZE + ((mut_hi << 8 | mut_lo) %
(payload.len() - NONCE_SIZE));
        payload[idx] ^= mut_xor;
    }
    let tag = fuzz_retag_block_v8(&payload, &sk, aad);
    payload.extend_from_slice(&tag);
    let _ = decrypt_bytes_v8(&payload, &key, &sk, aad);
});
```

/rust/fuzz/fuzz_targets/unauth_decode.rs

```
    // Target 1: pub fn decrypt — unauthenticated token slice + nonce.
    // Result<Vec<u32>, String>; must never panic on any tokens, any
key.
    let _ = decrypt(nonce, &tokens, &key);

    // Target 2: decrypt_raw — tag-first (safer surface), fed
arbitrary
    // bytes. Should always return Err from the parse floor or the tag
```

```
60      // check; must never panic.
61      let _ = decrypt_raw(data, &key, b"");
62
63      // Target 3: decrypt_str — base64-decode then decrypt_bytes.
Should
64      // report base64 or auth failure, never panic.
65      if let Ok(s) = std::str::from_utf8(data) {
66          let _ = decrypt_str(s, &key, b"");
67      }
```

CVF-V5-27: open Category: **procedural**

> **Summary**
> **The only evidence that the Rust implementation compares tags in constant time is still one weak timing test of the whole decryption call, run on a commit that was not delivered and not repeated in CI.**
> To stop an attacker from learning secret values by measuring how long operations take, the code that checks the authentication tag of a message must take the same time whatever the input. The only evidence offered for this is a single timing experiment on the whole decryption function, in which the comparison itself accounts for a tiny fraction of the measured time, so the experiment could easily miss a leak. The experiment was also run on a different version of the code from the one delivered, and nothing repeats it automatically when the code changes. The result does not show a leak, but it is too weak to support the claim that there is none.

This is the part of CVF-26 that the fixes did not resolve.

The fixes resolved the following: §8.4 of the paper (`napseq-eprint-v3.tex` lines 2252-2259) now cites the delivered Rust harness and its attestation, instead of a C study that was not delivered.

The following problems remain:

- `examples/dudect_harness.rs` (lines 29-69) runs dudect, i.e. a statistical test that compares running times for two classes of inputs, on the entire `decrypt_bytes_v8_key` call, with the first tag byte flipped in one class and the last tag byte flipped in the other. The comparator is therefore measured only inside an HMAC over the whole payload. The attestation itself puts the signal at about 0.1% of the measured window, so the pass (one run, $|t| = 1.113$ over 29.7 M samples) is low-power evidence, i.e. a test that would be unlikely to detect a leak of that size.
- The key is random and is not fixed between runs (`NapqesKey::generate()`), and the `_key` path skips `validate_key`, which is the one operation that the paper lists as not constant-time.

- The attestation names commit `6db914e` plus uncommitted changes in the working tree, and not the delivered commit `acbf9d0`.
- The harness for the comparator alone is deferred (the client's report on the fixes, §6 item 3), and there is no CI gate.

Recommendation

Add a harness that times `ct_eq_bytes` alone, on equal-length inputs that differ in the first byte in one class and in the last byte in the other.

Run it on the delivered commit, record the result in the attestation, and add the run to CI with a fixed |t| threshold. Also state in the paper that the measurement of the whole call is low-power evidence.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-27 — Fixed in e3ea69d, 9a2cbec (harness) and fecb8dd (recorded run). A comparator-only bench times `ct_eq_bytes` on operands that differ in the first versus the last byte, with a pinned key: |t| = 0.785 over 7.6M samples. A positive control (an early-exit comparator) reaches |t| = 4,610, which shows the test has the power to detect the leak. The whole-call bench gives |t| = 1.96 over 13.0M samples, and the paper now calls it low-power. All runs are on the delivered commits, and CI gates the comparator bench at |t| < 4.5.

/rust/examples/dudect_harness.rs

```
29 fn fixed_key() -> &'static NapqesKey {
30     static KEY: OnceLock<NapqesKey> = OnceLock::new();
31     KEY.get_or_init(|| {
32         napqes::self_test::run_power_on_self_tests().expect("dudect:
POST failed");
33         NapqesKey::generate().expect("dudect: NapqesKey::generate
failed")
34     })
35 }
36
37 /// One valid v8 ciphertext produced once at startup. Both classes
derive
38 /// from this by cloning + corrupting one tag byte in-place; allocator
39 /// typically reuses the same slot, giving both classes similar cache
state.
40 fn base_ct() -> &'static Vec<u8> {
41     static CT: OnceLock<Vec<u8>> = OnceLock::new();
42     CT.get_or_init(|| {
43         encrypt_bytes_v8_key(
44             "napqes v8 dudect constant-time timing harness (CVF-26
retarget)",
45             fixed_key(),
46             b"",
47         )
```

```
        .expect("dudect: v8 encrypt failed")
    })
}

fn bench_tag_comparison(runner: &mut CtRunner, rng: &mut BenchRng) {
    let key = fixed_key();
    let base = base_ct();
    let n = base.len();

    let mut ct = base.clone();

    // Left: tag byte 0 wrong  (a hypothetical early-exit comparator would
    // fail immediately). Right: tag byte 31 wrong (would scan all 31 bytes).
    // With subtle::ConstantTimeEq the two must be timing-indistinguishable.
    if rng.gen::<bool>() {
        ct[n - TAG_SIZE] ^= 0xFF;
        runner.run_one(Class::Left, || decrypt_bytes_v8_key(&ct, key, b""));
    } else {
        ct[n - 1] ^= 0xFF;
        runner.run_one(Class::Right, || decrypt_bytes_v8_key(&ct, key, b""));
    }
}
```

CVF-V5-28: open Category: **algorithm**

Summary

The decryption algorithm in the specification now handles every input, but the remark that explains why still gives an argument that is false, and the new text uses symbols before defining them.

The decryption algorithm in the specification must say what happens for every possible input, including malformed ones. The fixes added the missing checks, so the algorithm itself is now complete, and the code was already correct before. However, the explanation next to the algorithm still gives the earlier reason for its completeness, which we showed to be wrong, and the new text uses some values before saying what they are. These problems affect only the specification, but they could mislead someone who writes a new implementation from it.

This is the part of CVF-27 that the fixes did not resolve. The code was already correct before the fixes and is unchanged: the checks are in `decrypt_core_v8` after the tag verification, and the negative test vectors W-N09, W-N10 and W-N12 were added.

The fixes resolved the following: step (7) now has the `t < a` and divisibility guards and the scalar-value check (`napseq-eprint-v3.tex` lines 992-1012), so `Dec` as specified is total, i.e. it returns either a plaintext or an error for every input.

The following problems remain, all of them in the paper:

- Remark 3.13 (lines 1015-1034) still says that steps (6) and (8) make step (7) total, which is the claim that CVF-27 showed to be false. Its map from checks to test vectors also omits W-N12.
- The sentence saying that the scheme is "indifferent to which subset of [0, 2^21)" is kept (lines 913-915), although the scalar check now makes the choice of subset matter.
- Step (7) uses `a` and `k` before defining them. It also defines `a` as the raw output of `Derive_0x01` instead of the reduced addend, so `t < a`, read literally, compares a 64-bit token with a 256-bit digest.
- The "What is not" list in §8.4 (lines 2242-2246) still names only steps (6) and (8).

Recommendation

Rewrite Remark 3.13 to say that totality comes from the explicit guards in step (7), and add W-N12 to its map of test vectors.

Also delete the sentence about the scheme being "indifferent to which subset". In step (7), define `k` and the reduced addend before the guards use them, and update §8.4 so that it lists the checks of step (7).

Client Comment

Client statement, not verified by ABDK.

CVF-V5-28 — Fixed in db499d4. Dec step (7) binds `k` and the reduced addend before the guards, and names `c` before the scalar check. Remark `rem:dec-structural`

now says totality comes from each step's guards (the false "steps (6) and (8) make step (7) total" is withdrawn), and its vector map adds W-N12. The "indifferent to which subset" sentence is deleted, and §8.4 lists the step (7) checks.

/docs/napseq-eprint-v3.tex

```
913         weaker is needed and nothing stronger is assumed: the
arithmetic
914         layer itself operates on the integer codepoint values and is
915         indifferent to which subset of $[0,2^{21})$ they range over.
```

same file

```
992 (7) else unmask $B=\widetilde{B}\oplus\mathrm{ks}(N)$, deserialise
    the
993 $N_{\mathrm{tok}}$ fixed-width tokens, and invert the token-emission
    loop
994 using the same $(\mathsf{sk}_{\mathrm{fmt}},N)$-derived
    noise-position oracle: skip at most
995 $\mathrm{MAX\_NOISE\_RUN}$ consecutive noise positions, then read one
    real
996 token $t$; if $t<a$ or $(t-a)\not\equiv 0\pmod{k}$ output $\bot$
997 (CVF-27: totality preconditions on the recovery); otherwise recover
    the
998 padded codepoint as
999 $(t-a)/k_{(\mathrm{real\_idx}\bmod K)+1}$; if the recovered value
    exceeds
1000 $\mathtt{0x10FFFF}$ or is a Unicode surrogate output $\bot$
1001 (also CVF-27, matching the codepoint-range check the reference
1002 implementations perform), where $a$ is the
1003 $\mathrm{real\_idx}$-th addend
1004 $\mathrm{Derive}_{0x01}(\mathsf{sk}_{\mathrm{fmt}},N,\mathrm{real\_idx})$
    and
1005 $\mathrm{real\_idx}$ counts real tokens from $0$ — exactly the prime
    and
1006 addend $\mathsf{Enc}$ used at that index; stop once $R$ real tokens
    have
1007 been extracted, discarding any trailing filler; (8) read the
     2-codepoint
1008 big-endian length prefix $(p_0,p_1)$; if $p_0>255$ or $p_1>255$
     output
1009 $\bot$ (CVF-43: canonical, byte-valued prefix), else set
     $n=256p_0+p_1$
1010 and, if $n>R-2$, output $\bot$
1011 (Remark~\ref{rem:dec-structural}); (9) output the $n$ codepoints
     following
1012 the prefix.
```

same file

```
\begin{remark}[The post-authentication structural checks]
\label{rem:dec-structural}
Step (3) depends only on $|C|$, a public quantity, and is performed
\emph{before} the tag is computed, so a ciphertext of illegal length
is
rejected without an HMAC evaluation and the rejection reveals nothing
the
adversary did not already know.
Steps (6) and (8) are performed \emph{after} the tag comparison of
step (5),
so they are never reachable on an input an adversary without
$\mathsf{sk}$
could construct, and they play no part in the INT-CTXT argument. They
are
specified nonetheless because step (7) is not total without them: $R$
is
recovered from $|C|$ and $n$ from the decrypted buffer, so a
malformed but
correctly tagged ciphertext — which a key holder can trivially
produce —
would otherwise drive an out-of-range read or an allocation sized by
a field
the caller did not choose. Divisibility by
$\mathrm{MAX\_NOISE\_RUN}+1$
alone does not imply step (6): $R=20$ passes step (3) and fails step
(6).
All three reference implementations enforce these checks in this
order;
vector \texttt{W-N06} of Section~\ref{sec:kats} pins step (3),
\texttt{W-N07} step (6), \texttt{W-N09}--\texttt{W-N10} the per-token
checks of step (7), and \texttt{W-N08} and \texttt{W-N11} step (8).
\end{remark}
```

CVF-V5-29: open Category: **flaw**

Summary

Several v8 functions still check the key on every call with a slow test whose running time depends on the secret key.

Before a key is used, the code checks that it is valid, and the time this check takes depends on the secret values in the key, so someone who can measure it learns a little about the key. The fixes added a key type that is checked only once, when it is created. However, the older functions that take a plain key are still offered and still check it on every message, and the streaming functions have no version that accepts the new key type at all. The check has also become slower, because keys are now larger. The leaked information does not concern a security parameter of the current scheme, so the impact is low.

This is the part of CVF-29 that the fixes did not resolve.

The fixes resolved the following: the new `NapqesKey` type validates the key once, at construction (`key.rs:54-57`), and its `_key` entry points skip validation. §8.4 of the paper now lists key validation as not constant-time.

The following problems remain:

- The v8 functions that take the key as a bare slice still validate it on every message (`lib.rs:1517-1530`, `1611-1625`), and they are not deprecated in favour of the `_key` variants.
- The streaming format has no `_key` variant at all. `encrypt_stream_ae_v8_with_nonce` (`lib.rs:1960-1975`), `decrypt_stream_ae_v8` (`lib.rs:2070-2082`) and `StreamV8Encryptor::new` (`lib.rs:2240-2271`) each validate the key on every stream.
- With the widened prime interval and the new default `K = 13`, one validation costs about 17,400 trial divisions on average (from 6,487 to 25,155, depending on the secret primes), compared with about 11,000 before the fixes.

The information exposed is at most about 12.5 bits per observation. We rate this finding low because, for the v8 format, this information concerns only the prime tuple, which is not a security parameter. The fixes also extended the same behaviour to the v7 decryptors, which is reported separately as CVF-V5-12.

Recommendation

Add variants of the streaming entry points that take a `NapqesKey`, and deprecate the bare-slice v8 functions in favour of the `_key` ones.

Also document in each bare-slice function that it validates the key on every call, with timing that depends on the secret key.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-29 — Fixed in b87e98c. `encrypt_stream_ae_v8_key`, `decrypt_stream_ae_v8_key` and `StreamV8Encryptor::new_key` take a `NapqesKey`. The bare-slice

v8 functions are `#[deprecated]`, and their rustdoc states that they validate on every call by trial division, with timing that depends on the secret primes.

/rust/src/lib.rs

```
pub fn encrypt_bytes_v8_with_profile(
    message: &str,
    primes: &[u64],
    sk: &[u8; SK_SIZE],
    aad: &[u8],
    pad_profile: PadProfile,
) -> Result<Vec<u8>, String> {
    // CVF-47: FIPS 140-3 gate (opt-in via the `fips_gate` feature).
    #[cfg(feature = "fips_gate")]
    self_test::require_post().map_err(|e| e.to_string())?;
    validate_key(primes)?;
    encrypt_bytes_v8_core(message, primes, sk, aad, pad_profile)
}

```

same file

```
pub fn decrypt_bytes_v8(
    ciphertext: &[u8],
    primes: &[u64],
    sk: &[u8; SK_SIZE],
    aad: &[u8],
) -> Result<String, String> {
    // CVF-47: FIPS 140-3 gate (opt-in via the `fips_gate` feature).
    #[cfg(feature = "fips_gate")]
    self_test::require_post().map_err(|e| e.to_string())?;
    validate_key(primes)?;
    decrypt_bytes_v8_core(ciphertext, primes, sk, aad)
}

/// CVF-29 fix: shared v8-decrypt core that skips per-message `validate_key`.
/// See [`encrypt_bytes_v8_core`] for the caller contract.
```

same file

```
pub(crate) fn encrypt_stream_ae_v8_with_nonce(
    plaintext: &str,
    primes: &[u64],
    sk: &[u8; SK_SIZE],
    nonce: &[u8; NONCE_SIZE],
    aad: &[u8],
    frame_codepoints: u32,
) -> Result<Vec<u8>, String> {
```

```
    validate_key(primes)?;
    validate_stream_v8_frame(frame_codepoints)?;

    let sk_fmt = Secret32::new(derive_format_subkey(sk, FORMAT_STREAM_AE_V8));
    encrypt_stream_ae_v8_inner(plaintext, primes, sk_fmt.as_ref(), nonce, aad, frame_codepoints)
}

fn encrypt_stream_ae_v8_inner(
```

same file

```
pub fn decrypt_stream_ae_v8(
    stream: &[u8],
    primes: &[u64],
    sk: &[u8; SK_SIZE],
    aad: &[u8],
) -> Result<String, String> {
    // CVF-47: FIPS 140-3 gate (opt-in via the `fips_gate` feature).
    #[cfg(feature = "fips_gate")]
    self_test::require_post().map_err(|e| e.to_string())?;
    validate_key(primes)?;
    let sk_fmt = Secret32::new(derive_format_subkey(sk, FORMAT_STREAM_AE_V8));
    decrypt_stream_ae_v8_inner(stream, primes, sk_fmt.as_ref(), aad)
}
```

same file

```
    pub fn new(
        primes: &[u64],
        sk: &[u8; SK_SIZE],
        aad: &[u8],
        frame_codepoints: u32,
    ) -> Result<(Self, Vec<u8>), String> {
        // CVF-47: FIPS 140-3 gate (opt-in via the `fips_gate` feature).
        #[cfg(feature = "fips_gate")]
        crate::self_test::require_post().map_err(|e| e.to_string())?;
        validate_key(primes)?;
        validate_stream_v8_frame(frame_codepoints)?;
        let nonce = generate_nonce_with_crng_check()?;
        let sk_fmt = derive_format_subkey(sk, FORMAT_STREAM_AE_V8);
        let mut header = Vec::with_capacity(1 + 4 + NONCE_SIZE);
        header.push(FORMAT_STREAM_AE_V8);
        header.extend_from_slice(&frame_codepoints.to_be_bytes());
        header.extend_from_slice(&nonce);
```

```
        Ok((
            Self {
                primes: primes.to_vec(),
                sk_fmt,
                aad: aad.to_vec(),
                nonce,
                f: frame_codepoints,
                chunk_idx: 0,
                total_real_cps: 0,
               cp_buf: Vec::with_capacity(frame_codepoints as usize),
                finished: false,
            },
            header,
        ))
    }
```

CVF-V5-30: open Category: **efficiency**

> **Summary**
> **Prime generation and primality testing still have three of the four inefficiencies reported earlier, and key generation still accepts any number of key elements.**
> Creating and checking a secret key involves finding and testing prime numbers. The fixes removed one of the four inefficiencies that we reported in this code, and they added a limit on the size of the keys that are accepted. However, the other three inefficiencies remain, and the function that creates new keys does not apply the same size limit. The cost is small at the default key size, and the problems mainly affect performance.

This is the part of CVF-41 that the fixes did not resolve.

The fixes resolved the following. `MAX_KEY_ELEMENTS = 128` is enforced, `validate_key` checks the length and the range of the key before testing primality, and the quadratic distinctness check was replaced by sorting followed by a single scan. The test `cvf41_validate_key_rejects_key_above_max_elements` fails on the code before the fixes.

The following problems remain:

- `is_prime` (`lib.rs:172-190`) is unchanged: it does not use a table of small primes, and it recomputes the square-root bound on every iteration instead of computing it once before the loop.

- `generate_prime_numbers` (`lib.rs:208-239`) still checks distinctness with a linear `contains` for each candidate, which is quadratic in `count`, and `count` is not capped. The missing cap is reported separately in CVF-V5-14.
- Validation on every message remains on every path other than the `_key` ones (see CVF-V5-29).

The sorting step that the fixes introduced also leaves a sorted copy of the key in memory without wiping it, which is reported separately in CVF-V5-11.

Recommendation

In `is_prime`, compute the square-root bound once, and trial-divide first by a small table of primes.

In `generate_prime_numbers`, cap `count` at `MAX_KEY_ELEMENTS`, and track membership with a sorted vector or a hash set instead of `contains`.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-30 — Fixed in `75de8cf`. `is_prime` first trial-divides by the odd primes below 59 and computes the square-root bound once; it is checked against a sieve. Key generation tracks membership by binary search in a sorted, wiped index, and its count is capped (CVF-V5-14).

/rust/src/lib.rs

```rust
pub fn is_prime(n: u64) -> bool {
    if n < 2 {
        return false;
    }
    if n == 2 {
        return true;
    }
    if n % 2 == 0 {
        return false;
    }
    let mut i: u64 = 3;
    while i.saturating_mul(i) <= n {
        if n % i == 0 {
            return false;
        }
        i += 2;
    }
    true
}
```

same file

```rust
pub fn generate_prime_numbers(count: usize, min_val: u64, max_val: u64) -> Result<Vec<u64>, String> {
    use rand::distributions::{Distribution, Uniform};
```

```
    if max_val <= min_val {
        return Err(format!(
            "generate_prime_numbers: empty range [{}, {}]",
            min_val, max_val
        ));
    }
    let mut rng = rand::thread_rng();
    let dist = Uniform::new_inclusive(min_val, max_val);
    let max_attempts = (count.max(1) as
u64).saturating_mul(KEYGEN_DRAWS_PER_PRIME);
    let mut primes: Vec<u64> = Vec::with_capacity(count);
    let mut attempts: u64 = 0;
    while primes.len() < count && attempts < max_attempts {
        // CVF-20: unbiased sample via rand::Uniform (Lemire
rejection),
        // replacing the modulo-bias line `min_val + (rng.next_u64()
% span)`.
        let num = dist.sample(&mut rng);
        if is_prime(num) && !primes.contains(&num) {
            primes.push(num);
        }
        attempts += 1;
    }
    if primes.len() < count {
        return Err(format!(
           "generate_prime_numbers: could not find {} distinct primes
in \
             [{}, {}] within {} draws; the range is too narrow or too
prime-poor \
             for the requested count.",
            count, min_val, max_val, max_attempts
        ));
    }
    Ok(primes)
}
```

CVF-V5-31: open Category: **architecture**

Summary

The code that produces the encrypted tokens is still copied by hand in several places, so every future change has to be made identically in each copy.

The central step of encryption, which turns a message into a sequence of encrypted tokens, is written out more than once in the code instead of being written once and reused. The fixes merged the copies for the older format, but they added a new copy for the streaming format, and another part of the older format is still repeated three times. Today all the copies behave in the same way. However, any future change has to be made identically in every place, and a missed copy would make two encryption paths behave differently.

This is the part of CVF-46 that the fixes did not resolve.

The fixes resolved the following: the four v7 copies were replaced by one function, `emit_tokens_v7`. A statement-by-statement comparison with the copies before the fixes shows that this extraction does not change behaviour, and the loop is checked by re-encrypting vectors V001 to V022 and by the KAT-1 self-test.

The following problems remain:

- The v8 block loop (`lib.rs:1569-1598`) and the streaming-chunk loop (`lib.rs:1805-1835`) are two separate hand-made copies of the same statements. The streaming copy was added by the fixes, which is exactly the situation that CVF-46 warned about: the next change to the emission rule must be made twice.
- The v7 sealing tail, i.e. the masking, the tag computation and the assembly of the output, is still copied three times (`lib.rs:1072-1080`, `1120-1128`, `1244-1252`).
- The restructuring that moves the two statements duplicated in both branches of the loop out of the branches was not done.

We rate this finding low because the number of copies fell from five to three.

Recommendation

Extract one `emit_tokens_v8` function, parameterised by the counters of the chunk, and use it in both the block and the streaming encryptors.

Similarly, extract one v7 sealing helper and use it in the three v7 encryptors. Also move the statements that are common to both branches of the loop out of the branches.

Client Comment

Client statement, not verified by ABDK.

CVF-V5-31 — Fixed in 25310e5. `emit_tokens_v8` is the single v8 emission loop for block mode and every stream chunk, and `seal_v7` is the single v7 sealing pipeline. In both emitters, the statements common to the noise and real branches are hoisted. Behaviour is unchanged: all three KAT corpora re-encrypt byte for byte.

/rust/src/lib.rs

```
    let blob = fixed_encode_tokens(&cypher);
    let ks = varint_keystream(&kb, &nonce, blob.len());
    let masked: Vec<u8> = blob.iter().zip(ks.iter()).map(|(a, b)| a ^ b).collect();
    let mut payload = Vec::with_capacity(NONCE_SIZE + masked.len());
    payload.extend_from_slice(&nonce);
    payload.extend_from_slice(&masked);
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
    payload.extend_from_slice(&tag);
    Ok(payload)
```

same file

```
    let blob = fixed_encode_tokens(&cypher);
    let ks = varint_keystream(&kb, &nonce, blob.len());
    let masked: Vec<u8> = blob.iter().zip(ks.iter()).map(|(a, b)| a ^ b).collect();
    let mut payload = Vec::with_capacity(NONCE_SIZE + masked.len());
    payload.extend_from_slice(&nonce);
    payload.extend_from_slice(&masked);
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
    payload.extend_from_slice(&tag);
    payload
```

same file

```
    let blob = fixed_encode_tokens(&cypher);
    let ks = varint_keystream(&kb, &nonce, blob.len());
    let masked: Vec<u8> = blob.iter().zip(ks.iter()).map(|(a, b)| a ^ b).collect();
    let mut payload = Vec::with_capacity(NONCE_SIZE + masked.len());
    payload.extend_from_slice(&nonce);
    payload.extend_from_slice(&masked);
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
    payload.extend_from_slice(&tag);
    Ok(payload)
```

same file

```
    for &c in &padded {
        let mut noise_run: u64 = 0;
        loop {
            if noise_run < MAX_NOISE_RUN && is_noise_pos_v8(sk_fmt, &nonce, ct_pos, noise_theta) {
                let k = primes[(real_idx % kk) as usize];
                let nc = derive_noise_char(sk_fmt, &nonce, ct_pos);
                let na = derive_noise_token_addend(sk_fmt, &nonce, ct_pos, k);
                cypher.push(nc * k + na);
```

```
                ct_pos += 1;
                noise_run += 1;
            } else {
                let k = primes[(real_idx % kk) as usize];
                let addend = derive_addend(sk_fmt, &nonce, real_idx,
k);
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }
    // V2-CVF2 fix: pad up to the fixed, bucket-only ceiling so
ciphertext
    // length never depends on the message-derived nonce's noise
realisation.
    let ceiling = (padded.len() as u64) * (MAX_NOISE_RUN + 1);
    while (cypher.len() as u64) < ceiling {
        let k = primes[(real_idx % kk) as usize];
        let nc = derive_noise_char(sk_fmt, &nonce, ct_pos);
        let na = derive_noise_token_addend(sk_fmt, &nonce, ct_pos, k);
        cypher.push(nc * k + na);
        ct_pos += 1;
    }
```

same file

```
    for &c in chunk_cps {
        let mut noise_run: u64 = 0;
        loop {
            if noise_run < MAX_NOISE_RUN
                && is_noise_pos_v8(sk_fmt, sub_nonce, ct_pos,
noise_theta)
            {
                let k = primes[(real_idx % kk) as usize];
                let nc = derive_noise_char(sk_fmt, sub_nonce, ct_pos);
                let na = derive_noise_token_addend(sk_fmt, sub_nonce,
ct_pos, k);
                cypher.push(nc * k + na);
                ct_pos += 1;
                noise_run += 1;
            } else {
                let k = primes[(real_idx % kk) as usize];
                let addend = derive_addend(sk_fmt, sub_nonce,
real_idx, k);
                cypher.push(c as u64 * k + addend);
```

```
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }

    let ceiling = f * (MAX_NOISE_RUN + 1);
    while (cypher.len() as u64) < ceiling {
        let k = primes[(real_idx % kk) as usize];
        let nc = derive_noise_char(sk_fmt, sub_nonce, ct_pos);
        let na = derive_noise_token_addend(sk_fmt, sub_nonce, ct_pos,
k);
        cypher.push(nc * k + na);
        ct_pos += 1;
    }
```

7. Conclusion

This report covers the Rust reference implementation of NAPQES. The version 4 paper was used as the reference against which the code was checked; the paper itself was audited separately, and its findings are reported in a separate document. Overall, the implementation is careful and close to the construction that the paper describes, and the client's fixes resolved most of the problems we found. The main remaining risks are in the legacy v7 format, which the crate still ships next to the current v8 format, and in new code added with the fixes.

Review of the baseline code. In the code at commit `b8c3054` we found thirty-seven problems: three major, seventeen moderate and seventeen minor (Section 5).

The most important one, CVF-11, is that the code and the specification compute different functions. The code passes the secret key through an extra step, a format subkey $\mathrm{sk}_{\mathrm{fmt}} = \mathrm{HMAC}\big(\mathrm{sk}, \texttt{0x0B} \| \texttt{0x01}\big)$, that the specification does not mention. We verified this experimentally: a re-implementation written from the paper alone reproduced none of the twelve delivered test vectors, and all twelve once this one step was added. The extra step is a good design choice, since it keeps different wire formats apart, and it causes no loss of security. However, an implementation written from the paper could not work with

the code, the paper's own conformance test could not be passed, and the security theorems were proved for a slightly different function. The client has fixed this by describing the step in the specification.

The other two major problems were in the code. In CVF-13, six legacy functions accepted an empty ciphertext as a valid encryption of the empty message under any key, so anyone could make a receiver accept an empty message without knowing the key, including through the crate's own network message layer. In CVF-18, the mandatory start-up self-test checked only the legacy v7 format, and its integrity test compared the crate's version number with a fixed value, so it detected no tampering and would block the module after the next change of version number. A related moderate finding, CVF-47, showed that the result of the self-test was never enforced.

Most of the moderate findings concern the legacy v7 format. It still uses the older key schedule with a single secret, which the v8 design was created to replace; it lacks the limits that make the ciphertext length depend only on the padding block; and it validates no key. Much of the delivered evidence, such as the fuzzing set-up, part of the known-answer tests and the statistical study in Section 9 of the paper, also exercises v7 instead of v8. A second group of moderate findings concerns claims about the implementation that the code did not support, for example the constant-time tag comparison, the upper bound on the key primes and the absence of panics. The minor findings are documentation and maintenance problems: outdated comments and README text, tests that cannot fail, unused zeroization helpers and similar inconsistencies between the specification and the code.

Review of the fixes. The client then fixed the code, and we reviewed commit `acbf9d0` (Section 6). Of the thirty-seven findings, twenty-five are fixed and twelve are partially fixed; none is unresolved and none has become worse. The specification now describes the scheme that the code computes: a re-implementation from the updated specification reproduced all twenty-two positive v8 test vectors. In the fixed code we report thirty findings, eight moderate and twenty-two minor. Twelve of them describe what remains of the partially fixed findings. Seventeen were introduced by the fixes, mostly by the new v8 streaming format, which reveals the exact length of the plaintext and is not described in the specification (CVF-V5-1, CVF-V5-2), by the widened interval for the key primes (CVF-V5-3), and by the new self-test gate, which is off by default (CVF-V5-8). The remaining one is an older problem reported for the first time.

The client has commented on every finding in Section 6 and reports a fix for each, in later commits up to `a2fbd13`. We have not reviewed these later fixes, and the status of each finding reflects commit `acbf9d0`. The Notes for Users at the start of the findings summarise what developers who use the crate should know.

Limits of this review. The Python and C implementations were never delivered to us, so every statement about the three implementations agreeing rests on the Rust code, the stored test vectors, and what the paper and the code comments say. The delivered repository is also much larger than the reviewed files: the key-exchange module, the network message layer, the protocol adapters and the programs were not audited, even though the reviewed crate exports some of them. We came across some of that code

during the review, and we recorded what we saw in the two sections that follow. Neither of them contains audited findings.

Overall assessment. The v8 encryption path is sound, and we found no way to break it from outside. Its main defect, the undocumented key step, has been resolved in the specification. The remaining risk is concentrated in the legacy v7 format, which shares the public interface and the key type with v8, and in the new streaming format added with the fixes. We recommend that the client decide explicitly whether v7 is still supported and, if it is not, remove it, and that the streaming format be specified and reviewed before it is used.

8. Out-of-Scope Observations (Code Not Reviewed)

The code discussed in this section was not part of the reviewed surface. None of the items below is an audited finding, none has been assessed to the standard the findings above were held to, and none should be treated as one. They are recorded because we encountered them incidentally while establishing the reach of findings that are in scope, and because a client is better served by seeing them than by not.

The two we would most want the client to look at first both sit in the transport and key-establishment code that the audited crate re-exports as public modules of itself.

The first concerns the prime interval. Section 3.1 of the paper carries the document's only **MUST**: implementations must draw key elements from $\mathcal{P}$, the primes in $[10^6,\ 9.9 \times 10^6]$. The audited file honours it. `src/kem.rs`, which does not, declares its own interval $[10^6,\ 1.5 \times 10^7)$ and samples thirteen key elements from it. By an independent sieve there are $892{,}206$ primes in that interval, of which $312{,}259$ — exactly 35.00% — lie above max $\mathcal{P}$. A tuple produced there therefore carries 4.55 out-of-interval elements on average, and the probability that all thirteen are in range is about 0.37%. The audited key-validation routine accepts every one of them, because it enforces no upper bound. This matters for a finding that is in scope: it converts that finding's overflow precondition from a hypothetical caller error into a routine product of the crate's own code. Read with `kem.rs` in view, the in-scope finding is no longer "a caller who supplies an oversized prime overflows the token multiplication" but "the crate's own key-derivation path produces out-of-interval keys on essentially every invocation, and only the size of the excess keeps it below the overflow threshold". The same function also samples by an unrejected modulo reduction, disagrees with the audited sampler on whether the interval is closed at the top, and runs an unbounded search loop with no failure path.

The second concerns replay protection. `ot_frame::unwrap_pdu` validates and commits the per-peer sequence counter before it authenticates the payload: the envelope header is

parsed, the counter is checked and written, and only then is a tag verified. An attacker who can inject a single frame on the wire, holding no key material, therefore advances that counter to any value they choose, after which every genuine frame from the real peer is rejected as a replay. Choosing the maximum value locks the affected peer out permanently; the state is process-global, so recovery requires restarting the gateway, and the attack can be repeated immediately. This holds for any frame whose tag fails, so it is independent of the empty-envelope authentication bypass reported above — though the two compose badly, because an empty envelope is accepted rather than merely counted. The standard remedy is to move the counter write after tag verification; since the sequence number is already covered by the associated data, a frame that authenticates has proved the sender knew the key when it chose that number, which is exactly the property the counter needs.

Separately, and because it bears directly on the reach of a major finding: `ot_frame::unwrap_pdu` reaches the legacy decryption entry point, so the empty-ciphertext authentication bypass reported above is reachable from the frame layer, and through it from the wire. That is part of why we assess that finding as major rather than moderate.

Four further observations are recorded in our working notes and can be supplied in full on request: the transport path derives no independent 256-bit secret at all and runs the pre-v8 single-secret key schedule, so no theorem in the paper applies to any traffic the gateway carries; the file-backed key store loads device key material from an unvalidated plaintext JSON file with no validation, no independent secret and no zeroization; a key-derivation salt still carries a v6 domain label in a v8 crate; and the generator that produced the Section 9 bitstream calls the legacy encryptor, which is where the code-side half of an in-scope finding would be fixed.

We would put one question to the client ahead of any of this. If the position is that these modules are demonstration or integration code rather than part of the product, then most of the above becomes a scoping and documentation matter rather than a security one. Nothing in the crate, the README or the paper currently says that, and the audited file exports all of them as public modules, so the artifacts do not presently support that reading.

9. Evidence Provenance: The Section 9 Statistical Report

This section records a discrepancy in a delivered evidence artifact. It is not an audited finding and we are not asserting a conclusion about it; we are asking the client to explain it.

Section 9 of the paper reports a NIST SP 800-22 statistical study. The delivered tree contains both the pipeline that runs such a study, `tests/sts_pipeline.py`, and a stored result, `tests/sts_report_v6.json`. The stored result cannot have been produced by that pipeline, and it reports an outcome the pipeline is incapable of producing.

The schemas do not match. Each per-test record the pipeline emits carries the fields `name`, `passed`, `skipped_nistrng_bug`, `p_value` and `p_values`, and its summary block counts `total`, `passed`, `failed` and `skipped_nistrng_bug`. Every record in the stored file instead carries `name`, `passed`, `skipped`, `p_value` and `note`; there is no `p_values` array anywhere in it, and its summary counts `skipped` rather than `skipped_nistrng_bug`. The granularity differs too: the stored file lists forty entries, breaking the random-excursion families out into one row per state in the manner of the original NIST suite, where the pipeline emits one row per test as its library returns them.

The substantive point is the second one. The pipeline declares a set `_NISTRNG_BROKEN_TESTS` naming eight tests — the discrete Fourier transform, linear complexity, non-overlapping template matching, serial, approximate entropy, random excursion, random excursion variant and Maurer's universal test — which it treats as unusable because of a defect in the library it drives, and which it counts as skipped rather than as passed or failed. Any genuine run of that pipeline must therefore report at least eight skips. The stored file reports every one of those eight as a clean pass, with `skipped` false on each record and a summary skip count of zero, and reports forty passes out of forty with no failures and no skips.

We draw no conclusion from this beyond the obvious one, that the stored artifact and the delivered pipeline are not the same evidence. The file is also labelled as version v6, two format revisions behind the scheme the paper specifies, which is consistent with an in-scope finding reporting that the Section 9 result is asserted of a scheme other than the one it was measured on. We ask the client to say which tool produced `sts_report_v6.json`, against which version of the scheme, and whether the eight tests the delivered pipeline considers unusable were in fact run.



ABDK
Consulting

About Us

Established in 2016, is a leading service provider in the space of blockchain development and audit. It has contributed to numerous blockchain projects, and coauthored some widely known blockchain primitives like Poseidon hash function.

The ABDK Audit Team, led by Mikhail Vladimirov and Dmitry Khovratovich, has conducted over 300 audits of blockchain projects in Solidity, Rust, Circom, C++, Javascript, and other languages.

Contacts

EMail

info@abdkconsulting.com

Website

abdk.consulting

X (former Twitter)

@ABDKConsulting

LinkedIn

company/abdk-consulting